DEVOPS ENGINEERING MASTERY

CLOUD ENGINEERING — GOOGLE CLOUD PLATFORM

Google Cloud Architecture, IAM, VPC, Compute Engine, Cloud Storage, Cloud SQL, GKE, Cloud Run, Serverless, Security & Production Cloud Operations

VOLUME 13 • FIRST EDITION

```
nahid@technova:~$ gcloud compute instances list
✓ NAME              ZONE          STATUS
✓ technova-web-01   asia-south1-a   RUNNING
nahid@technova:~$ gcloud container clusters list
```

GCP • IAM • VPC • Compute Engine • Cloud SQL • GKE • Cloud Run

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Cloud Engineering — Google Cloud Platform (GCP)

DEVOPS ENGINEERING MASTERY — Volume 13
Google Cloud Architecture, IAM, VPC, Compute Engine, Cloud Storage, Cloud SQL, GKE, Cloud Run, Serverless, Security ও Production Cloud Operations

First Edition — Published 9 November 2026.

Hands-on environment: Google Cloud Platform with the gcloud CLI and Cloud Shell, Terraform (Google provider) where IaC examples are used, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud chapters name the resources that incur charges and always end with a cleanup step — never leave cloud resources running after a lab. Official Google Cloud documentation is the authority; no pricing, quotas or service limits are stated as permanent facts in this book — check the official Google Cloud Pricing Calculator before running a lab.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Google Cloud project beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy gcloud commands blindly; read the explanation under each command and configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on production infrastructure: use a disposable Google Cloud project with billing budgets and alerts, never paste a real secret into an example, never commit a service account key or a secret to Git, and always delete the resources you create at the end of every lab.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **TROUBLESHOOTING** helps you fix a failure, and **LAB** marks hands-on work. Console steps and gcloud CLI steps are both given; the CLI step is the one to automate.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Infrastructure changes should first be practised in a disposable environment, never directly on production systems. The author and publisher accept no liability for data loss, service disruption or cloud charges caused by running commands without a backup, a plan review or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux concepts, has studied Linux server administration, networking, CI/CD, Docker, Kubernetes, Terraform

and Ansible, and has studied AWS and Azure fundamentals — and now wants practical Google Cloud Platform engineering, from a first project and VPC to a governed, multi-project production platform with GKE, Cloud Run, managed databases, observability, security and disaster recovery.

Learning outcomes

By the end of this book you will be able to: explain Google Cloud's global infrastructure, resource hierarchy and billing model; design organizations, folders, projects and IAM with least privilege and service accounts; build global VPC networks with subnets, firewall rules, Cloud NAT and private connectivity; run Compute Engine VMs and Managed Instance Groups with autoscaling; store data in Cloud Storage, Persistent Disk and Filestore with backup and recovery; operate Cloud SQL, AlloyDB, Firestore, Bigtable and Memorystore; deploy applications on App Engine, Cloud Run and GKE; build event-driven systems with Pub/Sub and Eventarc; run CI/CD with Cloud Build, Cloud Deploy and GitHub Actions; provision infrastructure with Terraform and Infrastructure Manager; monitor with Cloud Monitoring, Logging and Trace; secure workloads with Cloud KMS, Secret Manager, Cloud Armor and VPC Service Controls; and operate a production GCP platform with reliability, disaster recovery and FinOps discipline.

সূচিপত্র — Contents

PART I — GCP Foundations & Architecture

Chapter 01 — Cloud Computing and Google Cloud Global Infrastructure 7

Chapter 02 — Organizations, Folders, Projects and Governance 13

Chapter 03 — Cloud IAM and Identity Engineering 19

Chapter 04 — Google Cloud CLI and Automation 25

Chapter 05 — Shared Responsibility, Architecture and Billing 31

PART II — Networking & Compute Engineering

Chapter 06 — VPC Networks and Subnet Engineering 37

Chapter 07 — Routes, Firewall and Private Connectivity 44

Chapter 08 — DNS, Load Balancing and CDN 50

Chapter 09 — Compute Engine Engineering 57

Chapter 10 — Hybrid and Enterprise Networking 63

Chapter 11 — Managed Instance Groups and High Availability 69

PART III — Storage, Database & Application Services

Chapter 12 — Cloud Storage Engineering 77

Chapter 13 — Persistent Storage and Backup 87

Chapter 14 — Cloud SQL and AlloyDB 95

Chapter 15 — Firestore, Bigtable and Memorystore 104

Chapter 16 — Application Hosting and Edge Delivery 112

PART IV — Containers, Serverless & Delivery

Chapter 17 — Container Engineering with GKE 122

Chapter 18 — Serverless and Event-Driven Architecture 130

Chapter 19 — CI/CD with Google Cloud 136

Chapter 20 — Infrastructure as Code 142

PART V — Production GCP Engineering

Chapter 21 — Monitoring, Logging and Tracing 148

Chapter 22 — GCP Security Engineering 152

Chapter 23 — Reliability, Backup and Disaster Recovery 157

Chapter 24 — Production Architecture and FinOps 161

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150+ Terms) • HCL & Command References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Terraform Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, gcloud, IAM, VPC and Terraform references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

GCP Foundations & Architecture

Cloud Computing ↬ Google Cloud Global Infrastructure, Organization, Folder ↬ Project, Cloud IAM, gcloud CLI ↬ Automation, Shared Responsibility, Well-Architected Framework ↬ Billing।

অধ্যায় ১ • PART I — GCP Foundations & Architecture

Chapter 1 — Cloud Computing and Google Cloud Global Infrastructure

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ক্লাউড কম্পিউটিংয়ের সংজ্ঞা, বৈশিষ্ট্য ও সার্ভিস মডেল (IaaS, PaaS, SaaS) প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউডের মধ্যে পার্থক্য ও সিদ্ধান্তের মানদণ্ড বলতে; (৩) Google Cloud-এর Region, Zone, global ও regional রিসোর্সের পার্থক্য ব্যাখ্যা করতে; (৪) Google-এর গ্লোবাল নেটওয়ার্ক, fault domain, availability ও latency-র সম্পর্ক বিশ্লেষণ করতে; (৫) একটি ছোট অ্যাপ্লিকেশনের জন্য সঠিক Region ও Zone নির্বাচন করতে এবং (৬) ক্লাউড আর্কিটেকচারের মূল নীতিগুলো — elasticity, decoupling, automation, observability — বাস্তব নকশায় প্রয়োগ করতে।

১.২ পূর্বজ্ঞান ঝালাই

এই অধ্যায়ে ঢোকার আগে আপনার জানা থাকা ভালো: একটি সার্ভার কী — CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক ইন্টারফেসের সমন্বয়; IP ঠিকানা, DNS ও HTTP অনুরোধ কীভাবে কাজ করে; ভার্চুয়ালাইজেশন কী — একই ফিজিক্যাল মেশিনে একাধিক ভার্চুয়াল মেশিন চালানো; এবং Linux কমান্ড লাইনের প্রাথমিক ধারণা। আপনি যদি নিজের ল্যাপটপে Docker চালিয়ে থাকেন, তাহলে ক্লাউডের ধারণা বুঝতে সুবিধা হবে: Docker যেমন অ্যাপ্লিকেশনকে তার নির্ভরতাসহ প্যাকেজ করে, ক্লাউড তেমনি পুরো অবকাঠামোকে চাহিদা অনুযায়ী ভাড়া দেওয়ার ব্যবস্থা করে। মনে রাখবেন, ক্লাউড কোনো জাদু নয় — এটি অন্য কারও ডেটা সেন্টারে চলা কম্পিউটার, নেটওয়ার্ক ও স্টোরেজ, যা API দিয়ে নিয়ন্ত্রণ করা যায়। এই সরল সত্যটি মনে রাখলে পরের সব জটিল বিষয় সহজ হয়ে যাবে।

১.৩ ক্লাউড কম্পিউটিং কী এবং কেন এটি এলো

ঐতিহ্যগতভাবে একটি কোম্পানিকে নিজের সার্ভার কিনতে হতো, ডেটা সেন্টারে রাখতে হতো, বিদ্যুৎ, কুলিং, নেটওয়ার্ক ও হার্ডওয়্যার রক্ষণাবেক্ষণের দায়িত্ব নিতে হতো। চাহিদা বাড়লে নতুন সার্ভার কিনতে সপ্তাহ বা মাস লেগে যেত; চাহিদা কমলে কেনা সার্ভার অলস পড়ে থাকত। এই সমস্যার নাম *capacity planning* — ভবিষ্যতের চাহিদা অনুমান করে আগে থেকে সম্পদ কেনা। অনুমান ভুল হলে হয় খরচ বাড়ে, নয়তো ব্যবহারকারী সেবা পায় না। ক্লাউড কম্পিউটিং এই সমস্যার সমাধান দেয় তিনটি উপায়ে: **on-demand self-service** — ব্যবহারকারী নিজেই API বা কনসোল থেকে সম্পদ নিতে পারে; **elasticity** — চাহিদা অনুযায়ী সম্পদ বাড়ানো-কমানো যায়; এবং **pay-as-you-go** — যতটুকু ব্যবহার, ততটুকু খরচ।

ক্লাউডের পাঁচটি মৌলিক বৈশিষ্ট্য মনে রাখুন: on-demand self-service, broad network access (ইন্টারনেট দিয়ে যেকোনো জায়গা থেকে ব্যবহার), resource pooling (একাধিক গ্রাহকের মধ্যে সম্পদ ভাগ করে ব্যবহার, তবে যৌক্তিকভাবে বিচ্ছিন্ন), rapid elasticity, এবং measured service (ব্যবহার মাপা হয় ও বিল হয়)। এই বৈশিষ্ট্যগুলো একসাথে না থাকলে সেটি প্রকৃত ক্লাউড নয় — সেটি কেবল অন্যের সার্ভারে হোস্টিং।

> **CONCEPT:** ক্লাউডকে ভাবুন বিদ্যুৎ গ্রিডের মতো। আগে প্রতিটি কারখানাকে নিজের জেনারেটর চালাতে হতো; এখন গ্রিড থেকে যতটুকু দরকার ততটুকু বিদ্যুৎ নেওয়া যায়। ক্লাউডও তেমনি কম্পিউটিংকে একটি ইউটিলিটি বানিয়েছে — তবে একটি পার্থক্য আছে: বিদ্যুৎ ব্যবহার করলে ডেটা কোথায় থাকে, কে দেখে, কীভাবে সুরক্ষিত থাকে — এসব প্রশ্নের উত্তর আপনাকেই দিতে হয়। তাই ক্লাউড শেখা মানে শুধু সার্ভার চালানো শেখা নয়, দায়িত্ব ভাগাভাগি শেখা।

১.৪ সার্ভিস মডেল: IaaS, PaaS, SaaS

Infrastructure as a Service (IaaS) — যেমন Compute Engine ভার্চুয়াল মেশিন, Cloud Storage, VPC নেটওয়ার্ক। এখানে আপনি অপারেটিং সিস্টেম থেকে ওপরের সবকিছু নিয়ন্ত্রণ করেন; Google শুধু ফিজিক্যাল হার্ডওয়্যার, ভার্চুয়ালাইজেশন ও ডেটা সেন্টার দেখে। নিয়ন্ত্রণ সর্বোচ্চ, কিন্তু পরিচালনার দায়িত্বও সর্বোচ্চ। **Platform as a Service (PaaS)** — যেমন Cloud Run, App Engine, Cloud SQL। এখানে আপনি কোড ও ডেটা দেন; প্ল্যাটফর্ম রানটাইম, স্কেলিং, প্যাচিং দেখে। নিয়ন্ত্রণ কম, কিন্তু গতি বেশি। **Software as a Service (SaaS)** — যেমন Gmail, Google Workspace। আপনি শুধু ব্যবহার করেন; কিছুই পরিচালনা করেন না।

কোনটি কখন? একটি কাস্টম নেটওয়ার্ক টপোলজি বা বিশেষ কার্নেল দরকার হলে IaaS; দ্রুত প্রোডাক্ট বাজারে আনতে হলে PaaS; আর ইমেইল বা অফিস সফটওয়্যারের মতো সাধারণ কাজে SaaS। TechNova-র উদাহরণ নিন: তাদের ওয়েব API Cloud Run-এ (PaaS) চলে কারণ ট্রাফিক ওঠানামা করে; তাদের লিগ্যাসি মনোলিথ Compute Engine-এ (IaaS) চলে কারণ সেটির বিশেষ নেটওয়ার্ক সেটিং দরকার; আর তাদের অফিসের ইমেইল Google Workspace-এ (SaaS)। একই কোম্পানিতে তিনটি মডেলই থাকতে পারে — এতে কোনো ভুল নেই।

দিক	IaaS	PaaS	SaaS
উদাহরণ (GCP)	Compute Engine, VPC, Cloud Storage	Cloud Run, App Engine, Cloud SQL	Google Workspace, Gmail
আপনি নিয়ন্ত্রণ করেন	OS, রানটাইম, অ্যাপ, ডেটা	অ্যাপ, ডেটা	শুধু ব্যবহার ও কনফিগারেশন
পরিচালনার ভার	সবচেয়ে বেশি	মাঝারি	সবচেয়ে কম
স্কেলিং	আপনি করবেন	প্ল্যাটফর্ম করে	স্বয়ংক্রিয়
কখন ব্যবহার করবেন	বিশেষ নিয়ন্ত্রণ দরকার হলে	দ্রুত ডেলিভারি চাইলে	সাধারণ কাজে

১.৫ পাবলিক, প্রাইভেট ও হাইব্রিড ক্লাউড

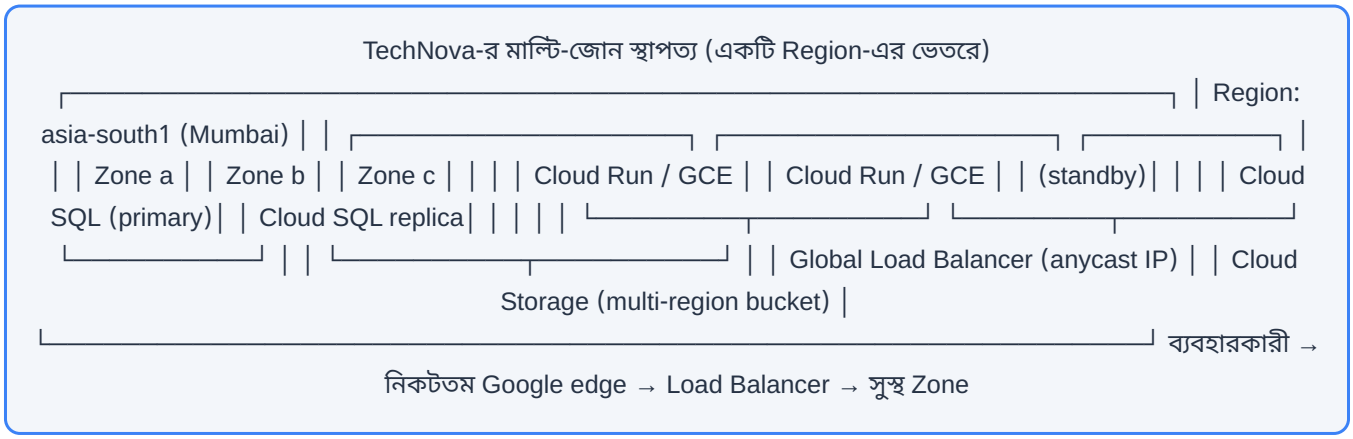
পাবলিক ক্লাউড হলো Google Cloud-এর মতো ভাগ করা অবকাঠামো — অনেক গ্রাহক একই ফিজিক্যাল সম্পদ ব্যবহার করেন, তবে ভার্চুয়ালভাবে সম্পূর্ণ বিচ্ছিন্ন থাকেন। সুবিধা: কম প্রাথমিক খরচ, দ্রুত স্কেল, বিশ্বজুড়ে উপস্থিতি। ঝুঁকি: ডেটা কোথায় থাকে ও কে পরিচালনা করে, তা নিয়ে নিয়ন্ত্রণ কম। **প্রাইভেট ক্লাউড** হলো শুধু একটি সংস্থার জন্য আলাদা অবকাঠামো — নিজের ডেটা সেন্টার বা ডেডিকেটেড হোস্টিং। নিয়ন্ত্রণ বেশি, খরচ বেশি, স্কেল ধীর। **হাইব্রিড ক্লাউড** হলো দুয়ের মিশ্রণ — সংবেদনশীল ডেটা প্রাইভেটে, পরিবর্তনশীল কাজ পাবলিকে। **মাল্টি-ক্লাউড** হলো একাধিক পাবলিক ক্লাউড একসাথে ব্যবহার করা — যেমন GCP ও অন্য প্রদানকারী।

সিদ্ধান্ত নেওয়ার মানদণ্ড: ডেটা সার্বভৌমত্ব ও নিয়ন্ত্রক বাধ্যবাধকতা (যেমন ব্যাংকিং বা স্বাস্থ্য ডেটা), latency (ব্যবহারকারীর কাছাকাছি Region আছে কি না), খরচের ধরন (স্থির খরচ বনাম পরিবর্তনশীল), এবং দলের দক্ষতা। একটি সাধারণ ভুল হলো "হাইব্রিড" শব্দটি ব্যবহার করে আসলে দুটি আলাদা পরিবেশ চালানো, যাদের মধ্যে কোনো সামঞ্জস্যপূর্ণ নেটওয়ার্ক, পরিচয় ব্যবস্থা বা পর্যবেক্ষণ নেই। হাইব্রিড সফল হয় তখনই, যখন পরিচয় (identity), নেটওয়ার্ক ও পর্যবেক্ষণ একইভাবে কাজ করে।

১.৬ Google Cloud-এর গ্লোবাল অবকাঠামো: Region ও Zone

Google Cloud-এর ভৌগোলিক এককগুলো বুঝুন। **Region** হলো একটি ভৌগোলিক এলাকায় অবস্থিত ডেটা সেন্টারের সমষ্টি — যেমন `asia-south1` (মুম্বাই), `asia-southeast1` (সিঙ্গাপুর), `us-central1` (আইওয়া)। প্রতিটি Region-এ সাধারণত তিন বা তার বেশি **Zone** থাকে — যেমন `asia-south1-a`, `asia-south1-b`, `asia-south1-c`। একটি Zone হলো একটি বা একাধিক ফিজিক্যাল ডেটা সেন্টার ভবন, যার নিজস্ব বিদ্যুৎ, কুলিং ও নেটওয়ার্ক আছে। Zone-গুলো একই Region-এর ভেতরে একে অপরের থেকে যথেষ্ট দূরে থাকে যাতে একটি ভবনের আগুন বা বিদ্যুৎ বিভ্রাট অন্য Zone-কে প্রভাবিত না করে, আবার যথেষ্ট কাছে থাকে যাতে Zone-গুলোর মধ্যে latency খুব কম (সাধারণত এক অঙ্কের মিলিসেকেন্ড) হয়।

এই বিভাজন কেন গুরুত্বপূর্ণ? কারণ ব্যর্থতা (failure) একটি বাস্তবতা — ডিস্ক নষ্ট হয়, বিদ্যুৎ যায়, নেটওয়ার্ক কাটা পড়ে। Region ও Zone-এর নকশা আপনাকে বলে: কোন ব্যর্থতা থেকে আপনি বাঁচতে পারবেন, আর কোনটি থেকে পারবেন না। একটি Zone-এর ভেতরে সবকিছু রাখলে একটি Zone-বিভ্রাটে আপনার পুরো সেবা বন্ধ হয়ে যাবে। দুটি Zone-এ ছড়িয়ে রাখলে একটি Zone গেলেও সেবা চলবে। দুটি Region-এ ছড়িয়ে রাখলে একটি পুরো Region-এর বিপর্যয়েও সেবা চলবে — তবে খরচ, ডেটা সিঙ্ক ও জটিলতা বাড়বে।



১.৭ গ্লোবাল, রিজিওনাল ও জোনাল রিসোর্স

প্রতিটি GCP রিসোর্সের একটি *scope* আছে। **জোনাল রিসোর্স** — যেমন Compute Engine VM, Persistent Disk — একটি নির্দিষ্ট Zone-এ থাকে। Zone গেলে সেটি অচল হয়। **রিজিওনাল রিসোর্স** — যেমন Regional Persistent Disk, Cloud SQL (regional), Managed Instance Group — একটি Region-এর একাধিক Zone জুড়ে থাকে; একটি Zone-বিভ্রাট সহ্য করতে পারে। **গ্লোবাল রিসোর্স** — যেমন VPC নেটওয়ার্ক, ফায়ারওয়াল নিয়ম, Global Load Balancer, Cloud DNS, IAM — পুরো Google Cloud জুড়ে একটিই; Region-বিভ্রাটেও এগুলো সাধারণত কাজ করে।

এই পার্থক্য নকশায় সরাসরি প্রভাব ফেলে। উদাহরণ: আপনি যদি একটি VM-এ ডিস্ক সংযুক্ত করেন, সেই ডিস্ক একই Zone-এ থাকতে হবে — অন্য Zone-এর ডিস্ক সংযুক্ত করা যায় না। আপনি যদি Cloud Storage bucket বানান, তার location ঠিক করতে হয়: regional, dual-region, না multi-region। Multi-region bucket ডেটা একাধিক Region-এ প্রতিলিপি করে — availability বেশি, কিন্তু খরচ বেশি ও ডেটা কোথায় থাকে তা কম নিয়ন্ত্রণে থাকে। ডেটা সার্বভৌমত্বের নিয়ম থাকলে regional bucket বেছে নিন; বৈশ্বিক কনটেন্ট ডেলিভারি চাইলে multi-region ভাবুন।

নামকরণের নিয়ম মনে রাখুন: Region-এর নাম যেমন `asia-south1`, Zone-এর নাম Region + একটি অক্ষর — `asia-south1-a`। gcloud কমান্ডে `--region` ও `--zone` ফ্ল্যাগ আলাদা; ভুল ফ্ল্যাগ দিলে রিসোর্স ভুল জায়গায় তৈরি হবে বা কমান্ড ব্যর্থ হবে।

১.৮ গ্লোবাল নেটওয়ার্কিং: Google-এর মেরুদণ্ড

Google-এর সবচেয়ে বড় শক্তিগুলোর একটি হলো তার নিজস্ব গ্লোবাল নেটওয়ার্ক — সমুদ্রের তলদেশের ফাইবার কেবল, বিশ্বজুড়ে edge location ও নিজস্ব ব্যাকবোন। ব্যবহারকারীর অনুরোধ প্রথমে নিকটতম Google edge (Point of Presence)-এ পৌঁছায়, তারপর Google-এর ব্যক্তিগত নেটওয়ার্ক দিয়ে গন্তব্য Region-এ যায় — পাবলিক ইন্টারনেটের ভিড় এড়িয়ে। এই কারণে GCP-তে ট্রাফিক প্রায়ই দ্রুত ও নির্ভরযোগ্য হয়।

এই নেটওয়ার্কের ওপর দাঁড়িয়ে আছে কয়েকটি মূল সেবা। **VPC (Virtual Private Cloud)** হলো আপনার নিজস্ব বিচ্ছিন্ন নেটওয়ার্ক — এটি গ্লোবাল: একটি VPC সব Region জুড়ে বিস্তৃত হতে পারে, আর subnet-গুলো regional। **Cloud Load Balancing** গ্লোবাল (HTTP(S), TCP/SSL proxy) বা regional (Network, Internal) হতে পারে — গ্লোবাল লোড ব্যালান্সার একটি anycast IP দিয়ে বিশ্বজুড়ে ট্রাফিক নিকটতম সুস্থ backend-এ পাঠায়। **Cloud CDN** কনটেন্ট edge-এ ক্যাশ করে। **Cloud DNS** গ্লোবাল DNS সেবা দেয়। **Cloud Interconnect / VPN** আপনার অন-প্রেমিসেস নেটওয়ার্ককে GCP-র সাথে যুক্ত করে।

নেটওয়ার্ক egress খরচের দিকে নজর দিন: একই Zone-এর ভেতরে ট্রাফিক সাধারণত সস্তা বা বিনামূল্যে; Zone-এর মধ্যে, Region-এর মধ্যে, এবং বিশেষ করে ইন্টারনেটে বেরিয়ে যাওয়া ট্রাফিকের খরচ বাড়ে। একটি অ্যাপ্লিকেশন যদি প্রতি অনুরোধে অন্য Region-এর ডেটাবেসে যায়, তাহলে latency ও egress — দুটোই বাড়বে। তাই নকশার নিয়ম: ডেটা ও কম্পিউট একসাথে রাখুন, ক্যাশ ব্যবহার করুন, আর অপ্রয়োজনীয় cross-region ট্রাফিক কমান।

> **সতর্কতা:** নেটওয়ার্ক egress ও লোড ব্যালান্সিং খরচ অনেক নতুন দলকে চমকে দেয়। একটি Global Load Balancer ও Cloud NAT চালু রেখে ভুলে গেলে, এমনকি কম ট্রাফিকেও মাসের বিল বাড়তে পারে। কোনো সেবা চালু করার আগে তার মূল্য নির্ধারণের পাতা ও Pricing Calculator দেখুন, এবং অধ্যায় ৫-এর বাজেট অ্যালার্ট শুরু থেকেই বসান।

১.৯ Fault Domain, Availability ও Latency

Fault domain হলো এমন একটি একক, যার ব্যর্থতা অন্য এককে প্রভাবিত করে না — একটি Zone একটি fault domain, একটি Region আরও বড় fault domain। ভালো স্থাপত্যের লক্ষ্য হলো একক fault domain-এর ওপর নির্ভর না করা। **Availability** মাপা হয় শতাংশে: ৯৯.৯% মানে মাসে প্রায় ৪৩ মিনিট ডাউনটাইম অনুমোদিত; ৯৯.৯৫% মানে প্রায় ২২ মিনিট; ৯৯.৯৯% মানে প্রায় ৪ মিনিট। প্রতিটি অতিরিক্ত ৯ (nine) পাওয়া কঠিন ও ব্যয়বহুল — কারণ তার জন্য আরও redundancy, আরও পরীক্ষা ও আরও স্বয়ংক্রিয় পুনরুদ্ধার দরকার। **SLA (Service Level Agreement)** হলো Google-এর প্রতিশ্রুতি — এটি পূরণ না হলে ক্রেডিট দেওয়া হয়, কিন্তু আপনার ব্যবহারকারীর ক্ষতি তা ফিরিয়ে দেয় না। তাই SLA-কে লক্ষ্য নয়, ন্যূনতম ভিত্তি হিসেবে দেখুন।

Latency হলো অনুরোধ-উত্তরের সময়। পদার্থবিজ্ঞান এখানে সীমা বেঁধে দেয়: আলো ফাইবারে প্রতি ১০০ কিমি-তে প্রায় ০.৫ মিলিসেকেন্ড নেয়, আর রাউটার, TLS handshake ও অ্যাপ্লিকেশন প্রসেসিং যোগ হয়। বাংলাদেশের ব্যবহারকারীর জন্য `asia-south1` (মুম্বাই) বা `asia-southeast1` (সিঙ্গাপুর) সাধারণত `us-central1`-এর চেয়ে কম latency দেয়। কিন্তু শুধু দূরত্ব নয় — আপনার অ্যাপ্লিকেশন যদি প্রতিটি পাতায় ২০টি ডেটাবেস কোয়েরি করে, তাহলে Region যত কাছেই হোক, latency বেশি থাকবে। তাই latency কমানোর কাজ শুরু হয় অ্যাপ্লিকেশন থেকে, Region নির্বাচন থেকে নয়।

RTO ও RPO দুটি গুরুত্বপূর্ণ ধারণা। RTO (Recovery Time Objective) — বিপর্যয়ের পর সেবা ফিরতে কত সময় লাগতে পারে; RPO (Recovery Point Objective) — কতটুকু ডেটা হারানো সহ্য করা যায়। একটি ব্যাংকিং সিস্টেমের RPO কয়েক সেকেন্ড হতে পারে; একটি ব্লগের RPO একদিনও চলতে পারে। আপনার ব্যাকআপ ও প্রতিলিপি নকশা RTO/RPO থেকেই নির্ধারিত হয়, উল্টো নয়।

১.১০ ক্লাউড আর্কিটেকচারের মূল নীতি

প্রথম নীতি: **ধরে নিন সবকিছু ব্যর্থ হবে।** VM থামবে, ডিস্ক নষ্ট হবে, Zone যাবে। তাই নকশা করুন পুনরুদ্ধারযোগ্য করে — stateless অ্যাপ্লিকেশন, একাধিক প্রতিলিপি, স্বয়ংক্রিয় health check। দ্বিতীয়: **loose coupling** — সেবাগুলো একে অপরের সাথে Pub/Sub, API বা কিউ দিয়ে কথা বলুক, সরাসরি ডেটাবেস ভাগ করে নয়; তাহলে একটি সেবা বদলালে অন্যটি ভাঙে না। তৃতীয়: **elasticity** — চাহিদা অনুযায়ী স্কেল করুন, আগে থেকে বড় সার্ভার কিনে বসে থাকবেন না। চতুর্থ: **automation** — হাতে ক্লিক করে নয়, কোড দিয়ে অবকাঠামো বানান (অধ্যায় ৪ দেখুন)। পঞ্চম: **observability** — লগ, মেট্রিক ও ট্রেস ছাড়া আপনি অন্ধ; Cloud Monitoring ও Cloud Logging শুরু থেকেই চালু করুন। ষষ্ঠ: **security by design** — least privilege, private নেটওয়ার্ক, এনক্রিপশন — পরে যোগ করার জিনিস নয়।

১.১১ Console ও gcloud দিয়ে শুরু

Google Cloud Console (console.cloud.google.com) হলো ওয়েব ইন্টারফেস — শেখার জন্য ও এককালীন কাজের জন্য ভালো। কিন্তু প্রোডাকশনে বারবার একই কাজ করতে হলে `gcloud` CLI ব্যবহার করুন — তা পুনরাবৃত্তিযোগ্য ও স্ক্রিপ্ট করা যায়। একটি প্রজেক্টে উপলব্ধ Region দেখতে:

```
gcloud compute regions list --project=technova-prod
gcloud compute zones list --filter="region:asia-south1" --project=technova-prod
gcloud compute machine-types list --zones=asia-south1-a --project=technova-prod
```

নিচের আউটপুটটি চিত্রকল্পের জন্য — আপনার প্রজেক্টে নাম ও সংখ্যা ভিন্ন হতে পারে। Console-এ একই তথ্য পাবেন Compute Engine → Settings → অথবা Cloud Shell থেকে উপরের কমান্ড চালিয়ে। মনে রাখবেন: `gcloud` চালানোর আগে আপনাকে প্রমাণীকরণ করতে হয় (অধ্যায় ৪), আর প্রতিটি কমান্ডে সঠিক প্রজেক্ট নির্দিষ্ট করতে হয় — ভুল প্রজেক্টে রিসোর্স বানানো নতুনদের সবচেয়ে সাধারণ ও ব্যয়বহুল ভুলগুলোর একটি।

১.১২ প্রোডাকশন দৃশ্যকল্প: TechNova-র প্রথম স্থাপত্য

TechNova বাংলাদেশ ও ভারতের ব্যবহারকারীদের জন্য একটি SaaS অ্যাপ বানাচ্ছে। তাদের সিদ্ধান্ত: প্রাথমিক Region `asia-south1` (মুম্বাই) — ব্যবহারকারীদের কাছাকাছি, খরচ যুক্তিসংগত। ওয়েব সেবা Cloud Run-এ, দুটি Zone জুড়ে; ডেটাবেস Cloud SQL regional (primary + standby); স্ট্যাটিক ফাইল Cloud Storage multi-region bucket + Cloud CDN; DNS Cloud DNS-এ। একটি Zone-বিভ্রাটে Cloud Run অন্য Zone-এ ট্রাফিক নেবে; ডেটাবেস failover করবে। তারা RTO ১৫ মিনিট ও RPO ১ মিনিট ঠিক করেছে, এবং প্রতি মাসে failover পরীক্ষা করে। তারা ইচ্ছা করেই শুরুতে মাল্টি-Region যায়নি — কারণ তাদের দল ছোট, ডেটা সিঙ্কের জটিলতা তাদের জন্য এখন বেশি। এই সিদ্ধান্তটি সঠিক: স্থাপত্য দলের আকার ও ঝুঁকির সাথে মিলতে হয়, শুধু তত্ত্বের সাথে নয়।

> **বাস্তব উদাহরণ:** একটি দল `us-central1`-এ অ্যাপ চালাত কারণ টিউটোরিয়ালে সেটিই ছিল। বাংলাদেশের ব্যবহারকারীরা ২৫০ মিলিসেকেন্ডের বেশি latency পাচ্ছিল। শুধু Region বদলে `asia-south1`-এ নেওয়ার পর latency অর্ধেকেরও কমে গেল — কোডে এক লাইনও বদলাতে হয়নি। Region নির্বাচন টিউটোরিয়ালের ডিফল্ট দিয়ে নয়, ব্যবহারকারীর অবস্থান দিয়ে করুন।

১.১৩ সাধারণ ভুল

- সবকিছু একটি Zone-এ রাখা, তারপর Zone-বিভ্রাটে অবাক হওয়া।

- টিউটোরিয়ালের ডিফল্ট Region (`us-central1`) ব্যবহার করা, ব্যবহারকারীর অবস্থান না দেখে।
- গ্লোবাল, রিজিওনাল ও জোনাল রিসোর্সের পার্থক্য না জেনে ডিস্ক বা IP ভুল জায়গায় বানানো।
- Availability-র "৯" বাড়ানোর জন্য অন্ধভাবে মাল্টি-Region করা — খরচ ও জটিলতা না মেপে।
- ব্যাকআপ আছে ধরে নেওয়া, কিন্তু কখনো restore পরীক্ষা না করা।
- Latency-র জন্য শুধু Region-কে দোষ দেওয়া, অথচ অ্যাপ প্রতি অনুরোধে অসংখ্য কোয়েরি করছে।

১.১৪ Troubleshooting

> **TROUBLESHOOTING:** "Resource is not available in this zone" — ওই Zone-এ ওই মেশিন টাইপ বা GPU নেই, অথবা কোটা শেষ। অন্য Zone বা মেশিন টাইপ চেষ্টা করুন, এবং `gcloud compute zones describe ZONE` দিয়ে দেখুন। "Disk must be in the same zone as the instance" — ডিস্ক ও VM-এর Zone মেলান। অ্যাপ ধীর মনে হলে প্রথমে Cloud Monitoring-এ latency ভাগ করুন: নেটওয়ার্ক, অ্যাপ, না ডেটাবেস — কোন স্তরে সময় যাচ্ছে? অনুমানে নয়, মাপে সিদ্ধান্ত নিন।

১.১৫ সারাংশ

ক্লাউড হলো চাহিদা অনুযায়ী, API-নিয়ন্ত্রিত, মাপযোগ্য কম্পিউটিং। IaaS দেয় নিয়ন্ত্রণ, PaaS দেয় গতি, SaaS দেয় সুবিধা — সঠিক মিশ্রণ কাজের ধরনে নির্ভর করে। Google Cloud-এর Region ও Zone ব্যর্থতা থেকে বাঁচার একক; গ্লোবাল নেটওয়ার্ক latency ও নির্ভরযোগ্যতার ভিত্তি। ভালো স্থাপত্য ব্যর্থতা ধরে নিয়ে নকশা করে, ডেটা ও কম্পিউট একসাথে রাখে, এবং RTO/RPO থেকে ব্যাকআপ ঠিক করে। পরের অধ্যায়ে আমরা দেখব এই অবকাঠামোর ওপরে সংস্থা, ফোল্ডার ও প্রজেক্ট কীভাবে সাজানো হয় — কারণ Region বেছে নেওয়ার আগেই ঠিক করতে হয় কে কী তৈরি করতে পারবে।

১.১৬ পর্যালোচনা প্রশ্ন

- IaaS, PaaS ও SaaS-এর মধ্যে নিয়ন্ত্রণ ও দায়িত্ব কীভাবে ভাগ হয়? TechNova-র একটি সেবার জন্য কোনটি বেছে নেবেন ও কেন?
- একটি Zone ও একটি Region-এর ব্যর্থতার প্রভাব কীভাবে আলাদা? আপনার অ্যাপ কোনটি সহ্য করতে পারে?
- গ্লোবাল, রিজিওনাল ও জোনাল রিসোর্সের তিনটি করে উদাহরণ দিন।
- ৯৯.৯% ও ৯৯.৯৯% availability-র মাসিক ডাউনটাইম কত? অতিরিক্ত "৯" পেতে কী খরচ হয়?
- RTO ও RPO-র পার্থক্য কী? একটি ই-কমার্স অর্ডার সিস্টেমের জন্য এগুলো কেমন হওয়া উচিত?

১.১৭ ব্যবহারিক অনুশীলন

- Cloud Shell-এ `gcloud compute regions list` চালিয়ে বাংলাদেশের নিকটতম তিনটি Region চিহ্নিত করুন ও latency-র যুক্তি লিখুন।
- একটি চিত্র আঁকুন: TechNova-র অ্যাপ `asia-south1`-এ দুটি Zone-এ কীভাবে চলবে, কোন রিসোর্স কোথায় থাকবে।
- Pricing Calculator-এ একটি ছোট VM, একটি regional ডিস্ক ও একটি লোড ব্যালান্সারের মাসিক খরচের অনুমান করুন (দাম পরিবর্তনশীল — অনুমান সংরক্ষণ করুন, সত্য ধরে নেবেন না)।

অধ্যায় ২ • PART I — GCP Foundations & Architecture

Chapter 2 — Organizations, Folders, Projects and Governance

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Google Cloud resource hierarchy — Organization, Folder, Project, Resource — ব্যাখ্যা করতে; (২) একটি প্রজেক্ট কী এবং কেন এটি বিলিং, IAM ও কোটার একক, তা বলতে; (৩) ফোল্ডার দিয়ে dev, staging ও production পরিবেশ সাজাতে; (৪) billing account ও organization policy-র ভূমিকা ব্যাখ্যা করতে; (৫) প্রজেক্ট পৃথকীকরণের সিদ্ধান্ত নিতে — কখন একটি প্রজেক্ট, কখন অনেক; এবং (৬) এন্টারপ্রাইজ governance-এর মূল নিয়ন্ত্রণগুলো (policy, audit, naming, labels) প্রয়োগ করতে।

২.২ পূর্বজ্ঞান ঝালাই

অধ্যায় ১ থেকে মনে করুন: Region ও Zone হলো ভৌগোলিক একক; এখানে আমরা শিখব সাংগঠনিক একক। একটি ফাইল সিস্টেমের সাথে তুলনা করুন: Organization হলো পুরো হার্ড ড্রাইভ, Folder হলো ডিরেক্টরি, Project হলো সাব-ডিরেক্টরি, আর Resource (VM, bucket, ডেটাবেস) হলো ফাইল। যেমন ফাইলের অনুমতি ডিরেক্টরি থেকে উত্তরাধিকারসূত্রে পাওয়া যায়, তেমনি GCP-তে IAM অনুমতি ও policy ওপরের স্তর থেকে নিচের স্তরে নামে। এই উত্তরাধিকার (inheritance) বুঝলে এই অধ্যায়ের অর্ধেক বোঝা হয়ে যাবে।

২.৩ Resource Hierarchy: কেন স্তর দরকার

একটি ছোট দলে একটি প্রজেক্টই যথেষ্ট হতে পারে। কিন্তু কোম্পানি বড় হলে প্রশ্ন ওঠে: কে কোন পরিবেশে কী করতে পারবে? dev-এর একজন ডেভেলপার কি production ডেটাবেস মুছতে পারবে? বিল কি দল অনুযায়ী ভাগ করা যাবে? একটি প্রজেক্টের সমস্যা কি অন্য প্রজেক্টকে প্রভাবিত করবে? Resource hierarchy এই প্রশ্নগুলোর উত্তর দেয় — এটি শুধু সাজানোর ব্যবস্থা নয়, এটি নিরাপত্তা, বিলিং ও পরিচালনার সীমানা।

স্তরগুলো ওপর থেকে নিচে: **Organization** — পুরো কোম্পানি (একটি Google Workspace বা Cloud Identity ডোমেইনের সাথে যুক্ত, যেমন `technova.example`)। **Folder** — Organization-এর ভেতরে বিভাগ বা পরিবেশ অনুযায়ী গোষ্ঠী (যেমন `Production`, `Development`)। Folder-এর ভেতরে আরও Folder থাকতে পারে। **Project** — কাজের মৌলিক একক; প্রতিটি রিসোর্স একটি প্রজেক্টের ভেতরে থাকে। **Resource** — VM, bucket, ডেটাবেস, নেটওয়ার্ক ইত্যাদি।

মনে রাখার নিয়ম: IAM policy ও organization policy ওপর থেকে নিচে উত্তরাধিকারসূত্রে যায় — Organization-এ দেওয়া অনুমতি সব Folder ও Project-এ প্রযোজ্য হয়, যদি না নিচে আরও সীমিত করা হয়। কিন্তু বিলিং ও কোটা প্রজেক্ট-স্তরে হিসাব হয়। আর একটি প্রজেক্টের রিসোর্স ডিফল্টভাবে অন্য প্রজেক্টের রিসোর্স দেখতে বা ব্যবহার করতে পারে না — এটি একটি গুরুত্বপূর্ণ বিচ্ছিন্নতা (isolation) সীমানা।

TechNova-র Resource Hierarchy Organization: technova.example ├── Folder: Production │ ├── Project: technova-prod-web (Cloud Run, Load Balancer) │ ├── Project: technova-prod-data (Cloud SQL, BigQuery) │ └── Project: technova-prod-shared (VPC, DNS, Logging) ├── Folder: Non-Production │ ├── Project: technova-staging │ └── Project: technova-dev-rahim (প্রতি ডেভেলপার আলাদা নয় — ছোট দলে একটি shared dev প্রজেক্টই যথেষ্ট হতে পারে) └── Folder: Security & Admin ├── Project: technova-security (Security Command Center, audit logs) └── Project: technova-billing-admin (বিলিং রিপোর্ট — সীমিত প্রবেশ)

২.৪ Organization: কোম্পানির মূল

Organization তৈরি হয় যখন কেউ Google Workspace বা Cloud Identity-তে একটি ডোমেইন যাচাই করে। Organization-এর সুপার অ্যাডমিন (Organization Admin) সবচেয়ে শক্তিশালী ভূমিকা — তিনি Folder ও Project বানাতে পারেন, IAM policy বদলাতে পারেন, billing account যুক্ত করতে পারেন। এই ভূমিকা খুব কম মানুষকে দিন, MFA বাধ্যতামূলক করুন, এবং এর ব্যবহার audit করুন। একটি সাধারণ ভুল হলো ব্যক্তিগত Gmail দিয়ে প্রোডাকশন Organization বানানো — এতে কর্মী চলে গেলে বা অ্যাকাউন্ট হারালে কোম্পানি তার অবকাঠামোর নিয়ন্ত্রণ হারায়। সবসময় কোম্পানির ডোমেইন (`technova.example`) দিয়ে Organization করুন।

Organization-এর ভেতরে **Organization Policy** (আগে Org Policy নামে পরিচিত) বসানো যায় — যেমন "কোনো VM-এর public IP থাকবে না", "শুধু নির্দিষ্ট Region-এ রিসোর্স বানানো যাবে", "Service account key তৈরি নিষিদ্ধ"। এই নীতিগুলো পুরো Organization জুড়ে প্রযোজ্য হয় এবং নিচের স্তর তা অমান্য করতে পারে না (কিছু ব্যতিক্রম ছাড়া)। এগুলোকে ভাবুন কোম্পানির সংবিধান হিসেবে — ব্যক্তিগত পছন্দের ওপরে সবার জন্য বাধ্যতামূলক নিয়ম।

২.৫ Project: কাজের মৌলিক একক

প্রতিটি GCP রিসোর্স একটি প্রজেক্টের অন্তর্গত। প্রজেক্টের তিনটি পরিচয় আছে: **Project Name** (মানুষের পড়ার মতো নাম, যেমন `technova-prod-web`), **Project ID** (বিশ্বজুড়ে অনন্য, একবার বানালে বদলানো যায় না — যেমন `technova-prod-web-123456`), এবং **Project Number** (Google-এর দেওয়া সংখ্যা)। কমান্ডে সাধারণত Project ID ব্যবহার হয়। Project ID বেছে নেওয়ার সময় অর্থবহ ও স্থায়ী নাম দিন — কারণ এটি বদলানো যায় না, আর URL ও লগে দেখা যায়।

প্রজেক্ট কেন গুরুত্বপূর্ণ? প্রথমত, **বিলিং**: প্রতিটি প্রজেক্ট একটি billing account-এর সাথে যুক্ত থাকে; বিল প্রজেক্ট অনুযায়ী ভাগ করা যায়। দ্বিতীয়ত, **IAM**: কে কোন প্রজেক্টে কী করতে পারবে, তা আলাদাভাবে ঠিক করা যায় — dev প্রজেক্টে ডেভেলপাররা এডিটর, production-এ শুধু দেখতে পারবে। তৃতীয়ত, **কোটা ও API**: প্রতিটি প্রজেক্টের নিজস্ব কোটা ও সক্রিয় API থাকে; dev-এ একটি পরীক্ষা production-এর কোটা খেয়ে ফেলতে পারে না। চতুর্থত, **বিচ্ছিন্নতা**: একটি প্রজেক্ট মুছে গেলে অন্য প্রজেক্টের রিসোর্স থাকে; একটি প্রজেক্টের নেটওয়ার্ক ডিফল্টভাবে অন্য প্রজেক্ট থেকে আলাদা।

প্রজেক্ট বানানো Console-এ: Navigation → IAM & Admin → Create a Project। gcloud-এ:

```
gcloud projects create technova-staging \
  --name="TechNova Staging" \
  --folder=123456789012
gcloud projects list --filter="name:technova*"
gcloud config set project technova-staging
```

প্রজেক্ট বানানোর পর API সক্রিয় করতে হয় — যেমন Compute Engine API না চালু থাকলে VM বানানো যাবে না। এটি একটি সাধারণ "permission denied" বিভ্রান্তির কারণ: IAM অনুমতি থাকলেও API বন্ধ থাকলে কাজ হয় না।

```
gcloud services enable compute.googleapis.com run.googleapis.com \
  --project=technova-staging
gcloud services list --enabled --project=technova-staging
```

২.৬ Folder: পরিবেশ ও দল সাজানো

Folder হলো Organization ও Project-এর মাঝখানের স্তর। এটি বাধ্যতামূলক নয় — ছোট দলে সরাসরি Organization-এর নিচে প্রজেক্ট রাখা যায় — কিন্তু দল বাড়লে Folder ছাড়া পরিচালনা অসম্ভব হয়ে যায়। সাধারণ প্যাটার্ন দুটি: **পরিবেশ অনুযায়ী** (Production / Staging / Development) এবং **দল বা পণ্য অনুযায়ী** (Payments / Web / Data)। বড় কোম্পানিতে দুটোই মেশানো হয়: প্রথমে দল, তার ভেতরে পরিবেশ — বা উল্টো। TechNova ছোট দল বলে পরিবেশ অনুযায়ী Folder বেছে নিয়েছে, কারণ তাদের প্রধান ঝুঁকি হলো dev-এর কাজ production-এ প্রভাব ফেলা।

Folder-এর শক্তি হলো policy inheritance: Production Folder-এ "public IP নিষিদ্ধ" নীতি বসালে তার ভেতরের সব প্রজেক্টে তা প্রযোজ্য হয়; Development Folder-এ সেই নীতি শিথিল রাখা যায় যাতে পরীক্ষা সহজ হয়। একইভাবে, Production Folder-এ শুধু সিনিয়র ইঞ্জিনিয়ারদের IAM দিন, Development Folder-এ সব ডেভেলপারকে। এভাবে একটি জায়গায় নিয়ম বদলালেই সব প্রজেক্টে তা কার্যকর হয় — প্রতিটি প্রজেক্টে আলাদাভাবে নিয়ম বসাতে হয় না।

> **CONCEPT:** Folder-কে ভাবুন একটি ভবনের তলা হিসেবে। Production তলা শুধু অনুমোদিত লোক ঢুকতে পারে, দরজায় তালা বেশি; Development তলা খোলামেলা। আপনি প্রতিটি ঘরে আলাদা তালা না লাগিয়ে তলার দরজায় নিয়ম বসান — এটাই inheritance-এর সুবিধা। তবে মনে রাখবেন: inheritance শুধু নিচের দিকে যায়; একটি প্রজেক্টের নিয়ম তার পাশের প্রজেক্টকে প্রভাবিত করে না।

২.৭ Billing Account ও খরচের ভাগ

Billing account হলো অর্থ প্রদানের একক — একটি কোম্পানির এক বা একাধিক billing account থাকতে পারে (যেমন আলাদা বিভাগের জন্য আলাদা)। প্রতিটি প্রজেক্ট ঠিক একটি billing account-এর সাথে যুক্ত থাকে। বিলিং অ্যাডমিন (Billing Account Administrator) পেমেন্ট পদ্ধতি ও বাজেট দেখেন, কিন্তু তিনি স্বয়ংক্রিয়ভাবে প্রজেক্টের রিসোর্স দেখতে বা বদলাতে পারেন না — বিলিং ও কারিগরি প্রবেশ আলাদা রাখা হয়েছে, এটাই নিরাপত্তার নীতি।

খরচ ভাগ করার জন্য **labels** ব্যবহার করুন: প্রতিটি রিসোর্সে `team=web`, `env=prod`, `cost-center=payments` লেবেল দিন। Billing export (BigQuery-তে) দিয়ে লেবেল অনুযায়ী খরচের রিপোর্ট বানানো যায়। লেবেল ছাড়া মাস শেষে শুধু জানবেন মোট বিল কত — কোন দল বা সেবা কত খরচ করেছে, তা জানবেন না। লেবেলের নামকরণ শুরু থেকেই ঠিক করুন ও সবাইকে মানতে বাধ্য করুন; পরে লেবেল যোগ করা যায়, কিন্তু আগের খরচের ইতিহাস ফিরে পাওয়া যায় না।

```
gcloud billing accounts list
gcloud billing projects link technova-staging \
  --billing-account=0X0X0X-0X0X0X-0X0X0X
gcloud projects add-iam-policy-binding technova-prod-web \
  --member="group:platform@technova.example" \
  --role="roles/viewer"
```

২.৮ প্রজেক্ট পৃথকীকরণ: কখন ভাগ করবেন, কখন নয়

একটি প্রজেক্টে সবকিছু রাখা সহজ, কিন্তু ঝুঁকিপূর্ণ: dev-এর ভুল production ডেটাবেস মুছে দিতে পারে, একটি পরীক্ষা production-এর কোটা শেষ করে দিতে পারে, আর বিল আলাদা করা যায় না। অন্যদিকে, অতিরিক্ত প্রজেক্টও খারাপ: প্রতিটি

প্রজেক্টে VPC, IAM, লগিং আলাদাভাবে সাজাতে হয়; প্রজেক্টের মধ্যে নেটওয়ার্ক যুক্ত করতে Shared VPC বা VPC Peering দরকার হয়; পরিচালনার ভার বাড়ে।

বাস্তব নির্দেশনা: **পরিবেশ আলাদা করুন** — dev, staging, production অন্তত আলাদা প্রজেক্টে রাখুন। **সংবেদনশীল ডেটা আলাদা করুন** — পেমেন্ট বা ব্যক্তিগত ডেটার প্রজেক্ট আলাদা রাখুন, যাতে কম লোকের প্রবেশ থাকে। **শেয়ার্ড সেবা আলাদা করুন** — VPC, DNS, কেন্দ্রীয় লগিং একটি shared প্রজেক্টে রাখুন। কিন্তু প্রতিটি মাইক্রোসার্ভিসের জন্য আলাদা প্রজেক্ট বানাবেন না, যদি না দল বা নিরাপত্তার সীমানা সত্যিই আলাদা হয়। TechNova-র ক্ষেত্রে: web, data ও shared — তিনটি production প্রজেক্ট; staging একটি; dev একটি। এই ভাগটি তাদের দলের আকারের সাথে মেলে।

২.৯ Development, Staging ও Production পরিবেশ

তিনটি পরিবেশের উদ্দেশ্য আলাদা। **Development** — দ্রুত পরীক্ষা, ভাঙা চলবে, ডেটা নকল, খরচ কম। **Staging** — production-এর মতো করে সাজানো, আসল ডেটার মতো নকল ডেটা, release-এর আগে শেষ পরীক্ষা। **Production** — আসল ব্যবহারকারী, আসল ডেটা, সবচেয়ে কঠোর নিয়ন্ত্রণ। একটি সাধারণ ভুল হলো staging-কে production-এর চেয়ে অনেক ছোট করে বানানো — তাহলে staging-এ পাশ করা release production-এ স্কেলের কারণে ব্যর্থ হয়। Staging-এর স্থাপত্য production-এর মতো হওয়া উচিত, আকার ছোট হতে পারে।

পরিবেশের মধ্যে প্রচার (promotion) কীভাবে হবে? কোড Git থেকে CI/CD দিয়ে যাবে: dev-এ merge হলে dev প্রজেক্টে deploy, tag হলে staging-এ, অনুমোদন পেলে production-এ। হাতে করে Console-এ ক্লিক করে production-এ deploy করা নিষিদ্ধ করুন — কারণ তাতে কে কী deploy করল, তার কোনো রেকর্ড থাকে না। এই বিষয়ে বিস্তারিত আসবে পরের ভলিউমে; এখানে মনে রাখুন: পরিবেশের পৃথকীকরণ শুধু প্রজেক্ট ভাগ করা নয়, deploy প্রক্রিয়াও ভাগ করা।

২.১০ Enterprise Governance: নীতি, নিরীক্ষা ও নামকরণ

Governance মানে নিয়ম বানানো নয় — নিয়ম যাতে স্বয়ংক্রিয়ভাবে মানা হয়, সেই ব্যবস্থা করা। মূল উপাদানগুলো: (১) **Organization Policy** — Region সীমা, public IP নিষেধ, service account key নিষেধ; (২) **IAM** — least privilege, গ্রুপ দিয়ে অনুমতি (ব্যক্তি দিয়ে নয়); (৩) **Audit Logs** — কে কী করল, Cloud Logging-এ সংরক্ষণ, কেন্দ্রীয় প্রজেক্টে সংগ্রহ; (৪) **নামকরণ ও লেবেল** — `technova-prod-web`-এর মতো ধারাবাহিক নাম, `env`, `team`, `owner` লেবেল; (৫) **বাজেট ও অ্যালার্ট** — অধ্যায় ৫-এ বিস্তারিত; (৬) **নিয়মিত পর্যালোচনা** — প্রতি মাসে IAM পর্যালোচনা: কে এখনো প্রবেশ পায়, যার দরকার নেই তার প্রবেশ সরান।

একটি বাস্তব পরামর্শ: governance শুরুতে কঠোর করবেন না, নইলে দল তা এড়িয়ে যাওয়ার পথ খুঁজবে। প্রথমে সবচেয়ে বিপজ্জনক জিনিসগুলো বন্ধ করুন — production-এ public IP, service account key তৈরি, অতিরিক্ত IAM — তারপর ধীরে ধীরে নিয়ম বাড়ান। আর প্রতিটি নীতির পাশে লিখুন কেন এটি আছে: "কেন public IP নিষিদ্ধ?" — কারণ গত বছর একটি ভুলে খোলা ডেটাবেস থেকে ডেটা বেরিয়ে গিয়েছিল। কারণ ছাড়া নিয়ম মানা হয় না।

> **সতর্কতা:** Organization Admin ও Billing Account Administrator ভূমিকা খুব কম মানুষকে দিন এবং তাদের অ্যাকাউন্টে MFA বাধ্যতামূলক করুন। একটি Organization Admin অ্যাকাউন্ট হাতছাড়া হলে পুরো কোম্পানির ক্লাউড নিয়ন্ত্রণ চলে যেতে পারে। Break-glass অ্যাকাউন্ট (জরুরি অ্যাকাউন্ট) থাকলে তার পাসওয়ার্ড ও MFA সুরক্ষিতভাবে সংরক্ষণ করুন এবং ব্যবহারের পর তাৎক্ষণিকভাবে পর্যালোচনা করুন।

২.১১ IaC দিয়ে প্রজেক্ট ও ফোল্ডার বানানো

হাতে Console-এ প্রজেক্ট বানানো শেখার জন্য ভালো, কিন্তু দল বাড়লে তা টেকসই নয় — কে কোন প্রজেক্ট বানিয়েছে, কোন API সক্রিয় করেছে, কোন লেবেল দিয়েছে, তার কোনো রেকর্ড থাকে না। তাই প্রজেক্ট ও ফোল্ডার Terraform-এর মতো Infrastructure as Code দিয়ে বানান। নিচের উদাহরণটি চিত্রকল্পের জন্য — আপনার Folder ID ও billing account ID ভিন্ন হবে:

```
resource "google_project" "staging" {
  name       = "TechNova Staging"
  project_id = "technova-staging-CHANGE-ME"
  folder_id  = "folders/123456789012"
  billing_account = "0X0X0X-0X0X0X-0X0X0X"
  labels = {
    env  = "staging"
    team = "platform"
  }
}
resource "google_project_service" "staging_apis" {
  project = google_project.staging.project_id
  service = "compute.googleapis.com"
}
```

Terraform-এর সুবিধা হলো `plan` — বদলানোর আগে দেখা যায় কী বদলাবে — আর Git-এ ইতিহাস থাকে: কে, কখন, কেন প্রজেক্টটি বানিয়েছে। তবে মনে রাখবেন: Terraform চালানো service account-কে Project Creator ও Billing Account User ভূমিকা দিতে হয় — সেই service account-কে সুরক্ষিত রাখুন (অধ্যায় ৩), আর Terraform state ফাইলে কোনো গোপন মান রাখবেন না। প্রজেক্ট মুছে ফেলার আগে `terraform destroy`-এর পরিকল্পনা পর্যালোচনা করুন — production প্রজেক্টের জন্য destroy সুরক্ষা (`deletion_policy` বা `prevent_destroy`) ব্যবহার করুন।

২.১২ প্রজেক্ট জীবনচক্র: তৈরি থেকে অবসর

প্রজেক্টের একটি জীবনচক্র আছে: তৈরি → সক্রিয় ব্যবহার → হিমায়িত (freeze) → মুছে ফেলা। একটি প্রজেক্ট যখন আর দরকার নেই, তখনই মুছে ফেলুন — নইলে তার VM, IP ও ডিস্ক বিল বাড়াতে থাকে। মুছে ফেলার আগে যাচাই করুন: (১) কোনো সক্রিয় রিসোর্স আছে কি — `gcloud asset list` বা Console-এ দেখুন; (২) কোনো গুরুত্বপূর্ণ ডেটা আছে কি — ব্যাকআপ নিন; (৩) অন্য প্রজেক্ট কি এর ওপর নির্ভর করে — যেমন Shared VPC বা DNS। মুছে ফেলা প্রজেক্ট ৩০ দিনের মধ্যে পুনরুদ্ধার করা যায়, তারপর চিরতরে যায়। তাই মুছে ফেলার অনুমতি শুধু অ্যাডমিনদের দিন, আর production প্রজেক্ট মুছে ফেলার আগে দুজনের অনুমোদন নিন। একটি ভালো অভ্যাস: প্রতি ত্রৈমাসিকে অব্যবহৃত প্রজেক্টের তালিকা পর্যালোচনা করুন — নামে `test-` বা `temp-` থাকা প্রজেক্টগুলো প্রথমে দেখুন।

২.১৩ সাধারণ ভুল

- ব্যক্তিগত Gmail দিয়ে production Organization বানানো।
- সবকিছু একটি প্রজেক্টে রাখা — dev-এর ভুলে production যাওয়া।
- ব্যক্তির নামে IAM দেওয়া, গ্রুপের নামে নয় — কর্মী চলে গেলে অনুমতি থেকে যায়।
- Project ID অর্থহীন বা পরিবর্তনশীল নামে বানানো (এটি বদলানো যায় না)।
- API সক্রিয় না করে "permission denied" নিয়ে সময় নষ্ট করা।
- লেবেল ছাড়া রিসোর্স বানানো — মাস শেষে খরচের উৎস খুঁজে না পাওয়া।

২.১৪ Troubleshooting

TROUBLESHOOTING: "You do not have permission to create projects" — আপনার Organization বা Folder-এ Project Creator ভূমিকা নেই; অ্যাডমিনকে বলুন। প্রজেক্ট দেখতে না পেলে `gcloud projects list` দিয়ে যাচাই করুন আপনি সঠিক অ্যাকাউন্টে আছেন কি না (`gcloud auth list`)। Billing account যুক্ত না থাকলে রিসোর্স বানানো যাবে না — `gcloud billing projects describe PROJECT_ID` দিয়ে দেখুন। প্রজেক্ট মুছে গেলে ৩০ দিনের মধ্যে পুনরুদ্ধার করা যায় (`gcloud projects undelete`), তারপর চিরতরে যায় — তাই production প্রজেক্ট মুছে ফেলার অনুমতি কাউকে দেবেন না।

২.১৫ সারাংশ

Organization, Folder, Project ও Resource — এই স্তরগুলো নিরাপত্তা, বিলিং ও পরিচালনার সীমানা। প্রজেক্ট হলো কাজের মৌলিক একক; Folder দিয়ে পরিবেশ ও দল সাজান; Organization Policy দিয়ে কোম্পানিজুড়ে নিয়ম বসান। Dev, staging ও production আলাদা প্রজেক্টে রাখুন, IAM গ্রুপ দিয়ে দিন, লেবেল দিয়ে খরচ ভাগ করুন। ভালো governance মানে কঠোর নিয়ম নয় — স্বয়ংক্রিয়ভাবে মানা হয় এমন নিয়ম। পরের অধ্যায়ে আমরা এই কাঠামোর ভেতরে IAM-এর গভীরে যাব — কে কী করতে পারবে, তা কীভাবে ঠিক হয়।

২.১৬ পর্যালোচনা প্রশ্ন

- Organization, Folder, Project ও Resource-এর মধ্যে IAM inheritance কীভাবে কাজ করে?
- একটি প্রজেক্টকে বিলিং, IAM ও কোটার একক বলা হয় কেন?
- কখন একটি প্রজেক্ট ভাগ করবেন, আর কখন একটিই রাখবেন? তিনটি উদাহরণ দিন।
- Billing account ও প্রজেক্টের সম্পর্ক কী? Billing অ্যাডমিন কি VM মুছতে পারেন?
- Organization Policy ও IAM Policy-র পার্থক্য কী?

২.১৭ ব্যবহারিক অনুশীলন

- একটি চিত্র আঁকুন: আপনার কাল্পনিক কোম্পানির Organization, Folder ও Project কাঠামো — dev, staging, production-সহ।
- gcloud-এ একটি পরীক্ষামূলক প্রজেক্ট বানান, দুটি API সক্রিয় করুন, তারপর লেবেলসহ একটি bucket বানান।
- আপনার প্রজেক্টের জন্য নামকরণ ও লেবেলের একটি ছোট নীতিমালা লিখুন (নাম, env, team, owner)।

অধ্যায় ৩ • PART I — GCP Foundations & Architecture

Chapter 3 — Cloud IAM and Identity Engineering

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) IAM-এর মূল ধারণা — principal, role, policy, permission — ব্যাখ্যা করতে; (২) Google অ্যাকাউন্ট, গ্রুপ ও service account-এর পার্থক্য বলতে; (৩) predefined, basic ও custom role-র মধ্যে সঠিকটি বেছে নিতে; (৪) IAM policy inheritance ও resource-level অনুমতি বিশ্লেষণ করতে; (৫) Workload Identity Federation ও service account impersonation নিরাপদে ব্যবহার করতে; (৬) least privilege প্রয়োগ করতে এবং (৭) সাধারণ IAM সমস্যা নির্ণয় করতে।

৩.২ পূর্বজ্ঞান ঝালাই

অধ্যায় ২ থেকে মনে করুন: Organization → Folder → Project → Resource স্তরে IAM উত্তরাধিকারসূত্রে যায়। এখানে আমরা শিখব সেই অনুমতি আসলে কী। একটি বাড়ির সাথে তুলনা করুন: principal হলো ব্যক্তি (কে), role হলো চাবির গোছা (কী করতে পারে), policy হলো দরজায় লাগানো তালিকা (কার কাছে কোন চাবি আছে), আর permission হলো একটি নির্দিষ্ট কাজ (দরজা খোলা)। "Editor" একটি চাবির গোছা, "compute.instances.create" একটি একক চাবি। কাউকে পুরো বাড়ির মালিকের চাবি দেওয়ার আগে ভাবুন — তার কি সত্যিই সব ঘরে ঢোকার দরকার আছে?

৩.৩ IAM প্রথম নীতি থেকে: কে, কী, কোথায়

Google Cloud IAM তিনটি প্রশ্নের উত্তর দেয়: **কে** (Who — principal), **কী করতে পারবে** (What — permission, role-এর ভেতরে গুচ্ছবদ্ধ), **কোন রিসোর্সে** (Where — কোন প্রজেক্ট, bucket, VM-এ)। এই তিনটি একসাথে একটি **IAM policy** তৈরি করে, যা একটি রিসোর্সের সাথে সংযুক্ত থাকে। যখন কেউ API অনুরোধ পাঠায়, Google প্রথমে প্রমাণীকরণ (authentication — আপনি কে, তা যাচাই) করে, তারপর অনুমোদন (authorization — আপনার ওই কাজের অনুমতি আছে কি না, তা যাচাই) করে। প্রমাণীকরণ ছাড়া অনুমোদন অর্থহীন, আর অনুমোদন ছাড়া প্রমাণীকরণ বিপজ্জনক।

IAM-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো এটি **allow-only** (শুধু অনুমতি দেয়, নিষেধ করে না) — কোনো explicit deny নেই (কিছু বিশেষ শর্ত ছাড়া)। অর্থাৎ, কাউকে অনুমতি না দিলে সে কাজটি করতে পারবে না; আলাদা করে নিষেধ করতে হয় না। আর অনুমতি **additive** — একাধিক role-এর অনুমতি যোগ হয়। কাউকে Viewer ও Editor — দুটো role দিলে সে Editor-এর সবকিছু করতে পারবে; Viewer তাকে সীমিত করবে না। তাই "Viewer দিয়ে Editor-কে সীমিত করব" — এই ধারণা ভুল।

৩.৪ Principal: মানুষ, গ্রুপ, সার্ভিস অ্যাকাউন্ট

Google অ্যাকাউন্ট — একজন মানুষের পরিচয় (যেমন `rahim@technova.example`)। **Google Group** — একাধিক মানুষের গোষ্ঠী (যেমন `platform@technova.example`)। IAM সবসময় গ্রুপকে দেওয়া উচিত, ব্যক্তিকে নয় — কারণ কর্মী যোগ দিলে বা চলে গেলে গ্রুপের সদস্যপদ বদলালেই হয়; প্রতিটি প্রজেক্টে গিয়ে অনুমতি বদলাতে হয় না। **Service account** — কোনো অ্যাপ্লিকেশন বা সেবার পরিচয় (যেমন `api-server@technova-prod.iam.gserviceaccount.com`)। এটি মানুষ নয়; এটি কোডের পরিচয়। একটি Cloud Run সেবা Cloud SQL-এ যুক্ত হতে চাইলে সে নিজের service account দিয়ে প্রমাণ করে সে কে। **Domain** — পুরো ডোমেইনের সবাই (যেমন `technova.example`) — খুব কম ব্যবহার করুন, কারণ এতে নতুন কর্মীও স্বয়ংক্রিয়ভাবে অনুমতি পেয়ে যায়।

Service account-এর দুটি গুরুত্বপূর্ণ বৈশিষ্ট্য: প্রথমত, এর কোনো পাসওয়ার্ড নেই — এর পরিচয় একটি কী (key) বা Google-এর পরিচালিত শংসাপত্র দিয়ে প্রমাণিত হয়। দ্বিতীয়ত, service account নিজে যেমন একটি principal (অন্য রিসোর্সে প্রবেশ পেতে পারে), তেমনি এটি একটি রিসোর্সও বটে (এর ওপর কে ব্যবহার করতে পারবে, তা IAM দিয়ে নিয়ন্ত্রণ করা যায়)। এই দ্বৈত ভূমিকা বুঝতে না পারলে impersonation বিভ্রান্তিকর মনে হবে।

৩.৫ Role: Basic, Predefined ও Custom

Basic role — Owner, Editor, Viewer। এগুলো পুরোনো ও খুব বিস্তৃত: Owner প্রায় সবকিছু করতে পারে, Editor প্রায় সবকিছু বানাতে ও বদলাতে পারে, Viewer শুধু দেখতে পারে। নতুন প্রজেক্টে এগুলো এড়িয়ে চলুন — কারণ Editor-কে দিলে সে IAM-ও বদলাতে পারে (নিজের অনুমতি বাড়িয়ে নিতে পারে)। শুধু শেখার সময় বা জরুরি পরিস্থিতিতে ব্যবহার করুন।

Predefined role — Google-এর বানানো নির্দিষ্ট কাজের role, যেমন `roles/compute.instanceAdmin` (VM পরিচালনা), `roles/storage.objectViewer` (bucket-এর ফাইল দেখা), `roles/cloudsql.client` (Cloud SQL-এ সংযোগ)। এগুলোই দৈনন্দিন কাজে ব্যবহার করুন — এগুলো least privilege-এর কাছাকাছি। **Custom role** — আপনি নিজে permission-এর তালিকা দিয়ে বানান, যখন কোনো predefined role ঠিক মেলে না। যেমন TechNova-র ডেভেলপারদের VM দেখা ও পুনরায় চালু করার অনুমতি দরকার, কিন্তু মুছে ফেলার নয় — তখন একটি custom role বানানো যায়। Custom role বানানোর আগে দেখুন predefined role-তে কাজ হয় কি না; custom role পরিচালনার ভার বাড়ায়।

Role-এর ধরন	উদাহরণ	কখন ব্যবহার করবেন	সতর্কতা
Basic	Owner, Editor, Viewer	শেখা, জরুরি কাজ	খুব বিস্তৃত; production-এ এড়ান
Predefined	roles/storage.objectViewer	প্রায় সব দৈনন্দিন কাজ	মাঝে মাঝে প্রয়োজনের চেয়ে বেশি দেয়
Custom	নিজের বানানো role	predefined মেলে না	পরিচালনা ও পরীক্ষার ভার

৩.৬ IAM Policy, Inheritance ও Resource-Level অনুমতি

একটি IAM policy হলো bindings-এর তালিকা: প্রতিটি binding বলে "এই principal-কে এই role দাও"। Policy কোনো রিসোর্সের সাথে যুক্ত থাকে — Organization, Folder, Project, বা একটি নির্দিষ্ট bucket বা VM-এ। ওপরের স্তরের policy নিচের স্তরে উত্তরাধিকারসূত্রে যায়: Organization-এ দেওয়া Viewer সব প্রজেক্টে Viewer। কিন্তু নিচের স্তরে আরও role যোগ করা যায় — তাতে ওপরের অনুমতি কমে না।

Resource-level অনুমতি হলো সবচেয়ে সূক্ষ্ম নিয়ন্ত্রণ: একটি নির্দিষ্ট Cloud Storage bucket-এ কাউকে objectViewer দিন, পুরো প্রজেক্টে নয়। একটি নির্দিষ্ট VM-এ কাউকে instanceAdmin দিন। এভাবে "blast radius" কমে — একটি ভুল বা একটি আপস হওয়া অ্যাকাউন্ট পুরো প্রজেক্ট ধ্বংস করতে পারে না। তবে মনে রাখবেন: কিছু permission শুধু প্রজেক্ট-স্তরে কাজ করে (যেমন API সক্রিয় করা); সেগুলো bucket-স্তরে দিলে কাজ হবে না।

IAM Conditions দিয়ে অনুমতিতে শর্ত যোগ করা যায় — যেমন "শুধু কর্মঘণ্টায়", "শুধু নির্দিষ্ট IP থেকে", "শুধু `env=prod` লেবেলের রিসোর্সে"। এটি শক্তিশালী, তবে জটিল; প্রথমে role ও scope ঠিক করুন, তারপর শর্ত যোগ করুন।

IAM সিদ্ধান্তের প্রবাহ অনুরোধ (principal + কাজ + রিসোর্স) → Authentication: আপনি কে? (Google অ্যাকাউন্ট / service account) → Authorization: Policy দেখুন — Organization → Folder → Project → Resource → সব স্তরের role যোগ করুন (additive) → অনুমতি আছে? হ্যাঁ → কাজ করুন | না → 403 PERMISSION_DENIED → Audit Log-এ লিপিবদ্ধ করুন (কে, কী, কখন, কোথা থেকে)

৩.৭ Service Account: তৈরি, ব্যবহার ও সুরক্ষা

Service account বানানো Console-এ: IAM & Admin → Service Accounts → Create। gcloud-এ:

```
gcloud iam service-accounts create api-server \
  --display-name="TechNova API Server" \
  --project=technova-prod-web
gcloud projects add-iam-policy-binding technova-prod-web \
  --member="serviceAccount:api-server@technova-prod-web.iam.gserviceaccount.com" \
  --role="roles/cloudsql.client"
```

Service account ব্যবহারের সবচেয়ে নিরাপদ উপায় হলো Google-এর পরিচালিত পরিচয় ব্যবহার করা — যেমন Compute Engine VM বা Cloud Run সেবার সাথে service account সংযুক্ত করা; তখন Google স্বয়ংক্রিয়ভাবে শংসাপত্র দেয় ও ঘোরায় (rotate করে), আপনাকে কোনো কী ফাইল রাখতে হয় না। **Service account key** (JSON ফাইল ডাউনলোড করা) হলো সবচেয়ে ঝুঁকিপূর্ণ পদ্ধতি: কী ফাইলটি যার হাতে যাবে, সে ওই service account-এর সব অনুমতি পাবে; Git-এ ভুলে কমিট হলে সেটি চিরতরে ফাঁস হয়ে যায়। কী ব্যবহার করতেই হলে: শুধু যখন বিকল্প নেই (যেমন বাইরের সিস্টেম), নিয়মিত ঘোরান, ব্যবহার audit করুন, আর Organization Policy দিয়ে কী তৈরি সীমিত করুন।

> **সতর্কতা:** Service account key JSON ফাইল কখনো Git, চ্যাট বা ইমেইলে পাঠাবেন না, কখনো স্ক্রিনশটে দেখাবেন না। একটি ফাঁস হওয়া কী দিয়ে আক্রমণকারী আপনার প্রজেক্টে ক্রিপ্টো মাইনিং চালাতে পারে — আর বিল আপনার। কী ফাঁস হয়েছে সন্দেহ হলে তাৎক্ষণিকভাবে কীটি মুছে ফেলুন (`gcloud iam service-accounts keys delete`), শুধু "পরে ঘোরাব" বলে অপেক্ষা করবেন না।

৩.৮ Workload Identity Federation: কী ছাড়া পরিচয়

আপনার অ্যাপ্লিকেশন যদি GCP-র বাইরে চলে — যেমন GitHub Actions, AWS, বা অন-প্রেমিসেস সার্ভারে — তাহলে আগে service account key ডাউনলোড করে দিতে হতো। **Workload Identity Federation (WIF)** এই কী-এর প্রয়োজন দূর করে: বাইরের identity provider (যেমন GitHub-এর OIDC token) Google-এর কাছে প্রমাণ করে, আর Google তার বিনিময়ে অস্থায়ী শংসাপত্র দেয়। কী কোথাও সংরক্ষণ করতে হয় না, ঘোরাতে হয় না, ফাঁস হওয়ার ঝুঁকি থাকে না।

কীভাবে কাজ করে: (১) GCP-তে একটি Workload Identity Pool ও Provider বানান (যেমন GitHub provider); (২) বাইরের পরিচয়কে একটি service account-এর সাথে ম্যাপ করুন — কোন GitHub repository, কোন branch, কোন service account ব্যবহার করতে পারবে, তা নির্দিষ্ট করুন; (৩) CI/CD-তে OIDC token দিয়ে `gcloud`-এ প্রমাণীকরণ করুন। GitHub Actions-এর উদাহরণে `google-github-actions/auth` অ্যাকশন এই কাজটি করে। নিয়ম: provider-এ attribute condition যতটা সম্ভব নির্দিষ্ট করুন — শুধু একটি repository, শুধু `main` branch — যাতে অন্য repository আপনার service account ব্যবহার করতে না পারে।

GCP-র ভেতরে একই ধারণার নাম **Workload Identity** (GKE-র জন্য): একটি Kubernetes Service Account-কে একটি Google Service Account-এর সাথে যুক্ত করা হয়; Pod তখন কী ছাড়াই Google API ব্যবহার করতে পারে। GKE ব্যবহার করলে এটি ব্যবহার করুন — Pod-এ কী ফাইল mount করা পুরোনো ও ঝুঁকিপূর্ণ পদ্ধতি।

৩.৯ Service Account Impersonation ও Just-in-Time প্রবেশ

Impersonation মানে একজন principal সাময়িকভাবে একটি service account-এর পরিচয় ধারণ করা — যেমন একজন ডেভেলপার CI/CD-র service account হয়ে deploy করা, বা একজন অ্যাডমিন একটি সীমিত service account হয়ে পরীক্ষা করা। এটি করতে `roles/iam.serviceAccountTokenCreator` ভূমিকা লাগে। সুবিধা: আসল কী ভাগ করতে হয় না; প্রতিটি impersonation audit log-এ থাকে। বিপদ: TokenCreator ভূমিকা যার আছে, সে ওই service account-এর সব অনুমতি ব্যবহার করতে পারে — তাই এটি শুধু বিশ্বস্ত মানুষ ও স্বয়ংক্রিয় সিস্টেমকে দিন।

```
gcloud iam service-accounts add-iam-policy-binding \
  deployer@technova-prod-web.iam.gserviceaccount.com \
  --member="user:rahim@technova.example" \
  --role="roles/iam.serviceAccountTokenCreator" \
  --project=technova-prod-web
gcloud auth print-access-token \
  --impersonate-service-account=deployer@technova-prod-web.iam.gserviceaccount.com
```

আরও ভালো পদ্ধতি হলো **just-in-time (JIT) প্রবেশ**: স্থায়ীভাবে শক্তিশালী ভূমিকা না দিয়ে, প্রয়োজনের সময় সাময়িকভাবে অনুমতি দেওয়া, কাজ শেষে তুলে নেওয়া। ছোট দলে এটি হাতে করা যায়; বড় দলে Privileged Access Manager (PAM) ব্যবহার করুন। নিয়মটি মনে রাখুন: স্থায়ী Owner-এর চেয়ে সাময়িক Editor অনেক নিরাপদ।

৩.১০ Least Privilege: তত্ত্ব থেকে অভ্যাস

Least privilege মানে: প্রত্যেককে তার কাজের জন্য যতটুকু দরকার, ঠিক ততটুকু অনুমতি দিন — তার বেশি নয়, কমও নয়। এটি বাস্তবায়নের অভ্যাস: (১) শুরুতে Viewer বা নির্দিষ্ট predefined role দিন, প্রয়োজনে বাড়ান — Editor দিয়ে শুরু করে কমাবেন না; (২) গ্রুপ ব্যবহার করুন; (৩) প্রজেক্ট-স্তরের বদলে resource-স্তরের অনুমতি ভাবুন; (৪) নিয়মিত পর্যালোচনা করুন — `Policy Analyzer` বা Recommender দেখুন কোন অনুমতি আসলে ব্যবহার হয়নি; (৫) service account-এর অনুমতিও একইভাবে পর্যালোচনা করুন — একটি service account-কে Editor দেওয়া একটি মানুষকে Editor দেওয়ার মতোই বিপজ্জনক।

একটি সাধারণ ভুল হলো "কাজ হচ্ছে না, তাই Owner দিয়ে দিই" — এটি সমস্যা লুকায়, সমাধান করে না। সঠিক পদ্ধতি: `gcloud`-এর ভুল বার্তা পড়ুন — কোন permission নেই, তা বার্তায় লেখা থাকে; Policy Troubleshooter (Console → IAM → Troubleshooter) দিয়ে দেখুন কোন policy বাধা দিচ্ছে; তারপর ঠিক সেই permission-সহ role দিন।

৩.১১ IAM Troubleshooting

TROUBLESHOOTING: 403 PERMISSION_DENIED পেলে: (১) `gcloud auth list` দিয়ে দেখুন আপনি কোন অ্যাকাউন্টে আছেন — ভুল অ্যাকাউন্ট খুব সাধারণ কারণ; (২) `gcloud projects get-iam-policy PROJECT_ID` দিয়ে দেখুন ওই principal-এর কী role আছে; (৩) API সক্রিয় আছে কি না দেখুন — IAM ঠিক থাকলেও বন্ধ API-তে 403 আসে; (৪) Policy Troubleshooter ব্যবহার করুন — এটি বলে দেবে কোন policy অনুমতি দিচ্ছে বা দিচ্ছে না; (৫) Service account-এর ক্ষেত্রে দেখুন service account-টি কোন প্রজেক্টের — অন্য প্রজেক্টের service account-কে এই প্রজেক্টে role দিতে হয়। মনে রাখবেন: IAM পরিবর্তন কার্যকর হতে কয়েক মিনিট সময় লাগতে পারে — বদলে সাথে সাথে পরীক্ষা করে "কাজ হচ্ছে না" বলবেন না।

৩.১২ প্রোডাকশন দৃশ্যকল্প: TechNova-র IAM নকশা

TechNova-র গ্রুপ: `platform@technova.example` (production প্রজেক্টে Compute Admin + Cloud SQL Admin), `developers@technova.example` (dev প্রজেক্টে Editor-এর বদলে নির্দিষ্ট role; staging-এ Viewer + Cloud Run Developer; production-এ শুধু Viewer), `billing@technova.example` (শুধু Billing Account Viewer)। Service account: `api-server` শুধু Cloud SQL Client ও Storage Object Viewer; `deployer` শুধু Cloud Run Admin, আর তার TokenCreator শুধু CI/CD-র WIF পরিচয়ের কাছে। কেউ সরাসরি Owner নয় — Organization Admin দুজন, MFA-সহ। প্রতি মাসের প্রথম সোমবার তারা IAM পর্যালোচনা করে: Recommender-এর "ব্যবহার হয়নি" role সরানো হয়।

বাস্তব উদাহরণ: একটি দল তাদের CI/CD-তে service account key GitHub secret-এ রেখেছিল। একটি ভুলে key-টি লগে ছাপা হয়ে যায়, আর কয়েক ঘণ্টার মধ্যে অজানা IP থেকে শত শত VM চালু হয় — বিল হাজার ডলার ছাড়িয়ে যায়। তারা key মুছে WIF-এ যায়, আর তারপর থেকে কোনো key কোথাও সংরক্ষণ করে না। শিক্ষা: key হলো স্থায়ী ঝুঁকি; federation হলো অস্থায়ী ও সীমিত পরিচয়।

৩.১৩ IAM নিরীক্ষা: Policy Analyzer ও Recommender

IAM একবার বসিয়ে ভুলে গেলে চলবে না — নিয়মিত নিরীক্ষা করতে হয়। **Policy Analyzer** (Console → IAM → Policy Analyzer) দিয়ে দেখুন কোন principal-এর কোন রিসোর্সে কী অনুমতি আছে, আর কোন অনুমতি আসলে ব্যবহার হয়নি। **IAM Recommender** বলে দেয় কোন role কমানো যায় — যেমন একজন ডেভেলপারকে Editor দেওয়া হয়েছে, কিন্তু তিনি গত ৯০ দিনে শুধু দেখার কাজ করেছেন — তাহলে Viewer যথেষ্ট। **Access Transparency ও Audit Logs** দেখায় কে, কখন, কী করেছে — বিশেষ করে IAM পরিবর্তন ও service account ব্যবহার। একটি ভালো অভ্যাস: প্রতি মাসে Recommender-এর পরামর্শ পর্যালোচনা করুন, আর প্রতি ত্রৈমাসিকে Organization-স্তরে কাদের Owner বা Organization Admin আছে, তা যাচাই করুন। মনে রাখবেন: Recommender-এর পরামর্শ একটি পরামর্শ — তা প্রয়োগের আগে যাচাই করুন যে ওই ব্যক্তির কাজে ওই অনুমতি সত্যিই লাগে না; ভুলে অনুমতি কমালে কাজ বন্ধ হয়ে যেতে পারে।

৩.১৪ IAM ও নেটওয়ার্ক: একসাথে নিরাপত্তা

IAM একা নিরাপত্তা দেয় না — এটি নেটওয়ার্ক নিয়ন্ত্রণের সাথে মিলে কাজ করে। উদাহরণ: একজন ডেভেলপারের Cloud SQL Client ভূমিকা থাকলেও, Cloud SQL ইনস্ট্যান্সটি যদি private IP-তে থাকে ও VPC-তে তার প্রবেশ না থাকে, তাহলে সে সংযোগ করতে পারবে না। আবার, VPC-তে প্রবেশ থাকলেও IAM না থাকলে সে ডেটাবেস দেখতে পারবে না। এই দুই স্তরের নিরাপত্তাকে *defense in depth* বলে — একটি স্তর ভাঙলেও আরেকটি রক্ষা করে। একইভাবে, Cloud Storage

bucket-এ IAM দিয়ে কে ফাইল দেখতে পারবে, তা ঠিক হয়; আর public access prevention দিয়ে নিশ্চিত করা হয় যে কেউ ভুলে bucket-টি public করে দিতে পারবে না। নতুনদের একটি সাধারণ ভুল হলো শুধু IAM দেখা আর ফায়ারওয়াল বা public access দেখা না — দুটোই দেখুন, কারণ আক্রমণকারী যেকোনো খোলা দরজা দিয়ে ঢোকে।

৩.১৫ সাধারণ ভুল

- সবাইকে Editor বা Owner দেওয়া — "সহজ হবে" বলে।
- ব্যক্তিকে IAM দেওয়া, গ্রুপকে নয়।
- Service account key ডাউনলোড করে Git-এ রাখা।
- Service account-কে Editor দেওয়া — "অ্যাপের তো সবকিছু লাগবে" বলে।
- IAM বদলে সাথে সাথে পরীক্ষা করে ব্যর্থ ভাবা (propagation সময় লাগে)।
- Audit log না দেখা — কে কী করল, তা না জানা।

৩.১৬ সারাংশ

IAM হলো তিনটি প্রশ্ন: কে, কী, কোথায়। Principal-কে role দিন, role-কে policy-তে বাঁধুন, policy-কে রিসোর্সে যুক্ত করুন। গ্রুপ ব্যবহার করুন, predefined role পছন্দ করুন, service account-এর কী এড়িয়ে Workload Identity Federation ব্যবহার করুন, আর least privilege-কে অভ্যাস বানান। IAM-এর ভুল ধীরে ধীরে বড় হয় — আজ দেওয়া অতিরিক্ত অনুমতি কালকের নিরাপত্তা ঘটনা হতে পারে। পরের অধ্যায়ে আমরা এই পরিচয় দিয়ে `gcloud` CLI ও স্বয়ংক্রিয়তা শিখব — কারণ IAM ঠিক থাকলেই automation নিরাপদ হয়।

৩.১৭ পর্যালোচনা প্রশ্ন

- Authentication ও authorization-এর পার্থক্য কী? একটি অনুরোধে এগুলো কোন ক্রমে হয়?
- Basic, predefined ও custom role-র মধ্যে কখন কোনটি বেছে নেবেন?
- Service account key-এর বদলে Workload Identity Federation কেন নিরাপদ?
- Impersonation কী এবং TokenCreator ভূমিকা কেন সতর্কতার সাথে দিতে হয়?
- একজন ডেভেলপার 403 পেলে আপনি কোন ক্রমে সমস্যা নির্ণয় করবেন?

৩.১৮ ব্যবহারিক অনুশীলন

- একটি পরীক্ষামূলক প্রজেক্টে একটি service account বানান, তাকে শুধু Storage Object Viewer দিন, এবং যাচাই করুন সে bucket বানাতে পারে না।
- Policy Troubleshooter দিয়ে একটি ইচ্ছাকৃত 403-এর কারণ খুঁজে বের করুন।
- আপনার প্রজেক্টের IAM policy দেখে তিনটি অতিরিক্ত অনুমতি চিহ্নিত করুন ও কীভাবে কমাবেন, তা লিখুন।

অধ্যায় ৪ • PART I — GCP Foundations & Architecture

Chapter 4 — Google Cloud CLI and Automation

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Google Cloud SDK ও `gcloud` CLI ইনস্টল ও হালনাগাদ করতে; (২) ব্যবহারকারী ও service account দিয়ে নিরাপদে প্রমাণীকরণ করতে; (৩) configuration, প্রজেক্ট, Region ও Zone পরিচালনা করতে; (৪) Cloud Shell কার্যকরভাবে ব্যবহার করতে; (৫) environment variable ও স্ক্রিপ্ট দিয়ে পুনরাবৃত্তিযোগ্য কাজ করতে; (৬) SDK ও API দিয়ে স্বয়ংক্রিয়তা বানাতে; এবং (৭) শংসাপত্র-সংক্রান্ত সাধারণ সমস্যা নির্ণয় করতে।

৪.২ পূর্বজ্ঞান ঝালাই

আপনার জানা থাকা ভালো: Linux/macOS টার্মিনালে কমান্ড চালানো, environment variable (`export KEY=value`) কী, JSON ফাইলের গঠন, এবং অধ্যায় ৩-এর IAM ধারণা — principal, service account, role। `gcloud` হলো Google Cloud API-র কমান্ড-লাইন আবরণ (wrapper): আপনি যখন `gcloud compute instances create` চালান, তখন `gcloud` আপনার শংসাপত্র দিয়ে Compute Engine API-তে একটি HTTPS অনুরোধ পাঠায়। Console-এ ক্লিক করলেও একই API-ই ডাকা হয় — শুধু মাধ্যম আলাদা। এই সত্যটি বুঝলে CLI আর "আলাদা জগৎ" মনে হবে না।

৪.৩ Google Cloud SDK কী

Google Cloud SDK হলো কয়েকটি টুলের সমষ্টি। মূল টুল **`gcloud`** — প্রজেক্ট, Compute Engine, Cloud Storage, IAM, Cloud Run ইত্যাদি পরিচালনা করে। **`gsutil`** — Cloud Storage-এর জন্য বিশেষায়িত টুল (এখন অনেক কাজ `gcloud storage` দিয়েও হয়)। **`bq`** — BigQuery-র জন্য। **`kubectl`** — GKE ক্লাস্টার পরিচালনার জন্য SDK-র সাথে আসে। আর **client libraries** — Python, Go, Node.js, Java ইত্যাদি ভাষা থেকে Google Cloud API ব্যবহারের লাইব্রেরি।

SDK ইনস্টল করা: Linux-এ Google-এর apt/yum repository বা ইনস্টলার স্ক্রিপ্ট দিয়ে; macOS-এ Homebrew বা ইনস্টলার দিয়ে; Windows-এ ইনস্টলার দিয়ে। ইনস্টলের পর যাচাই করুন:

```
gcloud --version
gcloud components list
gcloud components update
```

নিয়মিত `gcloud components update` চালান — নতুন সেবা ও সংশোধন আসে। তবে production স্ক্রিপ্ট চালানো মেশিনে হঠাৎ আপডেট করবেন না; প্রথমে পরীক্ষামূলক পরিবেশে যাচাই করুন। একটি নির্দিষ্ট সংস্করণে আটকে রাখতে চাইলে Docker ইমেজ (`google/cloud-sdk:slim`) ব্যবহার করুন, যেখানে সংস্করণ নির্দিষ্ট করা যায়।

৪.৪ প্রমাণীকরণ: আপনি কে, তা প্রমাণ করা

নিজের কাজের জন্য (ইন্টারঅ্যাক্টিভ ব্যবহার) প্রমাণীকরণ:

```
gcloud auth login
gcloud auth list
gcloud auth application-default login
```

`gcloud auth login` ব্রাউজারে Google অ্যাকাউন্টে সাইন-ইন করায় ও `gcloud` কমান্ডের জন্য শংসাপত্র সংরক্ষণ করে। `gcloud auth application-default login` (ADC — Application Default Credentials) আলাদা: এটি আপনার কোড (যেমন Python client library) যখন API ডাকে, তখন ব্যবহৃত হয়। দুটোকে গুলিয়ে ফেলবেন না: প্রথমটি CLI-র জন্য, দ্বিতীয়টি অ্যাপ্লিকেশন কোডের জন্য।

Service account দিয়ে প্রমাণীকরণ (স্বয়ংক্রিয় সিস্টেমের জন্য):

```
gcloud auth activate-service-account \
  --key-file=/path/to/CHANGE-ME-key.json \
  --project=technova-prod-web
```

কিন্তু মনে রাখুন অধ্যায় ৩-এর সতর্কতা: key ফাইল ব্যবহার করা ঝুঁকিপূর্ণ। CI/CD-তে Workload Identity Federation ব্যবহার করুন (অধ্যায় ৩.৮), Compute Engine বা Cloud Run-এ attached service account ব্যবহার করুন, আর স্থানীয় পরীক্ষায় ADC ব্যবহার করুন। Key ফাইল ব্যবহার করতেই হলে: ফাইলটি `~/.config/gcloud/`-এর বাইরে সুরক্ষিত জায়গায় রাখুন, `chmod 600` করুন, কখনো Git-এ কমিট করবেন না, আর কাজ শেষে `gcloud auth revoke` করুন।

> **সতর্কতা:** `gcloud auth login` দিয়ে পাওয়া শংসাপত্র আপনার ল্যাপটপে সংরক্ষিত থাকে। ল্যাপটপ হারালে বা অন্য কেউ ব্যবহার করলে সেই শংসাপত্র দিয়ে আপনার প্রজেক্টে প্রবেশ করা যায়। ল্যাপটপে ডিস্ক এনক্রিপশন ও স্ক্রিন লক চালু রাখুন, আর শেয়ার্ড মেশিনে কাজ শেষে `gcloud auth revoke --all` চালান।

৪.৫ Configuration: প্রজেক্ট, Region ও অ্যাকাউন্ট

`gcloud`-এর **configuration** হলো সেটিংসের একটি নামযুক্ত সেট: কোন অ্যাকাউন্ট, কোন প্রজেক্ট, কোন Region, কোন Zone। ডিফল্ট configuration-এর নাম `default`। একাধিক প্রজেক্টে কাজ করলে আলাদা configuration বানান — তাতে ভুল প্রজেক্টে কমান্ড চালানোর ঝুঁকি কমে:

```
gcloud config configurations create technova-prod
gcloud config configurations activate technova-prod
gcloud config set account rahim@technova.example
gcloud config set project technova-prod-web
gcloud config set compute/region asia-south1
gcloud config set compute/zone asia-south1-a
gcloud config list
gcloud config configurations list
```

একটি কমান্ডে সাময়িকভাবে প্রজেক্ট বদলাতে `--project=PROJECT_ID` ফ্ল্যাগ ব্যবহার করুন — এটি configuration বদলায় না, শুধু ওই কমান্ডের জন্য প্রযোজ্য। স্ক্রিপ্টে সবসময় `--project` স্পষ্টভাবে দিন; configuration-এর ওপর নির্ভর করবেন না — কারণ স্ক্রিপ্টটি অন্য মেশিনে চললে configuration আলাদা হতে পারে।

৪.৬ Cloud Shell: ব্রাউজারে টার্মিনাল

Cloud Shell হলো Google Cloud Console-এর ভেতরে ব্রাউজার-ভিত্তিক টার্মিনাল — একটি ছোট VM, যেখানে `gcloud`, `kubectl`, `git`, `docker` আগে থেকে ইনস্টল করা থাকে, আর আপনার হোম ডিরেক্টরি (৫ GB) স্থায়ীভাবে সংরক্ষিত থাকে। এটি শেখার জন্য, দ্রুত পরীক্ষার জন্য ও ল্যাপটপে SDK ইনস্টল না থাকলে কাজের জন্য চমৎকার। সীমাবদ্ধতা: এটি একটি ছোট ephemeral VM — দীর্ঘস্থায়ী ভারী কাজ, বড় বিল্ড, বা production deploy-এর একমাত্র মাধ্যম হিসেবে ব্যবহার করবেন না; আর এতে সংবেদনশীল ফাইল (যেমন service account key) রাখবেন না। Cloud Shell খুলতে Console-এর ওপরের টার্মিনাল আইকনে ক্লিক করুন, অথবা `shell.cloud.google.com`-এ যান।

Cloud Shell-এ একটি দরকারি অভ্যাস: কাজের শুরুতে `gcloud config list` দিয়ে দেখুন কোন প্রজেক্ট সক্রিয় আছে — কারণ Cloud Shell Console-এর নির্বাচিত প্রজেক্ট অনুসরণ করে, আর ভুল প্রজেক্টে কমান্ড চালানো সহজ।

৪.৭ Environment Variable ও স্ক্রিপ্ট: পুনরাবৃত্তিযোগ্য কাজ

বারবার একই মান টাইপ না করে environment variable ব্যবহার করুন:

```
export PROJECT_ID=technova-staging
export REGION=asia-south1
export BUCKET_NAME=technova-staging-assets-CHANGE-ME
gcloud config set project "$PROJECT_ID"
gsutil mb -p "$PROJECT_ID" -l "$REGION" "gs://$BUCKET_NAME"
```

একটি স্ক্রিপ্টে নিরাপত্তা যোগ করুন: শুরুতে `set -euo pipefail` দিন (ভুল হলে স্ক্রিপ্ট থেমে যাবে), প্রজেক্ট ID যাচাই করুন, আর `--quiet` ব্যবহারের আগে ভাবুন — এটি নিশ্চিতকরণ প্রশ্ন ছাড়াই কাজ করে, তাই ধ্বংসাত্মক কমান্ডে (`delete`) ব্যবহার করবেন না। একটি উদাহরণ — bucket বানানো ও সংস্করণ (versioning) চালু করা:

```
#!/bin/bash
set -euo pipefail
PROJECT_ID="${PROJECT_ID:-technova-staging}"
REGION="${REGION:-asia-south1}"
BUCKET="technova-staging-assets-CHANGE-ME"
gcloud storage buckets create "gs://$BUCKET" \
  --project="$PROJECT_ID" --location="$REGION" \
  --uniform-level-access
gcloud storage buckets update "gs://$BUCKET" --versioning
```

স্ক্রিপ্টে কখনো পাসওয়ার্ড, key বা token সরাসরি লিখবেন না। গোপন মান Secret Manager থেকে নিন (অধ্যায় ৫-এ ধারণা, বিস্তারিত পরের ভলিউমে) অথবা environment variable হিসেবে বাইরে থেকে দিন। স্ক্রিপ্ট Git-এ রাখলে প্রথমে `grep -r "CHANGE-ME\|BEGIN PRIVATE KEY" .` দিয়ে যাচাই করুন কোনো গোপন মান নেই।

৪.৮ SDK ও API দিয়ে স্বয়ংক্রিয়তা

CLI ছাড়াও আপনি সরাসরি API ব্যবহার করতে পারেন। Python client library-র উদাহরণ:

```
from google.cloud import storage
client = storage.Client(project="technova-staging")
bucket = client.bucket("technova-staging-assets-CHANGE-ME")
print([blob.name for blob in bucket.list_blobs(max_results=10)])
```

এই কোড ADC ব্যবহার করে — অর্থাৎ আপনি `gcloud auth application-default login` করলে এটি আপনার পরিচয়ে চলে; Compute Engine-এ চললে attached service account ব্যবহার করে। REST API সরাসরি ডাকতে চাইলে `curl` ও access token ব্যবহার করুন:

```
ACCESS_TOKEN="$(gcloud auth print-access-token)"
curl -s -H "Authorization: Bearer $ACCESS_TOKEN" \
  "https://storage.googleapis.com/storage/v1/b?project=technova-staging"
```

Access token স্বল্পস্থায়ী (সাধারণত ১ ঘণ্টা) — এটি লগে ছাপাবেন না, ফাইলে সংরক্ষণ করবেন না। স্বয়ংক্রিয়তায় `gcloud auth print-access-token` বারবার ডাকার বদলে client library ব্যবহার করুন — লাইব্রেরি নিজেই token নবায়ন করে।

Automation-এর স্তর হাতে Console-এ ক্লিক → শেখা ও এককালীন কাজ gcloud কমান্ড → পুনরাবৃত্তিযোগ্য কাজ, স্ক্রিপ্ট Bash / Python স্ক্রিপ্ট → একাধিক ধাপের কাজ Terraform / IaC → পুরো অবকাঠামো কোড হিসেবে (পরের ভলিউমে বিস্তারিত) CI/CD (WIF) → Git push → স্বয়ংক্রিয় deploy, কোনো key ছাড়া নিয়ম: যত ওপরে যাবেন, তত কম হাতের কাজ, তত বেশি পরীক্ষা দরকার।

৪.৯ নিরাপদ স্বয়ংক্রিয়তা: নীতিমালা

স্বয়ংক্রিয়তা শক্তিশালী — তাই ভুলও দ্রুত ছড়ায়। একটি স্ক্রিপ্টে ভুল প্রজেক্ট ID থাকলে সেটি production-এর সব VM মুছে দিতে পারে। নিরাপত্তার নিয়ম: (১) ধ্বংসাত্মক কমান্ডে (`delete`, `remove-iam-policy-binding`) সবসময় নিশ্চিতকরণ রাখুন; (২) স্ক্রিপ্টে প্রজেক্ট ID হার্ডকোড না করে যাচাই করুন — যেমন production প্রজেক্টে চলার আগে একটি সতর্কতা দেখান; (৩) automation-এর service account-কে least privilege দিন — deploy-এর service account-কে IAM বদলানোর অনুমতি দেবেন না; (৪) প্রতিটি স্বয়ংক্রিয় কাজের লগ রাখুন; (৫) প্রথমে `--dry-run` বা পরীক্ষামূলক প্রজেক্টে চালান, যেখানে সম্ভব। Terraform-এর `plan` এই কারণেই মূল্যবান — এটি দেখায় কী বদলাবে, বদলানোর আগে।

CONCEPT: Automation-এর লক্ষ্য "ক্লিক বাঁচানো" নয় — লক্ষ্য হলো একই কাজ প্রতিবার একইভাবে হওয়া। হাতে ক্লিক করলে একবার ভুল হয়; স্ক্রিপ্টে ভুল থাকলে প্রতিবার ভুল হয়। তাই স্ক্রিপ্ট লেখার পর তাকে কোডের মতোই পরীক্ষা করুন: পরীক্ষামূলক প্রজেক্টে চালান, আউটপুট পড়ুন, আর অন্য কাউকে দিয়ে পর্যালোচনা করান।

৪.১০ Credential Troubleshooting

TROUBLESHOOTING: "You do not have an active account" — `gcloud auth login` করুন। "Permission denied" — অধ্যায় ৩-এর ক্রম অনুসরণ করুন: অ্যাকাউন্ট, প্রজেক্ট, role, API। "Application Default Credentials not found" — `gcloud auth application-default login` করুন, অথবা `GOOGLE_APPLICATION_CREDENTIALS` environment variable-টি সঠিক JSON ফাইলের দিকে নির্দেশ করছে কি না দেখুন। "Token has expired" — `gcloud auth login` আবার করুন; service account key-র ক্ষেত্রে key-টি মুছে ফেলা হয়েছে কি না দেখুন। "Quota exceeded" — এটি IAM নয়, কোটার সমস্যা (অধ্যায় ৫ দেখুন)। আর একটি সাধারণ সমস্যা: একাধিক configuration-এর মধ্যে গুলিয়ে যাওয়া — `gcloud config configurations list` দিয়ে দেখুন কোনটি সক্রিয়, আর স্ক্রিপ্টে `--project` স্পষ্ট করুন।

৪.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র দৈনন্দিন CLI ব্যবহার

TechNova-র দল প্রতিদিন Cloud Shell বা নিজের ল্যাপটপে `gcloud` ব্যবহার করে। তাদের নিয়ম: প্রত্যেকের দুটি configuration — `technova-dev` ও `technova-prod` (prod সক্রিয় করতে সচেতনভাবে `activate` করতে হয়); স্ক্রিপ্টে সবসময় `--project`; deploy শুধু CI/CD থেকে WIF দিয়ে, কারও ল্যাপটপ থেকে নয়; আর `gcloud` আউটপুট JSON হিসেবে নিয়ে স্ক্রিপ্টে ব্যবহার করা হয় (`--format=json`) যাতে আউটপুটের গঠন নির্ভরযোগ্য থাকে। একটি উদাহরণ:

```
gcloud run services list --project=technova-prod-web \
  --region=asia-south1 --format="table(name,status.latestCreatedRevision)"
gcloud logging read "resource.type=cloud_run_revision" \
  --project=technova-prod-web --limit=10 --format=json
```

৪.১২ gcloud-এর আউটপুট: ফিল্টার, ফরম্যাট ও অটোমেশন

`gcloud`-এর আসল শক্তি বেরিয়ে আসে যখন আপনি এর আউটপুটকে স্ক্রিপ্টে ব্যবহার করেন। `--format` ফ্ল্যাগ দিয়ে আউটপুটের আকার ঠিক করুন — `table` মানুষের পড়ার জন্য, `json` ও `yaml` স্ক্রিপ্টের জন্য, `value` একটি মাত্র মান নেওয়ার জন্য। `--filter` দিয়ে ফলাফল ছাঁকুন:

```
gcloud compute instances list --project=technova-staging \
  --filter="status:RUNNING AND zone:asia-south1-a" \
  --format="table(name, machineType, status)"
gcloud compute instances list --project=technova-staging \
  --format="value(name)" | while read -r vm; do
    echo "Checking $vm"
  done
gcloud storage buckets list --project=technova-staging --format=json
```

একটি দরকারি অভ্যাস: স্ক্রিপ্টে `--format=json` ব্যবহার করুন ও `jq` দিয়ে মান বের করুন — মানুষের পড়ার মতো `table` আউটপুটের গঠন সংস্করণে বদলাতে পারে, কিন্তু JSON-এর গঠন স্থিতিশীল থাকে। আর `gcloud`-এর প্রতিটি কমান্ডের সাহায্য দেখতে `--help` ব্যবহার করুন — যেমন `gcloud run deploy --help` — এতে সব ফ্ল্যাগ ও উদাহরণ পাবেন, আর ইন্টারনেটে খুঁজতে হবে না।

৪.১৩ API কোটা, রেট লিমিট ও পুনঃচেষ্টা

স্বয়ংক্রিয় স্ক্রিপ্ট যখন শত শত API অনুরোধ পাঠায়, তখন **rate limit** (প্রতি মিনিটে কত অনুরোধ) ও **quota**-র মুখোমুখি হয়। `gcloud` সাধারণত 429 (Too Many Requests) পেলে নিজে থেকে কিছুক্ষণ অপেক্ষা করে আবার চেষ্টা করে, কিন্তু আপনার নিজের স্ক্রিপ্টে সেই যুক্তি যোগ করুন: exponential backoff — প্রথমে ১ সেকেন্ড, তারপর ২, ৪, ৮ সেকেন্ড অপেক্ষা করুন। আর একসাথে অনেক অনুরোধ না পাঠিয়ে ব্যাচ করুন — যেমন ১০০টি VM একসাথে না বানিয়ে ১০টি করে বানান। একটি সাধারণ ভুল হলো স্ক্রিপ্টে `for` লুপে দ্রুত API ডাকা, তারপর 429 পেয়ে স্ক্রিপ্ট ব্যর্থ হওয়া — আর অর্ধেক কাজ হয়ে যাওয়া। স্ক্রিপ্টকে *idempotent* বানান — একই স্ক্রিপ্ট দুবার চালালে দ্বিতীয়বার যেন নতুন রিসোর্স না বানিয়ে বলে "আগেই আছে" — তাহলে ব্যর্থতার পর আবার চালানো নিরাপদ হয়।

৪.১৪ gcloud Alpha, Beta ও নতুন সেবা

`gcloud`-এর অনেক কমান্ড তিনটি পর্যায়ে থাকে: **alpha** (পরীক্ষামূলক, যেকোনো সময় বদলাতে পারে), **beta** (প্রায় স্থিতিশীল, কিন্তু এখনো বদলাতে পারে), ও **GA** (সাধারণভাবে উপলব্ধ, স্থিতিশীল)। যেমন `gcloud alpha run ...` বা `gcloud beta ...`। শেখার সময় alpha/beta ব্যবহার করা যায়, কিন্তু production স্ক্রিপ্টে alpha কমান্ড ব্যবহার করবেন

না — কারণ পরের সংস্করণে তার ফ্ল্যাগ বা আউটপুট বদলে গেলে স্ক্রিপ্ট ভেঙে যাবে। নতুন সেবা চেষ্টা করার সময় প্রথমে তার ডকুমেন্টেশনে দেখুন সেটি GA কি না, আর alpha হলে কী সীমাবদ্ধতা আছে। একটি দরকারি অভ্যাস: স্ক্রিপ্টের শুরুতে মন্তব্য হিসেবে লিখুন কোন `gcloud` সংস্করণে এটি পরীক্ষা করা হয়েছে (`gcloud --version`), যাতে পরে কেউ বুঝতে পারে কেন একটি কমান্ড কাজ করছে না। আর `gcloud components install alpha beta` দিয়ে alpha/beta কম্পোনেন্ট ইনস্টল করা যায় — তবে শুধু পরীক্ষামূলক মেশিনে, production মেশিনে নয়।

৪.১৫ সাধারণ ভুল

- ভুল প্রজেক্টে কমান্ড চালানো — `--project` না দিয়ে।
- Service account key Git-এ কমিট করা বা চ্যাটে পাঠানো।
- `gcloud auth login` ও ADC-কে গুলিয়ে ফেলা।
- ধ্বংসাত্মক কমান্ডে `--quiet` ব্যবহার করা।
- স্ক্রিপ্টে গোপন মান হার্ডকোড করা।
- Cloud Shell-কে production deploy-এর মাধ্যম বানানো।

৪.১৬ সারাংশ

`gcloud` হলো Google Cloud API-র কমান্ড-লাইন মুখ — Console-এ যা করা যায়, CLI-তেও তা করা যায়, তবে পুনরাবৃত্তিযোগ্যভাবে। Configuration দিয়ে প্রজেক্ট ও Region পরিচালনা করুন, প্রমাণীকরণে ADC ও WIF-কে প্রাধান্য দিন, key ফাইল এড়ান, আর স্ক্রিপ্টকে কোডের মতো পরীক্ষা করুন। পরের অধ্যায়ে আমরা দেখব এই সবকিছুর দায়িত্ব ও খরচের ভাগ — Shared Responsibility, Well-Architected Framework ও Billing — কারণ সঠিকভাবে চালাতে পারলেই হবে না, সঠিকভাবে দায়িত্ব নিতে ও খরচ নিয়ন্ত্রণ করতে হবে।

৪.১৭ পর্যালোচনা প্রশ্ন

- `gcloud auth login` ও `gcloud auth application-default login`-এর পার্থক্য কী?
- একাধিক প্রজেক্টে কাজ করলে configuration কীভাবে ভুল কমায়?
- CI/CD-তে service account key-র বদলে কী ব্যবহার করবেন ও কেন?
- স্ক্রিপ্টে ধ্বংসাত্মক কমান্ড নিরাপদ করার তিনটি উপায় বলুন।
- Access token লগে ছাপানো কেন বিপজ্জনক?

৪.১৮ ব্যবহারিক অনুশীলন

- নিজের মেশিনে SDK ইনস্টল করুন, দুটি configuration বানান (dev ও prod-এর নামে), এবং `gcloud config list` দিয়ে যাচাই করুন।
- একটি Bash স্ক্রিপ্ট লিখুন যা একটি bucket বানায়, versioning চালু করে, এবং প্রজেক্ট ID ভুল হলে থেমে যায়।
- Cloud Shell-এ `gcloud logging read` দিয়ে আপনার প্রজেক্টের শেষ ১০টি লগ দেখুন।

অধ্যায় ৫ • PART I — GCP Foundations & Architecture

Chapter 5 — Shared Responsibility, Architecture and Billing

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Shared Responsibility Model ব্যাখ্যা করতে — Google কী দেখে, আপনি কী দেখেন; (২) Google Cloud Well-Architected Framework-এর পাঁচটি স্তম্ভ প্রয়োগ করতে; (৩) Cloud Billing কীভাবে কাজ করে, তা ব্যাখ্যা করতে; (৪) বাজেট, অ্যালার্ট ও খরচের রিপোর্ট বসাতে; (৫) লেবেল, কোটা ও Sustained Use / Committed Use-এর মতো খরচ নিয়ন্ত্রণের উপায় ব্যবহার করতে; এবং (৬) অপ্রত্যাশিত বিলের কারণ নির্ণয় করতে।

৫.২ পূর্বজ্ঞান ঝালাই

আগের অধ্যায়গুলো থেকে মনে করুন: IaaS/PaaS/SaaS-এ দায়িত্বের ভাগ আলাদা (অধ্যায় ১); প্রজেক্ট হলো বিলিংয়ের একক (অধ্যায় ২); IAM হলো কে কী করতে পারবে (অধ্যায় ৩); আর `gcloud` দিয়ে সবকিছু স্বয়ংক্রিয় করা যায় (অধ্যায় ৪)। এই অধ্যায়ে আমরা সেই সব জ্ঞানকে একত্র করে দুটি প্রশ্নের উত্তর দেব: "কোন দায়িত্ব কার?" এবং "খরচ কীভাবে নিয়ন্ত্রণে রাখব?" — এই দুটি প্রশ্নের উত্তর না জানলে ভালো স্থাপত্যও ব্যর্থ হয়।

৫.৩ Shared Responsibility Model: দায়িত্বের ভাগ

ক্লাউডে নিরাপত্তা একটি যৌথ দায়িত্ব। Google দেখে **"cloud-এর নিরাপত্তা"** (security *of* the cloud): ফিজিক্যাল ডেটা সেন্টার, হার্ডওয়্যার, ভার্চুয়ালাইজেশন স্তর, নেটওয়ার্ক অবকাঠামো, আর পরিচালিত সেবার (যেমন Cloud SQL, Cloud Storage) অন্তর্নিহিত প্ল্যাটফর্ম। আপনি দেখেন **"cloud-এর ভেতরের নিরাপত্তা"** (security *in* the cloud): আপনার ডেটা, আপনার অ্যাপ্লিকেশনের কোড, IAM নীতি, ফায়ারওয়াল নিয়ম, এনক্রিপশন কী-এর ব্যবহার, আর আপনার কনফিগারেশন।

এই ভাগ সার্ভিস মডেল অনুযায়ী বদলায়। IaaS-এ (Compute Engine) Google শুধু হার্ডওয়্যার ও হাইপারভাইজার দেখে; OS প্যাচ, ফায়ারওয়াল, অ্যাপ — সব আপনার। PaaS-এ (Cloud Run) Google রানটাইম ও প্যাচিং দেখে; আপনার কোড ও ডেটা আপনার। SaaS-এ (Gmail) Google প্রায় সবকিছু দেখে; আপনি শুধু ব্যবহারকারী ও শেয়ারিং সেটিং দেখেন। একটি বিপজ্জনক ভুল ধারণা হলো "ক্লাউডে গেলে নিরাপত্তা Google-এর দায়িত্ব" — না। Google ডেটা সেন্টারের দরজা পাহারা দেয়; কিন্তু আপনার bucket-টি public করে রাখলে, বা service account key ফাঁস করলে, সেটি আপনার দায়িত্ব। বেশিরভাগ ক্লাউড নিরাপত্তা ঘটনা ঘটে গ্রাহকের কনফিগারেশনের ভুলে, Google-এর অবকাঠামোর ভুলে নয়।

> **CONCEPT:** ভাড়া বাড়ির সাথে তুলনা করুন। বাড়িওয়ালা (Google) ভবনের কাঠামো, মূল দরজা ও বিদ্যুৎ দেখে; আপনি (গ্রাহক) নিজের ফ্ল্যাটের তালা, জানালা ও ভেতরের জিনিস দেখেন। বাড়িওয়ালা ভালো হলেই আপনার ফ্ল্যাট নিরাপদ হয় না — যদি আপনি দরজা খোলা রেখে যান। Shared Responsibility মানে এই ভাগটি স্পষ্টভাবে জানা ও নিজের অংশটি পালন করা।

৫.৪ Google Cloud Well-Architected Framework: পাঁচটি স্তম্ভ

Well-Architected Framework হলো ভালো স্থাপত্যের জন্য Google-এর নির্দেশিকা — পাঁচটি স্তম্ভ (pillar) নিয়ে গঠিত। **(১) Operational Excellence** — কাজের পদ্ধতি: IaC, CI/CD, স্বয়ংক্রিয় পরীক্ষা, পরিবর্তনের রেকর্ড, runbook। প্রশ্ন:

"আমরা কি পরিবর্তন নিরাপদে ও পুনরাবৃত্তিযোগ্যভাবে করতে পারি?" **(২) Security** — least privilege IAM, private নেটওয়ার্ক, এনক্রিপশন (at rest ও in transit), Secret Manager, audit log, নিয়মিত পর্যালোচনা। প্রশ্ন: "আমরা কি আক্রমণ ও ভুল থেকে ডেটা রক্ষা করতে পারি?" **(৩) Reliability** — মাল্টি-Zone স্থাপত্য, health check, স্বয়ংক্রিয় পুনরুদ্ধার, ব্যাকআপ ও restore পরীক্ষা, RTO/RPO। প্রশ্ন: "ব্যর্থতা এলেও কি সেবা চলবে?" **(৪) Performance Efficiency** — সঠিক মেশিন টাইপ, ক্যাশিং, CDN, ডেটা ও কম্পিউট একসাথে রাখা, লোড পরীক্ষা। প্রশ্ন: "আমরা কি সম্পদ দক্ষভাবে ব্যবহার করছি?" **(৫) Cost Optimization** — সঠিক আকার (right-sizing), অলস রিসোর্স বন্ধ করা, বাজেট অ্যালার্ট, Committed Use Discount। প্রশ্ন: "আমরা কি প্রয়োজনের বেশি খরচ করছি?"

Framework ব্যবহারের নিয়ম: এটি একটি চেকলিস্ট নয়, একটি চিন্তার কাঠামো। প্রতিটি বড় সিদ্ধান্তের আগে পাঁচটি স্তম্ভ দিয়ে যাচাই করুন — একটি স্তম্ভের উন্নতি অন্য স্তম্ভের ক্ষতি করছে কি না দেখুন। যেমন মাল্টি-Region করলে reliability বাড়ে, কিন্তু খরচ ও জটিলতা বাড়ে। সর্বোচ্চ নিরাপত্তা করলে operational গতি কমতে পারে। ভালো স্থাপত্য হলো এই বিনিময়গুলো (trade-off) সচেতনভাবে করা — অজান্তে নয়।

স্তম্ভ	মূল প্রশ্ন	TechNova-র উদাহরণ
Operational Excellence	পরিবর্তন কি নিরাপদ ও পুনরাবৃত্তিযোগ্য?	Terraform + CI/CD, runbook
Security	ডেটা ও প্রবেশ কি সুরক্ষিত?	Least privilege, private VPC, Secret Manager
Reliability	ব্যর্থতায় কি সেবা চলবে?	মাল্টি-Zone, ব্যাকআপ পরীক্ষা
Performance Efficiency	সম্পদ কি দক্ষভাবে ব্যবহৃত হচ্ছে?	সঠিক মেশিন টাইপ, CDN, ক্যাশ
Cost Optimization	খরচ কি নিয়ন্ত্রণে?	বাজেট অ্যালার্ট, অলস VM বন্ধ

৫.৫ নিরাপত্তা ও নির্ভরযোগ্যতা: একসাথে ভাবুন

নিরাপত্তা ও নির্ভরযোগ্যতা আলাদা বিষয় নয়। একটি আক্রমণ নির্ভরযোগ্যতা নষ্ট করে; একটি ভুল কনফিগারেশন নিরাপত্তা নষ্ট করে। মূল অভ্যাসগুলো: **defense in depth** — একটি স্তর ভাঙলেও আরেকটি স্তর রক্ষা করে (VPC ফায়ারওয়াল + IAM + এনক্রিপশন + audit); **least privilege** (অধ্যায় ৩); **private by default** — রিসোর্স ডিফল্টভাবে private রাখুন, public করুন শুধু প্রয়োজনে ও সচেতনভাবে; **এনক্রিপশন** — Google ডিফল্টভাবে ডেটা at rest এনক্রিপ্ট করে; সংবেদনশীল ডেটার জন্য Cloud KMS দিয়ে নিজের কী (CMEK) ব্যবহার করুন; **Secret Manager** — পাসওয়ার্ড, API কী, টোকেন কোডে বা environment variable-এ নয়, Secret Manager-এ রাখুন; **পর্যবেক্ষণ** — Cloud Monitoring-এ অ্যালার্ট (CPU, ত্রুটির হার, latency), Cloud Logging-এ audit log, আর Security Command Center দিয়ে ঝুঁকি দেখুন।

নির্ভরযোগ্যতার জন্য: health check ও autohealing (অসুস্থ VM স্বয়ংক্রিয়ভাবে বদলে যাওয়া), মাল্টি-Zone স্থাপত্য, ব্যাকআপ ও restore পরীক্ষা, আর **chaos-এর জন্য প্রস্তুতি** — Zone failover পরীক্ষা করুন, যাতে আসল বিপর্যয়ে প্রথমবার না হয়। একটি দল যারা কখনো failover পরীক্ষা করেনি, তারা আসলে জানে না তাদের failover কাজ করে কি না।

৫.৬ Cloud Billing কীভাবে কাজ করে

Google Cloud-এর বিলিং মডেল বুঝুন। প্রতিটি সেবার মূল্য নির্ধারণ আলাদা: Compute Engine প্রতি সেকেন্ডে (ন্যূনতম ১ মিনিট) VM চালানোর জন্য; Cloud Storage প্রতি GB সংরক্ষণ ও প্রতি অপারেশনের জন্য; Cloud Run প্রতি অনুরোধ ও CPU/মেমরি ব্যবহারের জন্য; BigQuery প্রতি TB স্ক্যানের জন্য। এছাড়া **নেটওয়ার্ক egress** — ডেটা GCP থেকে বাইরে গেলে খরচ হয়; **লগিং ingestion** — Cloud Logging-এ লগ ঢোকালে খরচ হয় (বিনামূল্যের সীমা আছে); আর **load balancing** — প্রতি ঘণ্টায় ও প্রতি GB প্রক্রিয়াজাত ডেটার জন্য খরচ হয়।

দাম **পরিবর্তনশীল** — Google মাঝে মাঝে দাম বদলায়, নতুন সেবা আসে, বিনামূল্যের সীমা বদলায়। তাই এই বইয়ে কোনো নির্দিষ্ট দামকে চিরন্তন সত্য হিসেবে লিখে রাখা হয়নি। কোনো সেবা চালু করার আগে তার মূল্য নির্ধারণের পাতা দেখুন ও **Google Cloud Pricing Calculator** -এ অনুমান করুন। Free trial বা free credits-এর যোগ্যতা, মেয়াদ ও শর্তও বদলাতে পারে — সাইন-আপের সময়ের শর্তই প্রযোজ্য।

খরচ কমানোর বৈধ উপায়: **Sustained Use Discount** — VM দীর্ঘক্ষণ চালালে স্বয়ংক্রিয় ছাড় (কিছু মেশিন টাইপে); **Committed Use Discount (CUD)** — ১ বা ৩ বছরের জন্য নির্দিষ্ট ব্যবহারের প্রতিশ্রুতি দিলে বড় ছাড় (শুধু যখন ব্যবহার নিশ্চিত, তখনই); **right-sizing** — প্রয়োজনের চেয়ে বড় VM না নেওয়া; **preemptible / Spot VM** — বাধাযোগ্য কাজের জন্য সস্তা VM (যেকোনো সময় বন্ধ হতে পারে); **অলস রিসোর্স বন্ধ করা** — ব্যবহার না হওয়া VM, IP, ডিস্ক, লোড ব্যালান্সার মুছে ফেলা। একটি সাধারণ ভুল হলো CUD কিনে ফেলা, অথচ ব্যবহার নিশ্চিত নয় — তখন ছাড়ের বদলে অপচয় হয়।

> **সতর্কতা:** সবচেয়ে ব্যয়বহুল সেবাগুলোর দিকে নজর দিন: GKE ক্লাস্টার (কন্ট্রোল প্লেন ও নোড), Cloud NAT (প্রতি ঘণ্টা + প্রতি GB), Cloud SQL (চালু থাকলেই খরচ, ব্যবহার না হলেও), লোড ব্যালান্সার, লগিং ingestion (বেশি লগ লিখলে), আর নেটওয়ার্ক egress (বিশেষ করে অন্য Region বা ইন্টারনেটে)। একটি Cloud NAT বা লোড ব্যালান্সার পরীক্ষার জন্য চালু করে ভুলে গেলে মাসের বিল চমকে দিতে পারে। পরীক্ষার রিসোর্সের জন্য আলাদা প্রজেক্ট ব্যবহার করুন ও পরীক্ষা শেষে মুছে ফেলুন।

৫.৭ বাজেট, অ্যালার্ট ও খরচের রিপোর্ট

বিলিং নিয়ন্ত্রণের প্রথম ধাপ: **বাজেট** বসানো। Console-এ Billing → Budgets & alerts → Create budget: কোন billing account বা প্রজেক্ট, কত টাকা (মাসিক), আর কত শতাংশে অ্যালার্ট যাবে (যেমন ৫০%, ৯০%, ১০০%)। অ্যালার্ট শুধু জানায় — এটি খরচ থামায় না। খরচ থামাতে চাইলে অ্যালার্টের সাথে Cloud Functions বা অন্য স্বয়ংক্রিয়তা যুক্ত করতে হয় (যেমন বাজেট ছাড়ালে non-production VM বন্ধ করা) — তবে production-এ এটি সতর্কতার সাথে করুন, কারণ হঠাৎ VM বন্ধ করলে সেবা যাবে।

দ্বিতীয় ধাপ: **Billing export** চালু করুন — প্রতিদিনের খরচের বিস্তারিত তথ্য BigQuery-তে যাবে; সেখান থেকে লেবেল, সেবা ও প্রজেক্ট অনুযায়ী রিপোর্ট বানান। তৃতীয় ধাপ: **নিয়মিত পর্যালোচনা** — প্রতি সপ্তাহে Billing → Reports দেখুন: কোন সেবার খরচ হঠাৎ বেড়েছে? কোন প্রজেক্টের খরচ বেশি? একটি ছোট অভ্যাস: প্রতি সোমবার সকালে ১০ মিনিট বিলিং রিপোর্ট দেখা — মাস শেষে চমকানোর চেয়ে অনেক সস্তা।

```
gcloud billing budgets list --billing-account=0X0X0X-0X0X0X-0X0X0X
gcloud billing budgets create --billing-account=0X0X0X-0X0X0X-0X0X0X \
  --display-name="TechNova monthly cap" --budget-amount=100
gcloud recommender recommendations list \
  --project=technova-prod-web --location=global \
  --recommender=google.compute.instance.MachineTypeRecommender
```

উপরের budget-amount মানটি একটি উদাহরণ — আপনার মুদ্রা ও সীমা অনুযায়ী ঠিক করুন। Recommender-এর পরামর্শ (যেমন "এই VM ছোট করা যায়") দেখুন, তবে অন্ধভাবে মানবেন না — প্রথমে যাচাই করুন VM-টি আসলেই কম ব্যবহৃত হয় কি না।

৫.৮ লেবেল, কোটা ও সীমা

Labels (অধ্যায় ২.৭) হলো খরচ ভাগের মূল উপায়: প্রতিটি রিসোর্সে `env`, `team`, `cost-center`, `owner` লেবেল দিন। লেবেল ছাড়া আপনি জানবেন মোট বিল কত, কিন্তু কোন দল কত খরচ করেছে তা জানবেন না। **Quotas** হলো প্রতিটি প্রজেক্টে প্রতিটি সেবার ব্যবহারের সীমা — যেমন একসাথে কতটি VM, কতটি IP, প্রতি মিনিটে কতটি API অনুরোধ। কোটা দুটি কারণে আছে: ভুল বা আক্রমণে হঠাৎ হাজার VM চালু হয়ে বিল না বাড়ে, আর Google-এর সম্পদ সবার মধ্যে ভাগ হয়। কোটা বাড়াতে Console-এ IAM & Admin → Quotas থেকে অনুরোধ করুন — এতে সময় লাগতে পারে, তাই production-এর আগে কোটা যাচাই করুন।

কোটা ও বাজেটের পার্থক্য মনে রাখুন: কোটা সীমা দেয় কতটুকু ব্যবহার করা যায়; বাজেট সতর্ক করে কত খরচ হচ্ছে। কোটা শেষ হলে নতুন রিসোর্স বানানো যাবে না (ভুল বার্তায় "quota exceeded" আসবে); বাজেট ছাড়ালে অ্যালার্ট আসবে, কিন্তু রিসোর্স চলতে থাকবে। দুটোই আগে থেকে বসান।

৫.৯ অপ্রত্যাশিত বিলের তদন্ত: ধাপে ধাপে

বিল হঠাৎ বেশি এলে আতঙ্কিত হবেন না — পদ্ধতিগতভাবে তদন্ত করুন। ধাপ ১: Billing → Reports-এ যান, সময়সীমা বদলে দেখুন খরচ কবে বেড়েছে, কোন সেবা ও কোন প্রজেক্টে। ধাপ ২: সেই সেবার বিস্তারিত দেখুন — যেমন Cloud Storage-এ কোন bucket-এর আকার বেড়েছে, Compute Engine-এ কোন VM চলছে। ধাপ ৩: Cloud Monitoring ও Logging দেখুন — কোনো অস্বাভাবিক ট্রাফিক, ত্রুটি, বা নতুন রিসোর্স তৈরি হয়েছে কি? ধাপ ৪: Audit log দেখুন কে কী বানিয়েছে — হয়তো কেউ পরীক্ষার জন্য বড় VM চালু করে ভুলে গেছে। ধাপ ৫: কারণ খুঁজে পেলে তা ঠিক করুন (অলস রিসোর্স মুছুন, লগের পরিমাণ কমান, egress কমান) এবং ভবিষ্যতের জন্য বাজেট অ্যালার্ট ও কোটা বসান।

সাধারণ কারণগুলো: ভুলে চালু রাখা VM বা GKE ক্লাস্টার; অতিরিক্ত verbose লগিং; অন্য Region-এ ডেটাবেস থেকে বারবার বড় ডেটা টানা; Cloud NAT দিয়ে অপ্রয়োজনীয় egress; পুরোনো snapshot ও ডিস্ক জমে যাওয়া; আর সবচেয়ে গুরুতর: আপস হওয়া শংসাপত্র দিয়ে অজানা VM চালু করা — এ ক্ষেত্রে প্রথমে শংসাপত্র বাতিল করুন (অধ্যায় ৩.৭), তারপর বিল দেখুন।

> **TROUBLESHOOTING:** "Quota exceeded" পেলে: কোন কোটা শেষ, তা বার্তায় দেখুন; Console → Quotas-এ গিয়ে ব্যবহার দেখুন; প্রয়োজনে বৃদ্ধির অনুরোধ করুন — আর অনুরোধের সময় ব্যবসায়িক যুক্তি লিখুন। বিল বেশি মনে হলে প্রথমে Billing Reports-এ সেবা অনুযায়ী ভাগ করুন — অনুমানে নয়। আর একটি সেবার দাম নিয়ে সন্দেহ হলে Pricing Calculator-এ সেই সেবার বর্তমান দাম যাচাই করুন; এই বইয়ের কোনো দামকে চূড়ান্ত ধরে নেবেন না।

৫.১০ প্রোডাকশন দৃশ্যকল্প: TechNova-র খরচ নিয়ন্ত্রণ

TechNova-র নিয়ম: প্রতিটি প্রজেক্টে মাসিক বাজেট ও ৫০% / ৯০% / ১০০% অ্যালার্ট; Billing export BigQuery-তে চালু; প্রতি সোমবার ১০ মিনিটের বিলিং পর্যালোচনা; সব রিসোর্সে `env`, `team`, `owner` লেবেল বাধ্যতামূলক; dev প্রজেক্টের VM রাতে ও সপ্তাহান্তে স্বয়ংক্রিয়ভাবে বন্ধ হয়; লগিং-এ শুধু প্রয়োজনীয় log রাখা হয়, debug লগ production-এ বন্ধ; আর Cloud SQL ও GKE-র মতো ব্যয়বহুল সেবা চালু করার আগে Pricing Calculator-এ অনুমান করা হয়। একবার তাদের বিল হঠাৎ বেড়ে গিয়েছিল — তদন্তে দেখা গেল একটি পরীক্ষামূলক Cloud NAT ভুলে চালু ছিল। তারা NAT মুছে বাজেট অ্যালার্ট আরও কমিয়ে দিল। শিক্ষা: বিলিং নিয়ন্ত্রণ একটি এককালীন কাজ নয়, একটি সাপ্তাহিক অভ্যাস।

বাস্তব উদাহরণ: একটি স্টার্টআপ তাদের অ্যাপের প্রতিটি অনুরোধে বিস্তারিত debug লগ লিখত। ট্রাফিক বাড়ার সাথে সাথে লগিং ingestion-এর বিল অ্যাপের VM-এর বিল ছাড়িয়ে গেল। তারা debug লগ বন্ধ করে শুধু ত্রুটি ও গুরুত্বপূর্ণ ঘটনা লগ করতে শুরু করল — বিল অর্ধেকেরও কমে গেল, আর পর্যবেক্ষণও সহজ হলো। শিক্ষা: লগ লেখাও খরচ — কী লগ করছেন, তা ভাবুন।

৫.১১ সাধারণ ভুল

- "নিরাপত্তা Google-এর দায়িত্ব" ভেবে নিজের কনফিগারেশন না দেখা।
- বাজেট অ্যালার্ট না বসিয়ে মাস শেষে বিল দেখা।
- পরীক্ষার রিসোর্স (VM, NAT, লোড ব্যালান্সার) চালু রেখে ভুলে যাওয়া।
- CUD কিনে ফেলা, অথচ ব্যবহার নিশ্চিত নয়।
- লেবেল ছাড়া রিসোর্স বানানো — খরচের উৎস খুঁজে না পাওয়া।
- দামকে চিরন্তন সত্য ধরে নেওয়া — Pricing Calculator না দেখা।
- কোটা না যাচাই করে production launch করা।

৫.১২ সারাংশ

Shared Responsibility মানে Google অবকাঠামো দেখে, আপনি নিজের ডেটা, কোড ও কনফিগারেশন দেখেন। Well-Architected Framework-এর পাঁচটি স্তম্ভ — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization — প্রতিটি সিদ্ধান্ত যাচাইয়ের কাঠামো। বিলিং নিয়ন্ত্রণে রাখতে বাজেট ও অ্যালার্ট বসান, লেবেল দিন, Billing export চালু করুন, অলস রিসোর্স মুছুন, আর দাম সবসময় Pricing Calculator-এ যাচাই করুন। Part I এখানেই শেষ — আপনি এখন GCP-র ভিত্তি জানেন: অবকাঠামো, সংস্থা, পরিচয়, CLI ও দায়িত্ব। Part II-এ আমরা এই ভিত্তির ওপর Compute, Storage ও Networking-এর গভীরে যাব।

৫.১৩ পর্যালোচনা প্রশ্ন

- IaaS, PaaS ও SaaS-এ Shared Responsibility কীভাবে বদলায়? তিনটি উদাহরণ দিন।
- Well-Architected Framework-এর পাঁচটি স্তম্ভ কী? একটি মাল্টি-Region সিদ্ধান্তে এগুলো কীভাবে পরস্পরবিরোধী হতে পারে?
- বাজেট অ্যালার্ট ও কোটার পার্থক্য কী? কোনটি খরচ থামায়?
- Cloud NAT, Cloud SQL ও লগিং ingestion কেন ব্যয়বহুল হতে পারে?
- বিল হঠাৎ বেড়ে গেলে আপনি কোন ক্রমে তদন্ত করবেন?

৫.১৪ ব্যবহারিক অনুশীলন

- আপনার পরীক্ষামূলক প্রজেক্টে একটি মাসিক বাজেট ও ৫০% / ৯০% অ্যালার্ট বসান।
- Billing Reports-এ গিয়ে সবচেয়ে ব্যয়বহুল তিনটি সেবা চিহ্নিত করুন ও একটি করে কমানোর উপায় লিখুন।
- TechNova-র জন্য একটি Well-Architected পর্যালোচনা লিখুন: প্রতিটি স্তম্ভে একটি শক্তি ও একটি দুর্বলতা।
- Pricing Calculator-এ একটি Cloud SQL ইনস্ট্যান্স ও একটি GKE ক্লাস্টারের মাসিক খরচের অনুমান করুন (দাম পরিবর্তনশীল — অনুমান হিসেবে সংরক্ষণ করুন)।

PART II • DEVOPS ENGINEERING MASTERY

Networking & Compute Engineering

VPC Network ↬ Subnet, Route ↬ Firewall, Private Connectivity, DNS, Load Balancing ↬ CDN, Compute Engine, Hybrid Networking, Managed Instance Group ↬ High Availability

অধ্যায় ৬ • PART II — Networking & Compute Engineering

Chapter 6 — VPC Networks and Subnet Engineering

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) VPC কী সমস্যা সমাধান করে তা প্রথম নীতি থেকে ব্যাখ্যা করতে; (২) গ্লোবাল VPC ও আঞ্চলিক সাবনেটের মধ্যে সম্পর্ক আঁকতে; (৩) CIDR পরিকল্পনা করে একটি প্রোডাকশন নেটওয়ার্ক ডিজাইন করতে; (৪) auto mode ও custom mode VPC-র পার্থক্য বিচার করে সঠিকটি বেছে নিতে; (৫) Shared VPC ও VPC peering কখন ব্যবহার করবেন — আর কখন করবেন না — তা সিদ্ধান্ত নিতে; (৬) Console, gcloud CLI এবং Terraform দিয়ে VPC তৈরি ও পরিচালনা করতে।

৬.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

এই অধ্যায়ে ঢোকার আগে আপনার জানা উচিত: IP ঠিকানা ও CIDR নোটেশন (যেমন `10.0.0.0/16` মানে প্রথম ১৬ বিট নেটওয়ার্ক অংশ), রাউটিং টেবিলের ধারণা, এবং GCP-তে প্রজেক্ট ও রিজিয়নের ধারণা। মনে রাখবেন: পাবলিক IP ইন্টারনেটে রাউটযোগ্য, প্রাইভেট IP (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) শুধু ব্যক্তিগত নেটওয়ার্কের ভেতরে ব্যবহৃত হয়। GCP-র ভেতরে আপনার ওয়ার্কলোডগুলো প্রাইভেট IP-তে চলে; পাবলিক IP শুধু প্রয়োজন হলে যুক্ত হয়।

৬.৩ প্রথম নীতি: VPC কী সমস্যা সমাধান করে?

কল্পনা করুন TechNova-র কোনো নেটওয়ার্ক নেই — প্রতিটি ভার্চুয়াল মেশিন সরাসরি ইন্টারনেটে বসে আছে। কোনো সীমানা নেই, কোনো নিয়ন্ত্রণ নেই: ডাটাবেস সার্ভারটিও যে কেউ পৌঁছাতে পারে, অ্যাপ সার্ভার থেকে ডাটাবেসে যাওয়া ট্রাফিকও পাবলিক ইন্টারনেট ঘুরে যায়। এটাই সমস্যা। VPC (Virtual Private Cloud) হলো আপনার নিজস্ব, বিচ্ছিন্ন, সফটওয়্যার-সংজ্ঞায়িত নেটওয়ার্ক — Google-এর ফিজিক্যাল নেটওয়ার্কের ওপরে আপনার জন্য আলাদা করা একটি লজিক্যাল সীমানা। এর ভেতরে আপনিই ঠিক করেন কোন IP রেঞ্জ ব্যবহার হবে, কোন সাবনেট কোথায় থাকবে, কোন ট্রাফিক কোথায় যেতে পারবে, আর কোনটি আটকে যাবে।

VPC-র অস্তিত্বের কারণ তিনটি: **বিচ্ছিন্নতা (isolation)** — আপনার ট্রাফিক অন্য গ্রাহকের ট্রাফিক থেকে আলাদা; **নিয়ন্ত্রণ (control)** — রাউটিং ও ফায়ারওয়াল দিয়ে ট্রাফিকের পথ আপনি ঠিক করেন; এবং **সংযোগ (connectivity)** — আপনার অন-প্রেমিসেস ডাটা সেন্টার, অন্য VPC, বা Google-এর ম্যানেজড সার্ভিসের সঙ্গে নিরাপদ সংযোগ। GCP-তে VPC একটি গ্লোবাল রিসোর্স: একটি VPC একাধিক রিজিয়ন জুড়ে বিস্তৃত হতে পারে, কিন্তু এর সাবনেটগুলো আঞ্চলিক।

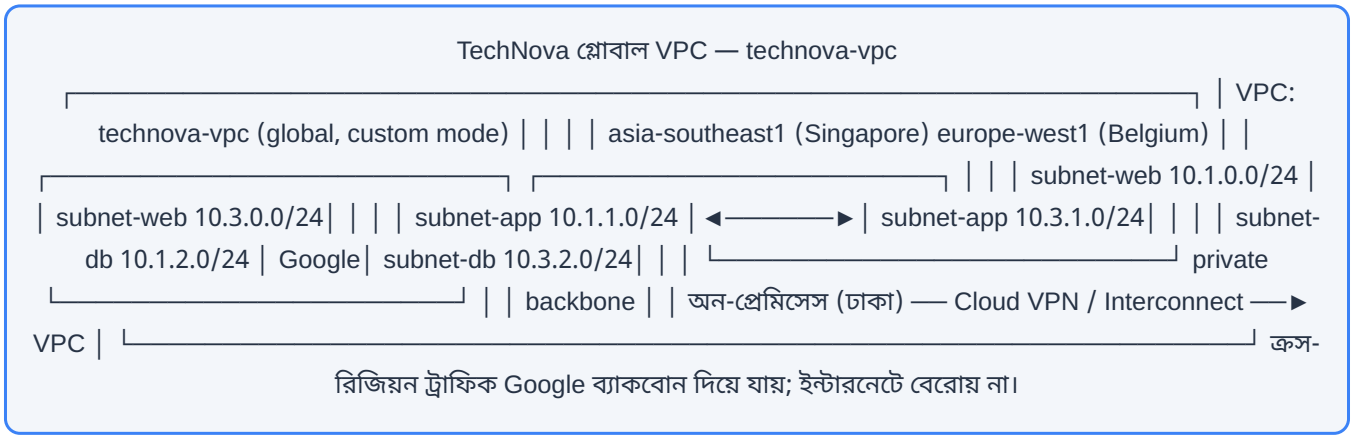
> **CONCEPT:** VPC কোনো ফিজিক্যাল কেবল বা সুইচ নয় — এটি Google-এর সফটওয়্যার-সংজ্ঞায়িত নেটওয়ার্কিং (SDN) কন্ট্রোল প্লেনের একটি লজিক্যাল বিমূর্ততা। আপনি যখন সাবনেট বানান, তখন কোনো হার্ডওয়্যার বসানো হয় না; কন্ট্রোল প্লেন শুধু রাউটিং ও ফায়ারওয়াল নিয়মগুলো পুরো ফ্লিট জুড়ে প্রোগ্রাম করে দেয়।

৬.৪ গ্লোবাল VPC আর্কিটেকচার

GCP-র VPC গ্লোবাল — AWS-র আঞ্চলিক VPC-র সঙ্গে এটাই বড় পার্থক্য। একটি VPC-র সাবনেট `asia-southeast1`, `europe-west1` ও `us-central1`-এ একসঙ্গে থাকতে পারে, আর এই সাবনেটগুলোর মধ্যে ট্রাফিক Google-এর প্রাইভেট

ব্যাকবোন দিয়ে যায় — পাবলিক ইন্টারনেট দিয়ে নয়। এর মানে: সিঙ্গাপুরের একটি VM থেকে ফ্রাঙ্কফুর্টের VM-এ পৌঁছাতে ট্রাফিক ইন্টারনেটে বেরোয় না, এনক্রিপ্টেড প্রাইভেট পথে যায়।

তবে গ্লোবাল হওয়ার মানে এই নয় যে সবকিছু এক জায়গায়। সাবনেট আঞ্চলিক: একটি সাবনেট একটি মাত্র রিজিয়নের অন্তর্গত, আর তার IP রেঞ্জ সেই রিজিয়নের জোনগুলোতে ভাগ হয়। রিজিয়ন ব্যর্থ হলে (যা বিরল, কিন্তু সম্ভব) অন্য রিজিয়নের সাবনেট আলাদাভাবে চলতে থাকে। তাই প্রোডাকশন ডিজাইনে অন্তত দুটি রিজিয়নে সাবনেট রাখা ভালো অভ্যাস — যদিও শুরুতে একটি রিজিয়নেই একাধিক জোনে ছড়ানো যথেষ্ট।



৬.৫ সাবনেট ও CIDR পরিকল্পনা

CIDR পরিকল্পনা হলো নেটওয়ার্ক ডিজাইনের সবচেয়ে কম খরচে ভুল করার জায়গা — কারণ একবার IP রেঞ্জ বরাদ্দ হয়ে গেলে তা বদলানো কঠিন। নিয়মগুলো মনে রাখুন: (১) প্রতিটি সাবনেটের রেঞ্জ একে অপরের সঙ্গে ওভারল্যাপ করবে না; (২) ভবিষ্যতের বৃদ্ধির জন্য জায়গা রাখুন — /24-এ মাত্র ২৫১টি ব্যবহারযোগ্য IP থাকে (Google প্রতি সাবনেটে ৪টি ঠিকানা সংরক্ষণ করে: নেটওয়ার্ক, গেটওয়ে, DNS, ব্রডকাস্ট); (৩) VPC peering বা VPN-এর জন্য পিয়ার নেটওয়ার্কের রেঞ্জের সঙ্গে ওভারল্যাপ চলবে না — তাই অন-প্রেমিসেসের রেঞ্জ জেনে নিন আগেই; (৪) GKE বা অন্য ম্যানেজড সার্ভিসের জন্য সেকেন্ডারি রেঞ্জ আলাদা করে রাখুন।

TechNova-র জন্য একটি বাস্তব পরিকল্পনা: VPC রেঞ্জ `10.0.0.0/16`। `asia-southeast1`-এ web `10.0.0.0/24`, app `10.0.1.0/24`, db `10.0.2.0/24`; `europe-west1`-এ একই কাঠামো `10.1.x.0/24` দিয়ে। GKE পডের জন্য সেকেন্ডারি রেঞ্জ `10.4.0.0/16` আলাদা। এইভাবে প্রতিটি স্তর (tier) আলাদা সাবনেটে থাকে — ফায়ারওয়াল নিয়ম লেখা সহজ হয়, আর ডাটাবেস সাবনেটে ইন্টারনেট রাউট না থাকলে সেটি স্বভাবতই বিচ্ছিন্ন থাকে।

সাবনেট	রিজিয়ন	CIDR	ব্যবহারযোগ্য IP	উদ্দেশ্য
subnet-web	asia-southeast1	10.0.0.0/24	২৫১	লোড ব্যালান্সার ব্যাকএন্ড, পাবলিক-মুখী VM
subnet-app	asia-southeast1	10.0.1.0/24	২৫১	অ্যাপ্লিকেশন সার্ভার, প্রাইভেট
subnet-db	asia-southeast1	10.0.2.0/24	২৫১	ডাটাবেস, ইন্টারনেট রাউট নেই
subnet-gke-pods	asia-southeast1	10.4.0.0/16 (secondary)	বড়	GKE পড IP

> **সতর্কতা:** সাবনেটের প্রাইমারি CIDR রেঞ্জ তৈরি হওয়ার পর বদলানো যায় না — শুধু প্রসারিত (expand) করা যায়। তাই ছোট রেঞ্জ দিয়ে শুরু করে পরে আটকে যাওয়ার চেয়ে প্রথমেই পর্যাপ্ত জায়গা রাখুন। আর RFC 1918-র বাইরের রেঞ্জ ব্যবহার করলে পরে VPN/Interconnect-এ ওভারল্যাপের ঝুঁকি বাড়ে।

৬.৬ Auto Mode বনাম Custom Mode

প্রতিটি GCP প্রজেক্টে ডিফল্টভাবে একটি auto mode VPC থাকে — `default` নেটওয়ার্ক। Auto mode-এ প্রতিটি রিজিয়নে Google স্বয়ংক্রিয়ভাবে একটি করে সাবনেট বানিয়ে দেয়, পূর্বনির্ধারিত রেঞ্জ সহ। শেখার জন্য বা দ্রুত পরীক্ষার জন্য এটি সুবিধাজনক। কিন্তু প্রোডাকশনে এটি প্রায় সবসময় ভুল পছন্দ: আপনি IP রেঞ্জ নিয়ন্ত্রণ করতে পারেন না, অপ্রয়োজনীয় রিজিয়নে সাবনেট তৈরি হয়, আর ফায়ারওয়াল নিয়মগুলো ডিফল্টভাবে ঢিলেঢালা থাকে। Custom mode VPC-তে আপনিই প্রতিটি সাবনেট বানান, রেঞ্জ ঠিক করেন, কোন রিজিয়নে সাবনেট থাকবে তা বেছে নেন। নিয়ম: প্রোডাকশনে সবসময় custom mode; auto mode শুধু ল্যাব বা দ্রুত প্রোটোটাইপের জন্য। একটি auto mode VPC-কে পরে custom mode-এ রূপান্তর করা যায়, কিন্তু উল্টোটা যায় না।

৬.৭ Shared VPC ও VPC Peering

বড় সংস্থায় অনেক প্রজেক্ট থাকে — প্রতিটিতে আলাদা VPC বানালে নেটওয়ার্ক ছিন্নভিন্ন হয়ে যায়। Shared VPC এই সমস্যার সমাধান: একটি host প্রজেক্ট VPC-র মালিক হয়, আর অন্য service প্রজেক্টগুলো সেই VPC-র সাবনেট ব্যবহার করে VM বানাতে পারে। নেটওয়ার্কের নিয়ন্ত্রণ (ফায়ারওয়াল, রাউট) কেন্দ্রীয় নেটওয়ার্ক দলের হাতে থাকে, আর অ্যাপ দলগুলো শুধু তাদের সাবনেটে VM বানায়। IAM দিয়ে ঠিক করা যায় কোন service প্রজেক্ট কোন সাবনেট ব্যবহার করতে পারবে।

VPC peering আলাদা জিনিস: দুটি VPC-কে সরাসরি যুক্ত করে, যাতে তাদের VM-গুলো প্রাইভেট IP-তে একে অপরের সঙ্গে কথা বলতে পারে। Peering non-transitive — A, B-র সঙ্গে peer করলে আর B, C-র সঙ্গে peer করলে A, C-তে পৌঁছাতে পারে না। আর peering-এ IP রেঞ্জ ওভারল্যাপ করা চলবে না। কখন ব্যবহার করবেন: দুটি আলাদা দলের বা আলাদা সংস্থার VPC যুক্ত করতে। কখন করবেন না: অনেক VPC-কে জালের মতো যুক্ত করতে (তখন hub-and-spoke বা Shared VPC ভালো), বা ট্রানজিট রাউটিং দরকার হলে — peering ট্রানজিট করে না।

> **CONCEPT:** Shared VPC বনাম Peering — Shared VPC হলো একটাই নেটওয়ার্ক অনেক প্রজেক্টে ভাগ করে নেওয়া (কেন্দ্রীয় নিয়ন্ত্রণ); Peering হলো দুটি আলাদা নেটওয়ার্কের মধ্যে সেতু (বিকেন্দ্রীকৃত)। নতুন ডিজাইনে Shared VPC-ই সাধারণত সঠিক শুরু।

৬.৮ Console ও gcloud দিয়ে VPC তৈরি

Console-এ: VPC network → VPC networks → Create VPC network → নাম দিন, Custom mode বেছে নিন, সাবনেট যোগ করুন (নাম, রিজিয়ন, CIDR), Firewall নিয়ম পরে যোগ করবেন। কিন্তু প্রোডাকশনে CLI/IaC-ই আসল পথ — কারণ তাতে পরিবর্তন রিভিউ করা যায়, পুনরাবৃত্তি করা যায়।

```
# Custom mode VPC তৈরি
gcloud compute networks create technova-vpc \
  --subnet-mode=custom \
  --bgp-routing-mode=regional \
  --mtu=1460
```

```
# সাবনেট তৈরি (প্রাইভেট Google Access চালু করে)
gcloud compute networks subnets create subnet-web \
  --network=technova-vpc \
  --region=asia-southeast1 \
  --range=10.0.0.0/24 \
  --enable-private-ip-google-access

gcloud compute networks subnets create subnet-app \
  --network=technova-vpc \
  --region=asia-southeast1 \
  --range=10.0.1.0/24 \
  --enable-private-ip-google-access

gcloud compute networks subnets create subnet-db \
  --network=technova-vpc \
  --region=asia-southeast1 \
  --range=10.0.2.0/24 \
  --enable-private-ip-google-access

# যাচাই
gcloud compute networks subnets list --network=technova-vpc
gcloud compute networks describe technova-vpc
```

উপরের আউটপুট উদাহরণস্বরূপ (illustrative) — আপনার প্রজেক্টের আসল আউটপুটে সাবনেটের নাম ও রেঞ্জ একইভাবে দেখাবে।

৬.৯ Terraform দিয়ে VPC (IaC)

```
resource "google_compute_network" "technova" {
  name                    = "technova-vpc"
  auto_create_subnetworks = false
  routing_mode            = "REGIONAL"
  mtu                     = 1460
}

resource "google_compute_subnetwork" "web" {
  name                     = "subnet-web"
  ip_cidr_range            = "10.0.0.0/24"
  region                   = "asia-southeast1"
  network                  = google_compute_network.technova.id
  private_ip_google_access = true
}

resource "google_compute_subnetwork" "app" {
  name                     = "subnet-app"
  ip_cidr_range            = "10.0.1.0/24"
  region                   = "asia-southeast1"
  network                  = google_compute_network.technova.id
  private_ip_google_access = true
}

resource "google_compute_subnetwork" "db" {
  name                     = "subnet-db"
  ip_cidr_range            = "10.0.2.0/24"
  region                   = "asia-southeast1"
```

```
  network                  = google_compute_network.technova.id
  private_ip_google_access = true
}
```

IaC-র সুবিধা: `terraform plan`-এ পরিবর্তন দেখে রিভিউ করা যায়, Git-এ ইতিহাস থাকে, আর একই কোড দিয়ে staging ও production-এ একই নেটওয়ার্ক বানানো যায়। Console-এ হাতে ক্লিক করে বানানো নেটওয়ার্কের কোনো ইতিহাস থাকে না।

৬.১০ প্রোডাকশন দৃশ্যকল্প: TechNova-র তিন-স্তর নেটওয়ার্ক

TechNova-র ওয়েব অ্যাপে তিনটি স্তর: web (লোড ব্যালান্সারের পেছনে), app (ব্যবসায়িক যুক্তি), db (PostgreSQL)। প্রতিটি স্তর আলাদা সাবনেটে। Web সাবনেটের VM-গুলোর পাবলিক IP নেই — লোড ব্যালান্সারই একমাত্র প্রবেশপথ। App সাবনেট থেকে শুধু web সাবনেট আসতে পারে, db সাবনেটে শুধু app সাবনেট আসতে পারে। ইন্টারনেটে যাওয়ার রাউট শুধু web ও app সাবনেটে (Cloud NAT দিয়ে, অধ্যায় ৭ দেখুন); db সাবনেটে কোনো ইন্টারনেট রাউট নেই — এমনকি আপডেটের জন্যও Private Google Access ব্যবহার হয়। এই ডিজাইনে একটি স্তর আপস হলেও আক্রমণকারী পরের স্তরে সরাসরি যেতে পারে না।

এই ডিজাইনকে আরও এগিয়ে নেওয়ার কথা ভাবুন। প্রথমত, staging ও production-এর নেটওয়ার্ক একই কাঠামোয় রাখুন — শুধু CIDR রেঞ্জ আলাদা (যেমন staging `10.10.0.0/16`, production `10.0.0.0/16`)। তাতে staging-এ পরীক্ষা করা পরিবর্তন production-এ একইভাবে আচরণ করে। দ্বিতীয়ত, IP ঠিকানা ব্যবস্থাপনা (IPAM) — একটি স্প্রেডশিট বা Terraform-এর locals-এ প্রতিটি সাবনেটের রেঞ্জ লিখে রাখুন; কেউ যেন অনুমানে নতুন সাবনেট না বানায়। তৃতীয়ত, ভবিষ্যতের সংযোগের কথা ভেবে VPC রেঞ্জ বাছুন: TechNova জানে ঢাকার ডাটা সেন্টার `192.168.0.0/16` ব্যবহার করে, তাই VPC-র জন্য `10.0.0.0/16` বেছে নিয়েছে — কোনো ওভারল্যাপ নেই, ভবিষ্যতে VPN বসাতে কোনো বাধা নেই। চতুর্থত, কোটা: প্রতিটি রিজিয়নে সাবনেটের সংখ্যা, প্রতি VPC-তে রাউটের সংখ্যা — এগুলোর সীমা আছে; বড় পরিকল্পনার আগেই কোটা দেখে নিন। আর শেষে, নেটওয়ার্ক পরিবর্তনের জন্য একটি নিয়ম: production VPC-তে হাতে কিছু বদলাবেন না — সব পরিবর্তন Terraform-এর pull request দিয়ে, রিভিউ করে, তারপর apply করুন। একটি ভুল CIDR বা ভুল রাউট পুরো সার্ভিস থামিয়ে দিতে পারে; রিভিউ সেই ঝুঁকি কমায়।

৬.১১ নিরাপত্তা ও খরচ

নিরাপত্তা: ডিফল্ট `default` VPC মুছে ফেলুন বা অন্তত তার ঢিলেঢালা ফায়ারওয়াল নিয়মগুলো শক্ত করুন; প্রতিটি সাবনেটে শুধু প্রয়োজনীয় ট্রাফিকের অনুমতি দিন; VPC Flow Logs চালু করুন (অধ্যায় ৭); প্রাইভেট সাবনেটে VM-এ পাবলিক IP দেবেন না। খরচ: VPC নিজে বিনামূল্যে, কিন্তু এর ওপর নির্ভরশীল জিনিসের খরচ আছে — ক্রস-রিজিয়ন ট্রাফিকের egress চার্জ, Cloud NAT-এর প্রতি-ঘণ্টা ও প্রতি-GB চার্জ, আর VPC Flow Logs-এর স্টোরেজ। বিলিং বাজেট ও অ্যালার্ট আগেই বসান। বিস্তারিত দামের জন্য Google Cloud Pricing Calculator দেখুন — দাম সময়ের সঙ্গে বদলায়, এই বইয়ে কোনো দাম চিরন্তন সত্য হিসেবে লেখা হয়নি।

নিরাপত্তার আরও দুটি স্তর মনে রাখুন। প্রথমত, IAM: VPC-তে কে সাবনেট বানাতে পারে, কে ফায়ারওয়াল বদলাতে পারে — এই অনুমতিগুলো আলাদা রোল দিয়ে দিন; নেটওয়ার্ক পরিবর্তনের অধিকার শুধু নেটওয়ার্ক দলের হাতে রাখুন, অ্যাপ দলের হাতে নয়। দ্বিতীয়ত, সংস্থা-স্তরের নীতি (Organization Policy): যেমন "প্রতিটি VM-এ পাবলিক IP নিষিদ্ধ" — এই নীতি বসালে কেউ ভুল করেও পাবলিক IP-যুক্ত VM বানাতে পারবে না। এটি একটি নিরাপত্তা জাল — ব্যক্তির সতর্কতার ওপর নির্ভর না করে, ব্যবস্থার ওপর নির্ভর করা। খরচের দিকে: egress কমানোর সবচেয়ে ভালো উপায় হলো ট্রাফিককে একই রিজিয়নের ভেতরে রাখা — অ্যাপ ও ডাটাবেস একই রিজিয়নে রাখুন, ক্রস-রিজিয়ন প্রতিলিপি (replication) শুধু যেখানে সত্যিই দরকার সেখানে। আর VPC Flow Logs-এর খরচ নিয়ন্ত্রণে sampling ব্যবহার করুন — সব প্যাকেটের লগ না নিয়ে

নমুনা নিন, আর লগের ধরে রাখার মেয়াদ (retention) সীমিত রাখুন। মনে রাখবেন: বিনামূল্যের VPC-র ওপর দাঁড়িয়ে ব্যয়বহুল স্থাপত্য বানানো খুব সহজ — তাই প্রতিটি নেটওয়ার্ক সিদ্ধান্তের সঙ্গে তার খরচের হিসাবও লিখে রাখুন।

৬.১২ সাধারণ ভুল

- Auto mode VPC প্রোডাকশনে ব্যবহার করা — পরে IP ওভারল্যাপে আটকে যাওয়া।
- অতিরিক্ত ছোট সাবনেট (/28, /29) বানিয়ে IP শেষ হয়ে যাওয়া — বিশেষ করে GKE-তে।
- সব স্তর এক সাবনেটে রাখা — ফায়ারওয়াল দিয়ে আলাদা করা যায় না।
- Peering-এ ওভারল্যাপিং CIDR — peering তৈরি হবে না, অথবা রাউটিং ভেঙে যাবে।
- ডিফল্ট VPC না মুছে তাতেই প্রোডাকশন চালানো।
- MTU ও BGP routing mode না বুঝে ডিফল্ট রেখে দেওয়া — হাইব্রিড সংযোগে পরে সমস্যা হয়।

৬.১৩ Troubleshooting

TROUBLESHOOTING: VM-এ IP পাওয়া যাচ্ছে না? প্রথমে দেখুন সাবনেটে ফাঁকা IP আছে কি না (`gcloud compute networks subnets describe`)। Peering কাজ করছে না? দুই দিকের peering-ই ACTIVE কি না দেখুন, আর CIDR ওভারল্যাপ আছে কি না যাচাই করুন — ওভারল্যাপ থাকলে রাউট এক্সচেঞ্জ হবে না। ক্রস-রিজিয়ন সংযোগ ধীর? মনে রাখবেন এটি ইন্টারনেট নয়, Google ব্যাকবোন — ল্যাটেন্সি ভৌগোলিক দূরত্বের কারণে, নেটওয়ার্কের দোষে নয়। Connectivity Tests (Network Intelligence Center) দিয়ে প্যাকেটের পথ যাচাই করুন।

৬.১৪ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

VPC-র ভেতরে প্যাকেট কীভাবে চলে, তা বুঝলে ডিজাইনের সিদ্ধান্তগুলো স্বাভাবিক মনে হবে। VM থেকে বেরোনো প্রতিটি প্যাকেট প্রথমে VPC-র রাউট টেবিলে যায় — গন্তব্য একই সাবনেটে হলে সরাসরি, অন্য সাবনেটে হলে Google ব্যাকবোন দিয়ে, ইন্টারনেটে হলে ডিফল্ট গেটওয়ে দিয়ে। তারপর ফায়ারওয়াল মূল্যায়ন হয় (অধ্যায় ৭), তারপর প্যাকেট গন্তব্য VM-এ পৌঁছায়। এই পুরো পথ সফটওয়্যারে চলে — কোনো ফিজিক্যাল রাউটার আপনার প্যাকেটের জন্য অপেক্ষা করে না। এর মানে: VPC-র ভেতরের ল্যাটেন্সি মূলত ভৌগোলিক দূরত্ব ও হোস্টের অবস্থানের ওপর নির্ভর করে, নেটওয়ার্ক যন্ত্রের সংখ্যার ওপর নয়।

VPC ব্যর্থ হলে কী হয়? VPC নিজে একটি আঞ্চলিক বা বৈশ্বিক একক ব্যর্থতার বিন্দু নয় — এটি কন্ট্রোল প্লেনের বিমূর্ততা, আর ডেটা প্লেন বিতরণ করা। একটি জোনের সমস্যা সেই জোনের VM-কে প্রভাবিত করে, কিন্তু VPC-র অন্য জোনের সাবনেট চলতে থাকে। আসল ঝুঁকি হলো কনফিগারেশন ভুল: ভুল ফায়ারওয়াল নিয়ম, ওভারল্যাপিং CIDR, বা ভুল রাউট — এগুলোই বাস্তবে নেটওয়ার্ক ডাউন করে। তাই পরিবর্তন সবসময় ছোট ছোট ধাপে, একটি সাবনেটে পরীক্ষা করে, তারপর বাকিগুলোতে ছড়ান।

পর্যবেক্ষণ: VPC Flow Logs চালু করুন প্রতিটি সাবনেটে — কোন VM থেকে কোথায় সংযোগের চেষ্টা হচ্ছে, কোনটি deny হচ্ছে, তা Cloud Logging-এ দেখা যায়। Network Intelligence Center-এর Connectivity Tests দিয়ে "A থেকে B-তে প্যাকেট যাবে কি?" — এই প্রশ্নের উত্তর প্যাকেট না পাঠিয়েই পাওয়া যায়। আর VPC-র কোটা নজরে রাখুন: প্রতি VPC-তে সাবনেটের সংখ্যা, রাউটের সংখ্যা, ফায়ারওয়াল নিয়মের সংখ্যা — সবকিছুর সীমা আছে; বড় ডিজাইনে আগে কোটা বাড়ানোর অনুরোধ করুন। বিলিং: VPC বিনামূল্যে হলেও এর সঙ্গে যুক্ত egress, NAT ও লগের খরচ আছে — Cloud Billing-এ নেটওয়ার্ক খরচ আলাদা করে দেখার রিপোর্ট বসান। বিকল্প: খুব ছোট কাজে একটি VPC-ই যথেষ্ট; অনেক

প্রজেক্টের সংস্থায় Shared VPC; আর একেবারে আলাদা পরিবেশ চাইলে আলাদা প্রজেক্ট ও আলাদা VPC — পিয়ারিং দিয়ে যুক্ত।

৬.১৫ সারাংশ

VPC হলো আপনার ক্লাউডের সীমানা। গ্লোবাল VPC, আঞ্চলিক সাবনেট, সতর্ক CIDR পরিকল্পনা, custom mode, আর প্রয়োজনে Shared VPC — এই পাঁচটি সিদ্ধান্তই আপনার নেটওয়ার্কের ভবিষ্যৎ ঠিক করে দেয়। সাবনেট দিয়ে স্তর আলাদা করুন, IP-র জায়গা ভবিষ্যতের জন্য রাখুন, আর সবকিছু IaC-তে লিখুন।

৬.১৬ পর্যালোচনা প্রশ্ন

- গ্লোবাল VPC-র সুবিধা কী? আঞ্চলিক VPC-র সঙ্গে পার্থক্য কোথায়?
- /24 সাবনেটে কতগুলো ব্যবহারযোগ্য IP থাকে, আর কেন ৪টি সংরক্ষিত?
- Shared VPC কখন Peering-এর চেয়ে ভালো?
- Peering কেন non-transitive? একটি উদাহরণ দিয়ে বোঝান।
- ডাটাবেস সাবনেটে ইন্টারনেট রাউট না রাখলে কী সুবিধা ও কী অসুবিধা?

৬.১৭ ব্যবহারিক অনুশীলন

- একটি custom mode VPC বানান: দুটি রিজিয়নে web/app/db সাবনেটসহ। gcloud দিয়ে করুন, তারপর Terraform-এ একই জিনিস লিখে `terraform plan` চালান।
- একটি /16 রেঞ্জকে ৬টি সাবনেটে ভাগ করে একটি ঠিকানা পরিকল্পনার টেবিল লিখুন — GKE-র জন্য সেকেন্ডারি রেঞ্জসহ।
- দুটি VPC-র মধ্যে peering বানান, একটিতে VM থেকে অন্যটির প্রাইভেট IP-তে ping করে যাচাই করুন, তারপর peering মুছে ফেলুন।

অধ্যায় ৭ • PART II — Networking & Compute Engineering

Chapter 7 — Routes, Firewall and Private Connectivity

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) VPC ফায়ারওয়াল নিয়ম কীভাবে কাজ করে তা ব্যাখ্যা করতে — priority, tag ও service account দিয়ে; (২) hierarchical firewall policy দিয়ে সংস্থা-জুড়ে নিয়ম চাপাতে; (৩) রাউটের ধরন (system-generated, custom, dynamic) বুঝে ট্রাফিকের পথ নিয়ন্ত্রণ করতে; (৪) Cloud NAT দিয়ে প্রাইভেট VM-কে নিরাপদে ইন্টারনেটে বের করতে; (৫) Private Google Access, Private Service Connect ও private services access-এর মধ্যে বেছে নিতে; (৬) নেটওয়ার্ক সমস্যার ক্ষেত্রে Connectivity Tests ও Flow Logs দিয়ে রোগ নির্ণয় করতে।

৭.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ৬ থেকে মনে রাখুন: VPC গ্লোবাল, সাবনেট আঞ্চলিক, আর প্রতিটি স্তর আলাদা সাবনেটে থাকে। আরও মনে রাখুন: GCP-তে ফায়ারওয়াল স্টেটফুল (stateful) — অর্থাৎ অনুমোদিত আউটবাউন্ড সংযোগের উত্তর (return traffic) স্বয়ংক্রিয়ভাবে ফিরতে পারে, আলাদা ইনবাউন্ড নিয়ম লাগে না। আর একটি কথা: GCP ফায়ারওয়াল VM-এর নেটওয়ার্ক ইন্টারফেসে প্রয়োগ হয়, সাবনেটের গেটওয়েতে নয় — তাই নিয়ম লিখতে হয় VM-কে লক্ষ্য করে, সাবনেটকে নয়।

৭.৩ প্রথম নীতি: ফায়ারওয়াল আসলে কী?

ফায়ারওয়াল কোনো যন্ত্র নয় — এটি একটি সিদ্ধান্তের তালিকা: কোন প্যাকেট যেতে পারবে, কোনটি পারবে না। GCP-তে প্রতিটি প্যাকেট VM-এ পৌঁছানোর আগেই ফায়ারওয়াল নিয়মের সঙ্গে মেলানো হয়। নিয়মের মূল উপাদান: দিক (ingress/egress), অগ্রাধিকার (priority — ছোট সংখ্যা মানে বেশি অগ্রাধিকার), অ্যাকশন (allow/deny), প্রোটোকল ও পোর্ট, উৎস/গন্তব্য (IP রেঞ্জ, নেটওয়ার্ক ট্যাগ, বা service account), আর লক্ষ্য (কোন VM-এ প্রযোজ্য)।

গুরুত্বপূর্ণ নীতি: **implicit deny** — কোনো নিয়ম না মিললে প্যাকেট আটকে যায়। আর দুটি implied নিয়ম সবসময় থাকে: egress allow-all (সব আউটবাউন্ড অনুমোদিত) ও ingress deny-all (সব ইনবাউন্ড নিষিদ্ধ)। তাই ডিফল্ট অবস্থায় VM বাইরে যেতে পারে, কিন্তু বাইরে থেকে কেউ ভেতরে আসতে পারে না — যতক্ষণ না আপনি স্পষ্টভাবে অনুমতি দেন।

> **CONCEPT:** GCP ফায়ারওয়াল distributed — প্রতিটি VM-এর পাশেই প্রয়োগ হয়, কোনো কেন্দ্রীয় choke point নেই। তাই ফায়ারওয়াল নিয়ম যোগ করলে কোনো একক যন্ত্রের ব্যান্ডউইথে আটকে যান না; নিয়মটি কন্ট্রোল প্লেন পুরো ফ্লিটে ছড়িয়ে দেয়।

৭.৪ ফায়ারওয়াল নিয়মের আর্কিটেকচার

ফায়ারওয়াল মূল্যায়নের ক্রম (ingress) ইন্টারনেট / অন্য VM ──► VPC ──► [priority 1000: allow-ssh-bastion] [priority 2000: allow-web-from-lb] [priority 5000: allow-app-from-web] [implied: deny-all ingress] ──► VM (tag: web / app / db) নিয়ম মেলানোর ক্রম: সবচেয়ে ছোট priority সংখ্যা আগে। প্রথম মিলেই সিদ্ধান্ত — allow হলে প্যাকেট যায়, deny হলে আটকায়। কোনো নিয়ম না মিললে implied deny-all কাজ করে।

লক্ষ্য নির্ধারণের দুটি উপায়: **নেটওয়ার্ক ট্যাগ** (যেমন `web-server`) ও **service account**। ট্যাগ সহজ, কিন্তু যে কেউ VM-এ ট্যাগ লাগাতে পারলে নিয়মের আওতায় ঢুকে যেতে পারে — ট্যাগ বদলানোর IAM অনুমতি তাই সতর্কভাবে দিতে হয়। Service account নিরাপদ: নিয়ম বলে "এই service account-ধারী VM থেকে আসা ট্রাফিক অনুমোদিত" — আর service account ব্যবহারের অনুমতি IAM দিয়ে নিয়ন্ত্রিত হয়। প্রোডাকশনে service account-ভিত্তিক নিয়মই সুপারিশ করা হয়।

৭.৫ Hierarchical Firewall Policies

অনেক প্রজেক্টের সংস্থায় প্রতিটি প্রজেক্টে আলাদা ফায়ারওয়াল নিয়ম লিখলে কেন্দ্রীয় নিরাপত্তা নীতি মানানো যায় না। Hierarchical firewall policy (HFP) সমাধান দেয়: সংস্থা বা ফোল্ডার স্তরে নীতি বসে, যা নিচের সব প্রজেক্টে প্রযোজ্য হয়। নীতিতে দুই ধরনের নিয়ম থাকে: allow/deny — আর গুরুত্বপূর্ণভাবে, **go-to-next** নয়, বরং deny নিয়ম প্রজেক্টের allow নিয়মকে ওভাররাইড করতে পারে। অর্থাৎ কেন্দ্রীয় দল বলতে পারে "কোনো প্রজেক্টেই পোর্ট 22 ইন্টারনেট থেকে খোলা যাবে না" — আর কোনো প্রজেক্টের স্থানীয় নিয়ম তা ভাঙতে পারবে না। মূল্যায়নের ক্রম: প্রথমে সংস্থার নীতি, তারপর ফোল্ডার, তারপর প্রজেক্টের VPC নিয়ম।

৭.৬ রাউট: ট্রাফিকের পথ কীভাবে ঠিক হয়

প্রতিটি VPC-তে একটি রাউট টেবিল থাকে। রাউটের উৎস তিন রকম: **system-generated** — সাবনেট রেঞ্জের মধ্যে ট্রাফিকের জন্য (সাবনেট-টু-সাবনেট সরাসরি যায়), আর ডিফল্ট ইন্টারনেট গেটওয়ের জন্য (0.0.0.0/0); **custom static route** — আপনি নিজে বানান, যেমন 0.0.0.0/0-কে একটি NAT গেটওয়ে বা VPN টানেলের দিকে পাঠানো; **dynamic route** — Cloud Router BGP-র মাধ্যমে শেখে (অধ্যায় ১০)। রাউট বাছাইয়ের নিয়ম: সবচেয়ে নির্দিষ্ট (longest prefix match) রাউট জেতে; সমান হলে priority দেখা হয়।

একটি সাধারণ ভুল ধারণা: ফায়ারওয়াল ট্রাফিকের পথ ঠিক করে। না — রাউট পথ ঠিক করে, ফায়ারওয়াল সেই পথে অনুমতি দেয় কি না ঠিক করে। প্যাকেট প্রথমে রাউট দিয়ে গন্তব্য খোঁজে, তারপর ফায়ারওয়াল দিয়ে অনুমতি পায়। দুটোর যেকোনো একটিতে আটকে গেলে সংযোগ ব্যর্থ হয় — আর troubleshooting-এ এটাই প্রথমে আলাদা করতে হয়।

৭.৭ Cloud NAT

প্রাইভেট VM-এর (পাবলিক IP নেই) ইন্টারনেটে যাওয়ার দরকার হয় — প্যাকেজ আপডেট, বাইরের API ডাকতে। Cloud NAT সেই সমস্যার সমাধান: এটি একটি আঞ্চলিক, ম্যানেজড NAT সার্ভিস। প্রাইভেট VM-এর আউটবাউন্ড প্যাকেট NAT গেটওয়ের পাবলিক IP দিয়ে বেরোয়, উত্তর ফিরে আসে একই পথে। কিন্তু ইনবাউন্ড সংযোগ কখনোই NAT দিয়ে ভেতরে আসতে পারে না — এটি একমুখী দরজা, যা নিরাপত্তার জন্য ভালো।

Cloud NAT আঞ্চলিক — প্রতিটি রিজিয়নে আলাদা NAT গেটওয়ে লাগে। এটি স্বয়ংক্রিয়ভাবে স্কেল করে, কোনো VM পরিচালনা করতে হয় না। খরচের দিকে নজর দিন: প্রতি ঘণ্টায় গেটওয়ে চার্জ আর প্রসেস করা প্রতি GB-তে ডেটা চার্জ — বড় ডাউনলোডের জন্য ব্যয়বহুল হতে পারে। বিকল্প: যেসব VM-এর সত্যিই ইন্টারনেট লাগে না, সেগুলোর জন্য NAT না বসিয়ে Private Google Access ব্যবহার করুন। আর মনে রাখবেন, NAT-এর মাধ্যমে যাওয়া ট্রাফিকের egress চার্জও আছে — দামের জন্য Google Cloud Pricing Calculator দেখুন।

```
# Cloud Router ও NAT তৈরি (asia-southeast1)
gcloud compute routers create technova-router \
  --network=technova-vpc --region=asia-southeast1

gcloud compute routers nats create technova-nat \
  --router=technova-router \
  --region=asia-southeast1 \
```

```
  --nat-all-subnet-ip-ranges \
  --enable-logging

# ফায়ারওয়াল নিয়ম: শুধু bastion থেকে SSH, শুধু LB থেকে HTTP
gcloud compute firewall-rules create allow-ssh-bastion \
  --network=technova-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:22 --source-ranges=10.0.0.0/24 --priority=1000 \
  --target-tags=bastion-ssh

gcloud compute firewall-rules create allow-web-from-lb \
  --network=technova-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:80,tcp:443 --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --priority=1000 --target-tags=web-server

# উপরের 130.211.0.0/22 ও 35.191.0.0/16 হলো Google লোড ব্যালান্সারের
# হেলথ-চেক ও প্রক্সি রেঞ্জ (ডকুমেন্টেশনে যাচাই করে নিন)
```

৭.৮ Private Google Access

Private Google Access (PGA) চালু থাকলে প্রাইভেট IP-র VM সরাসরি Google API ও সার্ভিসে (Cloud Storage, BigQuery, Container Registry ইত্যাদি) পৌঁছাতে পারে — ইন্টারনেট বা NAT ছাড়াই। সাবনেট স্তরে চালু হয় (`--enable-private-ip-google-access`)। এটি db সাবনেটের জন্য অপরিহার্য: ডাটাবেস সার্ভার ইন্টারনেটে না গিয়েও Cloud Storage থেকে ব্যাকআপ নিতে পারে। সীমাবদ্ধতা: এটি শুধু Google API-র জন্য; বাইরের কোনো সাইটের জন্য নয়।

৭.৯ Private Service Connect ও Private Services Access

এই দুটি প্রায়ই গুলিয়ে যায় — পার্থক্যটা পরিষ্কার করুন। **Private Services Access** হলো Google-এর ম্যানেজড সার্ভিস (যেমন Cloud SQL, Memorystore)-এর জন্য আপনার VPC থেকে একটি IP রেঞ্জ বরাদ্দ করে VPC peering-এর মাধ্যমে সংযোগ — সার্ভিসটি আপনার VPC-র ভেতরে প্রাইভেট IP পায়। **Private Service Connect (PSC)** আরও সাধারণ: যেকোনো সার্ভিস (Google-এর, বা অন্য প্রজেক্টের/সংস্থার, এমনকি আপনার নিজের) আপনার VPC-তে একটি প্রাইভেট IP এন্ডপয়েন্ট হিসেবে প্রকাশ পায়। ক্লায়েন্ট শুধু সেই এন্ডপয়েন্ট দেখে — সার্ভিসের আসল নেটওয়ার্ক দেখে না। PSC হলো আধুনিক সুপারিশ: Cloud SQL-এর নতুন ডিজাইনেও PSC-ই পথ।

> **বাস্তব উদাহরণ:** TechNova Cloud SQL ব্যবহার করে। Private Services Access দিয়ে Cloud SQL-কে `10.0.2.0/24` সাবনেটের পাশে প্রাইভেট IP দেওয়া হয়েছে — app সার্ভার সরাসরি প্রাইভেট IP-তে ডাটাবেসে যায়, কোনো পাবলিক IP নেই, কোনো ইন্টারনেট হপ নেই। পরে যখন TechNova অন্য প্রজেক্টের একটি অভ্যন্তরীণ API ব্যবহার করতে চাইল, তখন PSC দিয়ে সেই API-কে নিজের VPC-তে এন্ডপয়েন্ট হিসেবে নিয়ে এলো — API-র মালিক প্রজেক্টের নেটওয়ার্ক না দেখেই।

৭.১০ Terraform দিয়ে ফায়ারওয়াল ও NAT

```
resource "google_compute_firewall" "allow_app_from_web" {
  name      = "allow-app-from-web"
  network   = google_compute_network.technova.id
  direction = "INGRESS"
  priority  = 1000
  allow {
    protocol = "tcp"
```

```
    ports    = ["8080"]
  }
  source_tags = ["web-server"]
  target_tags = ["app-server"]
}

resource "google_compute_router_nat" "technova_nat" {
  name                               = "technova-nat"
  router                             = google_compute_router.technova.name
  region                             = "asia-southeast1"
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
  log_config {
    enable = true
    filter = "ERRORS_ONLY"
  }
}
```

৭.১১ প্রোডাকশন দৃশ্যকল্প: শূন্য-বিশ্বাস (zero-trust) নেটওয়ার্ক

TechNova-র নীতি: কোনো VM-এর পাবলিক IP নেই (bastion ছাড়া, আর সেটিও IAP দিয়ে — Identity-Aware Proxy, যাতে SSH-ও Google পরিচয় দিয়ে যায়)। প্রতিটি স্তরের মধ্যে শুধু প্রয়োজনীয় পোর্ট খোলা, service account দিয়ে লক্ষ্য নির্ধারণ, hierarchical policy দিয়ে পোর্ট 22 ইন্টারনেট থেকে নিষিদ্ধ। VPC Flow Logs সব সাবনেটে চালু — কোনো অস্বাভাবিক সংযোগের চেষ্টা হলে Cloud Logging-এ ধরা পড়ে। এই ডিজাইনে "ভেতরে ঢুকলেই সব পাওয়া যায়" — এমন ধারণা নেই; প্রতিটি হপে অনুমতি যাচাই হয়।

এই নীতিকে বাস্তবে টিকিয়ে রাখতে আরও কয়েকটি অভ্যাস দরকার। প্রথমত, ফায়ারওয়াল নিয়মের নামকরণের নিয়ম: `allow-<উৎস>-to-<গন্তব্য>-<পোর্ট>` — যেমন `allow-web-to-app-8080`। নাম দেখেই বোঝা যায় নিয়মটি কী করে; ছয় মাস পরেও কেউ বিভ্রান্ত হয় না। দ্বিতীয়ত, নিয়মিত অডিট: মাসে একবার Firewall Insights খুলে দেখুন কোন নিয়ম কখনো ব্যবহার হয়নি — অব্যবহৃত নিয়ম মুছে ফেলুন, কারণ প্রতিটি অতিরিক্ত নিয়মই একটি ভুলের সুযোগ। তৃতীয়ত, egress নিয়ন্ত্রণ: ডিফল্ট egress allow-all অনেক সংস্থায় রেখে দেওয়া হয়, কিন্তু সংবেদনশীল সাবনেটে (যেমন db) egress-ও সীমিত করুন — শুধু Google API ও নির্দিষ্ট গন্তব্যে যাওয়ার অনুমতি দিন। চতুর্থত, পরিবর্তনের পরীক্ষা: নতুন ফায়ারওয়াল নিয়ম প্রথমে একটি পরীক্ষামূলক VM-এ প্রয়োগ করুন, Connectivity Tests দিয়ে যাচাই করুন, তারপর production-এ ছড়ান। আর শেষে, জরুরি অবস্থার পরিকল্পনা: যদি কোনো আক্রমণ চলতে থাকে, তাহলে কোন নিয়মটি প্রথমে বসাবেন — সেই "emergency deny" নিয়মের priority ও প্রক্রিয়া আগেই লিখে রাখুন, বিপদের সময়ে ভেবে নয়।

৭.১২ নিরাপত্তা ও খরচ

নিরাপত্তা: কখনোই `0.0.0.0/0` থেকে সব পোর্ট খুলবেন না; SSH-র জন্য IAP বা bastion ব্যবহার করুন; ফায়ারওয়াল নিয়মে service account ব্যবহার করুন; নিয়মিত `gcloud compute firewall-rules list` দিয়ে অডিট করুন। খরচ: Cloud NAT প্রতি ঘণ্টা + প্রতি GB; VPC Flow Logs-এর লগ সংরক্ষণ; PSC এন্ডপয়েন্টের প্রতি-ঘণ্টা চার্জ। এগুলোর বিলিং বাজেটে অ্যালার্ট বসান।

নিরাপত্তার ক্ষেত্রে একটি সাধারণ ভুল হলো শুধু ingress নিয়ে ভাবা, egress নিয়ে না ভাবা। আক্রমণকারী যদি কোনোভাবে VM-এ ঢুকে যায়, তাহলে সে কী করতে পারে — তা নির্ভর করে VM কোথায় যেতে পারে তার ওপর। ডাটা চুরি করতে হলে আক্রমণকারীকে ডেটা বাইরে পাঠাতে হয় — egress সীমিত থাকলে সেটি কঠিন হয়। তাই সংবেদনশীল সাবনেটে egress নিয়মও লিখুন: শুধু নির্দিষ্ট গন্তব্য (যেমন Google API, নির্দিষ্ট আপডেট সার্ভার) অনুমোদিত, বাকি সব নিষিদ্ধ। আরেকটি স্তর: VPC Service Controls — এটি একটি পরিধি (perimeter) বানায় Google Cloud সার্ভিসের চারপাশে, যাতে অনুমোদিত

নেটওয়ার্কের বাইরে থেকে Cloud Storage বা BigQuery-তে ডেটা নেওয়া না যায় — এমনকি সঠিক credentials থাকলেও। বড় সংস্থায়, সংবেদনশীল ডেটা নিয়ে কাজ করলে, এটি বিবেচনা করুন। খরচের দিকে: NAT-এর বিল কমাতে দেখুন কোন VM-গুলো সবচেয়ে বেশি egress করছে — প্রায়ই দেখা যায় একটি ভুল কনফিগার করা অ্যাপ বারবার বড় ফাইল ডাউনলোড করছে, বা লগ বাইরে পাঠাচ্ছে। Flow Logs ও NAT লগ দিয়ে সেই উৎস খুঁজে বের করুন — খরচ কমানোর সেরা উপায় হলো অপ্রয়োজনীয় ট্রাফিক বন্ধ করা, সস্তা সার্ভিস খোঁজা নয়।

৭.১৩ সাধারণ ভুল

- Priority বুঝতে ভুল — বড় সংখ্যা মানে কম অগ্রাধিকার, উল্টোটা নয়।
- ট্যাগ-ভিত্তিক নিয়মে ট্যাগ বদলানোর IAM খোলা রাখা।
- ফায়ারওয়াল দিয়ে রাউটিং ঠিক করার চেষ্টা — রাউটের কাজ রাউটকে করতে দিন।
- Cloud NAT বসিয়ে ভাবা যে ইনবাউন্ডও আসবে — NAT একমুখী।
- লোড ব্যালান্সারের হেলথ-চেক রেঞ্জ (130.211.0.0/22, 35.191.0.0/16) না খুলে হেলথ চেক ব্যর্থ হওয়া।
- Private Google Access না চালিয়ে db সাবনেট থেকে Google API-তে যেতে না পারা।

আরেকটি প্রায়ই দেখা ভুল হলো ফায়ারওয়াল নিয়মের পরীক্ষা না করা। নিয়ম লেখার পর শুধু "কাজ করছে" দেখেই থেমে যাবেন না — "আটকাচ্ছে কি না" সেটিও পরীক্ষা করুন। যেমন, app সাবনেট থেকে db-তে 5432 পোর্ট খোলা থাকলে, web সাবনেট থেকে সরাসরি db-তে যাওয়ার চেষ্টা করে দেখুন সেটি আটকায় কি না। নেতিবাচক পরীক্ষা (negative test) ছাড়া আপনি জানতেই পারবেন না নিয়মটি আসলে কাজ করছে, না কি অন্য কোনো নিয়মের কারণে ট্রাফিক যাচ্ছে। আরেকটি অভ্যাস: প্রতিটি ফায়ারওয়াল নিয়মের description ফিল্ডে লিখে রাখুন কেন এই নিয়মটি আছে, কে অনুরোধ করেছিল, আর কবে পর্যালোচনা করা হবে। ছয় মাস পর কেউ যখন জিজ্ঞেস করবে "এই নিয়মটি কেন আছে?", তখন উত্তরটি নিয়মের সঙ্গেই থাকবে।

৭.১৪ Troubleshooting

TROUBLESHOOTING: সংযোগ ব্যর্থ হলে ধাপে ধাপে যান: (১) `ping` নয়, TCP-তে পরীক্ষা করুন — ICMP প্রায়ই বন্ধ থাকে; (২) Connectivity Tests দিয়ে প্যাকেটের পথ দেখুন — রাউটে আটকেছে না ফায়ারওয়ালে, তা বলে দেবে; (৩) VPC Flow Logs-এ দেখুন প্যাকেটটি deny হয়েছে কি না; (৪) VM-এর ভেতরে `ss -tlnp` দিয়ে সার্ভিস আদৌ শুনছে কি না দেখুন; (৫) NAT সমস্যায় NAT গেটওয়ের লগ ও পোর্ট বরাদ্দ দেখুন — অনেক সংযোগ একসঙ্গে খুললে পোর্ট শেষ হতে পারে।

৭.১৫ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

ফায়ারওয়াল নিয়ম কীভাবে প্রয়োগ হয়, তা জানলে troubleshooting অনেক সহজ হয়। GCP-তে ফায়ারওয়াল নিয়ম কন্ট্রোল প্লেন থেকে প্রতিটি হোস্টের Andromeda নেটওয়ার্ক স্ট্যাকে প্রোগ্রাম করা হয় — প্যাকেট VM-এর নেটওয়ার্ক ইন্টারফেসে পৌঁছানোর আগেই মূল্যায়ন হয়। নিয়ম বদলালে সেটি কয়েক সেকেন্ডের মধ্যে সব হোস্টে ছড়িয়ে যায়, কিন্তু "কয়েক সেকেন্ড" মানে পরিবর্তনের পরপরই পরীক্ষা করলে পুরোনো নিয়ম দেখতে পারেন — ধৈর্য ধরুন, তারপর পরীক্ষা করুন। আর একটি সূক্ষ্ম বিষয়: ফায়ারওয়াল স্টেটফুল বলে অনুমোদিত সংযোগের উত্তর প্যাকেটের জন্য আলাদা নিয়ম লাগে না — কিন্তু UDP-র মতো connectionless প্রোটোকলে স্টেটের সময়সীমা (timeout) থাকে; দীর্ঘ বিরতির পর প্রথম প্যাকেট নতুন সংযোগ হিসেবে দেখা হয়।

রাউটের ক্ষেত্রে মনে রাখুন: কাস্টম রাউটের next hop হতে পারে একটি VM (NAT গেটওয়ে হিসেবে), VPN টানেল, Interconnect attachment, বা ডিফল্ট ইন্টারনেট গেটওয়ে। Next hop VM মারা গেলে রাউটটি মৃত — তাই নিজের হাতে NAT VM বানানোর চেয়ে ম্যানেজড Cloud NAT ব্যবহার করা ভালো। আর রাউটের priority: একই গন্তব্যের দুটি রাউট থাকলে ছোট priority সংখ্যার রাউট জেতে — ফেলওভার ডিজাইনে প্রাইমারি রাউটের priority ১০০, ব্যাকআপের ২০০ রাখা সাধারণ প্যাটার্ন।

ব্যর্থতার দৃশ্য: Cloud NAT-এর পোর্ট শেষ হয়ে গেলে নতুন আউটবাউন্ড সংযোগ ব্যর্থ হয় — লক্ষণ হলো মাঝে মাঝে সংযোগ হয়, মাঝে মাঝে হয় না। সমাধান: NAT-এর জন্য আরও পাবলিক IP বরাদ্দ করা, বা সংযোগের সংখ্যা কমানো (connection pooling)। PSC এন্ডপয়েন্টের ক্ষেত্রে: এন্ডপয়েন্টের IP-তে পৌঁছানো গেলেও সার্ভিসের দিকের ফায়ারওয়াল বা IAM অনুমতি না থাকলে সংযোগ ব্যর্থ হয় — দুই দিকেই দেখতে হয়। পর্যবেক্ষণ: VPC Flow Logs-এ deny হওয়া প্যাকেটের হিসাব রাখুন; Cloud NAT-এর লগে পোর্ট বরাদ্দের ত্রুটি দেখুন; Firewall Insights দিয়ে অব্যবহৃত নিয়ম ও অতিরিক্ত খোলা নিয়ম খুঁজে বের করুন — এটি নিয়মিত পরিষ্কারের সেরা হাতিয়ার। বিকল্প: যেসব ক্ষেত্রে শুধু Google API-তে যেতে হয়, সেখানে NAT-এর বদলে Private Google Access; যেসব ক্ষেত্রে অন-প্রেমিসেসে যেতে হয়, সেখানে VPN/Interconnect — NAT কখনোই তার বিকল্প নয়।

৭.১৬ সারাংশ

রাউট পথ ঠিক করে, ফায়ারওয়াল অনুমতি দেয়, NAT প্রাইভেট VM-কে নিরাপদে বাইরে যেতে দেয়, আর PSC/PGA Google সার্ভিসকে ভেতরে নিয়ে আসে। এই চারটি একসঙ্গে মিলেই একটি প্রাইভেট, নিয়ন্ত্রিত নেটওয়ার্ক তৈরি হয়।

৭.১৭ পর্যালোচনা প্রশ্ন

- Implied deny-all ingress নিয়মটি কী? এটি না থাকলে কী হতো?
- ট্যাগ-ভিত্তিক ও service account-ভিত্তিক ফায়ারওয়াল নিয়মের নিরাপত্তা পার্থক্য কী?
- Cloud NAT দিয়ে ইনবাউন্ড সংযোগ কেন আসতে পারে না?
- Private Services Access ও Private Service Connect-এর পার্থক্য একটি উদাহরণ দিয়ে বোঝান।
- Hierarchical firewall policy কখন দরকার হয়?

৭.১৮ ব্যবহারিক অনুশীলন

- দুটি VM বানান (web ও app ট্যাগসহ) — এমন ফায়ারওয়াল নিয়ম লিখুন যাতে শুধু web থেকে app-এর 8080 পোর্টে যাওয়া যায়, অন্য কিছু নয়।
- Cloud NAT বসিয়ে পাবলিক IP-বিহীন VM থেকে `apt update` চালিয়ে দেখুন; তারপর NAT মুছে Private Google Access দিয়ে Cloud Storage-এ পৌঁছানোর চেষ্টা করুন।
- Connectivity Tests দিয়ে একটি ব্যর্থ সংযোগের কারণ খুঁজে বের করুন ও Flow Logs-এ প্রমাণ দেখুন।

অধ্যায় ৮ • PART II — Networking & Compute Engineering

Chapter 8 — DNS, Load Balancing and CDN

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Cloud DNS-এ পাবলিক ও প্রাইভেট জোন বানিয়ে রেকর্ড পরিচালনা করতে; (২) গ্লোবাল বনাম আঞ্চলিক লোড ব্যালান্সারের মধ্যে বেছে নিতে; (৩) এক্সটার্নাল ও ইন্টার্নাল, L4 ও L7 ব্যালান্সারের পার্থক্য ব্যাখ্যা করতে; (৪) হেলথ চেক, ব্যাকএন্ড সার্ভিস ও ব্যাকএন্ড বাকেটের ভূমিকা বুঝতে; (৫) Cloud CDN দিয়ে ক্যাশিং কৌশল ঠিক করতে; (৬) TLS সার্টিফিকেট ও WAF ধারণা দিয়ে ট্রাফিক সুরক্ষিত করতে; (৭) একটি ট্রাফিক-পরিচালনা সমাধান বেছে নেওয়ার সিদ্ধান্ত-কাঠামো ব্যবহার করতে।

৮.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

মনে রাখুন: DNS হলো নাম থেকে IP-তে অনুবাদের ব্যবস্থা — `app.technova.example` লিখলে রিজলভার জিজ্ঞেস করে কোন IP-তে যেতে হবে। A রেকর্ড নামকে IPv4-তে, AAAA IPv6-তে, CNAME এক নামকে আরেক নামে, MX মেইল সার্ভারে পাঠায়। TTL বলে দেয় রেকর্ড কতক্ষণ ক্যাশে থাকবে। আর লোড ব্যালান্সিং-এর মূল ধারণা: অনেক সার্ভারের সামনে একজন বিতরণকারী বসে, যে প্রতিটি অনুরোধকে সুস্থ সার্ভারের দিকে পাঠায়।

৮.৩ প্রথম নীতি: নাম, বিতরণ ও ক্যাশ — তিনটি আলাদা সমস্যা

ব্যবহারকারী যখন `app.technova.example` খোলে, তখন তিনটি প্রশ্নের উত্তর লাগে: (১) এই নামের IP কী? — DNS-এর কাজ। (২) অনেক সার্ভারের মধ্যে কোনটিতে অনুরোধ যাবে? — লোড ব্যালান্সারের কাজ। (৩) একই কনটেন্ট বারবার মূল সার্ভার থেকে আনার দরকার আছে কি? — CDN-এর কাজ। এই তিনটি আলাদা সমস্যা, আলাদা সার্ভিস — একটিকে আরেকটির বিকল্প ভাবা ভুল। DNS নাম দেয়, লোড ব্যালান্সার সিদ্ধান্ত নেয়, CDN দূরত্ব কমায়।

৮.৪ Cloud DNS

Cloud DNS হলো Google-এর ম্যানেজড, উচ্চ-উপলব্ধ DNS সার্ভিস — পাবলিক ও প্রাইভেট জোন দুটোই। পাবলিক জোন ইন্টারনেটের সবাইকে আপনার ডোমেইনের রেকর্ড দেয়; প্রাইভেট জোন শুধু আপনার VPC-র ভেতরের VM-গুলোকে দেয় — যেমন `db.internal.technova.example` শুধু ভেতর থেকে রিজলভ হয়। প্রাইভেট জোনের সুবিধা: অভ্যন্তরীণ নামের জন্য পাবলিক DNS-এ যেতে হয় না, আর অভ্যন্তরীণ IP ইন্টারনেটে ফাঁস হয় না। DNSSEC চালু করলে রেকর্ডের সত্যতা যাচাই হয় — কেউ মাঝপথে ভুয়া উত্তর দিতে পারে না।

DNS নীতি (DNS policy) দিয়ে VPC-র ভেতরের রিজলভারকে অন-প্রেমিসেস DNS-এর দিকে পাঠানো যায় (inbound/outbound forwarding) — হাইব্রিড পরিবেশে এটি অপরিহার্য, অধ্যায় ১০-এ আবার আসবে।

```
# পাবলিক জোন তৈরি
gcloud dns managed-zones create technova-public \
  --dns-name=technova.example. \
  --description="TechNova public zone" \
  --dnssec-state=on

# A রেকর্ড: অ্যাপের লোড ব্যালান্সার IP-তে
gcloud dns record-sets create app.technova.example. \
  --zone=technova-public --type=A --ttl=300 \
  --rrdatas=203.0.113.10
```

```
# প্রাইভেট জোন (শুধু VPC-র ভেতরে)
gcloud dns managed-zones create technova-private \
  --dns-name=internal.technova.example. \
  --description="TechNova private zone" \
  --visibility=private \
  --networks=technova-vpc

gcloud dns record-sets create db.internal.technova.example. \
  --zone=technova-private --type=A --ttl=300 \
  --rrdatas=10.0.2.10
```

সতর্কতা: `203.0.113.10` একটি উদাহরণ IP (TEST-NET-3) — আপনার আসল লোড ব্যালান্সারের IP এখানে বসবে। রেকর্ড বদলানোর পর TTL শেষ না হওয়া পর্যন্ত পুরোনো উত্তর ক্যাশে থাকতে পারে — তাই জরুরি পরিবর্তনের আগে TTL কমিয়ে রাখুন।

৮.৫ লোড ব্যালান্সিং আর্কিটেকচার

TechNova ট্রাফিক প্রবাহ ব্যবহারকারী ──► app.technova.example (Cloud DNS) ──► Global External HTTPS Load Balancer (anycast IP) ├── Cloud Armor (WAF) — ক্ষতিকর অনুরোধ আটকায় ├── TLS শেষ হয় এখানে (Google-managed সার্টিফিকেট) ├── URL map: /api/* → backend-api, /* → backend-web ├── backend-web ──► MIG (asia-southeast1, 3 VM) └── backend-api ──► MIG (asia-southeast1, 2 VM) ──► Cloud CDN: স্ট্যাটিক কনটেন্ট এজ থেকে দেয় ──► Cloud Storage বাকেট (স্ট্যাটিক সাইটের ব্যাকএন্ড বাকেট)

GCP-র লোড ব্যালান্সার সফটওয়্যার-সংজ্ঞায়িত ও গ্লোবাল: একটি global external load balancer-এর একটিমাত্র anycast IP থাকে, যা বিশ্বের যেকোনো প্রান্ত থেকে নিকটতম Google এজে পৌঁছায়। তারপর ট্রাফিক Google ব্যাকবোন দিয়ে নিকটতম সুস্থ ব্যাকএন্ডে যায়। আঞ্চলিক ব্যালান্সার একটি রিজিয়নের ভেতরে কাজ করে — অভ্যন্তরীণ সার্ভিসের জন্য বা এক রিজিয়নে সীমাবদ্ধ অ্যাপের জন্য।

৮.৬ কোন ব্যালান্সার কখন?

ধরন	স্তর	ব্যবহার	উদাহরণ
Global external HTTPS LB	L7	পাবলিক ওয়েব/অ্যাপ, URL-ভিত্তিক রাউটিং	app.technova.example
Global external TCP/UDP LB	L4	TLS পাস-থ্রু, অ-HTTP ট্রাফিক	গেম সার্ভার, কাস্টম প্রোটোকল
Regional external LB	L4/L7	এক রিজিয়নে সীমাবদ্ধ পাবলিক সার্ভিস	আঞ্চলিক API
Internal HTTPS LB	L7	VPC-র ভেতরের মাইক্রোসার্ভিস	app → api (প্রাইভেট)
Internal TCP/UDP LB	L4	VPC-র ভেতরের TCP সার্ভিস	প্রাইভেট ডাটাবেস প্রক্সি

নিয়ম: পাবলিক ওয়েবের জন্য global external HTTPS LB; VPC-র ভেতরের সার্ভিস-টু-সার্ভিসের জন্য internal LB — তাতে ট্রাফিক কখনো ইন্টারনেটে বেরোয় না। L7 ব্যালান্সার HTTP বোঝে — URL, হেডার, কুকি দিয়ে রাউট করতে পারে, TLS শেষ করতে পারে; L4 ব্যালান্সার শুধু TCP/UDP দেখে — দ্রুত, কিন্তু কনটেন্ট-ভিত্তিক সিদ্ধান্ত নিতে পারে না।

৮.৭ হেলথ চেক ও ব্যাকএন্ড সার্ভিস

লোড ব্যালান্সার কীভাবে জানে কোন VM সুস্থ? হেলথ চেক দিয়ে: নির্দিষ্ট বিরতিতে প্রতিটি ব্যাকএন্ডের একটি পাথে (যেমন `/healthz`) অনুরোধ পাঠায়; সফল উত্তর এলে VM সুস্থ, ব্যর্থ হলে ট্রাফিক বন্ধ। হেলথ চেকের রেঞ্জ (130.211.0.0/22, 35.191.0.0/16) ফায়ারওয়ালে খুলতে হয় — না খুললে সব VM অসুস্থ দেখাবে, যদিও অ্যাপ ঠিক আছে। ব্যাকএন্ড সার্ভিস হলো ব্যাকএন্ডগুলোর দল + নীতি: কোন প্রোটোকল, session affinity আছে কি না, timeout কত, connection draining কতক্ষণ। **Connection draining** মানে: VM সরানোর সময় চলমান সংযোগগুলো শেষ হতে দেওয়া — হঠাৎ কেটে না দেওয়া।

> **CONCEPT:** হেলথ চেক অ্যাপের স্বাস্থ্য মাপে, VM-এর নয়। `/healthz` এন্ডপয়েন্টে শুধু "সার্ভার চলছে" নয় — ডাটাবেস সংযোগ ঠিক আছে কি না, তাও যাচাই করা উচিত। নইলে ডাটাবেস মরে গেলেও লোড ব্যালান্সার VM-কে সুস্থ ভেবে ট্রাফিক পাঠাতে থাকবে।

৮.৮ Cloud CDN

Cloud CDN লোড ব্যালান্সারের সঙ্গে যুক্ত হয়ে স্ট্যাটিক কনটেন্ট (ছবি, CSS, JS, ভিডিও) Google-এর বিশ্বজোড়া এজে ক্যাশ করে। ব্যবহারকারী ঢাকায় বসে অনুরোধ করলে কনটেন্ট ঢাকার কাছাকাছি এজ থেকে আসে — মূল সার্ভারে (origin) যেতে হয় না। ফলে ল্যাটেন্সি কমে, origin-এর লোড কমে। ক্যাশ কী (cache key) দিয়ে ঠিক করা যায় কোন অনুরোধ আলাদা ক্যাশ এন্ট্রি পাবে; cache mode দিয়ে ঠিক করা যায় origin-এর Cache-Control হেডার মানা হবে কি না। ডায়নামিক কনটেন্ট (ব্যবহারকারী-নির্দিষ্ট পেজ) ক্যাশ করবেন না — ভুল ব্যবহারকারীর ডেটা অন্যের কাছে যেতে পারে।

৮.৯ TLS, সার্টিফিকেট ও WAF

HTTPS-এর জন্য সার্টিফিকেট লাগে। Google-managed SSL সার্টিফিকেট সবচেয়ে সহজ: ডোমেইন যাচাই করে Google নিজেই সার্টিফিকেট দেয় ও নবায়ন করে — আপনাকে কিছু করতে হয় না। শুধু DNS-এ ডোমেইনটি লোড ব্যালান্সারের IP-তে পয়েন্ট করতে হয়। Self-managed সার্টিফিকেট তখনই লাগে যখন আপনার নিজস্ব CA বা বিশেষ প্রয়োজন আছে। TLS নীতি দিয়ে পুরোনো, দুর্বল TLS সংস্করণ নিষিদ্ধ করুন।

WAF (Web Application Firewall) হিসেবে GCP-তে Cloud Armor আছে: এটি লোড ব্যালান্সারের সামনে বসে SQL injection, XSS-এর মতো পরিচিত আক্রমণের প্যাটার্ন, নির্দিষ্ট দেশ বা IP থেকে আসা ট্রাফিক, আর rate limiting দিয়ে আটকায়। Cloud Armor কোনো জাদু নয় — এটি নিয়মের সেট; আপনার অ্যাপের দুর্বলতা এটি ঠিক করে দেয় না, কিন্তু পরিচিত আক্রমণের ঢেউ থামায়।

```
# Global external HTTPS LB: ব্যাকএন্ড, হেলথ চেক, URL map
gcloud compute health-checks create http technova-hc \
  --port=80 --request-path=/healthz \
  --check-interval=10s --timeout=5s \
  --healthy-threshold=2 --unhealthy-threshold=3

gcloud compute backend-services create backend-web \
  --protocol=HTTP --port-name=http --health-checks=technova-hc \
  --global --enable-cdn --cache-mode=CACHE_ALL_STATIC

gcloud compute backend-services add-backend backend-web \
  --instance-group=technova-mig --instance-group-zone=asia-southeast1-a \
  --global --balancing-mode=UTILIZATION --max-utilization=0.8
```

```
gcloud compute url-maps create technova-urlmap \
  --default-service=backend-web

# Google-managed সার্টিফিকেট
gcloud compute ssl-certificates create technova-cert \
  --domains=app.technova.example --global

gcloud compute target-https-proxies create technova-https-proxy \
  --url-map=technova-urlmap --ssl-certificates=technova-cert --global

gcloud compute forwarding-rules create technova-https-rule \
  --target-https-proxy=technova-https-proxy \
  --ports=443 --global
```

Console-এ একই কাজ: Load balancing → Create load balancer → Application Load Balancer (global external) → ধাপে ধাপে frontend, backend, routing ঠিক করুন। Console নতুনদের জন্য ভালো; কিন্তু পুনরাবৃত্তির জন্য উপরের কমান্ড বা Terraform-ই নির্ভরযোগ্য।

৮.১০ সিদ্ধান্ত-কাঠামো: ট্রাফিক সমাধান বেছে নেওয়া

- পাবলিক ওয়েব অ্যাপ, একাধিক রিজিয়ন → Global external HTTPS LB + Cloud CDN + Cloud Armor।
- শুধু এক রিজিয়ন, পাবলিক TCP সার্ভিস → Regional external LB।
- VPC-র ভেতরে মাইক্রোসার্ভিস → Internal LB (L7 হলে HTTP রাউটিং, L4 হলে TCP)।
- স্ট্যাটিক সাইট → Cloud Storage বাকেট + backend bucket + CDN (কোনো VM লাগে না)।
- TLS পাস-থ্রু দরকার (LB TLS ভাঙবে না) → TCP/SSL proxy LB।

৮.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র লঞ্চ

TechNova লঞ্চের দিন ট্রাফিক দশ গুণ বেড়ে গেল। Global LB স্বয়ংক্রিয়ভাবে ট্রাফিক দুই রিজিয়নের ব্যাকএন্ডে ছড়িয়ে দিল; CDN স্ট্যাটিক কনটেন্ট এজ থেকে দিল, origin-এর লোড বাড়ল না; একটি VM-এর ডাটাবেস সংযোগ নষ্ট হলে হেলথ চেক তাকে সরিয়ে দিল — ব্যবহারকারী কিছু টের পেল না। Cloud Armor একটি বট-আক্রমণের ঢেউ rate limit দিয়ে থামাল। এই পুরো ব্যবস্থার কোনো একক অংশই একা কাজটি করতে পারত না — DNS, LB, CDN ও WAF একসঙ্গে মিলেই resilience দেয়।

লঞ্চের আগে TechNova যা যা করেছিল, তা-ই আসল শিক্ষা। প্রথমে লোড টেস্ট: staging-এ কৃত্রিম ট্রাফিক দিয়ে দেখা হয়েছিল লোড ব্যালান্সার কত অনুরোধ সামলায়, ব্যাকএন্ডের ল্যাটেন্সি কোথায় বাড়ে, আর autoscaler কত দ্রুত স্কেল করে। দ্বিতীয়ত, DNS প্রস্তুতি: লঞ্চের এক সপ্তাহ আগে TTL ৩০০ সেকেন্ডে নামিয়ে রাখা হয়েছিল — যাতে জরুরি DNS পরিবর্তন দ্রুত ছড়ায়। তৃতীয়ত, CDN প্রি-ওয়ার্মিং: জনপ্রিয় পেজ ও ছবিগুলো আগেই এজে ক্যাশ করা হয়েছিল, যাতে প্রথম ব্যবহারকারীরা ধীর অভিজ্ঞতা না পান। চতুর্থত, Cloud Armor-এর নিয়ম পরীক্ষা: rate limit এমনভাবে বসানো হয়েছিল যাতে আসল ব্যবহারকারী আটকে না যায় — সীমা খুব কম রাখলে বৈধ ট্রাফিকও আটকায়, খুব বেশি রাখলে আক্রমণ থামে না। পঞ্চমত, পর্যবেক্ষণ: লঞ্চের দিন একটি ড্যাশবোর্ডে অনুরোধের হার, 5xx, ল্যাটেন্সি ও ব্যাকএন্ড স্বাস্থ্য একসঙ্গে দেখা হচ্ছিল — আর প্রতিটি মেট্রিকের অ্যালার্ট আগেই বসানো ছিল। লঞ্চের সাফল্য সেদিনের সিদ্ধান্ত নয় — আগের সপ্তাহগুলোর প্রস্তুতির ফল।

৮.১২ নিরাপত্তা ও খরচ

নিরাপত্তা: HTTP থেকে HTTPS-এ redirect বাধ্যতামূলক করুন; HSTS হেডার দিন; Cloud Armor-এ rate limiting বসান; প্রাইভেট জোনের রেকর্ড পাবলিক জোনে ফাঁস করবেন না। খরচ: লোড ব্যালান্সারের প্রতি-ঘণ্টা চার্জ + প্রসেস করা ডেটা; CDN-এ cache egress চার্জ (origin থেকে এজে যাওয়া ডেটা) — কিন্তু cache hit বেশি হলে মোট খরচ কমে, কারণ origin egress বাঁচে। এখানেও দাম বদলায় — Google Cloud Pricing Calculator-ই নির্ভরযোগ্য উৎস। ইন্টারনেট egress সবসময় নজরে রাখুন: এটি প্রায়ই সবচেয়ে বড় নেটওয়ার্ক খরচ।

নিরাপত্তার আরও একটি দিক: লোড ব্যালান্সার ব্যবহারকারীর আসল IP দেখে না — GFE-র IP দেখে। তাই IP-ভিত্তিক rate limiting বা ভৌগোলিক নিষেধাজ্ঞা বসাতে হলে Cloud Armor-কে `X-Forwarded-For` হেডারের আসল IP ব্যবহার করতে বলুন — নইলে আপনি GFE-র IP-কেই সীমিত করবেন, যা সব ট্রাফিককে প্রভাবিত করবে। আরেকটি বিষয়: TLS-এর ক্ষেত্রে সর্বনিম্ন সংস্করণ ঠিক করুন — TLS 1.0 ও 1.1 নিষিদ্ধ করুন; পুরোনো ক্লায়েন্ট হারানোর ঝুঁকি নিলেও দুর্বল এনক্রিপশন রাখা ঠিক নয়। সার্টিফিকেটের মেয়াদ নিয়ে ভাবতে হয় না Google-managed সার্টিফিকেটে — Google নিজে নবায়ন করে; কিন্তু self-managed সার্টিফিকেট ব্যবহার করলে মেয়াদ শেষের অ্যালার্ট বসান — মেয়াদোত্তীর্ণ সার্টিফিকেটে পুরো সাইট ডাউন হয়ে যায়, আর এটি ঘটে সবচেয়ে ব্যস্ত দিনেই। খরচের দিকে: বড় ফাইল (ভিডিও, সফটওয়্যার ডাউনলোড) CDN-এর মাধ্যমে দিলে cache hit বাড়ে, কিন্তু cache miss-এর খরচও আছে — origin থেকে এজে যাওয়া ডেটার চার্জ। তাই ক্যাশের TTL ঠিক করুন কনটেন্টের পরিবর্তনের হারের সঙ্গে মিলিয়ে: ঘন ঘন বদলানো কনটেন্টে ছোট TTL, স্থায়ী কনটেন্টে বড় TTL। আর মনে রাখবেন: একটি লোড ব্যালান্সার যত ছোটই হোক, তার প্রতি-ঘণ্টা চার্জ আছে — পরীক্ষার পরিবেশে অপ্রয়োজনীয় লোড ব্যালান্সার চালিয়ে রাখবেন না।

৮.১৩ সাধারণ ভুল

- হেলথ-চেক রেঞ্জ ফায়ারওয়ালে না খোলা — সব ব্যাকএন্ড "unhealthy"।
- `/` পাথে হেলথ চেক করা — ভারী হোমপেজ প্রতিবার লোড হয়; হালকা `/healthz` বানান।
- ডায়নামিক কনটেন্ট CDN-এ ক্যাশ করা।
- TTL অনেক বড় রাখা — DNS পরিবর্তন ঘণ্টার পর ঘণ্টা ছড়াতে দেরি হয়।
- একটিমাত্র রিজিয়নে ব্যাকএন্ড রেখে "global" LB-র সুবিধা না নেওয়া।
- সার্টিফিকেটের ডোমেইন যাচাই না হওয়া অবস্থায় LB চালু করে "কাজ করছে না" ভাবা — DNS প্রোপাগেশন সময় নেয়।

আরেকটি সাধারণ ভুল হলো লোড ব্যালান্সারের সেটিংস না বুঝে ডিফল্ট রেখে দেওয়া। যেমন session affinity: ডিফল্টভাবে বন্ধ থাকে — প্রতিটি অনুরোধ যেকোনো ব্যাকএন্ডে যেতে পারে। আপনার অ্যাপ যদি সেশন ডেটা VM-এর মেমোরিতে রাখে, তাহলে ব্যবহারকারী প্রতিটি অনুরোধে আলাদা VM-এ গিয়ে লগআউট হয়ে যাবে। সমাধান দুটি: session affinity চালু করা (যা স্কেলিংকে অসমান করে), অথবা সেশন ডেটা বাইরে রাখা (Memorystore বা ডাটাবেসে) — দ্বিতীয়টিই সঠিক। আরেকটি সেটিং: backend timeout — ডিফল্ট ৩০ সেকেন্ড। আপনার API-র কোনো অনুরোধ যদি নিয়মিত ৩০ সেকেন্ডের বেশি নেয়, তাহলে লোড ব্যালান্সার 502 দেবে, যদিও ব্যাকএন্ড ঠিকঠাক কাজ করছে। Timeout ঠিক করুন আপনার সবচেয়ে ধীর বৈধ অনুরোধের সঙ্গে মিলিয়ে — আর সেই সঙ্গে অ্যাপের ধীরতা কমানোর কাজও করুন, কারণ timeout বাড়ানো সমস্যার সমাধান নয়, শুধু লক্ষণ ঢাকা।

৮.১৪ Troubleshooting

TROUBLESHOOTING: 502 Bad Gateway? ব্যাকএন্ড সুস্থ কি না দেখুন (`gcloud compute backend-services get-health`) — অসুস্থ হলে হেলথ চেকের পথ ও ফায়ারওয়াল দেখুন। 404? URL map-এর রাউটিং দেখুন। DNS রিজলভ হচ্ছে না? `dig app.technova.example` দিয়ে দেখুন কোন নেমসার্ভার উত্তর দিচ্ছে — জোনের নেমসার্ভার ডোমেইন রেজিস্ট্রারে বসানো হয়েছে কি না যাচাই করুন। CDN-এ পুরোনো কনটেন্ট? Cache invalidation করুন — তবে মনে রাখবেন invalidation-এরও সীমা ও খরচ আছে, তাই versioned ফাইলনাম (যেমন `app.v2.js`) ব্যবহার করা ভালো অভ্যাস।

৮.১৫ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

লোড ব্যালান্সারের ভেতরে কী ঘটে, তা বুঝুন। Global external HTTPS লোড ব্যালান্সার আসলে দুটি অংশে বিভক্ত: প্রথমে Google Front End (GFE) — বিশ্বজোড়া এজে বসা প্রক্সি, যা TLS শেষ করে, Cloud Armor নিয়ম প্রয়োগ করে, আর URL map অনুযায়ী রাউট ঠিক করে; তারপর ট্রাফিক Google ব্যাকবোন দিয়ে ব্যাকএন্ডে যায়। এর মানে আপনার VM কখনো সরাসরি ইন্টারনেটের মুখোমুখি হয় না — GFE-ই ঢাল। কিন্তু এর একটি পরিণতি আছে: VM-এ পৌঁছানো অনুরোধের উৎস IP আসল ব্যবহারকারীর IP নয়, GFE-র IP — আসল IP পেতে `X-Forwarded-For` হেডার পড়তে হয়, আর অ্যাপের লগে সেটি ব্যবহার করতে হয়।

DNS-এর ব্যর্থতার ধরন আলাদা: Cloud DNS নিজে অত্যন্ত নির্ভরযোগ্য, কিন্তু ভুল রেকর্ড, ভুল TTL, বা রেজিস্ট্রারে ভুল নেমসার্ভার — এগুলোই আসল ব্যর্থতার কারণ। একটি সাধারণ বিপদ: ডোমেইন ট্রান্সফারের সময় পুরোনো নেমসার্ভার মুছে ফেলা, অথচ Cloud DNS-এ জোন তৈরি না করা — তখন ডোমেইনটি সম্পূর্ণ অদৃশ্য হয়ে যায়। তাই পরিবর্তনের আগে সবসময় `dig +trace` দিয়ে পুরো চেইন যাচাই করুন। CDN-এর ব্যর্থতা সাধারণত ক্যাশের ভুল: পুরোনো কনটেন্ট আটকে থাকা, বা ভুল cache key-এর কারণে এক ব্যবহারকারীর পেজ অন্যের কাছে যাওয়া। সমাধান: versioned URL, সঠিক Cache-Control হেডার, আর ডায়নামিক পেজে `Cache-Control: private, no-store` ।

পর্যবেক্ষণ: লোড ব্যালান্সারের Cloud Monitoring মেট্রিক — অনুরোধের সংখ্যা, ল্যাটেন্সি (ব্যাকএন্ড বনাম মোট), 5xx-এর হার — এগুলো দিয়ে অ্যালার্ট বসান। ব্যাকএন্ডের স্বাস্থ্য `get-health` দিয়ে দেখুন; হেলথ চেকের ব্যর্থতার কারণ প্রায়ই ফায়ারওয়াল বা অ্যাপের `/healthz` এন্ডপয়েন্ট। বিলিং: লোড ব্যালান্সারের চার্জের বড় অংশ ডেটা প্রসেসিং — বড় ফাইল ডাউনলোডের সার্ভিসে CDN-এর cache hit বাড়ানো সরাসরি বিল কমায়। বিকল্প: খুব ছোট অ্যাপে সরাসরি VM-এর পাবলিক IP (কখনোই সুপারিশ নয়), বা Cloud Run-এর মতো সার্ভারলেস সার্ভিস — যেখানে লোড ব্যালান্সিং অন্তর্নির্মিত।

৮.১৬ সারাংশ

DNS নাম দেয়, লোড ব্যালান্সার ট্রাফিক বিতরণ করে, CDN দূরত্ব কমায়, TLS ও Cloud Armor সুরক্ষা দেয়। সঠিক ব্যালান্সার বেছে নেওয়া, হেলথ চেক ঠিক করা, আর CDN-এ শুধু ক্যাশযোগ্য কনটেন্ট রাখা — এই তিনটি অভ্যাসই ট্রাফিক স্তরকে নির্ভরযোগ্য করে।

৮.১৭ পর্যালোচনা প্রশ্ন

- Global ও regional load balancer-এর পার্থক্য কখন গুরুত্বপূর্ণ হয়?
- L7 ব্যালান্সার L4 ব্যালান্সারের চেয়ে কী বেশি করতে পারে, আর তার মূল্য কী?
- হেলথ চেক কেন `/healthz` -এর মতো হালকা এন্ডপয়েন্টে করা উচিত?

- Cloud Armor কী আটকাতে পারে, আর কী পারে না?
- CDN cache hit বাড়লে খরচ কীভাবে কমে?

৮.১৮ ব্যবহারিক অনুশীলন

- একটি পাবলিক DNS জোন বানিয়ে A রেকর্ড যোগ করুন; `dig` দিয়ে যাচাই করুন। একটি প্রাইভেট জোন বানিয়ে VPC-র ভেতর থেকে রিজলভ করুন।
- দুটি VM-এর সামনে একটি global external HTTP LB বানান; একটি VM বন্ধ করে দেখুন ট্রাফিক অন্যটিতে যায় কি না।
- Cloud CDN চালু করে একটি ছবি দুবার অনুরোধ করুন — প্রথমবার MISS, দ্বিতীয়বার HIT হয় কি না response হেডার দিয়ে যাচাই করুন।

অধ্যায় ৯ • PART II — Networking & Compute Engineering

Chapter 9 — Compute Engine Engineering

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Compute Engine-এর আর্কিটেকচার ব্যাখ্যা করতে — হোস্ট, হাইপারভাইজার ও লাইভ মাইগ্রেশন; (২) মেশিন ফ্যামিলি ও টাইপের মধ্যে বেছে নিতে; (৩) VM ইমেজ, Persistent Disk ও Hyperdisk-এর পার্থক্য বুঝতে; (৪) ইনস্ট্যান্স লাইফসাইকেল, স্টার্টআপ স্ক্রিপ্ট ও মেটাডেটা ব্যবহার করতে; (৫) SSH ও OS Login দিয়ে নিরাপদে VM-এ ঢুকতে; (৬) নিরাপদ VM কনফিগারেশন বানাতে ও সাধারণ সমস্যার রোগ নির্ণয় করতে।

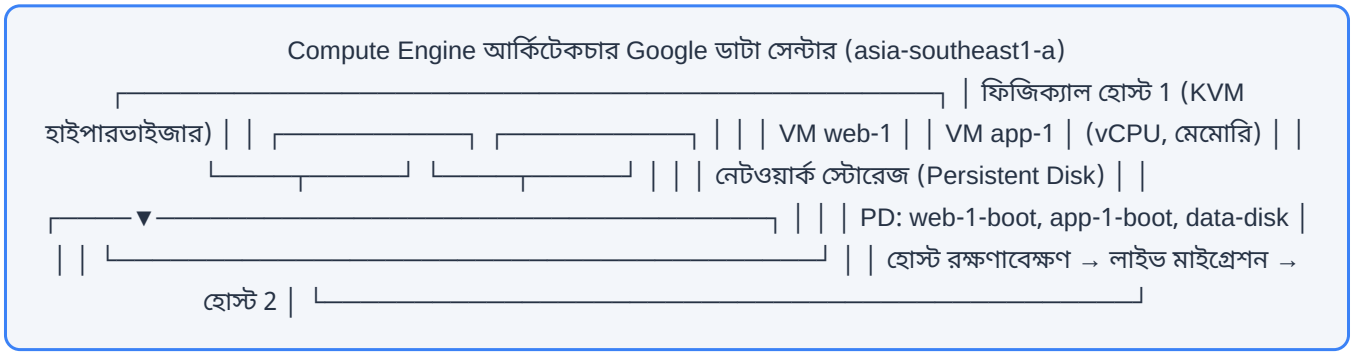
৯.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আপনি জানেন: VM হলো ফিজিক্যাল সার্ভারের ওপরে চলা ভার্চুয়াল সার্ভার; vCPU হলো ভার্চুয়াল প্রসেসর কোর, মেমোরি হলো RAM; ডিস্ক হলো স্থায়ী সংরক্ষণ। GCP-তে VM-কে বলে instance, আর সেটি চলে একটি জোনে (যেমন `asia-southeast1-a`)। মনে রাখুন: জোন হলো একটি রিজিয়নের ভেতরের আলাদা ফিজিক্যাল অবস্থান — একটি জোনের সমস্যা অন্য জোনকে প্রভাবিত করে না।

৯.৩ প্রথম নীতি: Compute Engine কীভাবে কাজ করে

Google-এর ডাটা সেন্টারে ফিজিক্যাল সার্ভার (হোস্ট) থাকে। KVM-ভিত্তিক হাইপারভাইজার সেই হোস্টকে ভাগ করে অনেক VM-এ — প্রতিটি VM নিজের vCPU, মেমোরি ও ডিস্ক নিয়ে আলাদা অপারেটিং সিস্টেম চালায়। আপনার VM-এর ডিস্ক আসলে নেটওয়ার্ক-সংযুক্ত স্টোরেজ — হোস্টের লোকাল ডিস্ক নয় (Local SSD ছাড়া)। এর ফলে VM এক হোস্ট থেকে আরেক হোস্টে সরানো যায় — এটাই **লাইভ মাইগ্রেশন**: হোস্ট রক্ষণাবেক্ষণের সময় আপনার VM রিবুট ছাড়াই অন্য হোস্টে চলে যায়, আপনি টেরও পান না।

এই আর্কিটেকচারের মূল্য: VM-এর ডিস্ক হোস্ট থেকে আলাদা বলে হোস্ট মারা গেলেও ডিস্ক বেঁচে থাকে; আর লাইভ মাইগ্রেশনের কারণে Google রক্ষণাবেক্ষণ করতে পারে আপনার সার্ভিস না থামিয়ে। কিন্তু এর সীমাও আছে: নেটওয়ার্ক ডিস্কের ল্যাটেন্সি লোকাল ডিস্কের চেয়ে বেশি — তাই অত্যন্ত কম ল্যাটেন্সির কাজে Local SSD বা Hyperdisk-এর কথা ভাবতে হয়।



৯.৪ মেশিন ফ্যামিলি ও মেশিন টাইপ

মেশিন ফ্যামিলি হলো কাজের ধরন অনুযায়ী VM-এর শ্রেণি। সাধারণ উদ্দেশ্য (general-purpose): E2 (সাশ্রয়ী, ছোট কাজ), N2/N2D (ভারসাম্যপূর্ণ), N4 (নতুন প্রজন্ম), C4 (কম্পিউট-অপ্টিমাইজড, বেশি CPU), M সিরিজ (মেমোরি-অপ্টিমাইজড

— বড় ডাটাবেস, SAP)। এছাড়া আছে GPU-যুক্ত A সিরিজ (AI/ML), আর Tau T2A (Arm-ভিত্তিক, সাশ্রয়ী)। মেশিন টাইপ যেমন `e2-standard-4` মানে ৪ vCPU, ১৬ GB মেমোরি।

বেছে নেওয়ার নিয়ম: প্রথমে কাজের ধরন দেখুন — CPU-ভারী (ভিডিও এনকোডিং) হলে C4, মেমোরি-ভারী (ইন-মেমোরি ক্যাশ) হলে M সিরিজ, সাধারণ ওয়েব সার্ভার হলে E2 বা N2। তারপর আকার: শুরুতে ছোট নিন, মনিটরিং দেখে বাড়ান — VM-এর আকার পরে বদলানো যায় (থামিয়ে)। আর খরচ কমাতে: **Spot VM** (আগে Preemptible) — Google যেকোনো সময় ফিরিয়ে নিতে পারে, কিন্তু দাম অনেক কম; ব্যাচ কাজ, বিল্ড, পরীক্ষার জন্য আদর্শ, প্রোডাকশন ওয়েব সার্ভারের জন্য নয়। **Committed use discount** — এক বা তিন বছরের প্রতিশ্রুতিতে ছাড়; স্থায়ী লোডের জন্য।

ফ্যামিলি	বৈশিষ্ট্য	উপযুক্ত কাজ
E2	সাশ্রয়ী, শেয়ার্ড-কোর বিকল্প	ছোট ওয়েব সার্ভার, ডেভ, পরীক্ষা
N2 / N4	ভারসাম্যপূর্ণ CPU-মেমোরি	সাধারণ অ্যাপ সার্ভার
C4	বেশি CPU, কম মেমোরি অনুপাত	কম্পিউট-ভারী কাজ
M সিরিজ	বড় মেমোরি	বড় ডাটাবেস, ইন-মেমোরি অ্যানালিটিক্স
A সিরিজ (GPU)	GPU যুক্ত	AI প্রশিক্ষণ, ইনফারেন্স

৯.৫ VM ইমেজ

ইমেজ হলো VM-এর বুট ডিস্কের ছবি — অপারেটিং সিস্টেম ও সফটওয়্যারসহ। Google পাবলিক ইমেজ দেয় (Debian, Ubuntu, Windows Server, Container-Optimized OS)। নিজের ইমেজ বানানো যায়: একটি VM ঠিকঠাক সেট আপ করে তার কাস্টম ইমেজ তৈরি করুন — তারপর সেই ইমেজ থেকে যত খুশি VM বানানো যায়, প্রতিটি একইভাবে কনফিগারড। ইমেজ ফ্যামিলি (যেমন `ubuntu-2204-lts`) ব্যবহার করলে সবসময় সর্বশেষ প্যাচ পাওয়া যায়। প্রোডাকশনে কাস্টম ইমেজ + স্টার্টআপ স্ক্রিপ্টের সমন্বয়ই সঠিক পথ: ইমেজে স্থায়ী জিনিস, স্ক্রিপ্টে পরিবর্তনশীল জিনিস।

৯.৬ Persistent Disk ও Hyperdisk

Persistent Disk (PD) হলো নেটওয়ার্ক-সংযুক্ত ব্লক স্টোরেজ: standard (HDD, সস্তা, ধীর), balanced (SSD, ভারসাম্য), SSD (দ্রুত), extreme (খুব দ্রুত)। ডিস্ক VM থেকে আলাদা করে রাখা যায়, স্ন্যাপশট নেওয়া যায়, আকার বাড়ানো যায় (কমানো যায় না)। **Hyperdisk** হলো নতুন প্রজন্ম: Balanced ও Extreme ধরন, আরও নমনীয় IOPS/থ্রুপুট কনফিগারেশন, আর মাল্টি-রাইটের মতো সুবিধা কিছু ক্ষেত্রে। নতুন ডিজাইনে Hyperdisk Balanced-ই সাধারণত সঠিক শুরু — তবে আপনার মেশিন টাইপ ও রিজিয়নে এর উপলব্ধতা যাচাই করে নিন।

স্ন্যাপশট হলো ডিস্কের সময়-ভিত্তিক ব্যাকআপ — ইনক্রিমেন্টাল, তাই প্রথমটির পর বাকিগুলো সস্তা। স্ন্যাপশট শিডিউল বসান: প্রতিদিন রাতে, ধরে রাখার নীতি (retention) সহ। আর একটি কথা: বুট ডিস্ক মুছে ফেললে VM-ও শেষ — ডিস্কের `auto-delete` সেটিং দেখে নিন।

৯.৭ ইনস্ট্যান্স লাইফসাইকেল

VM-এর অবস্থা: PROVISIONING → STAGING → RUNNING → (STOPPING → TERMINATED)। RUNNING অবস্থায় আপনি থামাতে (stop), রিবুট করতে, বা মুছতে পারেন। থামানো VM-এর vCPU/মেমোরির চার্জ বন্ধ হয়, কিন্তু ডিস্কের চার্জ চলতে থাকে। মুছে ফেলা VM ফিরে আসে না — ডিস্ক auto-delete না হলে শুধু ডিস্ক বেঁচে থাকে। হোস্ট

রক্ষণাবেক্ষণের সময় দুটি আচরণ: MIGRATE (লাইভ মাইগ্রেশন — ডিফল্ট) বা TERMINATE (VM থামে, পরে আবার চালু হয়)। প্রোডাকশনে MIGRATE রাখুন; বিশেষ ক্ষেত্রে (যেমন GPU VM) TERMINATE লাগতে পারে।

৯.৮ স্টার্টআপ স্ক্রিপ্ট ও মেটাডেটা

মেটাডেটা হলো VM-এর সঙ্গে যুক্ত কী-ভ্যালু তথ্য — যেমন প্রজেক্ট-ব্যাপী SSH কী, বা ইনস্ট্যান্স-নির্দিষ্ট কনফিগারেশন। স্টার্টআপ স্ক্রিপ্ট হলো মেটাডেটায় রাখা একটি স্ক্রিপ্ট, যা VM বুট হওয়ার সময় চলে — সফটওয়্যার ইনস্টল, কনফিগারেশন লেখা, সার্ভিস চালু করা। এটি VM-কে পুনরুৎপাদনযোগ্য করে: একই স্ক্রিপ্ট দিয়ে দশটি VM একইভাবে তৈরি হয়। সিক্রেট (পাসওয়ার্ড, API কী) কখনোই স্টার্টআপ স্ক্রিপ্টে লিখবেন না — Secret Manager থেকে VM চালু হওয়ার পর নিন, আর স্ক্রিপ্টে CHANGE-ME-... ধরনের প্লেসহোল্ডার ব্যবহার করুন।

```
# VM তৈরি: e2-standard-2, Ubuntu, স্টার্টআপ স্ক্রিপ্টসহ, পাবলিক IP ছাড়া
gcloud compute instances create web-1 \
  --zone=asia-southeast1-a \
  --machine-type=e2-standard-2 \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --network=technova-vpc \
  --subnet=subnet-web \
  --no-address \
  --tags=web-server \
  --metadata=startup-script='#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl enable nginx
systemctl start nginx' \
  --metadata=app-env=production \
  --maintenance-policy=MIGRATE \
  --scopes=https://www.googleapis.com/auth/cloud-platform

# ডিস্ক যোগ করা
gcloud compute disks create data-disk-1 \
  --zone=asia-southeast1-a --size=100GB --type=pd-balanced

gcloud compute instances attach-disk web-1 \
  --disk=data-disk-1 --zone=asia-southeast1-a

# স্ন্যাপশট শিডিউল
gcloud compute resource-policies create snapshot-schedule daily-backup \
  --region=asia-southeast1 \
  --max-retention-days=14 \
  --on-source-disk-delete=keep-auto-snapshots \
  --daily-schedule --start-time=02:00
```

৯.৯ SSH ও OS Login

VM-এ ঢোকার নিরাপদ উপায় তিনটি: (১) `gcloud compute ssh` — আপনার Google পরিচয় দিয়ে অস্থায়ী SSH কী বানিয়ে ঢোকে; (২) **OS Login** — IAM-এর সঙ্গে যুক্ত SSH ব্যবস্থা: কে কোন VM-এ ঢুকতে পারবে তা IAM রোল দিয়ে নিয়ন্ত্রিত হয়, SSH কী Google অ্যাকাউন্টের সঙ্গে বাঁধা থাকে; (৩) **IAP tunneling** — পাবলিক IP ছাড়াই, Identity-Aware Proxy দিয়ে SSH: `gcloud compute ssh --tunnel-through-iap`। প্রোডাকশনে OS Login + IAP-ই সুপারিশ: কোনো পাবলিক IP নেই, কোনো স্থায়ী SSH কী ছড়িয়ে নেই, আর কে ঢুকেছে তার অডিট লগ থাকে।

> **সতর্কতা:** প্রজেক্ট-ব্যাপী মেটাডেটায় SSH কী বসানো পুরোনো পদ্ধতি — এতে যে কেউ সেই কী দিয়ে সব VM-এ ঢুকতে পারে। OS Login ব্যবহার করুন, আর `enable-oslogin=TRUE` প্রজেক্ট মেটাডেটায় বসান। পাসওয়ার্ড-ভিত্তিক SSH কখনোই চালু করবেন না।

৯.১০ নিরাপদ VM কনফিগারেশন

- পাবলিক IP দেবেন না — IAP দিয়ে ঢুকুন।
- Service account-এ least privilege: VM-কে শুধু যে API-গুলো দরকার, সেগুলোর অনুমতি দিন; `cloud-platform` স্কোপ দিলেও IAM রোলে সীমাবদ্ধ রাখুন।
- Shielded VM চালু করুন: Secure Boot, vTPM, integrity monitoring — বুট প্রক্রিয়া কেউ বদলাতে পারলে ধরা পড়ে।
- OS patch management চালু করুন — নিয়মিত প্যাচ, রিবুটের সময়সূচীসহ।
- Serial console অ্যাক্সেস সীমিত রাখুন; এটি ভাঙা VM-এর শেষ দরজা।
- Confidential VM বিবেচনা করুন — যদি মেমোরিতে ডেটা এনক্রিপ্টেড থাকা দরকার হয়।

৯.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র অ্যাপ সার্ভার

TechNova-র app সার্ভার: কাস্টম ইমেজ থেকে তৈরি, e2-standard-4, Hyperdisk Balanced বুট ডিস্ক, আলাদা data ডিস্ক, স্টার্টআপ স্ক্রিপ্টে অ্যাপ ডাউনলোড ও চালু, OS Login + IAP দিয়ে অ্যাক্সেস, দৈনিক স্ন্যাপশট। VM-টি একটি Managed Instance Group-এ (অধ্যায় ১১) — একা নয়, দলের অংশ হিসেবে। কোনো VM মারা গেলে গ্রুপটি নতুন VM বানায় — একক VM-এর ওপর নির্ভরতা নেই।

এই VM-কে প্রোডাকশনে নেওয়ার আগে TechNova আরও কয়েকটি বিষয় নিশ্চিত করেছিল। প্রথমত, সাইজিং: staging-এ লোড টেস্ট করে দেখা হয়েছিল e2-standard-2 যথেষ্ট কি না — দেখা গেল CPU ৮০% ছাড়িয়ে যায়, তাই e2-standard-4 নেওয়া হলো; অনুমানে নয়, মাপে সিদ্ধান্ত। দ্বিতীয়ত, ইমেজের জীবনচক্র: কাস্টম ইমেজের সংস্করণ নম্বর দেওয়া হয় (v1, v2...), আর প্রতিটি সংস্করণের বিল্ড স্ক্রিপ্ট Git-এ থাকে — কোন VM কোন ইমেজ থেকে বানানো, তা সবসময় জানা যায়। তৃতীয়ত, ব্যাকআপ পরীক্ষা: স্ন্যাপশট নেওয়া হয়, কিন্তু সেটি থেকে VM ফিরিয়ে আনার পরীক্ষাও মাসে একবার করা হয় — ব্যাকআপ যা কখনো পরীক্ষা করা হয়নি, তা ব্যাকআপ নয়, শুধু আশা। চতুর্থত, প্যাচের সময়সূচী: OS patch management দিয়ে প্রতি মাসের প্রথম রবিবার রাত ২টায় প্যাচ বসে, আর MIG rolling update-এর মাধ্যমে VM-গুলো ধীরে ধীরে বদলায় — ব্যবহারকারী টের পায় না। পঞ্চমত, খরচের হিসাব: VM-টি সারা মাস চলে বলে committed use discount-এর আওতায় আনা হয়েছিল — এক বছরের প্রতিশ্রুতিতে উল্লেখযোগ্য ছাড়। এই পাঁচটি অভ্যাস — মাপা সাইজিং, সংস্করণযুক্ত ইমেজ, পরীক্ষিত ব্যাকআপ, নিয়মিত প্যাচ, আর প্রতিশ্রুতি-ভিত্তিক ছাড় — একটি VM-কে "চলছে" থেকে "প্রোডাকশন-যোগ্য"-তে নিয়ে যায়।

৯.১২ নিরাপত্তা ও খরচ

নিরাপত্তা: উপরের তালিকা ছাড়াও — VM-এর service account-এর কী কখনোই VM-এর ভেতরে ফাইলে রাখবেন না; Workload Identity বা attached service account ব্যবহার করুন। খরচ: VM প্রতি সেকেন্ডে বিল হয় (সর্বনিম্ন এক মিনিট); থামানো VM-এর ডিস্কের চার্জ চলতে থাকে — অপ্রয়োজনীয় ডিস্ক ও স্ন্যাপশট মুছুন; Spot VM ও committed use discount দিয়ে বড় ছাড় পাওয়া যায়; sustained use discount এখন অনেক ক্ষেত্রে স্বয়ংক্রিয়ভাবে প্রযোজ্য হয়। দামের জন্য Google Cloud Pricing Calculator দেখুন।

নিরাপত্তার ক্ষেত্রে একটি প্রায়ই অবহেলিত বিষয়: VM-এর ভেতরের সফটওয়্যার। অপারেটিং সিস্টেম প্যাচ করা থাকলেও অ্যাপের নির্ভরতা (dependency) পুরোনো থাকলে দুর্বলতা থেকে যায় — নিয়মিত vulnerability স্ক্যান চালান, আর Container Analysis বা Artifact Registry-র স্ক্যানিং ব্যবহার করুন। আরেকটি বিষয়: guest OS-এর লগ — কে SSH করেছে, কখন, কোথা থেকে — এই তথ্য Cloud Logging-এ পাঠান; কোনো অস্বাভাবিক লগইন (যেমন মাঝরাতে, অচেনা IP থেকে) হলে অ্যালার্ট বসান। খরচের দিকে: সবচেয়ে বড় সাশ্রয় প্রায়ই সঠিক সাইজিং থেকে আসে — একটি e2-standard-8 VM যেখানে e2-standard-2-ই যথেষ্ট, সেখানে মাসে চার গুণ বিল। Cloud Monitoring-এর recommendation (যেমন rightsizing সুপারিশ) নিয়মিত দেখুন — Google নিজেই বলে দেয় কোন VM ছোট করা যায়। আর একটি কৌশল: উন্নয়ন ও পরীক্ষার VM-গুলো কাজের সময়ের বাইরে বন্ধ রাখুন — একটি শিডিউল দিয়ে রাত ৮টায় থামিয়ে সকাল ৮টায় চালু করলে বিল প্রায় দুই-তৃতীয়াংশ কমে। এই ছোট অভ্যাসগুলো একসঙ্গে মিলে বড় সাশ্রয় হয় — আর সাশ্রয়ের টাকা নতুন ফিচারে খরচ করা যায়।

৯.১৩ সাধারণ ভুল

- সব VM-এ পাবলিক IP দেওয়া — "সুবিধার জন্য"।
- একটিমাত্র জোনে সব VM রাখা — জোনের সমস্যায় পুরো সার্ভিস ডাউন।
- স্ন্যাপশট ছাড়া ডাটা ডিস্ক চালানো।
- স্টার্টআপ স্ক্রিপ্টে সিক্রেট লেখা।
- মেশিন টাইপ না মেপে বড় নেওয়া — মনিটরিং না দেখে অনুমানে সাইজিং।
- বুট ডিস্কের auto-delete না বুঝে VM মুছে ডেটা হারানো।

আরেকটি প্রায়ই অবহেলিত বিষয়: VM-এর ভেতরের সময় (clock)। VM-এর ঘড়ি ভুল থাকলে TLS সার্টিফিকেট যাচাই ব্যর্থ হয়, লগের সময় মেলে না, আর ডাটাবেসের টাইমস্ট্যাম্প গোলমাল করে। Google-এর guest agent NTP সিঙ্ক করে, কিন্তু কাস্টম ইমেজে agent না থাকলে ঘড়ি ধীরে ধীরে সরে যায়। তাই কাস্টম ইমেজ বানালে guest agent ইনস্টল আছে কি না যাচাই করুন। আরেকটি বিষয়: VM-এর লোকাল SSD বনাম PD — Local SSD দ্রুত, কিন্তু VM মুছলে বা হোস্ট বদলালে ডেটা হারায়; এটি শুধু অস্থায়ী ডেটা (ক্যাশ, scratch) এর জন্য। গুরুত্বপূর্ণ ডেটা কখনোই Local SSD-তে রাখবেন না — PD বা Hyperdisk-এ রাখুন, যার স্ন্যাপশট নেওয়া যায়। এই পার্থক্যটি না জানলে একটি VM মুছে পুরো ডেটাসেট হারানোর ঘটনা খুবই সাধারণ।

৯.১৪ Troubleshooting

> **TROUBLESHOOTING:** VM-এ SSH হচ্ছে না? প্রথমে দেখুন VM RUNNING কি না, ফায়ারওয়ালে 22 নম্বর পোর্ট (বা IAP রেঞ্জ 35.235.240.0/20) খোলা কি না, আর OS Login চালু থাকলে আপনার IAM রোল (`roles/compute.osLogin`) আছে কি না। VM বুট হচ্ছে না? Serial console আউটপুট দেখুন (`gcloud compute instances get-serial-port-output`) — ডিস্ক পূর্ণ হলে বা স্টার্টআপ স্ক্রিপ্টে ভুল থাকলে সেখানেই ধরা পড়ে। ডিস্ক পূর্ণ? `df -h` দিয়ে দেখুন; PD-র আকার বাড়ানো যায় VM না থামিয়েই, তবে ফাইলসিস্টেমও বাড়াতে হয় (`resize2fs`)।

৯.১৫ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

VM বুট হওয়ার সময় কী ঘটে? হাইপারভাইজার VM-কে vCPU ও মেমোরি বরাদ্দ করে, বুট ডিস্ক (PD বা Hyperdisk) সংযুক্ত করে, নেটওয়ার্ক ইন্টারফেস বসায়, আর guest OS চালু হয়। তারপর guest agent (Google Guest Agent)

মেটাডেটা সার্ভার (`169.254.169.254`) থেকে তথ্য নেয় — স্টার্টআপ স্ক্রিপ্ট, SSH কী, OS Login তথ্য। এই মেটাডেটা সার্ভার শুধু VM-এর ভেতর থেকে পৌঁছানো যায় — এটি প্রতিটি VM-এর একটি বিশেষ লিঙ্ক-লোকাল ঠিকানা। মেটাডেটা সার্ভারে পৌঁছানো না গেলে স্টার্টআপ স্ক্রিপ্ট চলবে না, OS Login কাজ করবে না — তাই ফায়ারওয়াল বা প্রক্সি কনফিগারেশনে এই ঠিকানা বন্ধ করবেন না।

লাইভ মাইগ্রেশনের ভেতরের কথা: হোস্ট রক্ষণাবেক্ষণের সময় VM-এর মেমোরি ধীরে ধীরে নতুন হোস্টে কপি করা হয়, শেষ মুহূর্তে VM কয়েক মিলিসেকেন্ডের জন্য থামে, তারপর নতুন হোস্টে চলতে থাকে — ডিস্ক যেহেতু নেটওয়ার্ক-সংযুক্ত, তাই ডিস্ক সরানোর দরকার হয় না। কিন্তু মেমোরি-ভারী, দ্রুত বদলানো VM-এ মাইগ্রেশন বেশি সময় নিতে পারে — আর খুব বড় VM-এ মাইগ্রেশন ব্যর্থ হলে VM রিবুট হতে পারে। তাই মিশন-ক্রিটিক্যাল সার্ভিস একক VM-এর ওপর রাখবেন না — অধ্যায় ১১-এর MIG-ই সমাধান।

পর্যবেক্ষণ: Cloud Monitoring-এ VM-এর CPU, মেমোরি, ডিস্ক I/O ও নেটওয়ার্ক দেখুন; Ops Agent ইনস্টল করলে অ্যাপের লগও এক জায়গায় আসে। ডিস্কের ব্যবহার ৮০% ছাড়ালে অ্যালার্ট বসান — ডিস্ক পূর্ণ হওয়া VM ব্যর্থতার সবচেয়ে সাধারণ কারণগুলোর একটি, অথচ সবচেয়ে সহজে প্রতিরোধযোগ্য। বিলিং: VM-এর চার্জ প্রতি সেকেন্ডে — তাই পরীক্ষার VM রাতে বন্ধ রাখুন, বা স্ক্রিপ্ট দিয়ে স্বয়ংক্রিয়ভাবে থামান। অপ্রয়োজনীয় স্ন্যাপশট ও পুরোনো ইমেজ মুছুন — এগুলো চুপচাপ বিল বাড়ায়। বিকল্প: স্থায়ী সার্ভারের জন্য Compute Engine; পরিবর্তনশীল লোডের জন্য MIG; আর পুরোপুরি পরিচালিত পথ চাইলে Cloud Run বা GKE — যেখানে VM পরিচালনার দায়িত্ব Google-এর।

৯.১৬ সারাংশ

Compute Engine হলো নমনীয়, লাইভ-মাইগ্রেশন-সক্ষম ভার্চুয়াল সার্ভার। সঠিক মেশিন ফ্যামিলি, কাস্টম ইমেজ, Hyperdisk/PD, স্টার্টআপ স্ক্রিপ্ট, আর OS Login + IAP — এই পাঁচটি মিলে একটি VM-কে প্রোডাকশন-যোগ্য করে। আর মনে রাখবেন: একক VM কখনোই প্রোডাকশন স্থাপত্য নয় — অধ্যায় ১১-এ দেখবেন কীভাবে সেগুলোকে দলে সাজাতে হয়।

৯.১৭ পর্যালোচনা প্রশ্ন

- লাইভ মাইগ্রেশন কেন সম্ভব? এতে ডিস্কের ভূমিকা কী?
- Spot VM কখন ব্যবহার করবেন, কখন করবেন না?
- Persistent Disk ও Hyperdisk-এর পার্থক্য কী?
- OS Login প্রজেক্ট-মেটাডেটা SSH কী-এর চেয়ে নিরাপদ কেন?
- থামানো VM-এর কোন খরচ চলতে থাকে, কোনটি বন্ধ হয়?

৯.১৮ ব্যবহারিক অনুশীলন

- পাবলিক IP ছাড়া একটি VM বানিয়ে IAP দিয়ে SSH করুন; স্টার্টআপ স্ক্রিপ্টে nginx ইনস্টল করুন।
- একটি কাস্টম ইমেজ তৈরি করে তা থেকে দ্বিতীয় VM বানান — দুটি VM-এর কনফিগারেশন একই হয় কি না যাচাই করুন।
- একটি ডিস্কের স্ন্যাপশট নিয়ে তা থেকে নতুন ডিস্ক বানিয়ে অন্য VM-এ লাগান।

অধ্যায় ১০ • PART II — Networking & Compute Engineering

Chapter 10 — Hybrid and Enterprise Networking

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) হাইব্রিড সংযোগের প্রয়োজন ব্যাখ্যা করতে — কখন VPN যথেষ্ট, কখন Interconnect দরকার; (২) HA VPN ও Classic VPN-এর পার্থক্য বুঝতে; (৩) Dedicated ও Partner Interconnect-এর মধ্যে বেছে নিতে; (৪) Cloud Router ও BGP কীভাবে রাউট শেখে তা ব্যাখ্যা করতে; (৫) হাইব্রিড DNS ডিজাইন করতে; (৬) এন্টারপ্রাইজ নেটওয়ার্ক আর্কিটেকচার আঁকতে ও সংযোগের সমস্যার রোগ নির্ণয় করতে।

১০.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ৬-৭ থেকে মনে রাখুন: VPC গ্লোবাল, রাউট ট্রাফিকের পথ ঠিক করে, আর BGP-র মাধ্যমে শেখা রাউটকে বলে dynamic route। আরও মনে রাখুন: IPsec হলো ইন্টারনেটের ওপর এনক্রিপ্টেড টানেলের প্রোটোকল; BGP (Border Gateway Protocol) হলো নেটওয়ার্কগুলোর মধ্যে রাউট ঘোষণা করার প্রোটোকল। এই দুটি মিলেই হাইব্রিড নেটওয়ার্কিং চলে।

১০.৩ প্রথম নীতি: হাইব্রিড কেন দরকার?

কোনো সংস্থাই একদিনে পুরোপুরি ক্লাউডে যায় না। TechNova-র ঢাকায় একটি ডাটা সেন্টার আছে — সেখানে পুরোনো ডাটাবেস চলে; নতুন অ্যাপ GCP-তে। দুই জায়গার সার্ভারকে একে অপরের সঙ্গে কথা বলতে হবে — যেন তারা একটাই নেটওয়ার্ক। পাবলিক ইন্টারনেট দিয়ে কথা বলালে তিনটি সমস্যা: নিরাপত্তা (ট্রাফিক খোলা পথে যায়), নির্ভরযোগ্যতা (ইন্টারনেটের ভিড়ে ল্যাটেন্সি ওঠানামা করে), আর খরচ (egress)। হাইব্রিড সংযোগ এই তিনটি সমস্যার উত্তর দেয় — প্রাইভেট, স্থিতিশীল, পূর্বানুমেয় পথে।

> **CONCEPT:** হাইব্রিড সংযোগের মূল প্রশ্ন একটাই: "আপনার ট্রাফিক কতটা গুরুত্বপূর্ণ?" মাঝে মাঝে অল্প ডেটা গেলে VPN যথেষ্ট; প্রতিদিন টেরাবাইট ডেটা, কম ল্যাটেন্সি ও SLA দরকার হলে Interconnect। সিদ্ধান্তটি প্রযুক্তিগত নয় — ব্যবসায়িক।

১০.৪ Cloud VPN ও HA VPN

Cloud VPN ইন্টারনেটের ওপর IPsec টানেল বানায় — আপনার অন-প্রেমিসেস VPN গেটওয়ে ও Google-এর VPN গেটওয়ের মধ্যে এনক্রিপ্টেড সংযোগ। **HA VPN** হলো আধুনিক সুপারিশ: প্রতিটি টানেলের দুটি ইন্টারফেস, দুটি পাবলিক IP, ৯৯.৯৯% SLA (সঠিকভাবে কনফিগার করলে)। Classic VPN (একটিমাত্র টানেল, একটিমাত্র IP) এখন পুরোনো — নতুন ডিজাইনে ব্যবহার করবেন না। HA VPN-এর ব্যান্ডউইথ টানেল-প্রতি সীমাবদ্ধ — বেশি ব্যান্ডউইথ দরকার হলে একাধিক টানেল যোগ করুন।

VPN-এর সীমাবদ্ধতা মনে রাখুন: এটি ইন্টারনেটের ওপর দিয়ে যায় — তাই ল্যাটেন্সি ও ব্যান্ডউইথ ইন্টারনেটের ওপর নির্ভরশীল; কোনো ব্যান্ডউইথের SLA নেই; আর বড়, স্থায়ী ডেটা স্থানান্তরের জন্য ব্যয়বহুল হতে পারে (egress চার্জ)। কিন্তু এর সুবিধা: দ্রুত বসানো যায় (ঘণ্টার মধ্যে), কোনো ফিজিক্যাল কাজ লাগে না, খরচ কম।

১০.৫ Cloud Interconnect

Interconnect হলো Google-এর নেটওয়ার্কের সঙ্গে সরাসরি ফিজিক্যাল সংযোগ — ইন্টারনেট বাইপাস করে। দুই ধরন: **Dedicated Interconnect** — আপনার ডাটা সেন্টার থেকে Google-এর এজ লোকেশনে সরাসরি ফাইবার (১০ Gbps বা ১০০ Gbps); **Partner Interconnect** — একটি সার্ভিস প্রোভাইডারের মাধ্যমে সংযোগ (৫০ Mbps থেকে ১০ Gbps পর্যন্ত ভার্চুয়াল সার্কিট)। Dedicated দরকার যখন আপনার নিজের ফাইবার আছে ও সর্বোচ্চ ব্যান্ডউইথ চাই; Partner দরকার যখন আপনার ডাটা সেন্টার Google এজের কাছে নয়, বা ছোট ব্যান্ডউইথ দিয়ে শুরু করতে চান।

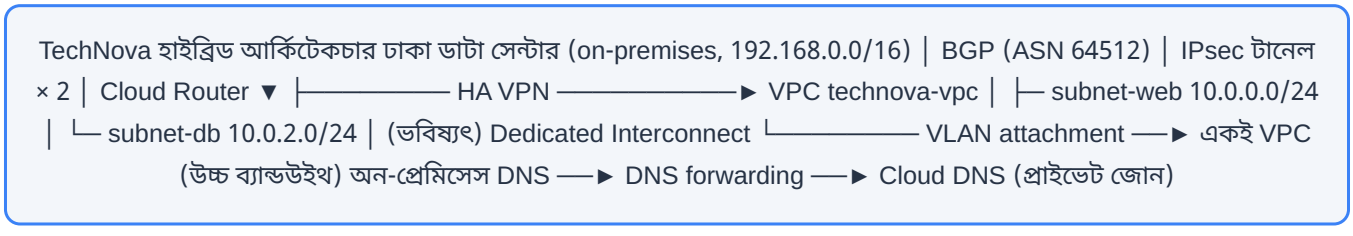
Interconnect-এর জন্য **VLAN attachment** (যাকে Interconnect attachment-ও বলে) লাগে — এটি আপনার VPC-কে Interconnect-এর সঙ্গে যুক্ত করে। প্রতিটি attachment একটি নির্দিষ্ট রিজিয়নের VPC-র সঙ্গে যুক্ত হয়। উচ্চ উপলব্ধতার জন্য দুটি আলাদা এজ লোকেশনে দুটি সংযোগ বসান — একটিতে সমস্যা হলে অন্যটি চলবে।

বৈশিষ্ট্য	HA VPN	Partner Interconnect	Dedicated Interconnect
পথ	ইন্টারনেট (IPsec)	প্রোভাইডারের প্রাইভেট নেটওয়ার্ক	সরাসরি ফাইবার
ব্যান্ডউইথ	টানেল-প্রতি সীমিত	৫০ Mbps–১০ Gbps	১০/১০০ Gbps
SLA	৯৯.৯৯% (সঠিক কনফিগে)	৯৯.৯%–৯৯.৯৯%	৯৯.৯৯%
বসাতে সময়	ঘণ্টা	দিন–সপ্তাহ	সপ্তাহ–মাস
উপযুক্ত	শুরু, অস্থায়ী, ছোট লোড	মাঝারি লোড, দূরবর্তী DC	বড়, স্থায়ী, মিশন-ক্রিটিক্যাল

১০.৬ Cloud Router ও BGP

Cloud Router হলো Google-এর ম্যানেজড BGP স্পিকার — এটি আপনার অন-প্রেমিসেস রাউটারের সঙ্গে BGP সেশন করে রাউট আদান-প্রদান করে। আপনার VPC-র সাবনেট রেঞ্জ BGP দিয়ে অন-প্রেমিসেসকে জানানো হয়; অন-প্রেমিসেসের রেঞ্জ BGP দিয়ে GCP-কে জানানো হয়। ফলে দুই দিকের রাউট টেবিল স্বয়ংক্রিয়ভাবে আপডেট হয় — নতুন সাবনেট বানালে হাতে রাউট লিখতে হয় না।

BGP-র মূল ধারণা: **ASN** (Autonomous System Number) — প্রতিটি নেটওয়ার্কের পরিচয় নম্বর; Google-এর Cloud Router-এর জন্য আপনি একটি প্রাইভেট ASN বেছে নেন (যেমন 64512)। **MED ও AS path** দিয়ে ঠিক করা যায় কোন পথটি পছন্দের — দুটি সংযোগ থাকলে একটিকে প্রাইমারি, অন্যটিকে ব্যাকআপ বানানো যায়। আর **advertised routes** দিয়ে নিয়ন্ত্রণ করা যায় GCP কোন রেঞ্জ ঘোষণা করবে — প্রয়োজনে শুধু নির্দিষ্ট সাবনেট ঘোষণা করুন, পুরো VPC নয়।




```
# Cloud Router + HA VPN তৈরি
gcloud compute routers create technova-hybrid-router \
  --network=technova-vpc --region=asia-southeast1 \
  --asn=64512

gcloud compute vpn-gateways create technova-ha-vpn \
  --network=technova-vpc --region=asia-southeast1
```

```
# টানেল (অন-প্রেমিসেস গেটওয়ের IP 203.0.113.1 উদাহরণ)
gcloud compute vpn-tunnels create tunnel-0 \
  --peer-gcp-gateway=technova-ha-vpn \
  --region=asia-southeast1 \
  --ike-version=2 \
  --shared-secret=CHANGE-ME-SHARED-SECRET \
  --router=technova-hybrid-router \
  --vpn-gateway-interface=0 \
  --peer-external-gateway=onprem-gw \
  --peer-external-gateway-interface=0

# BGP পিয়ার (অন-প্রেমিসেস রাউটারের IP ও ASN উদাহরণ)
gcloud compute routers add-bgp-peer technova-hybrid-router \
  --region=asia-southeast1 \
  --peer-name=onprem-peer-0 \
  --interface=tunnel-0-iface-0 \
  --peer-ip-address=169.254.0.2 \
  --peer-asn=64513 \
  --advertised-route-priority=100
```

> **সতর্কতা:** উপরের `CHANGE-ME-SHARED-SECRET` একটি প্লেসহোল্ডার — আসল শেয়ার্ড সিক্রেট কখনোই কোডে, চ্যাটে বা Git-এ লিখবেন না; Secret Manager-এ রাখুন। আর `203.0.113.1` ও `169.254.0.2` উদাহরণ ঠিকানা — আপনার আসল গেটওয়ের ঠিকানা বসবে।

১০.৭ হাইব্রিড DNS

হাইব্রিড পরিবেশে নামের সমস্যা: GCP-র VM অন-প্রেমিসেসের সার্ভারের নাম জানবে কীভাবে, আর উল্টোটা? সমাধান DNS forwarding: Cloud DNS-এ outbound forwarding নীতি বসিয়ে নির্দিষ্ট ডোমেইন (যেমন `corp.technova.example`) অন-প্রেমিসেস DNS সার্ভারের দিকে পাঠান; আর অন-প্রেমিসেস DNS-এ GCP-র প্রাইভেট জোনের (যেমন `internal.technova.example`) জন্য forwarding বসান। এতে দুই দিকের নাম দুই দিক থেকে রিজলভ হয় — কোনো পাবলিক DNS জড়িত নয়।

১০.৮ এন্টারপ্রাইজ নেটওয়ার্ক আর্কিটেকচার

বড় সংস্থার সাধারণ প্যাটার্ন: একটি hub VPC (বা Shared VPC host প্রজেক্ট) — যেখানে Interconnect/VPN এসে পড়ে; আর spoke VPC-গুলো hub-এর সঙ্গে যুক্ত। Google-এর সুপারিশকৃত প্যাটার্নে **Network Connectivity Center** hub হিসেবে কাজ করে — VPN, Interconnect, আর VPC-গুলোকে এক জায়গায় যুক্ত করে। নিয়ম: সংযোগের প্রবেশপথ এক জায়গায় কেন্দ্রীভূত করুন; প্রতিটি spoke-এ আলাদা VPN বসাবেন না — তাতে রাউটিং জটিল হয়, আর নিরাপত্তা নীতি ছড়িয়ে যায়।

Hub-and-spoke ডিজাইনের বিস্তারিত দিকগুলো ভেবে দেখুন। Hub VPC-তে সাধারণত কোনো অ্যাপ চলে না — সেখানে শুধু সংযোগের সরঞ্জাম থাকে: VPN গেটওয়ে, Interconnect attachment, আর ভাগ করা সার্ভিস (যেমন কেন্দ্রীয় DNS, প্রক্সি)। Spoke VPC-গুলোতে অ্যাপ চলে — প্রতিটি দলের বা পরিবেশের (production, staging) জন্য আলাদা spoke। Spoke-গুলোর মধ্যে ট্রাফিক hub দিয়ে যায়, আর hub-এর ফায়ারওয়াল নীতিতে ঠিক করা হয় কোন spoke কোন spoke-এর সঙ্গে কথা বলতে পারবে। এই ডিজাইনের সুবিধা: নিরাপত্তা নীতি এক জায়গায়, রাউটিং এক জায়গায়, আর নতুন spoke যোগ করা সহজ — শুধু hub-এর সঙ্গে যুক্ত করলেই হয়। অসুবিধা: hub একটি কেন্দ্রীয় নির্ভরতা — hub-এর সমস্যা সব spoke-কে প্রভাবিত করে; তাই hub-এর সংযোগে redundancy (দুটি VPN টানেল, দুটি Interconnect লিঙ্ক) অপরিহার্য।

আরেকটি বিবেচনা: hub দিয়ে যাওয়া ট্রাফিকের egress — spoke-থেকে-spoke ট্রাফিক hub দিয়ে গেলে অতিরিক্ত হপ হয়; খুব বেশি spoke-থেকে-spoke ট্রাফিক থাকলে সরাসরি peering বিবেচনা করুন। TechNova-র মতো মাঝারি সংস্থার জন্য শুরুতে একটি Shared VPC-ই যথেষ্ট; hub-and-spoke তখনই দরকার যখন প্রজেক্টের সংখ্যা দশের বেশি হয়ে যায়, বা আলাদা দলের নেটওয়ার্ক সম্পূর্ণ আলাদা রাখতে হয়।

১০.৯ প্রোডাকশন দৃশ্যকল্প: TechNova-র মাইগ্রেশন

TechNova ধাপে ধাপে মাইগ্রেট করছে: প্রথমে HA VPN দিয়ে ঢাকার ডাটা সেন্টার যুক্ত করল — অ্যাপ GCP-তে, ডাটাবেস এখনও অন-প্রেমিসেসে। ল্যাটেন্সি মেপে দেখল গড়ে ৩০ মিলিসেকেন্ড — অ্যাপের জন্য গ্রহণযোগ্য। ডেটা স্থানান্তরের পরিমাণ বাড়তে থাকায় ছয় মাস পর Partner Interconnect-এ গেল — ল্যাটেন্সি কমল, egress খরচ কমল। ডাটাবেস মাইগ্রেট হওয়ার পর VPN শুধু ব্যাকআপ পথ হিসেবে রইল। শিক্ষা: হাইব্রিড একটি স্থায়ী অবস্থা হতে পারে, কিন্তু সংযোগের ধরন লোডের সঙ্গে বদলাতে হয়।

মাইগ্রেশনের ধাপগুলো আরও বিস্তারিতভাবে দেখুন, কারণ এখানেই বেশিরভাগ দল ভুল করে। প্রথম ধাপ ছিল আবিষ্কার: ঢাকার ডাটা সেন্টারে কোন সার্ভার আছে, কোনটি কার সঙ্গে কথা বলে, কত ডেটা যায় — এই মানচিত্র ছাড়া সংযোগ বসানো অন্ধের মতো কাজ। দ্বিতীয় ধাপ: নেটওয়ার্ক পরিকল্পনা — CIDR ওভারল্যাপ যাচাই, BGP ASN ঠিক করা, DNS forwarding-এর নকশা। তৃতীয় ধাপ: VPN দিয়ে ছোট পরিসরে পরীক্ষা — প্রথমে একটি অ-গুরুত্বপূর্ণ অ্যাপ, তারপর ধীরে ধীরে বাকিগুলো। চতুর্থ ধাপ: পর্যবেক্ষণ ও মাপ — ল্যাটেন্সি, throughput, packet loss — প্রতিদিন দেখা; সংখ্যা না দেখে "মনে হচ্ছে ঠিক আছে" বলা চলে না। পঞ্চম ধাপ: Interconnect-এর সিদ্ধান্ত — যখন মাসিক egress বিল VPN-এর খরচ ছাড়িয়ে গেল, তখনই Interconnect-এর ব্যবসায়িক যুক্তি তৈরি হলো। আর শেষ ধাপ: প্রস্থান পরিকল্পনা — ডাটাবেস মাইগ্রেট হওয়ার পর অন-প্রেমিসেসের কী হবে, VPN কখন বন্ধ হবে, তা আগেই ঠিক করা ছিল। মাইগ্রেশন একটি প্রজেক্ট নয় — এটি একটি যাত্রা, যার প্রতিটি ধাপে মাপ, পরীক্ষা ও ফিরে যাওয়ার পথ থাকে।

১০.১০ নিরাপত্তা ও খরচ

নিরাপত্তা: VPN-এর শেয়ার্ড সিক্রেট শক্তিশালী ও নিয়মিত ঘোরানো (rotate) করুন; BGP-তে শুধু প্রয়োজনীয় রেঞ্জ ঘোষণা করুন — পুরো ইন্টারনেট রাউট ঘোষণা করবেন না; Interconnect-এর VLAN attachment-এ ফায়ারওয়াল নিয়ম প্রযোজ্য হয় — অন-প্রেমিসেস থেকে আসা ট্রাফিকও VPC ফায়ারওয়ালের আওতায়। খরচ: VPN টানেলের প্রতি-ঘণ্টা চার্জ + egress; Interconnect-এর পোর্ট চার্জ + attachment চার্জ + egress (Interconnect egress সাধারণত ইন্টারনেট egress-এর চেয়ে সস্তা — তবে দাম বদলায়, Google Cloud Pricing Calculator দেখুন)।

নিরাপত্তার ক্ষেত্রে হাইব্রিডের সবচেয়ে বড় ঝুঁকি হলো "ভেতরের নেটওয়ার্ক নিরাপদ" — এই ধারণা। অন-প্রেমিসেস থেকে আসা ট্রাফিককে বিশ্বাস করবেন না — VPC ফায়ারওয়াল নিয়ম অন-প্রেমিসেসের উৎসের ক্ষেত্রেও প্রযোজ্য, তাই শুধু প্রয়োজনীয় পোর্ট খুলুন, আর service account বা ট্যাগ দিয়ে লক্ষ্য নির্ধারণ করুন। আরেকটি ঝুঁকি: BGP-র মাধ্যমে ভুল রাউট ঘোষণা — অন-প্রেমিসেসের রাউটার যদি ভুল করে পুরো ইন্টারনেটের রাউট ঘোষণা করে, তাহলে GCP-র ট্রাফিক ভুল পথে যেতে পারে। তাই Cloud Router-এ advertised routes সীমিত করুন, আর অন-প্রেমিসেসের দিকেও route filtering বসান। খরচের দিকে: Interconnect-এর সিদ্ধান্ত নেওয়ার আগে তিনটি সংখ্যা মাপুন — বর্তমান মাসিক egress, প্রত্যাশিত বৃদ্ধি, আর Interconnect-এর মোট মাসিক খরচ (পোর্ট + attachment + egress)। অনেক ক্ষেত্রে দেখা যায় ছোট লোডে VPN-ই সস্তা, আর Interconnect তখনই লাভজনক যখন মাসিক ডেটা স্থানান্তর একটি নির্দিষ্ট সীমা ছাড়ায়। এই হিসাবটি Pricing Calculator-এ করে, লিখে রাখুন — ছয় মাস পর আবার মাপুন; লোড বদলালে সিদ্ধান্তও বদলাতে পারে। আর মনে রাখবেন: Interconnect বসাতে সময় লাগে (সপ্তাহ থেকে মাস) — তাই সিদ্ধান্তটি লোডের সংকটের সময়ে নেবেন না, আগেই নিন।

১০.১১ সাধারণ ভুল

- CIDR ওভারল্যাপ — অন-প্রেমিসেস ও VPC-র রেঞ্জ মিলে গেলে রাউটিং ভেঙে যায়।
- Classic VPN দিয়ে নতুন ডিজাইন শুরু করা।
- একটিমাত্র টানেল/সংযোগের ওপর নির্ভর করা — কোনো redundancy নেই।
- BGP ছাড়া স্ট্যাটিক রাউটে সব করা — নতুন সাবনেটে হাতে রাউট লিখতে হয়।
- MTU না মেলানো — VPN টানেলে MTU 1460-র কম রাখতে হয়; না হলে প্যাকেট ভেঙে যায়।
- DNS forwarding না বসিয়ে IP দিয়ে কাজ চালানো — পরে রক্ষণাবেক্ষণ অসম্ভব হয়।

১০.১২ Troubleshooting

TROUBLESHOOTING: টানেল উঠছে না? IKE সংস্করণ (v2) ও শেয়ার্ড সিক্রেট দুই দিকে এক কি না দেখুন; অন-প্রেমিসেস ফায়ারওয়ালে UDP 500/4500 খোলা কি না দেখুন। টানেল উঠেছে কিন্তু ট্রাফিক যাচ্ছে না? BGP সেশন ESTABLISHED কি না দেখুন (`gcloud compute routers get-status`) — BGP না উঠলে রাউট শেখা হয়নি। ট্রাফিক যাচ্ছে কিন্তু ধীর? MTU দেখুন, আর পথটি VPN না Interconnect দিয়ে যাচ্ছে তা traceroute দিয়ে যাচাই করুন। DNS রিজলভ হচ্ছে না? Forwarding নীতির লক্ষ্য সার্ভার পৌঁছানো যাচ্ছে কি না — ফায়ারওয়ালে DNS পোর্ট 53 খোলা আছে কি না দেখুন।

১০.১৩ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

BGP সেশন কীভাবে কাজ করে, তা জানলে হাইব্রিড troubleshooting অনেক সহজ হয়। Cloud Router ও অন-প্রেমিসেস রাউটার প্রথমে TCP সংযোগ করে (পোর্ট 179), তারপর OPEN মেসেজে ASN ও ক্ষমতা বিনিময় করে, তারপর রাউট ঘোষণা করে — UPDATE মেসেজে। সেশন ভেঙে গেলে সব শেখা রাউট মুছে যায় — তাই BGP সেশনের স্বাস্থ্যই হাইব্রিড সংযোগের স্বাস্থ্য। Cloud Router-এর স্ট্যাটাসে দেখা যায় প্রতিটি পিয়ারের অবস্থা: ESTABLISHED মানে সব ঠিক, IDLE বা ACTIVE মানে সংযোগই হচ্ছে না — তখন প্রথমে অন-প্রেমিসেস ফায়ারওয়ালে BGP পোর্ট খোলা আছে কি না দেখুন।

VPN টানেলের ভেতরে: IPsec দুটি পর্যায়ে কাজ করে — IKE (Phase 1) নিরাপদ চ্যানেল বানায়, তারপর ESP (Phase 2) আসল ডেটা এনক্রিপ্ট করে। দুই দিকের কনফিগারেশন (এনক্রিপশন অ্যালগরিদম, DH গ্রুপ, লাইফটাইম) না মিললে টানেল উঠবে না — আর ভুলের বার্তা প্রায়ই অস্পষ্ট হয়। তাই Google-এর ডকুমেন্টেশনের সুপারিশকৃত সেটিংস ব্যবহার করুন, নিজের পছন্দের অ্যালগরিদম নয়। Interconnect-এর ক্ষেত্রে ব্যর্থতার ধরন আলাদা: ফিজিক্যাল লিঙ্কের সমস্যা, VLAN attachment-এর ভুল কনফিগারেশন, বা BGP-র ভুল ASN — প্রতিটির লক্ষণ আলাদা, আর Cloud Interconnect-এর ডায়াগনস্টিক পেজে লিঙ্কের অবস্থা দেখা যায়।

পর্যবেক্ষণ: VPN টানেলের throughput ও packet loss, BGP সেশনের আপটাইম, Interconnect-এর লিঙ্ক ব্যবহার — এগুলো Cloud Monitoring-এ দেখুন ও অ্যালার্ট বসান। নিয়মিত ফেলওভার পরীক্ষা করুন: প্রাইমারি সংযোগ বন্ধ করে দেখুন ব্যাকআপ পথে ট্রাফিক যায় কি না — আসল বিপদের দিনে প্রথম পরীক্ষা করবেন না। বিলিং: Interconnect-এর egress ইন্টারনেটের চেয়ে সস্তা হলেও শূন্য নয় — বড় ডেটা স্থানান্তরের আগে Pricing Calculator-এ হিসাব করুন। বিকল্প: অস্থায়ী সংযোগের জন্য VPN; স্থায়ী, বড় লোডের জন্য Interconnect; আর একাধিক ক্লাউডের মধ্যে সংযোগের জন্য Network Connectivity Center বা তৃতীয় পক্ষের সমাধান বিবেচনা করুন।

১০.১৪ সারাংশ

HA VPN দ্রুত শুরু, Interconnect স্থায়ী সমাধান; Cloud Router ও BGP রাউটিংকে স্বয়ংক্রিয় করে; হাইব্রিড DNS নামের সেতু বানায়। সংযোগের ধরন লোড ও গুরুত্ব দিয়ে বেছে নিন — আর সবসময় redundancy রাখুন।

১০.১৫ পর্যালোচনা প্রশ্ন

- HA VPN Classic VPN-এর চেয়ে ভালো কেন?
- Dedicated ও Partner Interconnect-এর মধ্যে কখন কোনটি বেছে নেবেন?
- BGP ছাড়া হাইব্রিড নেটওয়ার্ক চালালে কী সমস্যা হয়?
- হাইব্রিড DNS-এ forwarding কেন দরকার?
- Network Connectivity Center কী সমস্যা সমাধান করে?

১০.১৬ ব্যবহারিক অনুশীলন

- দুটি VPC-র মধ্যে HA VPN-এর মতো করে একটি ল্যাব টানেলের কনফিগারেশন লিখুন (আসলে দুটি VPC peering নয়, VPN কনফিগারেশন)।
- একটি Cloud Router বানিয়ে তার BGP স্ট্যাটাস দেখুন; advertised routes বদলে কী হয় তা পর্যবেক্ষণ করুন।
- একটি হাইব্রিড DNS forwarding নীতির নকশা আঁকুন: কোন ডোমেইন কোন দিকে যাবে, তা টেবিলে লিখুন।

অধ্যায় ১১ • PART II — Networking & Compute Engineering

Chapter 11 — Managed Instance Groups and High Availability

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) ইনস্ট্যান্স টেমপ্লেট দিয়ে পুনরুৎপাদনযোগ্য VM সংজ্ঞা লিখতে; (২) Managed Instance Group (MIG) বানিয়ে VM-এর দল পরিচালনা করতে; (৩) autoscaling নীতি (CPU, লোড ব্যালান্সার ব্যবহার, কাস্টম মেট্রিক) ঠিক করতে; (৪) autohealing দিয়ে অসুস্থ VM স্বয়ংক্রিয়ভাবে বদলাতে; (৫) আঞ্চলিক MIG দিয়ে জোন-ব্যর্থতা সহ্য করতে; (৬) rolling update ও canary কৌশলে নতুন সংস্করণ ছাড়তে; (৭) উচ্চ উপলব্ধতা ও ধারণক্ষমতা পরিকল্পনা করতে।

১১.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

অধ্যায় ৯ থেকে মনে রাখুন: একক VM মারা যেতে পারে — হোস্টের সমস্যা, ডিস্কের সমস্যা, অ্যাপ ক্র্যাশ। অধ্যায় ৮ থেকে: লোড ব্যালান্সার হেলথ চেক দিয়ে সুস্থ ব্যাকএন্ডে ট্রাফিক পাঠায়। এই অধ্যায়ে এই দুটি ধারণা এক হয়: লোড ব্যালান্সারের সামনে VM-এর একটি দল, যার আকার ও স্বাস্থ্য Google নিজে দেখাশোনা করে।

১১.৩ প্রথম নীতি: একক সার্ভার কেন যথেষ্ট নয়

একটি VM যতই ভালো হোক, তার ব্যর্থতার সম্ভাবনা শূন্য নয়। একটি VM-এর উপলব্ধতা ৯৯.৯% হলেও বছরে প্রায় ৯ ঘণ্টা ডাউনটাইম হয়। কিন্তু তিনটি VM তিনটি জোনে থাকলে — তিনটি একসঙ্গে মারা যাওয়ার সম্ভাবনা অত্যন্ত কম। এটাই উচ্চ উপলব্ধতার গণিত: ব্যর্থতাকে দূর করা যায় না, ব্যর্থতার প্রভাবকে ছড়িয়ে দেওয়া যায়। MIG এই নীতির যান্ত্রিক রূপ: আপনি বলেন "আমার তিনটি একই রকম VM চাই", আর MIG সেই সংখ্যাটি ধরে রাখে — একটি মারা গেলে নতুন বানায়, লোড বাড়লে আরও বানায়, নতুন সংস্করণ এলে ধীরে ধীরে বদলায়।

> **CONCEPT:** MIG-এর মূল ধারণা হলো "ঘোষণামূলক অবস্থা" (desired state)। আপনি কাঙ্ক্ষিত অবস্থা ঘোষণা করেন — তিনটি VM, এই টেমপ্লেট, এই হেলথ চেক — আর MIG বাস্তব অবস্থাকে ক্রমাগত কাঙ্ক্ষিত অবস্থার দিকে টেনে আনে। এটি Kubernetes-এর Deployment-এর মতোই ধারণা, শুধু VM স্তরে।

১১.৪ ইনস্ট্যান্স টেমপ্লেট

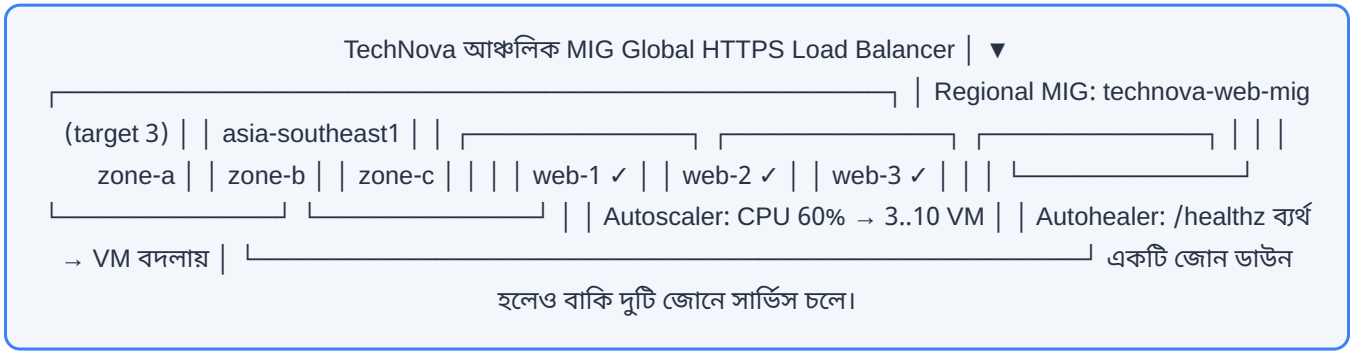
টেমপ্লেট হলো VM-এর নীলনকশা: মেশিন টাইপ, ইমেজ, ডিস্ক, নেটওয়ার্ক, ট্যাগ, মেটাডেটা (স্টার্টআপ স্ক্রিপ্টসহ), service account — সব এক জায়গায়। টেমপ্লেট অপরিবর্তনীয় (immutable): তৈরি হওয়ার পর বদলানো যায় না — নতুন সংস্করণের জন্য নতুন টেমপ্লেট বানাতে হয়। এটাই ভালো: প্রতিটি ডিপ্লয়মেন্টের ঠিক কোন সংজ্ঞা ব্যবহার হয়েছে, তা ইতিহাসে থাকে।

```
# ইনস্ট্যান্স টেমপ্লেট তৈরি
gcloud compute instance-templates create technova-web-template-v1 \
  --machine-type=e2-standard-2 \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --network=technova-vpc \
```

```
  --subnet=subnet-web \
  --no-address \
  --tags=web-server \
  --metadata=startup-script='#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl enable nginx
systemctl start nginx' \
  --maintenance-policy=MIGRATE

# টেমপ্লেট দেখা
gcloud compute instance-templates describe technova-web-template-v1
```

১১.৫ Managed Instance Group আর্কিটেকচার



MIG দুই ধরনের: **zonal** (এক জোনে) ও **regional** (একাধিক জোনে)। প্রোডাকশনে সবসময় regional — জোন-ব্যর্থতা সহ্য করতে। MIG-এর VM-গুলো একই টেমপ্লেট থেকে বানানো হয়, তাই তারা একই রকম — কোনো VM-এ হাতে কিছু বদলালে পরের বার সেটি হারিয়ে যাবে। নিয়ম: VM-এ হাতে বদলাবেন না; টেমপ্লেট বদলে নতুন করে ডিপ্লয় করুন। এটাকে বলে **immutable infrastructure**।

১১.৬ Autoscaling

Autoscaler লোড দেখে VM-এর সংখ্যা বাড়ায়-কমায়। সংকেত (signal) তিন রকম: **CPU utilization** (যেমন গড় CPU ৬০% হলে বাড়াও), **load balancer utilization** (প্রতি VM-এ অনুরোধের চাপ), আর **custom metric** (যেমন কিউয়ের দৈর্ঘ্য — Cloud Monitoring থেকে যেকোনো মেট্রিক)। নীতি ঠিক করার সময় মনে রাখুন: **cool-down period** — নতুন VM চালু হতে সময় লাগে, সেই সময়ে আবার স্কেল সিদ্ধান্ত নেওয়া উচিত নয়; **min/max** — সর্বনিম্ন (সবসময় চালু থাকা) ও সর্বোচ্চ (খরচের সীমা) সংখ্যা; আর **scale-in controls** — হঠাৎ অনেক VM কমিয়ে ফেলা আটকাতে।

সতর্কতা: autoscaling খরচের ঢাল — max সীমা না বসালে ট্রাফিকের ঢেউয়ে বিলের ঢেউ আসবে। বিলিং অ্যালার্ট ও max replicas — দুটোই বসান। আর স্টার্টআপ স্ক্রিপ্ট ধীর হলে (যেমন বড় প্যাকেজ ডাউনলোড) স্কেল-আপ ধীর হয় — তাই কাস্টম ইমেজে আগেই সফটওয়্যার বসিয়ে রাখুন, স্ক্রিপ্টে শুধু কনফিগারেশন করুন।

```
# আঞ্চলিক MIG তৈরি (৩টি VM, ৩ জোনে ছড়ানো)
gcloud compute instance-groups managed create technova-web-mig \
  --template=technova-web-template-v1 \
  --size=3 \
  --region=asia-southeast1 \
  --health-check=technova-hc \
  --initial-delay=120
```

```
# Autoscaler: CPU 60% লক্ষ্য, 3–10 VM
gcloud compute instance-groups managed set-autoscaling technova-web-mig \
  --region=asia-southeast1 \
  --min-num-replicas=3 \
  --max-num-replicas=10 \
  --target-cpu-utilization=0.6 \
  --cool-down-period=90

# Autohealing: হেলথ চেক ব্যর্থ হলে VM বদলানো
gcloud compute instance-groups managed update technova-web-mig \
  --region=asia-southeast1 \
  --health-check=technova-hc \
  --initial-delay=120
```

১১.৭ Autohealing ও হেলথ চেক

Autohealing হলো MIG-এর স্ব-চিকিৎসা: হেলথ চেক ব্যর্থ হলে MIG VM-টিকে অসুস্থ ঘোষণা করে, মুছে ফেলে, নতুন VM বানায়। **Initial delay** গুরুত্বপূর্ণ — নতুন VM বুট হয়ে অ্যাপ চালু হতে সময় লাগে; সেই সময়ের আগেই হেলথ চেক ব্যর্থ বললে VM বারবার মুছে নতুন বানানোর চক্রে পড়বে (flapping)। আপনার অ্যাপের স্টার্টআপ সময় মেপে initial delay তার চেয়ে একটু বেশি রাখুন।

> **বাস্তব উদাহরণ:** TechNova-র একটি VM-এর ডিস্ক পূর্ণ হয়ে অ্যাপ থেমে গেল। লোড ব্যালান্সার তাকে ট্রাফিক দেওয়া বন্ধ করল, কিন্তু VM-টি তখনও চলছিল — কেউ খেয়াল করেনি। Autohealing থাকলে MIG হেলথ চেক ব্যর্থ দেখে VM-টিকে বদলে দিত — সমস্যাটি স্বয়ংক্রিয়ভাবে সেরে যেত, আর on-call প্রকৌশলী শুধু লগে দেখতেন একটি VM বদলানো হয়েছে।

১১.৮ Rolling Update ও Canary

নতুন সংস্করণ ছাড়ার সময় সব VM একসঙ্গে বদলাবেন না। **Rolling update**: MIG ধীরে ধীরে পুরোনো VM বদলে নতুন টেমপ্লেটের VM বানায় — একবারে একটি বা দুটি করে, প্রতিটির হেলথ চেক পাস হওয়ার পর পরেরটি। **Max surge** বলে অতিরিক্ত কত VM একসঙ্গে বানানো যাবে; **max unavailable** বলে একসঙ্গে কত VM ডাউন থাকতে পারবে। **Canary**: প্রথমে একটি ছোট ভগ্নাংশ (যেমন ১০%) নতুন সংস্করণে চালান, মনিটরিং দেখুন, ঠিক থাকলে বাকিগুলো বদলান। ভুল হলে rollback — পুরোনো টেমপ্লেটে ফিরে যান।

```
# নতুন টেমপ্লেট v2 বানানোর পর rolling update
gcloud compute instance-groups managed rolling-action start-update technova-web-mig \
  --region=asia-southeast1 \
  --version=template=technova-web-template-v2 \
  --max-surge=1 \
  --max-unavailable=1

# আপডেটের অগ্রগতি দেখা
gcloud compute instance-groups managed describe technova-web-mig \
  --region=asia-southeast1

# সমস্যা হলে rollback (পুরোনো টেমপ্লেটে ফেরত)
gcloud compute instance-groups managed rolling-action start-update technova-web-mig \
```

```
  --region=asia-southeast1 \
  --version=template=technova-web-template-v1
```

১১.৯ উচ্চ উপলব্ধতা ও ধারণক্ষমতা পরিকল্পনা

উচ্চ উপলব্ধতার নকশা: (১) আঞ্চলিক MIG — কমপক্ষে দুটি জোনে; (২) লোড ব্যালান্সার — global, যাতে রিজিয়ন-ব্যর্থতাতেও ট্রাফিক অন্য রিজিয়নে যায়; (৩) ডাটাবেস — আলাদা উচ্চ-উপলব্ধতা কনফিগারেশন (Cloud SQL HA); (৪) স্টেটলেস অ্যাপ — VM-এ কোনো গুরুত্বপূর্ণ ডেটা রাখবেন না, সেশন ও ফাইল বাইরে (Memorystore, Cloud Storage) রাখুন। ধারণক্ষমতা পরিকল্পনা: স্বাভাবিক লোডের ২ গুণ সামলানোর ক্ষমতা রাখুন (N+1 নয়, N×2); লোড টেস্ট দিয়ে যাচাই করুন — autoscaler কত দ্রুত স্কেল করে, তা মেপে দেখুন, অনুমানে নয়।

SLO-র কথা মনে রাখুন: ৯৯.৯% উপলব্ধতা মানে মাসে প্রায় ৪৩ মিনিট ডাউনটাইমের বাজেট (error budget)। সেই বাজেটের ভেতরে rolling update, জোন-ব্যর্থতা ও ভুল ডিপ্লয়মেন্ট — সবকিছু ধরতে হবে।

১১.১০ Terraform দিয়ে MIG

```
resource "google_compute_instance_template" "web" {
  name         = "technova-web-template-v1"
  machine_type = "e2-standard-2"
  disk {
    source_image = "ubuntu-os-cloud/ubuntu-2204-lts"
    auto_delete  = true
    boot         = true
  }
  network_interface {
    network    = google_compute_network.technova.id
    subnetwork = google_compute_subnetwork.web.id
  }
  metadata_startup_script = file("startup.sh")
  tags = ["web-server"]
  lifecycle {
    create_before_destroy = true
  }
}

resource "google_compute_region_instance_group_manager" "web" {
  name               = "technova-web-mig"
  region             = "asia-southeast1"
  base_instance_name = "web"
  target_size        = 3
  version {
    instance_template = google_compute_instance_template.web.id
  }
  auto_healing_policies {
    health_check      = google_compute_health_check.web.id
    initial_delay_sec = 120
  }
}

resource "google_compute_region_autoscaler" "web" {
  name   = "technova-web-autoscaler"
  region = "asia-southeast1"
  target = google_compute_region_instance_group_manager.web.id
  autoscaling_policy {
```

```
    min_replicas    = 3
    max_replicas    = 10
    cooldown_period = 90
    cpu_utilization {
      target = 0.6
    }
  }
}
```

১১.১১ প্রোডাকশন দৃশ্যকল্প: TechNova-র ব্ল্যাক ফ্রাইডে

TechNova-র বছরের সবচেয়ে বড় বিক্রির দিন: সকাল ৯টায় ট্রাফিক স্বাভাবিকের পাঁচ গুণ। Autoscaler CPU ৬০% ছাড়াতেই VM বাড়াতে শুরু করল — ৩ থেকে ৮-এ গেল ১০ মিনিটে। একটি জোনে হোস্টের সমস্যা হলো — সেই জোনের দুটি VM মারা গেল, কিন্তু আঞ্চলিক MIG অন্য জোনে নতুন VM বানিয়ে সংখ্যা ধরে রাখল। দুপুরে নতুন সংস্করণের rolling update চালানো হলো — max-unavailable=1 থাকায় ব্যবহারকারী কিছু টের পেল না। দিন শেষে ট্রাফিক কমতেই autoscaler VM কমিয়ে ৩-এ ফিরিয়ে আনল — খরচও কমল। এই পুরো দিনটিতে কোনো মানুষ VM-এর সংখ্যা নিয়ে ভাবেনি — MIG ভেবেছে।

এই দিনটি সফল হওয়ার পেছনে লুকিয়ে ছিল আগের মাসগুলোর প্রস্তুতি। প্রথমত, ধারণক্ষমতা পরীক্ষা: বিক্রির এক মাস আগে লোড টেস্ট করে দেখা হয়েছিল ১০ গুণ ট্রাফিকে কী হয় — দেখা গেল autoscaler-এর cool-down period ৯০ সেকেন্ড রাখায় স্কেল-আপ ধীর হচ্ছে, তাই সেটি ৬০ সেকেন্ডে নামানো হলো, আর min replicas ৩ থেকে ৫-এ বাড়ানো হলো — যাতে স্কেল-আপের আগেই যথেষ্ট ধারণক্ষমতা থাকে। দ্বিতীয়ত, স্টার্টআপ সময়: কাস্টম ইমেজে অ্যাপ আগে থেকে বসানো ছিল বলে নতুন VM ৯০ সেকেন্ডে সার্ভিস দিতে শুরু করত — স্টার্টআপ স্ক্রিপ্টে বড় ডাউনলোড থাকলে সেটি ৫ মিনিট লাগত, আর স্কেল-আপ দেরিতে হতো। তৃতীয়ত, ডাটাবেসের সীমা: অ্যাপ সার্ভার বাড়লেও ডাটাবেসের সংযোগের সংখ্যা সীমিত — তাই connection pooling বসানো হয়েছিল, নইলে নতুন VM-গুলো ডাটাবেসে সংযোগই পেত না। চতুর্থত, খরচের সীমা: max replicas ১০ রাখা হয়েছিল — ট্রাফিক আরও বাড়লে সার্ভিস ধীর হতো, কিন্তু বিলের বিস্ফোরণ হতো না; এই সীমাটি ব্যবসায়িক সিদ্ধান্ত ছিল, প্রযুক্তিগত নয়। পঞ্চমত, মানুষের প্রস্তুতি: সেদিন on-call দলে দুজন ছিলেন, rollback-এর নির্দেশ লেখা ছিল, আর যোগাযোগের চ্যানেল ঠিক ছিল। অটোস্কেলিং যন্ত্রের কাজ করে — কিন্তু তার সীমা, তার প্রস্তুতি, আর তার ব্যর্থতার পরিকল্পনা মানুষের কাজ।

১১.১২ নিরাপত্তা ও খরচ

নিরাপত্তা: টেমপ্লেটে least-privilege service account দিন — MIG-এর সব VM একই service account পায়, তাই সেটি বেশি অনুমতি পেলে সব VM-ই বেশি অনুমতি পায়। স্টার্টআপ স্ক্রিপ্টে সিক্রেট রাখবেন না। খরচ: autoscaler-এর max সীমা বসান; নিয়মিত unused টেমপ্লেট ও পুরোনো ইমেজ মুছুন; Spot VM দিয়ে MIG-ও চালানো যায় (ব্যাচ কাজের জন্য) — তবে প্রোডাকশন ওয়েবের জন্য নয়। বিলিং অ্যালার্ট বসান — স্কেল-আপের ঢেউ বিলে ধরা পড়ার আগেই জানতে পারবেন।

নিরাপত্তার ক্ষেত্রে MIG-এর একটি বিশেষত্ব মনে রাখুন: VM-গুলো একই রকম, তাই একটি VM-এর দুর্বলতা সব VM-এর দুর্বলতা। একটি প্যাচ বাকি থাকলে সব VM-ই অরক্ষিত — তাই প্যাচের জন্য MIG-এর rolling update ব্যবহার করুন: নতুন টেমপ্লেটে প্যাচ করা ইমেজ, তারপর ধীরে ধীরে বদল। আরেকটি বিষয়: MIG-এর VM-গুলো স্বয়ংক্রিয়ভাবে বানানো হয় বলে তাদের নাম ও IP পরিবর্তনশীল — কোনো সিস্টেম যেন VM-এর IP-তে নির্ভর না করে; সার্ভিস ডিসকভারির জন্য লোড ব্যালান্সার বা DNS নাম ব্যবহার করুন। খরচের দিকে: autoscaler-এর সিদ্ধান্তের ইতিহাস দেখুন — কোন সময়ে, কেন স্কেল হয়েছিল। অনেক সময় দেখা যায় রাতের ব্যাচ কাজ বা ভুল কনফিগার করা হেলথ চেকের কারণে অপ্রয়োজনীয় স্কেল-আপ হচ্ছে। আরেকটি সাশ্রয়: min replicas কমিয়ে, আর স্কেল-আপের গতি বাড়িয়ে — রাতের কম লোডে ৩টির বদলে ২টি VM চালানো, আর ট্রাফিক বাড়লে দ্রুত বাড়ানো। কিন্তু সাবধান: min খুব কম রাখলে হঠাৎ ট্রাফিকের ঢেউয়ে সার্ভিস ধীর হয়ে

যাবে — সীমাটি লোড টেস্টের সংখ্যা দিয়ে ঠিক করুন, অনুমানে নয়। শেষ কথা: MIG-এর বিল দেখার সময় শুধু VM-এর বিল দেখবেন না — লোড ব্যালান্সার, NAT ও egress-ও স্কেলের সঙ্গে বাড়ে; পুরো স্থাপত্যের বিল একসঙ্গে দেখুন।

১১.১৩ সাধারণ ভুল

- Zonal MIG দিয়ে প্রোডাকশন চালানো — জোনের সমস্যায় সব শেষ।
- Initial delay খুব কম রাখা — নতুন VM বারবার মুছে যাওয়া (flapping)।
- স্টেটফুল অ্যাপ MIG-এ চালানো — VM বদলালে ডেটা হারানো।
- VM-এ হাতে বদলানো — পরের রিক্রিয়েশনে হারিয়ে যাওয়া।
- Max replicas না বসানো — বিলের বিস্ফোরণ।
- হেলথ চেকের পথে ভারী কাজ রাখা — হেলথ চেক নিজেই লোড বাড়ায়।

১১.১৪ Troubleshooting

TROUBLESHOOTING: MIG বারবার VM বানাচ্ছে আর মুছছে? Initial delay বাড়ান, স্টার্টআপ স্ক্রিপ্টের লগ দেখুন (`journalctl -u google-startup-scripts`), আর হেলথ চেকের পথটি VM-এর ভেতর থেকে curl করে দেখুন। Autoscaler স্কেল করছে না? মেট্রিকটি আসছে কি না দেখুন — CPU-ভিত্তিক স্কেলে VM-এ Cloud Monitoring এজেন্ট না থাকলেও CPU মেট্রিক আসে, কিন্তু কাস্টম মেট্রিকে এজেন্ট লাগে। Rolling update আটকে আছে? `describe` দিয়ে দেখুন কোন VM-টি হেলথ চেক পাস করছে না — প্রায়ই নতুন টেমপ্লেটের স্টার্টআপ স্ক্রিপ্টে ভুল থাকে।

১১.১৫ অভ্যন্তরীণ কার্যপ্রণালী, ব্যর্থতা ও পর্যবেক্ষণ

MIG-এর ভেতরে কী ঘটে? Autoscaler নির্দিষ্ট বিরতিতে মেট্রিক সংগ্রহ করে — CPU ব্যবহারের গড়, বা লোড ব্যালান্সারের প্রতি-VM চাপ। লক্ষ্য থেকে বিচ্যুতি দেখলে এটি কাঙ্ক্ষিত VM সংখ্যা পুনঃগণনা করে, কিন্তু cool-down period-এর মধ্যে দ্বিতীয় সিদ্ধান্ত নেয় না — নইলে নতুন VM চালু হওয়ার আগেই আরও VM চাওয়া হতো। এই হিসাবের একটি সূক্ষ্মতা: autoscaler গড় ব্যবহার দেখে, তাই একটি VM ১০০% আর দুটি VM ০% থাকলে গড় ৩৩% — স্কেল-আপ নাও হতে পারে। লোড অসমান হলে লোড ব্যালান্সারের balancing mode দেখুন — সেটি ঠিক না থাকলে autoscaler-এর সংকেতই ভুল হবে।

Autohealing-এর ক্ষেত্রে মনে রাখুন: হেলথ চেক VM-এর অ্যাপকে পরীক্ষা করে, কিন্তু VM-এর ডিস্ক পূর্ণ হওয়া বা মেমোরি লিকের মতো সমস্যা হেলথ চেকের পথে ধরা নাও পড়তে পারে — যদি `/healthz` শুধু "সার্ভার চলছে" বলে। তাই হেলথ চেক গভীর করুন: ডাটাবেস সংযোগ, ডিস্কের ফাঁকা জায়গা, মেমোরির ব্যবহার — এগুলো যাচাই করুন। আর একটি ব্যর্থতার ধরন: টেমপ্লেটের ভুল — যেমন স্টার্টআপ স্ক্রিপ্টে ভুল প্যাকেজের নাম — তখন প্রতিটি নতুন VM-ই ব্যর্থ হয়, আর MIG বারবার বানাতে থাকে। এর লক্ষণ: MIG-এর VM-গুলোর বয়স খুব কম, বারবার বদলাচ্ছে। সমাধান: টেমপ্লেট ঠিক করা, তারপর rolling update — MIG-কে একা ছেড়ে দিলে সে নিজে থেকে টেমপ্লেটের ভুল ঠিক করতে পারবে না।

পর্যবেক্ষণ: MIG-এর আকার, autoscaler-এর সিদ্ধান্তের ইতিহাস, আর VM বদলের ঘটনা — এগুলো Cloud Monitoring ও Logging-এ দেখুন। Rolling update-এর সময় প্রতিটি ধাপের হেলথ চেকের ফল দেখুন; কোনো ধাপে ব্যর্থতা হলে আপডেট থামে — সেটি একটি সুরক্ষা, ব্যর্থতা নয়। বিলিং: MIG-এর খরচ VM-এর খরচের সমান — কিন্তু autoscaler-এর max সীমা না থাকলে খরচের ঊর্ধ্বসীমা থাকে না। তাই max replicas-এর পাশাপাশি বিলিং বাজেটে অ্যালার্ট বসান। বিকল্প: VM-স্তরের অটোস্কেলিংয়ের বদলে GKE-র Horizontal Pod Autoscaler বা Cloud Run-এর স্বয়ংক্রিয় স্কেলিং — কনটেইনার বা

সার্ভারলেস স্থাপত্যে সেগুলোই সঠিক। আর মনে রাখুন: MIG স্টেটলেস অ্যাপের জন্য — ডাটাবেস বা স্টেটফুল সার্ভিসের জন্য নয়।

১১.১৬ সারাংশ

টেমপ্লেট সংজ্ঞা দেয়, MIG সংখ্যা ও স্বাস্থ্য ধরে রাখে, autoscaler লোডের সঙ্গে আকার বদলায়, autohealing অসুস্থ VM বদলায়, rolling update নিরাপদে নতুন সংস্করণ আনে। একক VM-এর যুগ শেষ — প্রোডাকশন মানে VM-এর দল, ঘোষণামূলকভাবে পরিচালিত।

১১.১৭ পর্যালোচনা প্রশ্ন

- টেমপ্লেট অপরিবর্তনীয় (immutable) হওয়া কেন ভালো?
- Regional MIG zonal MIG-এর চেয়ে ভালো কেন?
- Initial delay খুব কম হলে কী হয়?
- Rolling update-এ max surge ও max unavailable-এর ভূমিকা কী?
- Autoscaler-এর সংকেত হিসেবে কাস্টম মেট্রিক কখন দরকার হয়?

১১.১৮ ব্যবহারিক অনুশীলন

- একটি টেমপ্লেট ও আঞ্চলিক MIG বানান (৩টি VM); একটি VM মুছে দেখুন MIG নতুন VM বানায় কি না।
- Autoscaler বসিয়ে CPU লোড দিয়ে স্কেল-আপ পরীক্ষা করুন; তারপর লোড কমিয়ে স্কেল-ডাউন দেখুন।
- নতুন টেমপ্লেট (v2) বানিয়ে rolling update চালান; মাঝপথে একটি VM-এর হেলথ চেক ব্যর্থ করিয়ে দেখুন আপডেট থামে কি না।

PART III • DEVOPS ENGINEERING MASTERY

Storage, Database & Application Services

Cloud Storage, Persistent Disk ꩜ Backup, Cloud SQL ꩜ AlloyDB, Firestore, Bigtable ꩜ Memorystore, App Engine, Cloud Run ꩜ Edge Delivery

অধ্যায় ১২ • PART III — Storage, Database & Application Services

Chapter 12 — Cloud Storage Engineering

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Cloud Storage কী সমস্যা সমাধান করে তা first-principles দৃষ্টিকোণ থেকে ব্যাখ্যা করতে; bucket, object, storage class, object lifecycle, versioning ও retention policy ডিজাইন করতে; IAM, uniform bucket-level access, public access prevention ও encryption দিয়ে ডেটা সুরক্ষিত করতে; signed URL ও signed policy document দিয়ে নিরাপদ অস্থায়ী অ্যাক্সেস দিতে; replication, redundancy ও multi-region ডিজাইনের trade-off বুঝতে; এবং খরচ অপ্টিমাইজ করে production-grade object storage আর্কিটেকচার তৈরি করতে। আপনি Console ও gcloud CLI — দুই পথেই কাজ করতে পারবেন এবং Terraform দিয়ে bucket কে Infrastructure as Code হিসেবে পরিচালনা করতে পারবেন।

- Bucket ও object-এর মডেল, naming, location ও storage class ব্যাখ্যা করা
- Lifecycle rule, versioning, retention ও Object Hold দিয়ে ডেটা-জীবনচক্র নিয়ন্ত্রণ করা
- IAM, ACL, signed URL ও encryption (Google-managed, CMEK, CSEK) প্রয়োগ করা
- Redundancy, replication, consistency ও durability-এর বাস্তব অর্থ বোঝা
- Storage খরচের উপাদান — storage, operations, egress, retrieval — বিশ্লেষণ করে অপ্টিমাইজ করা

১২.২ Prerequisite Refresher

এই অধ্যায় শুরু করার আগে Volume 13-এর Part I ও Part II থেকে তিনটি ধারণা ঝালিয়ে নিন। প্রথম, GCP resource hierarchy: Organization → Folder → Project → Resource। Cloud Storage bucket একটি project-এর অধীন resource, কিন্তু bucket-এর নাম globally unique — পুরো Google Cloud-এ একটাই নাম থাকতে পারে। দ্বিতীয়, IAM: কে (principal) কোন resource-এ কী করতে পারে (role) — এটাই অ্যাক্সেস নিয়ন্ত্রণের ভিত্তি। তৃতীয়, region ও zone: region হলো ভৌগোলিক এলাকা (যেমন asia-south1), zone তার ভেতরের আলাদা data center। Cloud Storage-এর location ধারণা region/zone-এর চেয়ে আলাদা — এখানে location হতে পারে regional, dual-region বা multi-region। আপনি যদি Linux-এ ফাইল, HTTP verb (GET/PUT/DELETE) এবং JSON/YAML-এর সঙ্গে পরিচিত হন, তাহলে object storage বুঝতে সুবিধা হবে।

> **CONCEPT:** Object storage ফাইল সিস্টেম নয়। ফাইল সিস্টেমে hierarchical directory ও inode থাকে; object storage-এ প্রতিটি object একটি flat namespace-এ থাকে, তার একটি unique key (নাম), metadata ও ডেটা থাকে। “Folder” বলে যা দেখেন তা আসলে object নামের prefix মাত্র — যেমন `logs/2026/11/app.log` একটি object, `logs/` কোনো বাস্তব ডিরেক্টরি নয়। এই পার্থক্য বুঝলে listing, prefix-delimiter ও lifecycle নিয়ে বিভ্রান্তি কমে।

১২.৩ First Principles — কেন Object Storage দরকার?

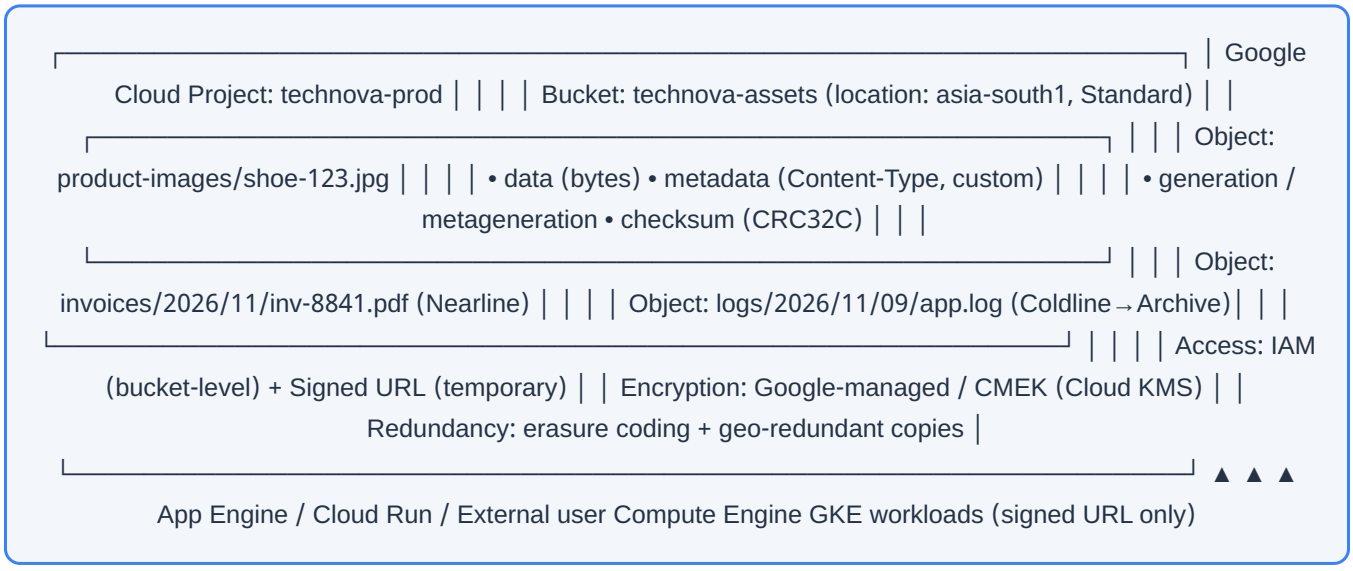
একটি অ্যাপ্লিকেশনে ডেটা তিন রকম: structured ডেটা (ডেটাবেজ), block ডেটা (VM-এর ডিস্ক) এবং unstructured ডেটা — ছবি, ভিডিও, ব্যাকআপ, লগ, static ওয়েবসাইট ফাইল, ML training ডেটা। Unstructured ডেটার বৈশিষ্ট্য হলো: আয়তন বিশাল ও অনির্দিষ্ট, অ্যাক্সেস প্যাটার্ন অনিয়মিত, প্রতিটি ফাইল স্বাধীনভাবে addressable হতে হয়, এবং হার্ডওয়্যার ব্যর্থতা

সত্ত্বেও ডেটা হারানো চলবে না। ঐতিহ্যবাহী ফাইল সার্ভার এই স্কেলে ভেঙে পড়ে — capacity বাড়াতে গেলে শাটডাউন, replication হাতে করতে হয়, আর একটিমাত্র সার্ভার নষ্ট হলে ডেটা ঝুঁকিতে পড়ে।

Cloud Storage এই সমস্যার উত্তর দেয় একটি managed, distributed object store দিয়ে। আপনি একটি bucket তৈরি করেন — একটি logical container — আর তাতে object রাখেন। Google ভেতরে ডেটাকে ছোট ছোট chunk-এ ভেঙে একাধিক ডিস্ক ও একাধিক facility-তে ছড়িয়ে রাখে, checksum দিয়ে integrity যাচাই করে, হার্ডওয়্যার নষ্ট হলে স্বয়ংক্রিয়ভাবে পুনর্গঠন করে। আপনার দায়িত্ব সরে যায় হার্ডওয়্যার থেকে নীতিতে: কোন ডেটা কোথায় থাকবে, কে দেখতে পারবে, কতদিন থাকবে, কখন সস্তা class-এ যাবে। এটাই cloud engineering-এর মূল পরিবর্তন — infrastructure পরিচালনা নয়, policy পরিচালনা।

TechNova-র উদাহরণ নিন। কোম্পানির e-commerce অ্যাপে প্রতিদিন হাজারো product ছবি আপলোড হয়, অর্ডারের PDF invoice তৈরি হয়, আর অ্যাপ্লিকেশন লগ জমা হয়। ছবি ও invoice ঘন ঘন পড়া হয়, লগ মাসে একবার audit-এ দেখা হয়, আর তিন বছরের পুরনো invoice প্রায় কখনোই খোলা হয় না। তিন রকম অ্যাক্সেস প্যাটার্নের জন্য তিন রকম খরচ-কাঠামো দরকার — এখানেই storage class ও lifecycle কাজে আসে।

১২.৪ আর্কিটেকচার — Bucket, Object ও Location



Bucket তৈরির সময় তিনটি সিদ্ধান্ত স্থায়ী প্রভাব ফেলে। প্রথম, নাম: globally unique, DNS-compatible, ছোট হাতের অক্ষর, সংখ্যা, hyphen ও dot ব্যবহার করা যায়। দ্বিতীয়, location: regional (একটি region, কম latency ও কম খরচ), dual-region (দুটি region-এ synchronous replication) বা multi-region (যেমন ASIA — একাধিক region জুড়ে)। Location একবার নির্ধারণ হলে পরিবর্তন করা যায় না — bucket পুনরায় তৈরি করে ডেটা সরাতে হয়। তৃতীয়, storage class: ডিফল্ট class নির্ধারণ করে দেয়, তবে প্রতিটি object আলাদা class-ও পেতে পারে।

Object-এর পরিচয় দুটি সংখ্যায়: generation (object-এর সংস্করণ) ও metageneration (metadata-এর সংস্করণ)। একই নামে নতুন object আপলোড করলে পুরনো generation প্রতিস্থাপিত হয় — versioning চালু না থাকলে পুরনোটি হারিয়ে যায়। প্রতিটি object-এ CRC32C ও MD5 checksum থাকে; Google নিয়মিত integrity scan চালায়।

১২.৫ Storage Classes — সঠিক Class বেছে নেওয়া

Storage Class	কখন ব্যবহার করবেন	অ্যাক্সেস প্যাটার্ন	মনে রাখার বিষয়

Standard	ঘন ঘন অ্যাক্সেস করা ডেটা, ওয়েবসাইট asset, active ডেটা	দৈনিক/ঘণ্টায়	সবচেয়ে বেশি storage খরচ, retrieval ফি নেই বললেই চলে
Nearline	মাসে একবার পড়া ব্যাকআপ, monthly report	মাসে ~১ বার	কম storage খরচ, retrieval ফি আছে, ন্যূনতম ৩০ দিন
Coldline	ত্রৈমাসিক/বাৎসরিক অ্যাক্সেস, DR ব্যাকআপ	বছরে কয়েকবার	আরও কম storage খরচ, retrieval ফি বেশি, ন্যূনতম ৯০ দিন
Archive	Compliance archive, দীর্ঘমেয়াদি সংরক্ষণ	বছরে ১ বার বা তার কম	সবচেয়ে কম storage খরচ, retrieval সবচেয়ে ব্যয়বহুল, ন্যূনতম ৩৬৫ দিন

ভুল class বেছে নেওয়া খরচের সবচেয়ে সাধারণ কারণ। Archive-এ রাখা ডেটা বারবার পড়লে retrieval ফি storage সাশ্রয়ের চেয়ে বেশি হয়ে যায়। আবার Standard-এ বছরের পর বছর untouched ডেটা রাখা অপচয়। সমাধান: হাতে class বদলাবেন না — lifecycle rule দিয়ে স্বয়ংক্রিয়ভাবে বদলান। আর দাম কখনো এই বই থেকে মুখস্থ করবেন না — দাম region ও সময়ের সঙ্গে বদলায়। যেকোনো সিদ্ধান্তের আগে **Google Cloud Pricing Calculator** (cloud.google.com/products/calculator) দিয়ে হিসাব করুন।

> **সতর্কতা:** Nearline/Coldline/Archive-এ ন্যূনতম সংরক্ষণ সময় (minimum storage duration) আছে। তার আগেই object মুছে ফেললে বা class বদলালে বাকি দিনের খরচ early deletion ফি হিসেবে দিতে হয়। Lifecycle rule লেখার সময় এই সময়সীমার সঙ্গে মিলিয়ে নিন — ৩০ দিনের কম বয়সী object-কে Coldline-এ পাঠাবেন না।

১২.৬ Object Lifecycle Management, Versioning ও Retention

Lifecycle rule হলো bucket-এর স্বয়ংক্রিয় housekeeping নীতি: condition মিললে action চলবে। যেমন — “বয়স ৩০ দিন হলে Nearline-এ নাও, ৯০ দিন হলে Coldline-এ, ৩৬৫ দিন হলে Archive-এ, ৭ বছর হলে মুছে ফেলো।” Condition হতে পারে বয়স, তৈরি হওয়ার তারিখ, storage class, live/noncurrent version, বা custom time। Noncurrent version বলতে versioning চালু থাকলে পুরনো সংস্করণকে বোঝায় — তার জন্য আলাদা rule লেখা যায়, যেমন পুরনো সংস্করণ ৩০ দিন পর মুছে ফেলো।

Versioning চালু করলে একই নামের প্রতিটি আপলোড আলাদা generation হিসেবে থাকে; ভুল করে overwrite বা delete করলে আগের সংস্করণ ফিরিয়ে আনা যায়। কিন্তু versioning খরচ বাড়ায় — প্রতিটি সংস্করণ আলাদা object হিসেবে বিল হয়। তাই versioning-এর সঙ্গে lifecycle rule জোড়া বাধ্যতামূলক। Retention policy আরও কঠোর: নির্দিষ্ট সময় পর্যন্ত কোনো object মুছে ফেলা বা পরিবর্তন করা যায় না — compliance ও audit-এর জন্য। Retention lock করলে সেই নীতি আর কমানো যায় না, এমনকি project owner-ও না; তাই lock করার আগে ভালো করে ভাবুন। Object Hold (temporary hold / event-based hold) কোনো নির্দিষ্ট object-কে সাময়িকভাবে মুছে ফেলা থেকে আটকায়, যেমন আইনি তদন্ত চলাকালীন।

```
# Lifecycle configuration (lifecycle.json) — TechNova logs bucket
{
  "lifecycle": {
    "rule": [
      { "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
        "condition": {"age": 30, "matchesStorageClass": ["STANDARD"]} },
      { "action": {"type": "SetStorageClass", "storageClass": "COLDLINE"},
        "condition": {"age": 90} },
```

```
      { "action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
        "condition": {"age": 365} },
      { "action": {"type": "Delete"},
        "condition": {"age": 2555, "matchesStorageClass": ["ARCHIVE"]} },
      { "action": {"type": "Delete"},
        "condition": {"age": 30, "isLive": false} }
    ]
  }
}
# Apply
gcloud storage buckets update gs://technova-logs --lifecycle-file=lifecycle.json
gcloud storage buckets update gs://technova-logs --versioning
gcloud storage buckets update gs://technova-logs --retention-period=365d
# Retention period দেখুন (lock করার আগে যাচাই করুন)
gcloud storage buckets describe gs://technova-logs --format="yaml(retentionPolicy)"
```

১২.৭ Console ও gcloud CLI — Bucket তৈরি ও পরিচালনা

Console-এ: Cloud Storage → Buckets → Create → নাম দিন (যেমন `technova-assets`), location type বেছে নিন (Region: asia-south1), default storage class Standard, access control-এ Uniform বেছে নিন, “Enforce public access prevention” চালু করুন, encryption-এ Google-managed বা CMEK বেছে নিন, তারপর Create। Console নতুনদের জন্য ভালো, কিন্তু পুনরাবৃত্তি ও audit-এর জন্য CLI-ই মানদণ্ড।

```
# Bucket তৈরি — region, class, uniform access, public access prevention সহ
gcloud storage buckets create gs://technova-assets \
  --location=asia-south1 \
  --default-storage-class=STANDARD \
  --uniform-bucket-level-access \
  --public-access-prevention

# Object আপলোড, তালিকা, ডাউনলোড (illustrative output)
gcloud storage cp ./shoe-123.jpg gs://technova-assets/product-images/
gcloud storage cp gs://technova-assets/product-images/shoe-123.jpg ./downloaded.jpg
gcloud storage ls gs://technova-assets/product-images/ --recursive
gcloud storage ls --long gs://technova-assets/   # size, class, generation দেখায়

# Object-এর metadata ও class বদলানো
gcloud storage objects update gs://technova-assets/invoices/inv-8841.pdf \
  --storage-class=NEARLINE
gcloud storage objects describe gs://technova-assets/product-images/shoe-123.jpg

# পুরো directory sync (rsync) — static site deploy-এর সাধারণ পদ্ধতি
gcloud storage rsync --recursive ./dist/ gs://technova-web-static/
# শুধু মুছে ফেলা ফাইল sync করতে চাইলে --delete-unmatched-destination-objects
# সাবধানে ব্যবহার করুন — ভুল source দিলে destination মুছে যেতে পারে।
```

নতুন `gcloud storage` কমান্ড গ্রুপটি পুরনো `gsutil`-এর উত্তরসূরি। পুরনো স্ক্রিপ্টে `gsutil` দেখলে ভয় পাবেন না — ধারণা একই, তবে নতুন কাজে `gcloud storage` ব্যবহার করুন। প্রতিটি কমান্ড চালানোর আগে `--dry-run` যেখানে সম্ভব সেখানে ব্যবহার করে দেখুন কী ঘটবে।

১২.৮ IAM ও Access Control — Least Privilege

Cloud Storage-এ অ্যাক্সেস নিয়ন্ত্রণের আধুনিক পদ্ধতি হলো **Uniform bucket-level access**: পুরো bucket-এর জন্য একটাই IAM policy, প্রতিটি object-এর আলাদা ACL নেই। এটি সুপারিশকৃত পদ্ধতি — কারণ ACL object-ভিত্তিক, audit করা কঠিন, আর ভুলে public হয়ে যাওয়ার ঝুঁকি বেশি। Uniform চালু করলে `roles/storage.objectViewer`, `roles/storage.objectCreator`, `roles/storage.objectAdmin`-এর মতো role bucket-level-এ বাঁধা হয়।

```
# App-এর service account-কে শুধু পড়া ও লেখার অনুমতি (least privilege)
gcloud storage buckets add-iam-policy-binding gs://technova-assets \
  --member="serviceAccount:app@technova-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
gcloud storage buckets add-iam-policy-binding gs://technova-assets \
  --member="serviceAccount:uploader@technova-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectCreator"

# Public access prevention যাচাই ও IAM policy দেখা
gcloud storage buckets describe gs://technova-assets \
  --format="yaml(iamConfiguration, publicAccessPrevention)"
gcloud storage buckets get-iam-policy gs://technova-assets

# কখনোই করবেন না: allUsers-কে objectViewer দেওয়া (public bucket)
# যদি সত্যিই public static asset দরকার হয়, Cloud CDN + signed URL বা
# Load Balancer backend bucket ব্যবহার করুন — সরাসরি public bucket নয়।
```

> **সতর্কতা:** `roles/storage.admin` বা `roles/owner` কাউকে দিলে তিনি bucket মুছে ফেলতে পারেন, IAM বদলাতে পারেন, public করতে পারেন। Application service account-কে কখনো admin role দেবেন না — শুধু তার প্রয়োজনীয় role দিন। আর `allUsers` / `allAuthenticatedUsers` বাঁধা আছে কি না তা নিয়মিত Cloud Asset Inventory বা Security Command Center দিয়ে যাচাই করুন।

১২.৯ Encryption — Google-Managed, CMEK ও CSEK

Cloud Storage-এ ডেটা rest-এ সবসময় encrypt করা থাকে — এটি ডিফল্ট, বন্ধ করা যায় না। পার্থক্য শুধু কে key পরিচালনা করে। **Google-managed encryption keys (GMEK)**: Google key তৈরি, rotate ও সংরক্ষণ করে — বেশিরভাগ কাজের জন্য যথেষ্ট। **Customer-managed encryption keys (CMEK)**: Cloud KMS-এ আপনার নিজের key দিয়ে encrypt — key rotate, disable ও destroy আপনি নিয়ন্ত্রণ করেন; compliance বা key-revocation প্রয়োজন হলে এটিই পথ। Key disable করলে সেই key দিয়ে encrypt করা object পড়া যায় না — এটি শক্তিশালী নিয়ন্ত্রণ, আবার ভুল করে disable করলে নিজের ডেটাও হারানোর ঝুঁকি। **Customer-supplied encryption keys (CSEK)**: আপনি নিজে key সরবরাহ করেন, Google তা সংরক্ষণ করে না — key হারালে ডেটা চিরতরে হারিয়ে যায়। CSEK শুধু বিশেষ ক্ষেত্রে ব্যবহার করুন। Transit-এ ডেটা সবসময় TLS দিয়ে সুরক্ষিত থাকে।

```
# CMEK দিয়ে bucket তৈরি (KMS key ring ও key আগে তৈরি থাকতে হবে)
gcloud storage buckets create gs://technova-sensitive \
  --location=asia-south1 \
  --default-kms-key=projects/technova-prod/locations/asia-south1/keyRings/technova-ring/
cryptoKeys/storage-key

# KMS key-কে Cloud Storage service agent-এর জন্য অনুমতি দেওয়া আবশ্যক
# (তা না হলে encrypt/decrypt ব্যর্থ হবে)
```

```
gcloud kms keys add-iam-policy-binding storage-key \
  --keyring=technova-ring --location=asia-south1 \
  --member="serviceAccount:service-PROJECT_NUMBER@gs-project-accounts.iam.gserviceaccount.com"
\
  --role="roles/cloudkms.cryptoKeyEncrypterDecrypter"
```

CMEK ব্যবহারের আগে key rotation schedule (যেমন ৯০ দিন) ও key destruction-এর প্রভাব নথিভুক্ত করুন। Production-এ sensitive bucket-এ CMEK + uniform access + public access prevention — এই তিনটি একসঙ্গে ব্যবহার করাই মানদণ্ড।

১২.১০ Signed URL ও Temporary Access

Private bucket-এর object কাউকে সাময়িকভাবে দিতে হলে bucket public করবেন না — signed URL ব্যবহার করুন। Signed URL হলো একটি URL যার সঙ্গে expiry time ও cryptographic signature জোড়া থাকে; যে কেউ সেই সময়ের মধ্যে URL খুলতে পারে, সময় শেষ হলে আর পারে না। এটি প্রযোজ্য: ব্যবহারকারীকে invoice PDF ডাউনলোড দিতে, মোবাইল অ্যাপ থেকে সরাসরি আপলোড নিতে (signed URL দিয়ে PUT), বা তৃতীয় পক্ষকে সীমিত সময়ের জন্য ফাইল দিতে।

```
# Signed URL তৈরি — 1 ঘণ্টার জন্য GET অনুমতি (service account key দিয়ে sign)
gcloud storage sign-url gs://technova-assets/invoices/inv-8841.pdf \
  --duration=1h \
  --http-verb=GET

# Python-এ signed URL (google-cloud-storage লাইব্রেরি) — সার্ভার থেকে তৈরি করুন
# from google.cloud import storage
# client = storage.Client()
# bucket = client.bucket("technova-assets")
# blob = bucket.blob("invoices/inv-8841.pdf")
# url = blob.generate_signed_url(version="v4", expiration=3600, method="GET")
# print(url)  # ক্লায়েন্টকে এই URL দিন, bucket public করবেন না
```

মনে রাখবেন: signed URL তৈরি করতে signing-এর জন্য service account-এর private key বা IAM signBlob অনুমতি দরকার। URL ফাঁস হলে expiry পর্যন্ত যে কেউ ব্যবহার করতে পারে — তাই expiry ছোট রাখুন (মিনিট থেকে কয়েক ঘণ্টা), আর সংবেদনশীল ডেটার ক্ষেত্রে প্রতিটি ডাউনলোডের জন্য নতুন URL তৈরি করুন। Signed policy document ব্যবহার করলে ব্রাউজার থেকে সরাসরি আপলোডে ফাইলের আকার ও content-type-এর সীমাও বেঁধে দেওয়া যায়।

১২.১১ Replication, Redundancy ও Consistency

Cloud Storage-এর durability আসে erasure coding ও geographic distribution থেকে। Regional bucket-এ ডেটা একই region-এর একাধিক zone-এ ছড়িয়ে থাকে; dual-region-এ দুটি region-এ; multi-region-এ আরও বিস্তৃতভাবে। এর ফলে একটি zone বা একটি data center নষ্ট হলেও ডেটা পাওয়া যায়। Google Cloud Storage-এর জন্য **eleven nines (99.999999999%) বার্ষিক durability** দাবি করে — অর্থাৎ ডেটা হারানোর সম্ভাবনা অত্যন্ত কম। কিন্তু durability আর availability এক নয়। Availability (SLA) নির্ভর করে location ও class-এর ওপর; multi-region সাধারণত regional-এর চেয়ে বেশি availability দেয়।

Consistency মডেল: Cloud Storage **strong consistency** দেয় — successful write-এর পরপরই পরবর্তী read নতুন ডেটাই দেখবে, list operation-ও নতুন object দেখাবে। এ কারণে object storage-কে distributed lock বা coordination-এর জন্য ব্যবহার করা ঝুঁকিপূর্ণ নয়, তবে high-frequency overwrite বা hot object-এর ক্ষেত্রে rate limit ও latency

বিবেচনা করুন। Cross-region replication আলাদাভাবে কনফিগার করতে হয় না — location type-ই replication নির্ধারণ করে। কিন্তু এক region থেকে অন্য region-এ ডেটা কপি করতে চাইলে Storage Transfer Service ব্যবহার করুন, হাতে `cp` নয় — Transfer Service checksum যাচাই করে, retry করে, আর বড় স্থানান্তরে নির্ভরযোগ্য।

> **CONCEPT:** Durability মানে ডেটা হারাবে না; availability মানে ডেটা পাওয়া যাবে; consistency মানে সবাই একই সংস্করণ দেখবে। একটি সিস্টেম তিনটিতেই শক্তিশালী হতে পারে, কিন্তু খরচ ও latency আলাদা। Multi-region বেছে নেওয়া মানে বেশি খরচের বিনিময়ে বেশি availability ও geo-redundancy কেনা — TechNova-র product ছবির জন্য তা যুক্তিযুক্ত হতে পারে, কিন্তু দৈনিক লগের জন্য regional-ই যথেষ্ট।

১২.১২ IaC — Terraform দিয়ে Bucket

```
# Terraform — Cloud Storage bucket (variables-এ CHANGE-ME মান বদলান)
resource "google_storage_bucket" "assets" {
  name                        = "technova-assets"
  location                    = "ASIA-SOUTH1"
  storage_class               = "STANDARD"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"
  force_destroy               = false # production-এ false রাখুন

  versioning { enabled = true }

  lifecycle_rule {
    condition { age = 30 }
    action { type = "SetStorageClass" storage_class = "NEARLINE" }
  }
  lifecycle_rule {
    condition { age = 90 }
    action { type = "SetStorageClass" storage_class = "COLDLINE" }
  }
  lifecycle_rule {
    condition { age = 365 }
    action { type = "SetStorageClass" storage_class = "ARCHIVE" }
  }
  lifecycle_rule {
    condition { age = 30 is_live = false }
    action { type = "Delete" }
  }

  # encryption { default_kms_key_name = google_kms_crypto_key.storage.id }
}
resource "google_storage_bucket_iam_member" "app_viewer" {
  bucket = google_storage_bucket.assets.name
  role   = "roles/storage.objectViewer"
  member = "serviceAccount:app@technova-prod.iam.gserviceaccount.com"
}
```

Terraform-এ bucket-এর নাম, location ও lifecycle এক জায়গায় সংজ্ঞায়িত থাকে — Console-এ হাতে বদলানো (click-ops) audit-এ ধরা পড়ে না। Production-এ সব bucket Terraform বা সমতুল্য IaC দিয়ে পরিচালনা করুন, state file-কে remote backend-এ (আলাদা bucket-এ, versioning সহ) রাখুন, আর `terraform plan` ছাড়া `apply` করবেন না।

১২.১৩ Production Scenario — TechNova-র Media ও Log Pipeline

TechNova-র architecture: ব্যবহারকারী ছবি আপলোড করে Cloud Run service-এ; service ছবি যাচাই করে `technova-assets` bucket-এ রাখে, thumbnail তৈরি করে `technova-thumbs` -এ রাখে। Invoice PDF তৈরি হয় Cloud Run job-এ, রাখা হয় `technova-invoices` -এ (Nearline lifecycle সহ)। অ্যাপ্লিকেশন লগ Cloud Logging থেকে sink হয়ে `technova-logs` -এ যায়, lifecycle অনুযায়ী Archive-এ যায় ও ৭ বছর পর মুছে যায়। প্রতিটি bucket-এ public access prevention চালু, uniform access চালু, versioning চালু (logs ছাড়া), আর Cloud Monitoring-এ bucket size ও object count-এর alert বসানো। ডাউনলোডের জন্য ব্যবহারকারীকে সরাসরি bucket URL দেওয়া হয় না — backend signed URL তৈরি করে দেয়। এই ডিজাইনে ছবি দ্রুত পাওয়া যায়, invoice সস্তায় সংরক্ষিত হয়, লগ compliance অনুযায়ী থাকে, আর কোনো bucket-ই ভুল করে public হয় না।

ব্যর্থতার পরিস্থিতি ভাবুন: একটি region-এ বিভ্রাট হলে regional bucket-এর object সেই region-এর ভেতরে অন্য zone থেকে পাওয়া যাবে, কিন্তু region-জুড়ে বিভ্রাট হলে পাওয়া যাবে না — সেজন্য business-critical asset-এর জন্য dual-region বা multi-region বিবেচনা করুন। Bucket মুছে গেলে ডেটা ফেরে না (যদি versioning বা backup না থাকে) — তাই production bucket-এ `force_destroy = false` ও IAM-এ delete অনুমতি সীমিত রাখুন।

১২.১৪ Security, Cost ও Common Mistakes

Security checklist: uniform bucket-level access; public access prevention enforced; least-privilege IAM; CMEK যেখানে compliance দরকার; signed URL-এর ছোট expiry; Cloud Audit Logs চালু (Data Access logs প্রয়োজনে); VPC Service Controls দিয়ে perimeter-এর বাইরে ডেটা যাওয়া আটকানো যেখানে প্রয়োজন। **Cost:** খরচের চারটি উপাদান — storage (GB-মাস), operations (Class A: write/list, Class B: read), retrieval (Nearline/Coldline/Archive থেকে পড়া) ও egress (internet-এ ডেটা বের করা — সবচেয়ে ব্যয়বহুল হতে পারে)। Egress কমাতে একই region-এর ভেতরে compute ও storage রাখুন, আর বড় ডাউনলোডের জন্য Cloud CDN ব্যবহার করুন (অধ্যায় ১৬)। Cost Explorer / Billing reports দিয়ে প্রতি bucket-এর খরচ আলাদা করে দেখুন, label ব্যবহার করুন (যেমন `team:platform, env:prod`)।

- **ভুল ১:** Public bucket করে static site চালানো — বদলে Load Balancer + backend bucket বা CDN ব্যবহার করুন।
- **ভুল ২:** Versioning চালু করে lifecycle না দেওয়া — পুরনো সংস্করণের বিল বাড়তেই থাকে।
- **ভুল ৩:** Archive-এ রাখা ডেটা বারবার পড়া — retrieval ফি সাশ্রয়কে ছাড়িয়ে যায়।
- **ভুল ৪:** Service account-কে storage admin দেওয়া — শুধু প্রয়োজনীয় role দিন।
- **ভুল ৫:** Lifecycle ছাড়া “একদিন মুছব” ভাবা — কেউ মুছে না, বিল বাড়ে।
- **ভুল ৬:** Bucket নামে কোম্পানির আসল গোপন তথ্য বা environment নামের ভুল — নাম global ও অনুমানযোগ্য হতে পারে।

১২.১৫ Troubleshooting

TROUBLESHOOTING: 403 Forbidden on GET: প্রথমে IAM দেখুন — principal-এর `storage.objects.get` আছে কি না; uniform access চালু থাকলে object ACL নয়, bucket IAM-ই নির্ণায়ক। Signed URL হলে expiry ও signing account যাচাই করুন। **BucketAlreadyExists:** নামটি global-এ অন্য কেউ ব্যবহার করছে — prefix বদলান (যেমন `technova-assets-prod-01`)। **Slow upload:** একটিমাত্র বড় object-এর পরিবর্তে parallel composite upload বা `gcloud storage cp`-এর স্বয়ংক্রিয় chunking ব্যবহার করুন; region-এর দূরত্ব যাচাই করুন। **Lifecycle কাজ করছে না:** rule-এর condition (age বনাম createdBefore) ও storage class মিলিয়ে দেখুন; lifecycle asynchronous — পরিবর্তন দেখতে ঘণ্টা লাগতে পারে। **Unexpected bill:** Billing-এ bucket-ভিত্তিক breakdown দেখুন; Class A operation (list) লুপে চলছে কি না, বা egress কোথা থেকে হচ্ছে তা যাচাই করুন।

১২.১৬ সারাংশ

Cloud Storage হলো managed object store: bucket হলো container, object হলো ডেটা + metadata, location ও storage class হলো durability, latency ও খরচের trade-off। Lifecycle, versioning ও retention দিয়ে ডেটার জীবনচক্র স্বয়ংক্রিয় করুন; uniform IAM, public access prevention ও encryption দিয়ে সুরক্ষা দিন; signed URL দিয়ে অস্থায়ী অ্যাক্সেস দিন; আর খরচের হিসাব Pricing Calculator দিয়ে যাচাই করুন। Console দিয়ে শিখুন, CLI ও Terraform দিয়ে production চালান।

১২.১৭ Review Questions

- Object storage ফাইল সিস্টেম থেকে কীভাবে আলাদা? Prefix “folder” আসলে কী?
- Standard, Nearline, Coldline ও Archive-এর মধ্যে কখন কোনটি বেছে নেবেন? Early deletion ফি কখন প্রযোজ্য?
- Uniform bucket-level access কেন fine-grained ACL-এর চেয়ে সুপারিশ করা হয়?
- Versioning চালু করলে lifecycle rule কেন বাধ্যতামূলক হয়ে যায়?
- CMEK, GMEK ও CSEK-এর মধ্যে key নিয়ন্ত্রণের পার্থক্য কী? Key disable করলে কী হয়?
- Signed URL কীভাবে কাজ করে? Expiry ছোট রাখা কেন গুরুত্বপূর্ণ?
- Storage খরচের চারটি উপাদান কী কী? Egress কমানোর দুটি উপায় লিখুন।

১২.১৮ Practical Exercises

- **Exercise ১:** `gcloud storage` দিয়ে `technova-assets-CHANGE-ME` নামে একটি regional bucket তৈরি করুন (uniform access + public access prevention সহ)। একটি ছবি আপলোড করে `ls --long` দিয়ে generation দেখুন।
- **Exercise ২:** Lifecycle JSON লিখে ৩০ দিনে Nearline ও ৯০ দিনে Coldline-এ যাওয়ার rule প্রয়োগ করুন। একটি object-এর class হাতে বদলে যাচাই করুন।
- **Exercise ৩:** একটি service account তৈরি করে শুধু `objectViewer` দিন; `objectCreator` ছাড়া আপলোড চেষ্টা করে 403 দেখুন, তারপর role যোগ করে আবার চেষ্টা করুন।

- **Exercise ৪:** ১ ঘণ্টার signed URL তৈরি করে ব্রাউজারে খুলুন; expiry-র ধারণা যাচাই করতে ছোট duration (যেমন ৬০ সেকেন্ড) দিয়ে পরীক্ষা করুন।
- **Exercise ৫:** Terraform দিয়ে উপরের bucket-টি পুনরায় তৈরি করুন (আগের bucket মুছে), `terraform plan`-এর আউটপুট পড়ে প্রতিটি resource বুঝুন। Pricing Calculator-এ ১০০ GB Standard বনাম Archive-এর মাসিক খরচের তুলনা করুন (দাম পরিবর্তনশীল — Calculator-এর ফলাফলই সত্য)।

অধ্যায় ১৩ • PART III — Storage, Database & Application Services

Chapter 13 — Persistent Storage and Backup

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Persistent Disk ও Hyperdisk-এর পার্থক্য ও ব্যবহার ব্যাখ্যা করতে; VM-এ disk attach, format, mount ও resize করতে; Filestore দিয়ে shared file storage ডিজাইন করতে; snapshot, image ও backup দিয়ে recovery পরিকল্পনা তৈরি করতে; storage performance (IOPS, throughput, latency)-এর trade-off বুঝতে; এবং production-এ ডেটা durability ও disaster recovery আর্কিটেকচার তৈরি করতে। Console ও gcloud CLI দুই পথেই কাজ করবেন, আর Terraform দিয়ে disk ও snapshot policy IaC হিসেবে লিখবেন।

- Persistent Disk (Standard, Balanced, SSD, Extreme) ও Hyperdisk (Balanced, Extreme, ML) বেছে নেওয়া
- Zonal বনাম regional disk, disk resize ও performance tuning
- Filestore (Basic, High Scale, Enterprise) দিয়ে NFS shared storage
- Snapshot schedule, snapshot chain, image ও instant snapshot
- Backup & DR: RPO/RTO নির্ধারণ, cross-region copy, restore পরীক্ষা

১৩.২ Prerequisite Refresher

অধ্যায় ১২-তে আপনি object storage শিখেছেন — object storage ফাইলকে পুরো একটি unit হিসেবে রাখে। Block storage তার উল্টো: ডিস্ককে fixed-size block-এ ভাগ করে, operating system সেখানে ফাইল সিস্টেম (ext4, XFS) তৈরি করে। VM-এর দৃষ্টিতে Persistent Disk একটি সাধারণ ডিস্ক — কিন্তু ভেতরে তা network-attached, replicated storage। Linux-এ `lsblk`, `mkfs.ext4`, `mount` ও `/etc/fstab` জানা থাকলে এই অধ্যায় সহজ হবে। আর অধ্যায় ১২-এর durability ধারণা মনে রাখুন: replication মানেই backup নয় — replication হার্ডওয়্যার ব্যর্থতা থেকে বাঁচায়, মানুষের ভুল (ভুল delete) থেকে নয়; সেজন্য snapshot দরকার।

> **CONCEPT:** Persistent Disk-এর “persistent” মানে হলো VM মুছে গেলেও ডিস্ক থেকে যায়। VM-এর boot disk মুছে ফেলার সময় ডিফল্ট আচরণ দেখে নিন — অনেক সময় VM মুছলে boot disk-ও মুছে যায় যদি auto-delete চালু থাকে। Data disk সাধারণত আলাদা রাখুন, যাতে VM পুনরায় তৈরি করলেও ডেটা অক্ষত থাকে।

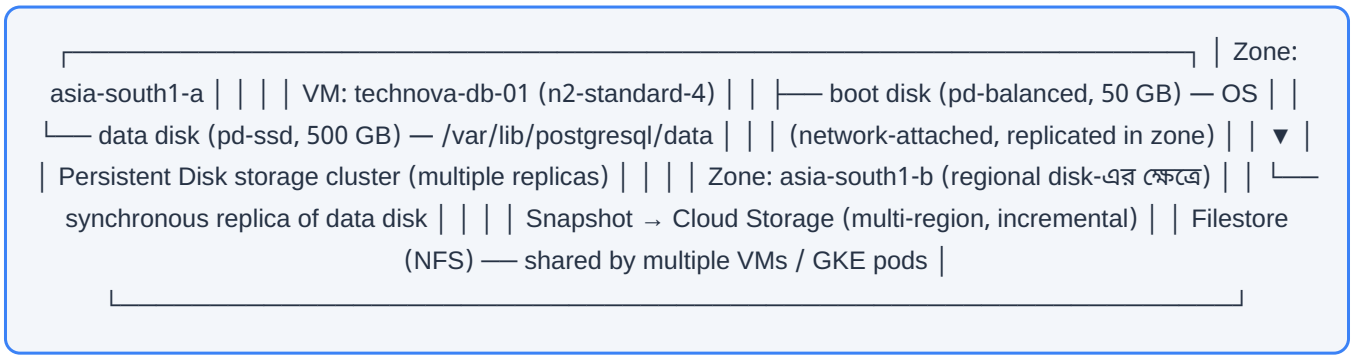
১৩.৩ First Principles — Block Storage কেন দরকার?

ডেটাবেজ, যেকোনো অ্যাপ্লিকেশন যা ফাইল সিস্টেমে লেখে, বা লিগ্যাসি সফটওয়্যার যা POSIX file API আশা করে — এদের সবার block storage দরকার। Object storage-এ আপনি `open()` করে random byte লিখতে পারেন না; block storage-এ পারেন। কিন্তু একটি VM-এর local SSD নষ্ট হলে ডেটা হারিয়ে যায় — সেজন্য Google Persistent Disk-কে network storage হিসেবে ডিজাইন করেছে: ডেটা VM-এর host মেশিনে নয়, storage cluster-এ থাকে, একাধিক replica সহ। VM এক zone থেকে অন্য zone-এ গেলেও (live migration) disk সংযোগ থাকে।

TechNova-র উদাহরণ: তাদের PostgreSQL চলে একটি Compute Engine VM-এ (পরে Cloud SQL-এ যাবে, অধ্যায় ১৪)। ডেটাবেজের ডেটা ডিরেক্টরি (`/var/lib/postgresql/data`) একটি Persistent Disk-এ mount করা। VM রিস্টার্ট

হলে ডেটা থাকে, VM মুছে গেলেও disk আলাদাভাবে থাকে। আর প্রতিদিন রাতে snapshot নেওয়া হয় — ভুল `DROP TABLE` হলে snapshot থেকে ফিরিয়ে আনা যায়।

১৩.৪ আর্কিটেকচার — Disk, VM ও Zone



Zonal disk একটি zone-এর ভেতরে replicate হয়; regional disk দুটি zone-এ synchronous replicate হয় — একটি zone নষ্ট হলেও disk অন্য zone-এ attach করা যায়। Regional disk-এর খরচ বেশি, latency সামান্য বেশি হতে পারে, কিন্তু availability বেশি। Disk-এর আকার ও type performance নির্ধারণ করে: বড় disk সাধারণত বেশি IOPS ও throughput পায় (pd-balanced ও pd-ssd-এ size-এর সঙ্গে performance বাড়ে)।

১৩.৫ Persistent Disk Types ও Hyperdisk

Disk Type	বৈশিষ্ট্য	কখন ব্যবহার করবেন
pd-standard	HDD-backed, সবচেয়ে সস্তা, কম IOPS	বড়, কম-অ্যাক্সেস করা ডেটা, sequential workload
pd-balanced	SSD-backed, ভালো price/performance	বেশিরভাগ সাধারণ workload, boot disk, অ্যাপ ডেটা
pd-ssd	উচ্চ IOPS ও throughput	ডেটাবেজ, latency-sensitive workload
pd-extreme	সবচেয়ে বেশি IOPS, provisioned IOPS	খুব demanding ডেটাবেজ, high-end workload
Hyperdisk Balanced	IOPS/throughput আলাদাভাবে provision করা যায়	নমনীয় performance tuning দরকার হলে
Hyperdisk Extreme / ML	খুব উচ্চ IOPS, ML workload-এর জন্য অপ্টিমাইজড	AI/ML, high-performance analytics

Hyperdisk-এর মূল সুবিধা হলো capacity ও performance আলাদা করা যায় — ছোট disk-এও বেশি IOPS কিনতে পারেন। Persistent Disk-এ তা হয় না; performance size-এর সঙ্গে বাঁধা। তবে Hyperdisk সব machine type ও সব region-এ পাওয়া যায় না — ব্যবহারের আগে ডকুমেন্টেশন যাচাই করুন। দাম অঞ্চল ও type অনুযায়ী বদলায়; Pricing Calculator দিয়ে হিসাব করুন।

> **সতর্কতা:** Disk type একবার তৈরি করার পর সব type-এ পরিবর্তন করা যায় না, আর live disk-এর type বদলাতে downtime লাগতে পারে। Production ডেটাবেজের জন্য শুরুতেই সঠিক type বেছে নিন, আর performance পরীক্ষা করুন load test দিয়ে — অনুমান দিয়ে নয়।

১৩.৬ Console ও gcloud CLI — Disk তৈরি, Attach, Mount

Console-এ: Compute Engine → Disks → Create Disk → নাম, zone, type, size দিন → Create। তারপর VM-এর Edit-এ গিয়ে Additional disks-এ attach করুন, অথবা VM তৈরির সময়ই disk যোগ করুন। CLI-তে পুরো প্রক্রিয়া স্ক্রিপ্ট করা যায় — এটাই production পদ্ধতি।

```
# Disk তৈরি ও VM-এ attach করা
gcloud compute disks create technova-db-data \
  --zone=asia-south1-a \
  --type=pd-ssd \
  --size=500GB
gcloud compute instances attach-disk technova-db-01 \
  --disk=technova-db-data \
  --zone=asia-south1-a

# VM-এর ভেতরে (SSH করে) — illustrative terminal session
# lsblk  # নতুন disk দেখুন, যেমন /dev/sdb
# sudo mkfs.ext4 -m 0 -F -E lazy_itable_init=0,lazy_journal_init=0,discard /dev/sdb
# sudo mkdir -p /var/lib/postgresql/data
# sudo mount -o discard,defaults /dev/sdb /var/lib/postgresql/data
# # স্থায়ী mount-এর জন্য /etc/fstab-এ UUID দিয়ে entry যোগ করুন
# sudo blkid /dev/sdb   # UUID দেখুন
# # /etc/fstab: UUID=CHANGE-ME-UUID /var/lib/postgresql/data ext4 discard,defaults,nofail 0 2
# df -h /var/lib/postgresql/data

# Regional disk তৈরি (দুটি zone-এ synchronous replica)
gcloud compute disks create technova-db-data-regional \
  --region=asia-south1 \
  --replica-zones=asia-south1-a,asia-south1-b \
  --type=pd-ssd \
  --size=500GB

# Disk resize (live, কোনো downtime ছাড়াই বাড়ানো যায়; কমানো যায় না)
gcloud compute disks resize technova-db-data --size=1000GB --zone=asia-south1-a
# তারপর VM-এর ভেতরে: sudo resize2fs /dev/sdb
# Disk-এর তথ্য দেখুন
gcloud compute disks describe technova-db-data --zone=asia-south1-a
gcloud compute disks list --filter="zone:asia-south1-a"
```

`/etc/fstab`-এ device নাম (`/dev/sdb`) নয়, UUID ব্যবহার করুন — রিবুটের পর device নাম বদলে যেতে পারে। `nofail` অপশন দিলে disk attach না হলেও VM বুট হবে; তা না দিলে VM বুট আটকে যেতে পারে।

১৩.৭ Filestore — Shared File Storage (NFS)

একাধিক VM বা GKE pod একই ফাইল সিস্টেম share করতে চাইলে Persistent Disk কাজ করে না — একটি disk একসঙ্গে একটিমাত্র VM-এ read-write mode-এ attach করা যায় (multi-writer কিছু ক্ষেত্রে ব্যতিক্রম)। Filestore হলো

managed NFS service: একটি Filestore instance তৈরি করলে একটি NFS share পান, যা অনেক ক্লায়েন্ট একসঙ্গে mount করতে পারে। ব্যবহার: shared media, content management, home directory, build artifact, GKE-তে ReadWriteMany volume।

Filestore Tier	বৈশিষ্ট্য	কখন ব্যবহার করবেন
Basic HDD / SSD	সাধারণ NFS, zonal	ছোট shared storage, dev/test
High Scale SSD	বড় capacity, উচ্চ throughput	বড় shared workload, media processing
Enterprise	Regional, উচ্চ availability, snapshot	Production shared storage, HA দরকার হলে

```
# Filestore instance তৈরি ও mount (illustrative)
gcloud filestore instances create technova-shared \
  --zone=asia-south1-a \
  --tier=BASIC_SSD \
  --file-share=name="shared",capacity=1TB \
  --network=name="default"
# Instance-এর IP দেখুন
gcloud filestore instances describe technova-shared --zone=asia-south1-a \
  --format="value(networks[0].ipAddresses[0])"
# VM-এর ভেতরে mount:
# sudo apt-get install -y nfs-common
# sudo mkdir -p /mnt/shared
# sudo mount 10.0.0.2:/shared /mnt/shared
# # /etc/fstab: 10.0.0.2:/shared /mnt/shared nfs defaults,_netdev 0 0
```

Filestore শুধু VPC-এর ভেতর থেকে অ্যাক্সেস করা যায় — public internet থেকে নয়। এটি নিরাপত্তার জন্য ভালো, কিন্তু on-premises থেকে অ্যাক্সেস করতে হলে VPN বা Interconnect দরকার। NFS-এর performance latency-sensitive ডেটাবেজের জন্য উপযুক্ত নয় — ডেটাবেজের জন্য block storage বা Cloud SQL ব্যবহার করুন।

১৩.৮ Snapshots, Images ও Backup

Snapshot হলো disk-এর point-in-time copy, incremental — প্রথম snapshot পুরো disk, পরেরগুলো শুধু বদলে যাওয়া block। Snapshot Cloud Storage-এ সংরক্ষিত হয়, multi-region redundancy সহ। Snapshot থেকে নতুন disk তৈরি করা যায়, অন্য zone বা region-এ disk পুনরুদ্ধার করা যায়, বা VM-এর boot disk হিসেবে ব্যবহার করা যায়। **Image** হলো bootable disk-এর template — একটি কনফিগার করা OS + সফটওয়্যার থেকে image বানিয়ে একই রকম VM বারবার তৈরি করা যায়। **Instant snapshot** হলো দ্রুত, in-zone snapshot — দ্রুত disk clone বা rollback-এর জন্য।

```
# হাতে snapshot নেওয়া
gcloud compute disks snapshot technova-db-data \
  --snapshot-names=technova-db-2026-11-09 \
  --zone=asia-south1-a

# Snapshot থেকে নতুন disk তৈরি (অন্য zone-এ)
gcloud compute disks create technova-db-restored \
  --source-snapshot=technova-db-2026-11-09 \
  --zone=asia-south1-b \
  --type=pd-ssd

# Snapshot schedule — প্রতিদিন রাত ২টায়, ১৪ দিন ধরে রাখা
```

```
gcloud compute resource-policies create snapshot-schedule technova-daily \
  --region=asia-south1 \
  --max-retention-days=14 \
  --on-source-disk-delete=keep-auto-snapshots \
  --daily-schedule --start-time=02:00
gcloud compute disks add-resource-policies technova-db-data \
  --resource-policies=technova-daily \
  --zone=asia-south1-a
# Schedule-এর snapshot দেখুন
gcloud compute snapshots list --filter="name:technova-db-data*"

# Image তৈরি (boot disk থেকে)
gcloud compute images create technova-app-image-v1 \
  --source-disk=technova-app-01 --source-disk-zone=asia-south1-a \
  --family=technova-app
# Image থেকে VM তৈরি
gcloud compute instances create technova-app-02 \
  --image-family=technova-app --image-project=technova-prod \
  --zone=asia-south1-a --machine-type=e2-medium
```

> **CONCEPT:** Snapshot incremental হলেও প্রতিটি snapshot স্বাধীনভাবে restore করা যায় — মাঝের snapshot মুছে ফেললেও পরের snapshot কাজ করে, কারণ Google ভেতরে প্রয়োজনীয় block ধরে রাখে। তাই পুরনো snapshot মুছে ফেলতে ভয় পাবেন না, তবে retention policy লেখার আগে restore পরীক্ষা করে নিন।

১৩.৯ Storage Performance — IOPS, Throughput, Latency

Performance-এর তিনটি মাপ: **IOPS** (প্রতি সেকেন্ডে operation সংখ্যা — ছোট random read/write-এর জন্য গুরুত্বপূর্ণ), **throughput** (MB/s — বড় sequential read/write-এর জন্য), **latency** (একটি operation-এর সময় — ডেটাবেজের জন্য সবচেয়ে গুরুত্বপূর্ণ)। pd-ssd-তে size বাড়লে IOPS ও throughput বাড়ে, একটি সীমা পর্যন্ত। VM-এর machine type-ও সীমা নির্ধারণ করে — ছোট VM বেশি disk performance পায় না, যত বড় disk-ই লাগান। Performance সমস্যা হলে প্রথমে দেখুন: disk utilization, VM-এর CPU, নাকি অ্যাপ্লিকেশনের query? `iostat`, `vmstat` ও Cloud Monitoring-এর disk metrics (read/write bytes, queue depth) দিয়ে নির্ণয় করুন।

Hyperdisk-এ IOPS ও throughput আলাদাভাবে provision করা যায় — ডেটাবেজের জন্য IOPS বেশি, throughput কম দরকার হলে সেটাই কিনুন। কিন্তু provisioned IOPS-এর খরচ বেশি — load test ছাড়া বেশি IOPS কিনবেন না। আর একটি সতর্কতা: snapshot নেওয়ার সময় disk-এর performance সাময়িকভাবে কমতে পারে; production ডেটাবেজের snapshot নেওয়ার আগে low-traffic সময় বেছে নিন, বা Cloud SQL-এর managed backup ব্যবহার করুন (অধ্যায় ১৪)।

১৩.১০ IaC — Terraform দিয়ে Disk ও Snapshot Policy

```
# Terraform — data disk + snapshot schedule
resource "google_compute_disk" "db_data" {
  name  = "technova-db-data"
  type  = "pd-ssd"
  zone  = "asia-south1-a"
  size  = 500
  labels = { team = "platform", env = "prod" }
}
resource "google_compute_resource_policy" "daily_snapshot" {
```

```
  name   = "technova-daily"
  region = "asia-south1"
  snapshot_schedule_policy {
    schedule { daily { start_time = "02:00" } }
    retention_policy {
      max_retention_days    = 14
      on_source_disk_delete = "KEEP_AUTO_SNAPSHOTS"
    }
  }
}
resource "google_compute_disk_resource_policy_attachment" "db_snapshot" {
  name = google_compute_resource_policy.daily_snapshot.name
  disk = google_compute_disk.db_data.name
  zone = "asia-south1-a"
}
resource "google_filestore_instance" "shared" {
  name     = "technova-shared"
  location = "asia-south1-a"
  tier     = "BASIC_SSD"
  file_shares { capacity_gb = 1024 name = "shared" }
  networks { network = "default" modes = ["MODE_IPV4"] }
}
```

১৩.১১ Production Scenario — Backup ও Disaster Recovery

TechNova-র RPO (Recovery Point Objective — কতটুকু ডেটা হারানো acceptable) ১ ঘণ্টা, RTO (Recovery Time Objective — কতক্ষণে ফিরতে হবে) ৪ ঘণ্টা। ডিজাইন: ডেটাবেজ disk-এ দৈনিক snapshot + transaction log backup; snapshot cross-region copy (Storage Transfer বা snapshot-এর multi-region storage); Filestore Enterprise (regional) shared asset-এর জন্য; আর প্রতি মাসে একটি restore drill — snapshot থেকে নতুন VM তৈরি করে ডেটা যাচাই করা। Drill ছাড়া backup পরিকল্পনা কাগজেই থাকে — restore কখনো পরীক্ষা না করলে জানবেন না তা কাজ করে কি না।

ব্যর্থতার দৃশ্য: (১) VM crash — disk অক্ষত থাকে, নতুন VM-এ attach করলেই চলে। (২) Zone বিভ্রাট — regional disk অন্য zone-এ attach করা যায়; zonal disk হলে snapshot থেকে অন্য zone-এ disk তৈরি করতে হয়। (৩) ভুল delete (`rm -rf` বা ভুল SQL) — snapshot থেকে ফিরিয়ে আনতে হয়; replication এখানে সাহায্য করে না, কারণ ভুলটি replicate হয়। (৪) Region বিভ্রাট — cross-region snapshot copy বা Cloud SQL-এর cross-region replica (অধ্যায় ১৪) দরকার।

১৩.১২ Security, Cost ও Common Mistakes

Security: Disk encryption at rest ডিফল্ট (Google-managed); CMEK ব্যবহার করলে Cloud KMS key দিয়ে encrypt করুন। Disk-এর IAM: কে disk attach/detach করতে পারে তা সীমিত করুন; snapshot share করার সময় project সীমিত রাখুন — public snapshot করবেন না। Filestore-এ IP-based access control ও VPC-এর ভেতরে সীমাবদ্ধতা ব্যবহার করুন। **Cost:** Disk-এর খরচ provisioned size অনুযায়ী, ব্যবহার অনুযায়ী নয় — ৫০০ GB disk-এ ১০০ GB ডেটা থাকলেও ৫০০ GB-এর বিল আসে। অব্যবহৃত disk, পুরনো snapshot, orphaned disk (VM মুছে গেছে কিন্তু disk রয়ে গেছে) নিয়মিত পরিষ্কার করুন। Snapshot-এর খরচ incremental, কিন্তু অনেক snapshot জমলে বিল বাড়ে — retention সীমিত রাখুন। দাম যাচাই করুন Pricing Calculator-এ।

- **ভুল ১:** Boot disk-এ ডেটা রাখা — VM মুছলে ডেটা হারায়। Data disk আলাদা রাখুন।

- **ভুল ২:** Snapshot না নিয়ে disk resize বা type বদলানো — আগে snapshot নিন।
- **ভুল ৩:** `/etc/fstab`-এ device নাম ব্যবহার করা — UUID ব্যবহার করুন।
- **ভুল ৪:** Backup নেওয়া কিন্তু restore পরীক্ষা না করা।
- **ভুল ৫:** Filestore-এ ডেটাবেজ চালানো — NFS latency ডেটাবেজের জন্য উপযুক্ত নয়।
- **ভুল ৬:** Orphaned disk ও snapshot ফেলে রাখা — মাসিক clean-up অটোমেট করুন।

১৩.১৩ Troubleshooting

TROUBLESHOOTING: Disk attach হচ্ছে না: disk ও VM একই zone-এ আছে কি না দেখুন; disk already অন্য VM-এ attached কি না যাচাই করুন। **Mount ব্যর্থ:** `dmesg | tail` ও `lsblk -f` দিয়ে দেখুন disk চেনা যাচ্ছে কি না; filesystem তৈরি হয়েছে কি না যাচাই করুন। **Disk full:** `df -h` ও `du -sh /*` দিয়ে বড় ফাইল খুঁজুন; resize করার আগে snapshot নিন। **Slow disk:** Cloud Monitoring-এ disk read/write latency ও queue depth দেখুন; disk size বাড়ানো বা type বদলানো (pd-balanced → pd-ssd) বিবেচনা করুন; VM-এর machine type-এর সীমাও যাচাই করুন। **Snapshot ব্যর্থ:** disk-এর resource policy ও quota যাচাই করুন; snapshot storage quota শেষ হয়েছে কি না দেখুন। **Filestore mount ব্যর্থ:** VPC firewall-এ NFS port (2049) খোলা আছে কি না ও instance IP সঠিক কি না যাচাই করুন।

১৩.১৪ সারাংশ

Persistent Disk হলো VM-এর network-attached block storage — zonal বা regional, type অনুযায়ী performance আলাদা, Hyperdisk-এ performance আলাদাভাবে provision করা যায়। Filestore হলো shared NFS storage। Snapshot incremental backup দেয়, image bootable template দেয়, আর restore drill ছাড়া কোনো backup পরিকল্পনাই সম্পূর্ণ নয়। Disk-এর খরচ provisioned size অনুযায়ী — অব্যবহৃত disk পরিষ্কার করুন, আর RPO/RTO অনুযায়ী snapshot schedule ডিজাইন করুন।

১৩.১৫ Review Questions

- Zonal ও regional disk-এর মধ্যে availability ও খরচের trade-off কী?
- pd-balanced, pd-ssd ও Hyperdisk Balanced-এর মধ্যে কখন কোনটি বেছে নেবেন?
- Snapshot incremental হলেও প্রতিটি snapshot স্বাধীনভাবে restore করা যায় কেন?
- Replication ও backup-এর পার্থক্য কী? ভুল delete থেকে কোনটি বাঁচায়?
- Filestore কখন ব্যবহার করবেন, আর কখন Persistent Disk?
- RPO ও RTO কী? TechNova-র RPO ১ ঘণ্টা হলে snapshot schedule কেমন হওয়া উচিত?
- Disk resize করার আগে কেন snapshot নেওয়া উচিত?

১৩.১৬ Practical Exercises

- **Exercise ১:** একটি ১০০ GB pd-balanced disk তৈরি করে VM-এ attach করুন, format ও mount করুন, `/etc/fstab`-এ UUID দিয়ে স্থায়ী করুন। VM রিবুট করে mount অক্ষত আছে কি না যাচাই করুন।

- **Exercise ২:** Disk-এ কিছু ফাইল লিখে snapshot নিন, ফাইল মুছে ফেলুন, snapshot থেকে নতুন disk তৈরি করে ফাইল ফিরিয়ে আনুন।
- **Exercise ৩:** Snapshot schedule তৈরি করে disk-এ attach করুন; পরের দিন snapshot তৈরি হয়েছে কি না যাচাই করুন।
- **Exercise ৪:** Disk-কে ১০০ GB থেকে ২০০ GB-তে resize করুন (আগে snapshot নিন), VM-এর ভেতরে filesystem resize করে যাচাই করুন।
- **Exercise ৫:** Filestore Basic instance তৈরি করে দুটি VM থেকে একই share mount করুন; একটি VM থেকে লেখা ফাইল অন্যটিতে দেখা যায় কি না যাচাই করুন। অব্যবহৃত disk ও snapshot তালিকা করে clean-up পরিকল্পনা লিখুন।

অধ্যায় ১৪ • PART III — Storage, Database & Application Services

Chapter 14 — Cloud SQL and AlloyDB

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — managed relational database কেন নিজে VM-এ ডেটাবেজ চালানোর চেয়ে আলাদা তা ব্যাখ্যা করতে; Cloud SQL ও AlloyDB-এর architecture, engine, high availability, read replica ও backup মডেল ডিজাইন করতে; private IP, IAM database authentication, SSL/TLS ও Secret Manager দিয়ে ডেটাবেজ সুরক্ষিত করতে; point-in-time recovery (PITR) পরীক্ষা করতে; এবং production-এ connection pooling, monitoring ও cost নিয়ন্ত্রণসহ ডেটাবেজ পরিচালনা করতে। Console ও gcloud CLI দুই পথেই কাজ করবেন, Terraform দিয়ে instance IaC হিসেবে লিখবেন।

- Cloud SQL (PostgreSQL, MySQL, SQL Server) বনাম AlloyDB (PostgreSQL-compatible) বেছে নেওয়া
- HA, read replica, cross-region replica ও failover ডিজাইন
- Automated backup, PITR, on-demand backup ও restore পরীক্ষা
- Private IP, Private Service Connect, Cloud SQL Auth Proxy ও IAM authentication
- Connection limit, pooling (PgBouncer/Cloud SQL Proxy) ও performance tuning

১৪.২ Prerequisite Refresher

অধ্যায় ১৩-তে আপনি Persistent Disk শিখেছেন — নিজে VM-এ PostgreSQL চালালে disk, backup, patching, replication সব আপনাকে করতে হয়। Cloud SQL সেই কাজগুলো managed service হিসেবে নেয়। SQL জানা থাকলে এই অধ্যায় সহজ হবে: table, index, transaction, ACID। মনে রাখুন ACID-এর অর্থ — Atomicity, Consistency, Isolation, Durability — relational database-এর মূল প্রতিশ্রুতি। আর অধ্যায় ১২-এর IAM ও encryption ধারণা এখানেও প্রযোজ্য: ডেটাবেজের অ্যাক্সেসও least-privilege দিয়ে নিয়ন্ত্রণ করতে হয়, পাসওয়ার্ড কখনো কোডে লেখা যায় না।

> **CONCEPT:** Managed database মানে “ডেটাবেজের যত্ন নেওয়ার দরকার নেই” নয় — মানে যত্নের ধরন বদলে যায়। Google patching, backup, HA ও storage পরিচালনা করে; কিন্তু schema ডিজাইন, index, query tuning, connection management ও অ্যাক্সেস নিয়ন্ত্রণ আপনার দায়িত্বই থাকে। Managed service-এও খারাপ query ডেটাবেজকে ধীর করে দিতে পারে।

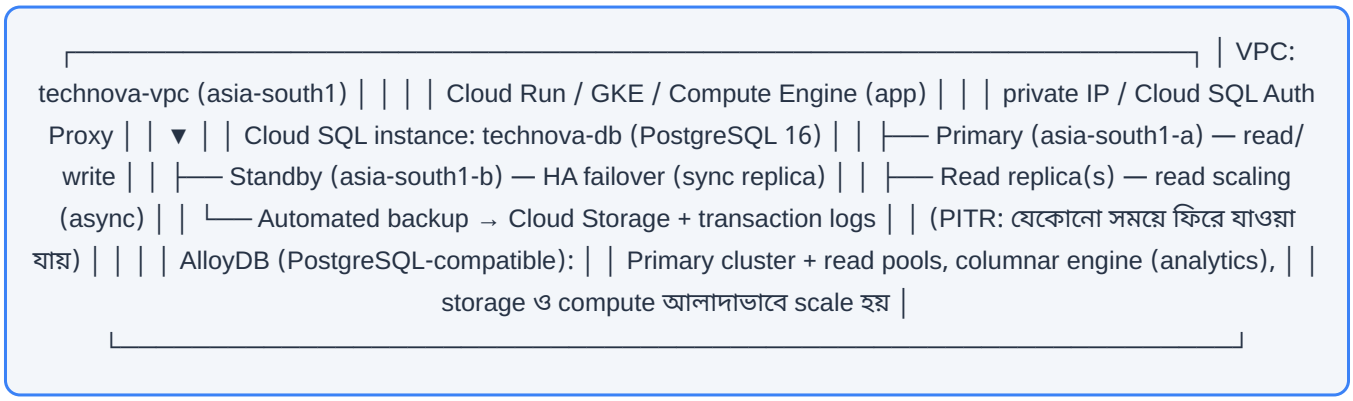
১৪.৩ First Principles — Managed Relational Database কেন?

নিজে ডেটাবেজ চালাতে গেলে যা যা করতে হয় ভাবুন: OS patch, database version upgrade, backup নেওয়া ও যাচাই করা, replication সেটআপ, failover পরীক্ষা, disk full হলে বাড়ানো, connection limit সামলানো। প্রতিটি কাজে ভুলের সুযোগ আছে, আর ভুলের মূল্য ডেটা হারানো। Cloud SQL এই operational কাজগুলো স্বয়ংক্রিয় করে — আপনি instance তৈরি করেন, Google ভেতরে VM, disk, replication ও backup পরিচালনা করে। আপনি মনোযোগ দেন schema, query ও অ্যাপ্লিকেশন লজিকে।

TechNova-র e-commerce-এ অর্ডার, পেমেন্ট ও inventory — এই ডেটা relational: অর্ডার ও পেমেন্টের মধ্যে consistency দরকার, transaction দরকার। এখানে PostgreSQL উপযুক্ত। তাদের analytics workload আলাদা —

সেখানে AlloyDB বা BigQuery বিবেচনা করা যায়। সঠিক ডেটাবেজ বেছে নেওয়া মানে workload-এর consistency, scale ও latency-এর প্রয়োজন বোঝা।

১৪.৪ আর্কিটেকচার — Cloud SQL কীভাবে কাজ করে



Cloud SQL instance-এর ভেতরে: একটি VM (আপনি দেখতে পান না), একটি Persistent Disk (ডেটা), আর Google-এর control plane যা backup, patching ও failover পরিচালনা করে। HA চালু করলে একটি standby instance অন্য zone-এ synchronous replication করে — primary নষ্ট হলে স্বয়ংক্রিয় failover হয়, সাধারণত এক মিনিটের মধ্যে। Read replica asynchronous — primary-তে লেখা কিছুক্ষণ পর replica-তে দেখা যায় (replication lag)। এই lag-এর কারণে read replica থেকে পড়া ডেটা সামান্য পুরনো হতে পারে — অ্যাপ্লিকেশন ডিজাইনে তা মাথায় রাখুন।

Supported engines: Cloud SQL-এ PostgreSQL, MySQL ও SQL Server পাওয়া যায়। প্রতিটি engine-এর সংস্করণ (version) Google সমর্থন করে — পুরনো সংস্করণ একসময় অবসর (deprecate) হয়, তাই সংস্করণের lifecycle নজরে রাখুন। **AlloyDB** হলো PostgreSQL-compatible managed service, যা Google-এর নিজস্ব storage ও compute architecture ব্যবহার করে — উচ্চ throughput, columnar engine দিয়ে analytics query দ্রুত করা, আর storage/compute আলাদাভাবে scale করার সুবিধা দেয়। AlloyDB সাধারণ PostgreSQL-এর চেয়ে বেশি demanding workload-এর জন্য ডিজাইন করা হয়েছে।

১৪.৫ Cloud SQL তৈরি — Console ও gcloud CLI

Console-এ: SQL → Create Instance → PostgreSQL বেছে নিন → instance ID, password, version, region (asia-south1), zone, machine type, storage type ও size দিন → Connectivity-তে Private IP বেছে নিন (নিচে ১৪.৭ দেখুন) → Create। Console সহজ, কিন্তু instance-এর প্রতিটি সেটিং IaC-তে লিখে রাখাই production মানদণ্ড।

```
# Cloud SQL PostgreSQL instance তৈরি (private IP সহ — আগে VPC peering সেটআপ দরকার)
gcloud sql instances create technova-db \
  --database-version=POSTGRES_16 \
  --region=asia-south1 \
  --tier=db-custom-2-7680 \
  --storage-type=SSD \
  --storage-size=100GB \
  --storage-auto-increase \
  --network=projects/technova-prod/global/networks/technova-vpc \
  --no-assign-ip \
  --root-password='CHANGE-ME-STRONG-PASSWORD'

# Instance-এর তথ্য ও IP দেখুন
gcloud sql instances describe technova-db --format="yaml(ipAddresses, settings)"
```

```
gcloud sql instances list

# Database ও user তৈরি
gcloud sql databases create technova_app --instance=technova-db
gcloud sql users create app_user --instance=technova-db \
  --password='CHANGE-ME-APP-PASSWORD'
# App user-কে শুধু প্রয়োজনীয় privilege দিন — superuser নয়
# psql-এ সংযোগ করে: GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
app_user;

# MySQL instance (উদাহরণ)
# gcloud sql instances create technova-mysql --database-version=MYSQL_8_0 \
#   --region=asia-south1 --tier=db-n1-standard-2 --storage-size=100GB
```

Root password ও app password কখনো terminal history বা কোডে রাখবেন না — তৈরি করার পর Secret Manager-এ সংরক্ষণ করুন (১৪.৮ দেখুন)। `--storage-auto-increase` চালু রাখলে disk full হয়ে ডেটাবেজ থেমে যাওয়ার ঝুঁকি কমে, তবে খরচের alert বসান — auto-increase বিল বাড়াতে পারে।

১৪.৬ High Availability ও Read Replicas

```
# HA চালু করা (standby অন্য zone-এ তৈরি হয়)
gcloud sql instances patch technova-db --availability-type=REGIONAL
# HA অবস্থা যাচাই
gcloud sql instances describe technova-db --format="value(settings.availabilityType)"

# Read replica তৈরি (read scaling-এর জন্য)
gcloud sql instances create technova-db-replica \
  --master-instance-name=technova-db \
  --region=asia-south1 \
  --tier=db-custom-2-7680
# Replica-র replication lag দেখুন (Cloud Monitoring-এ cloudsql.googleapis.com/database/
replication/replica_lag)

# Cross-region read replica (DR-এর জন্য, অন্য region-এ)
# gcloud sql instances create technova-db-dr \
#   --master-instance-name=technova-db --region=asia-southeast1 \
#   --tier=db-custom-2-7680
# Failover পরীক্ষা (maintenance window-এর বাইরে সাবধানে)
# gcloud sql instances failover technova-db
```

HA failover স্বয়ংক্রিয়, কিন্তু অ্যাপ্লিকেশনকে reconnect করতে হয় — connection pooler ও retry logic থাকতে হয়। Read replica-তে লেখা যায় না — লেখার চেষ্টা করলে error আসে। Cross-region replica disaster recovery-এর জন্য ভালো, কিন্তু replication lag বেশি হতে পারে ও egress খরচ লাগে। Failover পরীক্ষা production-এ করার আগে staging-এ করুন, আর maintenance window নির্ধারণ করুন।

> **সতর্কতা:** HA মানে backup নয়। HA zone বিভ্রাট থেকে বাঁচায়, কিন্তু ভুল `DELETE` বা `DROP TABLE` standby-তেও replicate হয়। সেজন্য backup ও PITR আলাদাভাবে দরকার — পরের section দেখুন।

১৪.৭ Private IP Connectivity ও Cloud SQL Auth Proxy

Cloud SQL-কে public IP দিয়ে internet থেকে খোলা রাখা নিরাপত্তা ঝুঁকি। সুপারিশকৃত পদ্ধতি: **Private IP** — VPC-এর ভেতরে private address দিয়ে সংযোগ, public IP বন্ধ। Private IP-এর জন্য VPC-তে private services access (VPC peering) সেটআপ করতে হয় — একবার করলে সব Cloud SQL instance সেই VPC থেকে private IP পায়। **Cloud SQL Auth Proxy** হলো একটি sidecar/process যা আপনার অ্যাপ্লিকেশন ও Cloud SQL-এর মধ্যে TLS-encrypted সংযোগ তৈরি করে, IAM দিয়ে authenticate করে — public IP থাকলেও proxy ছাড়া সংযোগ হয় না। Cloud Run, GKE ও Compute Engine থেকে proxy বা private IP — দুটোই ব্যবহার করা যায়; Cloud Run-এর জন্য Cloud SQL connection (Unix socket) সেটআপ সবচেয়ে সহজ।

```
# Private IP instance-এর জন্য VPC peering (একবার, project-এ)
# Console: VPC network → Private services access → Allocate IP range → Create connection
# CLI-তে address ও peering তৈরি:
gcloud compute addresses create google-managed-services-technova-vpc \
  --global --purpose=VPC_PEERING --prefix-length=16 \
  --network=technova-vpc
gcloud services vpc-peerings connect \
  --service=servicenetworking.googleapis.com \
  --ranges=google-managed-services-technova-vpc \
  --network=technova-vpc

# Cloud SQL Auth Proxy চালানো (local / VM-এ, illustrative)
# curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/
v2.XX.X/cloud-sql-proxy.linux.amd64
# chmod +x cloud-sql-proxy
# ./cloud-sql-proxy technova-prod:asia-south1:technova-db --port 5432
# তারপর psql -h 127.0.0.1 -U app_user -d technova_app
# Cloud Run-এ: --add-cloudsql-instances=technova-prod:asia-south1:technova-db
#   দিয়ে deploy করলে /cloudsql/technova-prod:asia-south1:technova-db socket পাওয়া যায়।
```

Private IP চালু করার পরও authorized networks-এ `0.0.0.0/0` রাখবেন না। SSL/TLS চালু রাখুন — Cloud SQL-এ server CA দিয়ে সংযোগ encrypt করা যায়। আর database user-এর জন্য IAM database authentication ব্যবহার করলে পাসওয়ার্ডের পরিবর্তে IAM token দিয়ে authenticate করা যায় — PostgreSQL ও MySQL-এ সমর্থিত, তবে সব অ্যাপ্লিকেশনে নয়।

১৪.৮ Database Security — IAM, Secret Manager ও Encryption

ডেটাবেজ নিরাপত্তার স্তরগুলো: **Network:** private IP, authorized networks সীমিত, VPC firewall। **Authentication:** প্রতিটি অ্যাপ্লিকেশনের জন্য আলাদা database user, least-privilege GRANT, IAM database authentication যেখানে সম্ভব। **Secrets:** পাসওয়ার্ড Secret Manager-এ রাখুন, কোডে বা environment variable-এ plaintext-এ নয়; Cloud Run/GKE থেকে Secret Manager-এর মাধ্যমে mount করুন। **Encryption:** at rest ডিফল্ট encrypt, CMEK ব্যবহার করলে Cloud KMS key দিয়ে encrypt করুন; in transit SSL/TLS। **Audit:** Cloud Audit Logs ও database log (pgAudit, Cloud SQL Insights) চালু করুন — কে কী query চালাল তা দেখা যায়।

```
# Secret Manager-এ DB password রাখা ও Cloud Run-এ ব্যবহার
echo -n 'CHANGE-ME-APP-PASSWORD' | gcloud secrets create technova-db-password \
  --data-file=-
gcloud secrets add-iam-policy-binding technova-db-password \
  --member="serviceAccount:app@technova-prod.iam.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```

```
# Cloud Run deploy-এ secret mount:
# gcloud run deploy technova-app --image=gcr.io/technova-prod/app:v1 \
#   --add-cloudsql-instances=technova-prod:asia-south1:technova-db \
#   --set-secrets=DB_PASSWORD=technova-db-password:latest \
#   --set-env-vars=DB_HOST=/cloudsql/technova-prod:asia-south1:technova-
db,DB_USER=app_user,DB_NAME=technova_app

# SSL enforce করা
gcloud sql instances patch technova-db --require-ssl
# CMEK দিয়ে instance তৈরি (KMS key আগে তৈরি করুন)
# gcloud sql instances create technova-db-cmek --database-version=POSTGRES_16 \
#   --disk-encryption-key=projects/technova-prod/locations/asia-south1/keyRings/technova-ring/
cryptoKeys/sql-key \
#   --region=asia-south1 --tier=db-custom-2-7680
```

সতর্কতা: Database password Git-এ commit করবেন না, Slack-এ পাঠাবেন না, screenshot-এ রাখবেন না। পাসওয়ার্ড ফাঁস হলে সঙ্গে সঙ্গে rotate করুন — Secret Manager-এ নতুন version যোগ করে অ্যাপ্লিকেশন পুনরায় deploy করুন। Root user দিয়ে অ্যাপ্লিকেশন চালাবেন না — root শুধু প্রশাসনিক কাজের জন্য।

১৪.৯ Backups ও Point-in-Time Recovery (PITR)

```
# Automated backup ও PITR চালু করা
gcloud sql instances patch technova-db \
  --backup-start-time=02:00 \
  --enable-point-in-time-recovery \
  --retained-backups-count=14 \
  --retained-transaction-log-days=7

# On-demand backup নেওয়া
gcloud sql backups create --instance=technova-db --description="before-migration-2026-11-09"
gcloud sql backups list --instance=technova-db

# PITR — নির্দিষ্ট সময়ে restore (নতুন instance-এ, মূল instance অক্ষত রেখে)
gcloud sql instances clone technova-db technova-db-pitr \
  --point-in-time="2026-11-09T10:30:00Z"
# Backup থেকে restore (মূল instance-এ — সাবধানে, ডেটা overwrite হয়)
# gcloud sql backups restore BACKUP_ID --restore-instance=technova-db
```

PITR কাজ করে automated backup + transaction log (WAL) দিয়ে — backup-এর পরের প্রতিটি transaction log সংরক্ষিত থাকে, তাই যেকোনো সময়ে ফিরে যাওয়া যায়। কিন্তু PITR-এর সীমা আছে: retained transaction log দিন সংখ্যা (যেমন ৭ দিন) — তার আগের সময়ে ফেরা যায় না। Restore সবসময় নতুন instance-এ (clone) পরীক্ষা করুন — মূল instance-এ restore করলে বর্তমান ডেটা হারিয়ে যায়। প্রতি মাসে একটি PITR drill করুন: clone করে ডেটা যাচাই করুন, তারপর clone মুছে ফেলুন।

১৪.১০ AlloyDB Concepts

AlloyDB PostgreSQL-compatible, কিন্তু ভেতরের architecture আলাদা: storage ও compute আলাদা স্তর, storage-এ intelligent caching, আর columnar engine analytics query-এর জন্য। এর ফলে একই instance-এ transactional (OLTP) ও analytical (OLAP) workload — দুটোই ভালো চলে। AlloyDB-তে primary cluster, read pool (read

replica-এর মতো), automated backup ও PITR আছে। AlloyDB সাধারণত Cloud SQL PostgreSQL-এর চেয়ে বেশি throughput ও কম latency দেয়, কিন্তু খরচ বেশি ও সেটআপ কিছুটা জটিল। কখন AlloyDB? যখন PostgreSQL-এর performance সীমায় পৌঁছে গেছে, analytics query transactional ডেটাবেজকে ধীর করছে, বা storage/compute আলাদাভাবে scale করতে চান। কখন নয়? ছোট workload, বাজেট সীমিত, বা SQL Server/MySQL দরকার হলে — AlloyDB শুধু PostgreSQL-compatible।

```
# AlloyDB cluster তৈরি (উদাহরণ — সঠিক flag ডকুমেন্টেশন দিয়ে যাচাই করুন)
gcloud alloydb clusters create technova-alloydb \
  --region=asia-south1 \
  --password='CHANGE-ME-STRONG-PASSWORD'
gcloud alloydb instances create technova-alloydb-primary \
  --cluster=technova-alloydb --region=asia-south1 \
  --instance-type=PRIMARY --cpu-count=4
# Read pool instance যোগ করা
# gcloud alloydb instances create technova-alloydb-reader \
#   --cluster=technova-alloydb --region=asia-south1 \
#   --instance-type=READ_POOL --cpu-count=4 --read-pool-node-count=2
```

AlloyDB-এর দাম ও উপলভ্যতা region অনুযায়ী বদলায় — Pricing Calculator দিয়ে Cloud SQL-এর সঙ্গে তুলনা করুন। Migration-এর জন্য Database Migration Service (DMS) ব্যবহার করুন — হাতে dump/restore করলে downtime বেশি হয়।

১৪.১১ Production Database Operations — Connection, Monitoring ও Cost

Production ডেটাবেজের সবচেয়ে সাধারণ ব্যর্থতা connection exhaustion: প্রতিটি অ্যাপ্লিকেশন instance অনেক connection খুলে রাখে, ডেটাবেজের max connection শেষ হয়ে যায়। সমাধান: connection pooler — PgBouncer (PostgreSQL) বা Cloud SQL-এর built-in connection management, আর অ্যাপ্লিকেশনে pool size সীমিত রাখা। Cloud Run-এর মতো serverless-এ প্রতিটি instance আলাদা pool খোলে — max instances সীমিত রাখুন বা pooler ব্যবহার করুন।

Monitoring: Cloud Monitoring-এ দেখুন — CPU, memory, disk usage, active connections, replication lag, query latency (Query Insights / Cloud SQL Insights)। Slow query log চালু করে নিয়মিত পর্যালোচনা করুন; missing index খুঁজে বের করুন। **Maintenance:** minor version upgrade-এর জন্য maintenance window নির্ধারণ করুন; major upgrade-এর আগে staging-এ পরীক্ষা করুন। **Cost:** খরচের উপাদান — machine type (vCPU/RAM), storage (GB), backup storage, network egress, HA-এর standby (প্রায় দ্বিগুণ)। অপ্রয়োজনীয় HA বা বড় machine type নেবেন না — load test দিয়ে সঠিক size নির্ধারণ করুন। Pricing Calculator-এ হিসাব করুন, আর billing alert বসান।

> **বাস্তব উদাহরণ:** TechNova-র Black Friday-এর আগে load test-এ দেখা গেল connection limit শেষ হয়ে যাচ্ছে। সমাধান ছিল machine type বাড়ানো নয় — PgBouncer যোগ করা ও Cloud Run-এর max instances ১০০ থেকে ৩০-এ কমানো। Connection ৮০০ থেকে ১২০-এ নেমে এলো, আর ডেটাবেজ স্থিতিশীল থাকল। Performance সমস্যার উত্তর সবসময় বড় instance নয় — প্রায়ই architecture।

১৪.১২ IaC — Terraform দিয়ে Cloud SQL

```
# Terraform — Cloud SQL PostgreSQL (private IP)
resource "google_sql_database_instance" "technova_db" {
  name             = "technova-db"
  database_version = "POSTGRES_16"
  region           = "asia-south1"
  settings {
    tier              = "db-custom-2-7680"
    availability_type = "REGIONAL"
    disk_type         = "PD_SSD"
    disk_size         = 100
    disk_autoresize   = true
    ip_configuration {
      ipv4_enabled    = false
      private_network = google_compute_network.technova_vpc.id
      require_ssl     = true
    }
    backup_configuration {
      enabled                        = true
      start_time                     = "02:00"
      point_in_time_recovery_enabled = true
      transaction_log_retention_days = 7
      backup_retention_settings {
        retained_backups = 14
      }
    }
    insights_config {
      query_insights_enabled  = true
      query_string_length     = 1024
      record_application_tags = true
    }
  }
  deletion_protection = true # production-এ true রাখুন
}
resource "google_sql_database" "app" {
  name     = "technova_app"
  instance = google_sql_database_instance.technova_db.name
}
resource "google_sql_user" "app_user" {
  name     = "app_user"
  instance = google_sql_database_instance.technova_db.name
  password = "CHANGE-ME-APP-PASSWORD" # বাস্তবে Secret Manager / random_password ব্যবহার করুন
}
```

`deletion_protection = true` production-এ অপরিহার্য — ভুল `terraform destroy` থেকে instance বাঁচায়। Password Terraform-এ plaintext-এ রাখবেন না — `random_password` resource বা Secret Manager ব্যবহার করুন।

১৪.১৩ Production Scenario — TechNova-র Order Database

TechNova-র order ডেটাবেজ: Cloud SQL PostgreSQL 16, REGIONAL HA, private IP, ১০০ GB SSD, automated backup ০২:০০-তে, PITR ৭ দিন, read replica একটি (reporting query-এর জন্য)। অ্যাপ্লিকেশন Cloud Run থেকে Auth Proxy-এর পরিবর্তে private IP + Secret Manager দিয়ে সংযোগ করে; connection pooling PgBouncer দিয়ে। Reporting query read replica-তে যায়, write primary-তে। প্রতি মাসে PITR drill, প্রতি ত্রৈমাসিকে

failover drill। Billing alert: মাসিক খরচ অনুমানের ৮০% ছাড়ালে নোটিফিকেশন। এই ডিজাইনে zone বিভ্রাটে failover হয়, ভুল delete-এ PITR থেকে ফেরা যায়, আর reporting load primary-কে ধীর করে না।

১৪.১৪ Security, Cost ও Common Mistakes

- **ভুল ১:** Public IP + `0.0.0.0/0` authorized network — private IP ব্যবহার করুন।
- **ভুল ২:** Root user দিয়ে অ্যাপ্লিকেশন চালানো — আলাদা least-privilege user তৈরি করুন।
- **ভুল ৩:** পাসওয়ার্ড কোডে বা env file-এ রাখা — Secret Manager ব্যবহার করুন।
- **ভুল ৪:** Backup চালু না করে production চালানো — backup ও PITR প্রথম দিন থেকেই চালু করুন।
- **ভুল ৫:** Connection pooling ছাড়া serverless থেকে সংযোগ — connection exhaustion হয়।
- **ভুল ৬:** Read replica থেকে সবসময় fresh ডেটা আশা করা — replication lag মাথায় রাখুন।
- **ভুল ৭:** Deletion protection বন্ধ রাখা — ভুল delete থেকে বাঁচায় না।

১৪.১৫ Troubleshooting

TROUBLESHOOTING: Connection refused / timeout: private IP হলে VPC peering ও firewall যাচাই করুন; Auth Proxy হলে proxy চলছে কি না ও IAM অনুমতি দেখুন; authorized networks যাচাই করুন। **Too many connections:** Cloud Monitoring-এ active connection দেখুন; pool size কমান, PgBouncer যোগ করুন, idle connection বন্ধ করুন। **Slow queries:** Query Insights-এ slow query খুঁজুন; `EXPLAIN ANALYZE` দিয়ে index যাচাই করুন; read replica-তে reporting সরান। **Disk full:** storage auto-increase চালু আছে কি না দেখুন; পুরনো backup ও log পরিষ্কার করুন। **Failover হচ্ছে না:** HA REGIONAL কি না যাচাই করুন; maintenance window-এর বাইরে failover পরীক্ষা করুন। **PITR restore ব্যর্থ:** transaction log retention দিন সংখ্যা যাচাই করুন — তার আগের সময়ে ফেরা যায় না।

১৪.১৬ সারাংশ

Cloud SQL হলো managed relational database — PostgreSQL, MySQL, SQL Server; AlloyDB হলো PostgreSQL-compatible উচ্চ-performance বিকল্প। HA zone বিভ্রাট থেকে বাঁচায়, read replica read scaling দেয়, backup ও PITR মানুষের ভুল থেকে বাঁচায় — তিনটি আলাদা সুরক্ষা, একটিকে অন্যটির বিকল্প ভাববেন না। Private IP, least-privilege user, Secret Manager ও SSL দিয়ে সুরক্ষা দিন; connection pooling দিয়ে scale করুন; আর restore drill ছাড়া backup-এ ভরসা করবেন না।

১৪.১৭ Review Questions

- HA, read replica ও backup — তিনটি কী কী ভিন্ন ব্যর্থতা থেকে বাঁচায়?
- Private IP কেন public IP + authorized network-এর চেয়ে নিরাপদ?
- Cloud SQL Auth Proxy কীভাবে কাজ করে? কখন এটি ব্যবহার করবেন?
- PITR কীভাবে কাজ করে? Transaction log retention-এর সীমা কী?
- AlloyDB কখন Cloud SQL PostgreSQL-এর চেয়ে উপযুক্ত?
- Connection exhaustion-এর কারণ কী? PgBouncer কীভাবে সাহায্য করে?

- Read replica-তে replication lag থাকলে অ্যাপ্লিকেশন ডিজাইনে কী বদলাতে হয়?

১৪.১৮ Practical Exercises

- **Exercise ১:** Cloud SQL PostgreSQL instance তৈরি করুন (private IP বা Auth Proxy), database ও least-privilege user তৈরি করুন, `psql` দিয়ে সংযোগ করে একটি table তৈরি ও query করুন।
- **Exercise ২:** Automated backup ও PITR চালু করুন; একটি table-এ ডেটা লিখে ভুল করে মুছে ফেলুন, clone দিয়ে PITR restore করে ডেটা ফিরিয়ে আনুন।
- **Exercise ৩:** Read replica তৈরি করে reporting query replica-তে চালান; replication lag Cloud Monitoring-এ দেখুন।
- **Exercise ৪:** Secret Manager-এ DB password রেখে Cloud Run থেকে সংযোগ করুন; পাসওয়ার্ড কোডে নেই তা যাচাই করুন।
- **Exercise ৫:** Terraform দিয়ে উপরের instance-টি পুনরায় তৈরি করুন (`deletion_protection = true` সহ)। Pricing Calculator-এ HA সহ ও ছাড়া মাসিক খরচের তুলনা করুন।

অধ্যায় ১৫ • PART III — Storage, Database & Application Services

Chapter 15 — Firestore, Bigtable and Memorystore

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — NoSQL ডেটাবেজ কখন relational-এর চেয়ে উপযুক্ত তা ব্যাখ্যা করতে; Firestore-এ document/collection মডেল, index ও query ডিজাইন করতে; Bigtable-এর wide-column architecture ও row-key ডিজাইন বুঝতে; Memorystore (Redis) দিয়ে caching ও session management করতে; consistency মডেল (strong বনাম eventual) ও scaling trade-off বিশ্লেষণ করতে; এবং workload অনুযায়ী সঠিক ডেটাবেজ বেছে নিতে। Console ও gcloud CLI দুই পথেই কাজ করবেন, আর client library দিয়ে Firestore ও Redis-এর উদাহরণ চালাবেন।

- NoSQL-এর ধরন: document, wide-column, key-value — কখন কোনটি
- Firestore data modeling, composite index, query সীমা ও security rules
- Bigtable row-key ডিজাইন, column family, performance ও cost
- Memorystore Redis: caching pattern, eviction, persistence, HA
- সঠিক ডেটাবেজ বেছে নেওয়ার সিদ্ধান্ত-কাঠামো

১৫.২ Prerequisite Refresher

অধ্যায় ১৪-তে আপনি relational database শিখেছেন — fixed schema, JOIN, ACID transaction। NoSQL সেই মডেল থেকে সরে আসে: schema নমনীয়, scale-out সহজ, কিন্তু JOIN ও জটিল transaction সীমিত। CAP theorem মনে রাখুন: একটি distributed system একসঙ্গে Consistency, Availability ও Partition tolerance — তিনটি পুরোপুরি দিতে পারে না; বাস্তবে partition ঘটলে consistency বা availability — একটিকে বেছে নিতে হয়। আর অধ্যায় ১২-এর object storage-এর কথা মনে করুন: Firestore ও Bigtable-ও managed service — patching ও replication Google করে, data modeling আপনার দায়িত্ব।

> **CONCEPT:** NoSQL মানে “No SQL” নয় — মানে “Not Only SQL”। অনেক NoSQL ডেটাবেজেই query language আছে, কিন্তু তাদের data model relational নয়। সঠিক প্রশ্ন “SQL বনাম NoSQL কোনটি ভালো?” নয় — প্রশ্ন হলো “এই workload-এর অ্যাক্সেস প্যাটার্ন কী, আর কোন মডেল তাতে স্বাভাবিকভাবে ফিট করে?”

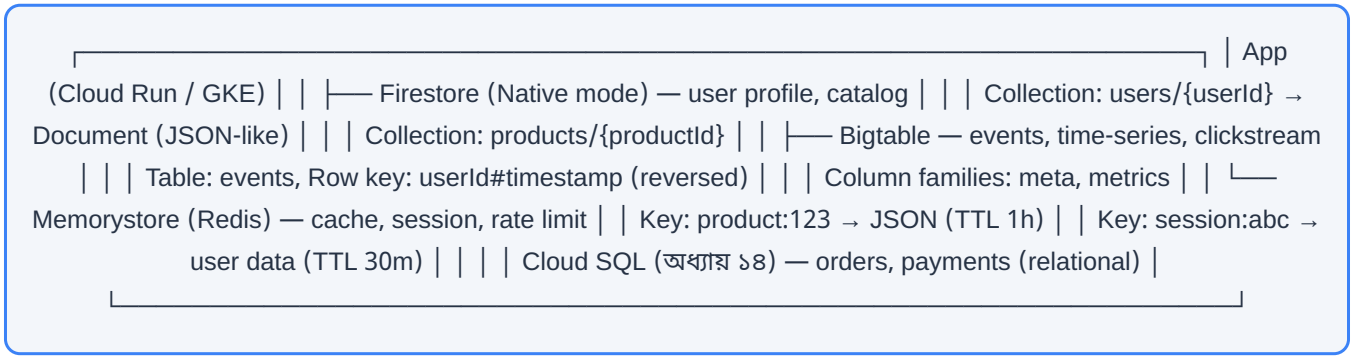
১৫.৩ First Principles — NoSQL কেন দরকার?

Relational database-এর শক্তি consistency ও JOIN, কিন্তু দুর্বলতা scale-out: একটি বড় table-কে একাধিক সার্ভারে ভাগ করা (sharding) জটিল, আর schema বদলাতে migration লাগে। কিছু workload-এ এই trade-off উল্টো হয়: (১) বিশাল write throughput — প্রতি সেকেন্ডে লাখো event, যেমন IoT sensor বা clickstream; (২) নমনীয় schema — প্রতিটি record-এর field আলাদা, যেমন user profile বা product catalog; (৩) global scale — বিশ্বজুড়ে ব্যবহারকারী, কম latency-তে read/write; (৪) simple access pattern — key দিয়ে পড়া, range scan, time-series। এই workload-গুলোর জন্য document, wide-column ও key-value মডেল তৈরি হয়েছে।

TechNova-র উদাহরণ: product catalog ও user profile — প্রতিটি product-এর attribute আলাদা, schema বদলায় ঘন ঘন — Firestore উপযুক্ত। Clickstream ও sensor ডেটা — সময় অনুযায়ী append, বিশাল আয়তন — Bigtable উপযুক্ত।

Product page-এর cache ও session — দ্রুত key-value অ্যাক্সেস — Memorystore Redis উপযুক্ত। আর অর্ডার ও পেমেন্ট — relational-ই থাকে (অধ্যায় ১৪)। একটিমাত্র ডেটাবেজ দিয়ে সব workload চালানো ভুল — polyglot persistence-ই বাস্তবতা।

১৫.৪ আর্কিটেকচার — তিনটি NoSQL সেবা একসঙ্গে



তিনটি সেবার scale মডেল আলাদা: Firestore serverless — throughput স্বয়ংক্রিয়ভাবে scale হয়, আপনি node পরিচালনা করেন না। Bigtable-এ cluster ও node provision করতে হয় — node বাড়ালে throughput বাড়ে। Memorystore-এ instance size (memory) ও tier (Basic/Standard) বেছে নিতে হয় — memory পূর্ণ হলে eviction policy কাজ করে।

১৫.৫ Firestore — Document Database

Firestore-এর মডেল: **Collection** → **Document** → **Field**। Document হলো JSON-like key-value map, field-এর value string, number, boolean, array, map বা nested object হতে পারে। Subcollection দিয়ে hierarchy তৈরি করা যায় — যেমন `users/{userId}/orders/{orderId}`। প্রতিটি document-এর সর্বোচ্চ আকার ১ MiB — বড় ফাইল Firestore-এ রাখবেন না, Cloud Storage-এ রেখে Firestore-এ শুধু URL রাখুন।

Firestore-এর দুটি mode আছে: **Native mode** (সুপারিশকৃত — real-time listener, mobile/web SDK, strong consistency) ও **Datastore mode** (পুরনো Datastore-এর সঙ্গে সামঞ্জস্যের জন্য)। নতুন project-এ Native mode ব্যবহার করুন। Firestore-এ query: single-field index স্বয়ংক্রিয়ভাবে তৈরি হয়; একাধিক field-এ filter + sort করতে **composite index** দরকার — Console বা CLI-তে তৈরি করতে হয়, না হলে query error দেয় ও index তৈরির লিঙ্ক দেয়।

```
# Firestore database তৈরি (Native mode, region asia-south1)
gcloud firestore databases create --location=asia-south1 --type=firestore-native
gcloud firestore databases list
gcloud firestore databases describe --database="(default)"

# Composite index তৈরি (CLI / Terraform-এ লিখে রাখুন)
gcloud firestore indexes composite create --collection-group=products \
  --query-scope=COLLECTION \
  --field-config=field-path=category,order=ascending \
  --field-config=field-path=price,order=ascending \
  --database="(default)"

# Firestore-এ ডেটা লেখা — Node.js (firebase-admin / @google-cloud/firestore)
# const { Firestore } = require('@google-cloud/firestore');
# const db = new Firestore({ projectId: 'technova-prod' });
# await db.collection('products').doc('shoe-123').set({
```

```
#   name: 'Running Shoe', category: 'footwear', price: 2999,
#   stock: 42, updatedAt: Firestore.FieldValue.serverTimestamp(),
# });
# // Query — category + price range (composite index দরকার)
# const snap = await db.collection('products')
#   .where('category', '==', 'footwear').where('price', '<', 5000)
#   .orderBy('price').limit(20).get();
```

Firestore-এর transaction: একটি transaction-এ একাধিক document read/write করা যায়, ACID guarantee সহ — কিন্তু transaction-এ document সংখ্যা ও সময় সীমিত। Batch write দিয়ে একসঙ্গে অনেক document লেখা যায়। Real-time listener (onSnapshot) দিয়ে client-কে live update দেওয়া যায় — chat, live dashboard-এর জন্য উপযুক্ত।

> **সতর্কতা:** Firestore-এ `where` + `orderBy` একসঙ্গে ব্যবহার করলে composite index লাগে — index ছাড়া query production-এ ব্যর্থ হবে। Index তৈরি হতে সময় লাগে — deploy-এর আগে index তৈরি করুন। আর collection-এ document সংখ্যা বেশি হলে `list` বা full scan করবেন না — pagination (cursor) ব্যবহার করুন।

১৫.৬ Firestore Security Rules ও Cost

Firestore-এর অ্যাক্সেস নিয়ন্ত্রণ করে **Security Rules** — কে কোন document পড়তে/লিখতে পারবে তা rule-এ লেখা হয়। Mobile/web client সরাসরি Firestore-এ সংযোগ করলে Security Rules-ই একমাত্র সুরক্ষা — ভুল rule মানে ডেটা ফাঁস। Server (Cloud Run/GKE) থেকে Admin SDK দিয়ে সংযোগ করলে IAM দিয়ে নিয়ন্ত্রণ করা হয় — Security Rules প্রযোজ্য হয় না। Production-এ client-side অ্যাক্সেসের জন্য Firebase Authentication + Security Rules ব্যবহার করুন, আর rule পরীক্ষা করুন emulator দিয়ে।

```
// Firestore Security Rules — উদাহরণ (firestore.rules)
// rules_version = '2';
// service cloud.firestore {
//   match /databases/{database}/documents {
//     // নিজের profile শুধু নিজে পড়তে/লিখতে পারবে
//     match /users/{userId} {
//       allow read, write: if request.auth != null && request.auth.uid == userId;
//     }
//     // products সবাই পড়তে পারবে, লিখতে পারবে শুধু admin (custom claim)
//     match /products/{productId} {
//       allow read: if true;
//       allow write: if request.auth != null && request.auth.token.admin == true;
//     }
//   }
// }
// Deploy: firebase deploy --only firestore:rules  (Firebase CLI)
// কখনোই করবেন না: allow read, write: if true;  — এটি পুরো database public করে দেয়।
```

Cost: Firestore-এর খরচ — document read/write/delete operation সংখ্যা, storage, network egress। ছোট ছোট read অনেকবার করলে বিল বাড়ে — প্রয়োজনে cache (Memorystore) বা denormalization ব্যবহার করুন। দাম Pricing Calculator-এ যাচাই করুন।

১৫.৭ Bigtable — Wide-Column Storage

Bigtable হলো wide-column NoSQL — বিশাল scale-এর জন্য ডিজাইন করা (Google-এর Search, Maps, Gmail-এর ভেতরে ব্যবহৃত প্রযুক্তির managed সংস্করণ)। মডেল: **Table → Row key → Column family → Column qualifier → Cell (timestamped version)**। প্রতিটি row একটি row key দিয়ে চিহ্নিত; row key-এর lexicographic order-এ ডেটা সংরক্ষিত হয় — তাই row-key ডিজাইনই performance নির্ধারণ করে।

Row-key ডিজাইনের নিয়ম: (১) Key এমনভাবে বানান যাতে read pattern অনুযায়ী পাশাপাশি থাকে — যেমন time-series-এর জন্য `userId#reversed-timestamp` (timestamp উল্টো করলে নতুন ডেটা আগে আসে)। (২) Hotspotting এড়ান — sequential key (যেমন timestamp একা) ব্যবহার করলে সব write একটিমাত্র node-এ যায়; key-এর শুরুতে hash বা userId যোগ করুন। (৩) Key ছোট রাখুন — প্রতিটি cell-এ key সংরক্ষিত হয়। Column family-তে ডেটা গ্রুপ করুন — যেমন `meta` (কম বদলায়) ও `metrics` (ঘন ঘন বদলায়) আলাদা family-তে রাখলে garbage collection আলাদাভাবে করা যায়। প্রতিটি cell-এ একাধিক timestamped version থাকতে পারে — পুরনো version-এর retention সেট করা যায়।

```
# Bigtable instance ও table তৈরি
gcloud bigtable instances create technova-events \
  --cluster=technova-events-c1 --cluster-zone=asia-south1-a \
  --cluster-num-nodes=3 --cluster-storage-type=SSD \
  --display-name="TechNova Events"
gcloud bigtable instances describe technova-events
# Table ও column family তৈরি (cbt CLI বা Console)
# cbt -project technova-prod -instance technova-events createtable events
# cbt -project technova-prod -instance technova-events createfamily events meta
# cbt -project technova-prod -instance technova-events createfamily events metrics
# Row লেখা (উদাহরণ):
# cbt -project technova-prod -instance technova-events set events 'user-42#9000000000'
meta:page=home metrics:clicks=1
# Row পড়া:
# cbt -project technova-prod -instance technova-events read events --prefix='user-42#' --
limit=10
```

Bigtable-এ JOIN নেই, secondary index নেই — query শুধু row key বা prefix দিয়ে। জটিল query দরকার হলে ডেটা BigQuery-তে export করে বিশ্লেষণ করুন। Bigtable-এর খরচ — node সংখ্যা, storage, network — Pricing Calculator-এ যাচাই করুন। ছোট workload-এর জন্য Bigtable ব্যয়বহুল — Firestore বা Cloud SQL যথেষ্ট হলে Bigtable নেবেন না।

১৫.৮ Memorystore — Redis Caching Concepts

Memorystore হলো managed Redis (ও Memcached) service — in-memory key-value store, মাইক্রোসেকেন্ড latency। ব্যবহার: cache, session store, rate limiter, leaderboard, queue। Redis-এ ডেটা memory-তে থাকে — ডিস্কে নয় (persistence চালু করলে RDB snapshot নেওয়া যায়)। Memory পূর্ণ হলে eviction policy কাজ করে — যেমন `allkeys-lru` (সবচেয়ে কম ব্যবহৃত key মুছে ফেলা) বা `volatile-lru` (শুধু TTL-ওয়ালা key-এর মধ্যে)। Cache-এর মূল নীতি: cache-এ যা আছে তা হারিয়ে গেলেও চলবে — source of truth ডেটাবেজেই থাকে।

```
# Memorystore Redis instance তৈরি (Standard tier — HA, replica সহ)
gcloud redis instances create technova-cache \
  --size=5 --region=asia-south1 \
  --tier=STANDARD --redis-version=redis_7_0 \
```

```
  --network=technova-vpc
gcloud redis instances describe technova-cache --region=asia-south1 \
  --format="value(host, port, currentLocationId)"

# Redis-এ সংযোগ (VM / Cloud Run থেকে VPC-এর ভেতরে, redis-cli বা client library)
# redis-cli -h 10.0.0.5 -p 6379 ping
# SET product:123 '{"name":"Running Shoe","price":2999}' EX 3600
# GET product:123
# Node.js (ioredis):
# const Redis = require('ioredis');
# const redis = new Redis({ host: '10.0.0.5', port: 6379 });
# const cached = await redis.get('product:123');
# if (!cached) {
#   const product = await db.collection('products').doc('123').get();
#   await redis.set('product:123', JSON.stringify(product.data()), 'EX', 3600);
# }
```

Caching pattern: **Cache-aside** (অ্যাপ্লিকেশন প্রথমে cache দেখে, না পেলে ডেটাবেজ থেকে নিয়ে cache-এ রাখে) সবচেয়ে সাধারণ। **Write-through** (লেখার সময় cache ও ডেটাবেজ দুটোতেই লেখা) consistency ভালো দেয়, কিন্তু লেখা ধীর হয়। Cache invalidation সবচেয়ে কঠিন অংশ — ডেটা বদলালে cache-এর পুরনো entry মুছে ফেলুন বা TTL ছোট রাখুন। Session store-এ TTL ব্যবহার করুন — session শেষ হলে key নিজে মুছে যাবে।

> **CONCEPT:** Cache-এর hit ratio দেখুন — cache hit বেশি মানে ডেটাবেজের load কম। Hit ratio কম হলে TTL বাড়ানো, key ডিজাইন বদলানো, বা cache-এর আকার বাড়ানো বিবেচনা করুন। কিন্তু cache কখনোই source of truth নয় — cache মুছে গেলে অ্যাপ্লিকেশন ডেটাবেজ থেকে ডেটা ফিরিয়ে আনতে পারতে হবে।

১৫.৯ Data Consistency, Scaling ও Performance

সেবা	Consistency	Scaling	Latency
Firestore	Strong (single-region / multi-region)	Serverless, স্বয়ংক্রিয়	মিলিসেকেন্ড
Bigtable	Strong (single-cluster), eventual (replication)	Node যোগ করে	মিলিসেকেন্ড (single-digit)
Memorystore Redis	Strong (single instance), async replica	Memory / shard বাড়িয়ে	সাব-মিলিসেকেন্ড
Cloud SQL	Strong (ACID)	Vertical + read replica	মিলিসেকেন্ড

Firestore-এ single-region strong consistency দেয়; multi-region-এও strong, তবে latency বেশি। Bigtable-এ replication চালু করলে replica-তে eventual consistency হয় — primary-তে লেখা কিছুক্ষণ পর replica-তে দেখা যায়। Redis-এ Standard tier-এ replica asynchronous — failover-এর সময় সামান্য ডেটা হারাতে পারে। প্রতিটি সেবার consistency মডেল অ্যাপ্লিকেশন ডিজাইনে প্রভাব ফেলে — যেমন, লেখার পরপরই পড়লে নতুন ডেটা দেখা যাবে কি না তা জানতে হবে।

১৫.১০ IaC — Terraform দিয়ে Firestore ও Memorystore

```
# Terraform — Firestore database ও composite index
resource "google_firestore_database" "technova" {
  project     = "technova-prod"
  name        = "(default)"
  location_id = "asia-south1"
  type        = "FIRESTORE_NATIVE"
}
resource "google_firestore_index" "products_category_price" {
  project    = "technova-prod"
  database   = google_firestore_database.technova.name
  collection = "products"
  fields {
    field_path = "category"
    order      = "ASCENDING"
  }
  fields {
    field_path = "price"
    order      = "ASCENDING"
  }
}
# Terraform — Memorystore Redis
resource "google_redis_instance" "cache" {
  name           = "technova-cache"
  tier           = "STANDARD_HA"
  memory_size_gb = 5
  region         = "asia-south1"
  redis_version  = "REDIS_7_0"
  authorized_network = google_compute_network.technova_vpc.id
}
```

Firestore Security Rules Terraform-এ নয়, Firebase CLI বা `google_firestore_document`-এর মাধ্যমে পরিচালনা করুন — rule file-কে version control-এ রাখুন। Bigtable-এর জন্য Terraform-এ `google_bigtable_instance` ও `google_bigtable_table` resource ব্যবহার করুন।

১৫.১১ Production Scenario — TechNova-র Polyglot Architecture

TechNova-র production: অর্ডার ও পেমেন্ট Cloud SQL PostgreSQL-এ; product catalog ও user profile Firestore-এ (real-time update, mobile app থেকে সরাসরি); clickstream ও IoT event Bigtable-এ (প্রতিদিন কোটি event, row key `userId#reversed-timestamp`); product page cache ও session Memorystore Redis-এ (TTL ১ ঘণ্টা)। Analytics-এর জন্য Bigtable থেকে Dataflow দিয়ে BigQuery-তে export করা হয়। এই ডিজাইনে প্রতিটি workload তার উপযুক্ত ডেটাবেজে চলে — একটি ডেটাবেজকে সব কাজে জোর করা হয় না। ব্যর্থতা: Redis down হলে অ্যাপ্লিকেশন ডেটাবেজ থেকে ডেটা নেয় (ধীর, কিন্তু চলে); Firestore down হলে catalog পড়া যায় না — সেজন্য critical read-এ cache fallback রাখা হয়; Bigtable-এ node বাড়িয়ে throughput বাড়ানো যায়।

১৫.১২ Security, Cost ও Common Mistakes

Security: Firestore-এ Security Rules + Firebase Auth; Bigtable-এ IAM (table-level); Memorystore-এ VPC-এর ভেতরে সীমাবদ্ধ, AUTH (Redis AUTH) চালু, TLS in-transit যেখানে সমর্থিত। কোনো NoSQL সেবাকেই public internet-এ খুলবেন না। **Cost:** Firestore-এ operation সংখ্যা — অপ্রয়োজনীয় read কমান, pagination ব্যবহার করুন;

Bigtable-এ node সংখ্যা — load অনুযায়ী scale করুন; Memorystore-এ memory size — eviction দেখে সঠিক size নির্ধারণ করুন। দাম Pricing Calculator-এ যাচাই করুন।

- **ভুল ১:** Firestore-এ `allow read, write: if true;` — পুরো database public।
- **ভুল ২:** Firestore document-এ বড় ফাইল রাখা — ১ MiB সীমা, Cloud Storage ব্যবহার করুন।
- **ভুল ৩:** Bigtable-এ sequential row key (timestamp একা) — hotspotting হয়।
- **ভুল ৪:** Redis-কে source of truth ভাবা — cache মুছে গেলে ডেটা হারায়।
- **ভুল ৫:** Composite index ছাড়া production query deploy করা।
- **ভুল ৬:** Cache invalidation না করা — পুরনো ডেটা দেখায়।
- **ভুল ৭:** ছোট workload-এ Bigtable ব্যবহার করা — খরচ বেশি।

১৫.১৩ Troubleshooting

TROUBLESHOOTING: Firestore query error (index): error message-এর লিঙ্ক থেকে composite index তৈরি করুন; index building অবস্থায় query ব্যর্থ হতে পারে — status যাচাই করুন। **Firestore permission denied:** Security Rules (client) বা IAM (server) যাচাই করুন; emulator দিয়ে rule পরীক্ষা করুন। **Bigtable slow read:** row-key ডিজাইন যাচাই করুন — hotspotting আছে কি না; node সংখ্যা ও CPU utilization দেখুন; scan-এর range সীমিত করুন। **Redis connection timeout:** VPC, authorized network ও IP যাচাই করুন; Redis instance-এর memory পূর্ণ হয়েছে কি না দেখুন; client-এর connection pool সীমিত করুন। **Redis eviction বেশি:** memory size বাড়ান বা TTL কমান; `INFO stats` দিয়ে evicted keys দেখুন। **Cache stale data:** invalidation logic যাচাই করুন; TTL ছোট করুন; write-এর সময় cache delete করুন।

১৫.১৪ সারাংশ

Firestore হলো serverless document database — নমনীয় schema, real-time, strong consistency। Bigtable হলো wide-column database — বিশাল write throughput, time-series, row-key ডিজাইনই performance। Memorystore Redis হলো in-memory cache — সাব-মিলিসেকেন্ড latency, কিন্তু source of truth নয়। সঠিক ডেটাবেজ বেছে নেওয়া মানে workload-এর অ্যাক্সেস প্যাটার্ন, consistency ও scale-এর প্রয়োজন বোঝা — একটি ডেটাবেজ দিয়ে সব কাজ হয় না।

১৫.১৫ Review Questions

- Document, wide-column ও key-value মডেলের মধ্যে পার্থক্য কী? প্রতিটির উদাহরণ দিন।
- Firestore-এ composite index কখন দরকার? Index ছাড়া query কী হয়?
- Bigtable-এ row-key ডিজাইন কেন এত গুরুত্বপূর্ণ? Hotspotting কী?
- Redis-এ cache-aside ও write-through pattern-এর পার্থক্য কী?
- Firestore Security Rules-এ `allow read, write: if true;` কেন বিপজ্জনক?
- কোন workload-এ Cloud SQL, কোনটিতে Firestore, কোনটিতে Bigtable বেছে নেবেন? যুক্তি দিন।
- Cache-এর hit ratio কম হলে কী করবেন?

১৫.১৬ Practical Exercises

- **Exercise ১:** Firestore Native database তৈরি করে `products` collection-এ ১০টি document লিখুন; category + price query চালিয়ে composite index তৈরি করুন।
- **Exercise ২:** Firestore Security Rules লিখুন — নিজের profile শুধু নিজে লিখতে পারবে, products সবাই পড়তে পারবে — emulator দিয়ে পরীক্ষা করুন।
- **Exercise ৩:** Bigtable instance তৈরি করে time-series row key (`userId#reversed-timestamp`) দিয়ে ১০০টি row লিখুন; prefix scan করে পড়ুন। Sequential key দিয়ে hotspotting-এর ধারণা লিখুন।
- **Exercise ৪:** Memorystore Redis instance তৈরি করে cache-aside pattern-এর ছোট Node.js স্ক্রিপ্ট লিখুন; TTL পরীক্ষা করুন।
- **Exercise ৫:** একটি সিদ্ধান্ত-তালিকা তৈরি করুন: অর্ডার, product catalog, clickstream, session, chat message — প্রতিটির জন্য কোন ডেটাবেজ (Cloud SQL / Firestore / Bigtable / Redis) ও কেন, তা লিখুন।

অধ্যায় ১৬ • PART III — Storage, Database & Application Services

Chapter 16 — Application Hosting and Edge Delivery

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — App Engine, Cloud Run ও static hosting-এর মধ্যে trade-off ব্যাখ্যা করতে; Cloud Run-এ containerized অ্যাপ্লিকেশন deploy করতে; Cloud Load Balancing, Cloud CDN, Cloud DNS ও Certificate Manager দিয়ে HTTPS architecture ডিজাইন করতে; custom domain, SSL certificate ও DNS record পরিচালনা করতে; এবং production-এ hosting খরচ, scaling ও নিরাপত্তা নিয়ন্ত্রণ করতে। Console ও gcloud CLI দুই পথেই কাজ করবেন, Terraform দিয়ে load balancer ও DNS IaC হিসেবে লিখবেন।

- App Engine (Standard/Flexible) বনাম Cloud Run বনাম GKE — কখন কোনটি
- Cloud Run: service, revision, traffic splitting, concurrency, scaling
- Global Load Balancer, backend service, health check ও backend bucket
- Cloud CDN: cache, cache key, invalidation ও signed URL
- Cloud DNS, Certificate Manager ও HTTPS (TLS) architecture

১৬.২ Prerequisite Refresher

Part I ও Part II থেকে মনে রাখুন: VPC, firewall, IAM ও Cloud Run-এর প্রাথমিক ধারণা। HTTP/HTTPS-এর মূল বিষয়: DNS নামকে IP-তে রূপান্তর করে, TLS handshake-এ certificate যাচাই হয়, তারপর encrypted সংযোগে request যায়। Container ধারণা: Dockerfile দিয়ে image তৈরি, image-কে registry-তে রাখা, registry থেকে deploy করা। অধ্যায় ১২-এর Cloud Storage bucket এখানে আবার আসবে — static hosting-এর backend হিসেবে। আর অধ্যায় ১৪-এর Cloud SQL সংযোগ এখানে কাজে লাগবে — অ্যাপ্লিকেশনকে ডেটাবেজের সঙ্গে যুক্ত করতে।

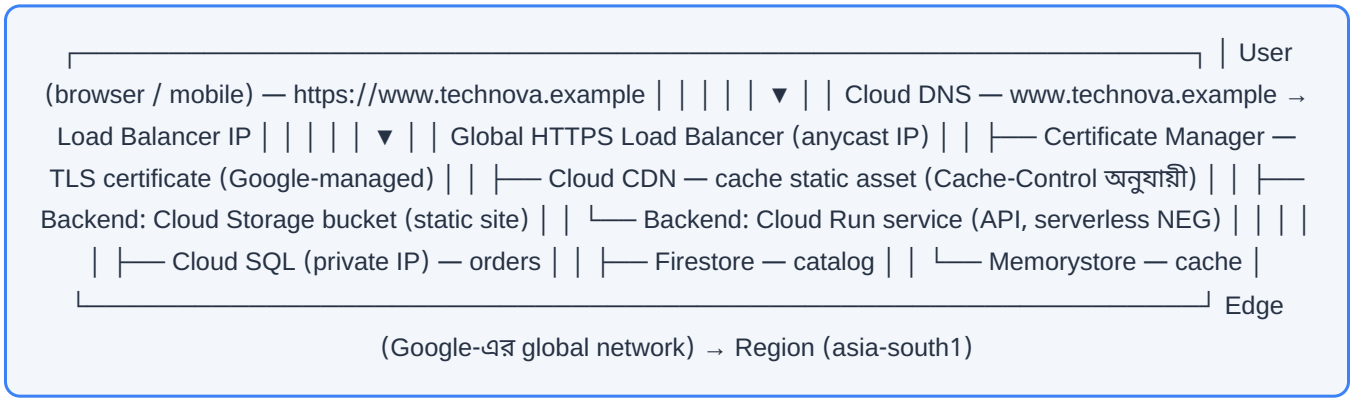
> **CONCEPT:** Hosting মানে শুধু “কোড চালানো” নয় — মানে traffic গ্রহণ করা, scale করা, TLS দেওয়া, DNS পরিচালনা করা, static asset দ্রুত দেওয়া, আর ব্যর্থতায় অন্য region-এ যাওয়া। একটি ভালো hosting architecture-এ এই সব স্তর আলাদা — প্রতিটি স্তর স্বাধীনভাবে বদলানো যায়।

১৬.৩ First Principles — Application Hosting-এর সমস্যা

একটি ওয়েব অ্যাপ্লিকেশনকে যা যা করতে হয়: ব্যবহারকারীর request গ্রহণ করা, কোড চালানো, ডেটাবেজের সঙ্গে কথা বলা, static ফাইল (ছবি, CSS, JS) দেওয়া, আর traffic বাড়লে scale করা। নিজে VM-এ চালালে আপনাকেই করতে হয়: OS patch, load balancer সেটআপ, TLS certificate renew, autoscaling, health check। Managed hosting এই কাজগুলো নেয় — আপনি কোড ও container দেন, Google বাকিটা পরিচালনা করে। কিন্তু managed-এরও trade-off আছে: নিয়ন্ত্রণ কম, platform-এর সীমা মানতে হয়, আর ভুল কনফিগারেশনে খরচ বাড়তে পারে।

TechNova-র উদাহরণ: তাদের marketing site static (HTML/CSS/JS) — Cloud Storage + CDN-ই যথেষ্ট। E-commerce API containerized — Cloud Run উপযুক্ত (scale-to-zero, pay-per-use)। একটি পুরনো monolith যা App Engine-এ চলে — সেখানে App Engine-ই সহজ। তিনটি workload, তিনটি hosting পছন্দ — একটিমাত্র প্ল্যাটফর্মে জোর করা ভুল।

১৬.৪ আর্কিটেকচার — Edge থেকে App পর্যন্ত



এই architecture-এ প্রতিটি স্তরের দায়িত্ব আলাদা: DNS নাম রূপান্তর করে, Load Balancer traffic গ্রহণ ও বিতরণ করে, CDN static asset cache করে, Cloud Run কোড চালায়, ডেটাবেজ ডেটা রাখে। ব্যবহারকারী সবচেয়ে কাছের Google edge location থেকে response পায় — latency কমে। একটি স্তর নষ্ট হলে অন্য স্তর কাজ করে — যেমন Cloud Run-এর একটি revision ব্যর্থ হলে traffic আগের revision-এ ফেরানো যায়।

১৬.৫ App Engine Overview

App Engine হলো Google-এর প্রথম PaaS — কোড upload করলেই চলে, server পরিচালনা করতে হয় না। **Standard environment:** নির্দিষ্ট runtime (Python, Node.js, Java, Go ইত্যাদি), দ্রুত scale, scale-to-zero, সীমিত flexibility — sandbox-এর ভেতরে চলে। **Flexible environment:** Docker container চলে, যেকোনো runtime, বেশি নিয়ন্ত্রণ, কিন্তু scale ধীর ও খরচ বেশি (সবসময় অন্তত একটি instance চলে)। নতুন কাজে Cloud Run সাধারণত App Engine-এর চেয়ে নমনীয় — কারণ Cloud Run যেকোনো container চালায়, যেকোনো ভাষায়, আর scale-to-zero দেয়। App Engine এখনো উপযুক্ত: পুরনো App Engine অ্যাপ, বা যেখানে platform-এর built-in service (Task Queue, Memcache) ব্যবহার করা হয়েছে। Migration-এর সময় App Engine থেকে Cloud Run-এ যাওয়ার খরচ ও সুবিধা যাচাই করুন।

```
# App Engine — app.yaml উদাহরণ (Standard, Python)
# runtime: python312
# service: default
# instance_class: F1
# automatic_scaling:
#   max_instances: 10
#   min_instances: 0
# handlers:
# - url: /static
#   static_dir: static
# Deploy:
# gcloud app deploy app.yaml --project=technova-prod
# gcloud app browse --project=technova-prod
```

১৬.৬ Cloud Run — Containerized Hosting

Cloud Run হলো serverless container platform: আপনি container image দেন, Google তা চালায়, request অনুযায়ী scale করে, request না থাকলে scale-to-zero করে (খরচ বন্ধ)। প্রতিটি deploy একটি **revision** তৈরি করে — immutable; traffic revision-গুলোর মধ্যে ভাগ করা যায় (যেমন ৯০% পুরনো, ১০% নতুন — canary deploy)।

Concurrency হলো একটি container instance একসঙ্গে কত request সামলাবে — বেশি concurrency মানে কম instance, কম খরচ, কিন্তু CPU/memory-এর চাপ বেশি। **Min instances** দিলে cold start কমে, কিন্তু সবসময় খরচ হয়।

```
# Cloud Run-এ deploy — Artifact Registry থেকে image
gcloud run deploy technova-api \
  --image=asia-south1-docker.pkg.dev/technova-prod/technova-repo/api:v1 \
  --region=asia-south1 \
  --platform=managed \
  --allow-unauthenticated \
  --min-instances=0 --max-instances=10 \
  --concurrency=80 \
  --memory=512Mi --cpu=1 \
  --set-env-vars=NODE_ENV=production \
  --add-cloudsql-instances=technova-prod:asia-south1:technova-db \
  --set-secrets=DB_PASSWORD=technova-db-password:latest

# Service-এর তথ্য, revision ও traffic দেখুন
gcloud run services describe technova-api --region=asia-south1
gcloud run revisions list --service=technova-api --region=asia-south1

# Canary — নতুন revision-এ 10% traffic
gcloud run deploy technova-api \
  --image=asia-south1-docker.pkg.dev/technova-prod/technova-repo/api:v2 \
  --region=asia-south1 --no-traffic
gcloud run services update-traffic technova-api --region=asia-south1 \
  --to-revisions=LATEST=10
# সব traffic নতুন revision-এ সরানো
gcloud run services update-traffic technova-api --region=asia-south1 \
  --to-latest

# Cloud Run Job (একবার চলা task — যেমন invoice generation)
# gcloud run jobs create technova-invoice-job \
#   --image=asia-south1-docker.pkg.dev/technova-prod/technova-repo/invoice:v1 \
#   --region=asia-south1 --tasks=1
# gcloud run jobs execute technova-invoice-job --region=asia-south1
```

Cloud Run-এ `--allow-unauthenticated` দিলে যে কেউ service-এ request করতে পারে — public API-এর জন্য ঠিক, কিন্তু internal service-এর জন্য নয়। Internal service-এ unauthenticated বন্ধ করে IAM দিয়ে `roles/run.invoker` দিন, আর Load Balancer-এর পেছনে রাখুন। Cold start কমাতে min-instances ১ রাখা যায়, কিন্তু খরচের হিসাব করুন — Pricing Calculator-এ দেখুন।

> **সতর্কতা:** Cloud Run-এ প্রতিটি instance-এর CPU শুধু request চলাকালীন বরাদ্দ থাকে (ডিফল্ট) — background task বা long-running process চালালে CPU throttle হতে পারে। Background কাজের জন্য Cloud Run Job বা `--cpu-throttling=false` (সবসময় CPU বরাদ্দ) বিবেচনা করুন — দ্বিতীয়টির খরচ বেশি।

১৬.৭ Static Hosting Architecture

Static site (HTML/CSS/JS, ছবি) host করার সবচেয়ে সহজ ও সস্তা পদ্ধতি: Cloud Storage bucket + Load Balancer backend bucket + Cloud CDN। Bucket-এ ফাইল রাখুন, Load Balancer-এর backend bucket হিসেবে যোগ করুন, CDN চালু করুন। ব্যবহারকারী Load Balancer-এর IP-তে আসে, CDN cache থেকে ফাইল পায় — origin

bucket-এ প্রতিবার যাওয়া লাগে না। **Firebase Hosting** আরেকটি বিকল্প — static site + serverless function, সহজ deploy, Google-managed TLS — ছোট site-এর জন্য উপযুক্ত।

```
# Static site deploy — Storage bucket-এ sync + cache header
gcloud storage rsync --recursive ./dist/ gs://technova-web-static/
# Cache-Control header দেওয়া (HTML কম, asset বেশি cache)
gcloud storage objects update gs://technova-web-static/index.html \
  --cache-control="public, max-age=300"
gcloud storage objects update gs://technova-web-static/assets/app.js \
  --cache-control="public, max-age=31536000, immutable"
# Bucket-কে website হিসেবে কনফিগার (সরাসরি bucket URL-এর জন্য — production-এ LB ব্যবহার করুন)
gcloud storage buckets update gs://technova-web-static \
  --web-main-page-suffix=index.html --web-error-page=404.html
```

Static asset-এর Cache-Control সঠিকভাবে দেওয়া performance-এর জন্য অত্যন্ত গুরুত্বপূর্ণ: versioned asset (যেমন `app.abc123.js`) দীর্ঘ cache, HTML ছোট cache — নতুন deploy-এ HTML বদলালে ব্যবহারকারী নতুন asset পায়।

১৬.৮ Cloud Load Balancing

Cloud Load Balancing হলো global, managed load balancer — একটি anycast IP দিয়ে বিশ্বজুড়ে traffic গ্রহণ করে, সবচেয়ে কাছের backend-এ পাঠায়। **Global external Application Load Balancer** (HTTP/HTTPS) — ওয়েব অ্যাপ্লিকেশনের জন্য সবচেয়ে সাধারণ। Backend হতে পারে: Cloud Run (serverless NEG), GKE, Compute Engine instance group, Cloud Storage bucket (backend bucket), বা internet NEG। **Health check** দিয়ে backend-এর সুস্থতা যাচাই করা হয় — অসুস্থ backend-এ traffic যায় না। **URL map** দিয়ে path অনুযায়ী routing করা যায় — যেমন `/api/*` Cloud Run-এ, `/*` bucket-এ।

```
# Load Balancer তৈরি — CLI-তে ধাপে ধাপে (Console-এ wizard-ও আছে)
# 1) Serverless NEG (Cloud Run-এর জন্য)
gcloud compute network-endpoint-groups create technova-api-neg \
  --region=asia-south1 --network-endpoint-type=SERVERLESS \
  --cloud-run-service=technova-api
# 2) Backend service
gcloud compute backend-services create technova-api-backend \
  --global --load-balancing-scheme=EXTERNAL_MANAGED --protocol=HTTPS
gcloud compute backend-services add-backend technova-api-backend \
  --global --network-endpoint-group=technova-api-neg \
  --network-endpoint-group-region=asia-south1
# 3) Backend bucket (static site)
gcloud compute backend-buckets create technova-web-backend \
  --gcs-bucket-name=technova-web-static --enable-cdn
# 4) URL map — /api/* → API, বাকি → static
gcloud compute url-maps create technova-urlmap \
  --default-service=technova-web-backend
gcloud compute url-maps add-path-matcher technova-urlmap \
  --path-matcher-name=api-matcher --default-service=technova-web-backend \
  --path-rules="/api/*=technova-api-backend"
# 5) HTTPS proxy + forwarding rule (certificate পরের section-এ)
# gcloud compute target-https-proxies create technova-https-proxy \
#   --url-map=technova-urlmap --ssl-certificates=technova-cert
# gcloud compute forwarding-rules create technova-https-rule \
#   --global --target-https-proxy=technova-https-proxy \
#   --ports=443 --address=LB_STATIC_IP
```

Load Balancer-এর জন্য static IP reserve করুন — IP বদলে গেলে DNS বদলাতে হয়। Health check-এর path ও interval সঠিকভাবে নির্ধারণ করুন — খুব ঘন ঘন health check backend-এর load বাড়ায়, খুব কম হলে ব্যর্থতা দেরিতে ধরা পড়ে।

১৬.৯ Cloud CDN — Edge Caching

Cloud CDN Load Balancer-এর সঙ্গে যুক্ত হয়ে static ও cacheable content edge location-এ cache করে — ব্যবহারকারী কাছের cache থেকে পায়, origin-এ যাওয়া লাগে না। Cache নিয়ন্ত্রণ করে `Cache-Control` header — origin যা বলে CDN তাই মানে। **Cache invalidation** দিয়ে পুরনো cache মুছে ফেলা যায় — deploy-এর পর `/assets/*` invalidate করুন। **Signed URL / signed cookie** দিয়ে private content-কে CDN দিয়ে দেওয়া যায় — শুধু অনুমোদিত ব্যবহারকারী cache থেকে পায়। CDN-এর খরচ — cache egress ও request সংখ্যা — Pricing Calculator-এ যাচাই করুন; তবে CDN সাধারণত origin egress-এর চেয়ে সস্তা ও দ্রুত।

```
# CDN চালু করা (backend bucket বা backend service-এ)
gcloud compute backend-buckets update technova-web-backend --enable-cdn
gcloud compute backend-services update technova-api-backend --global --enable-cdn
# Cache invalidation (deploy-এর পর)
gcloud compute url-maps invalidate-cdn-cache technova-urlmap \
  --path="/assets/*" --async
# CDN status দেখুন
gcloud compute backend-buckets describe technova-web-backend \
  --format="yaml(cdnPolicy)"
```

> **CONCEPT:** CDN cache hit ratio দেখুন — hit বেশি মানে origin-এর load কম ও latency কম। Hit ratio কম হলে Cache-Control header যাচাই করুন — `no-store` বা `private` থাকলে CDN cache করে না। Dynamic API response (যেমন user-specific ডেটা) cache করবেন না — ভুল ব্যবহারকারী অন্যের ডেটা দেখতে পারে।

১৬.১০ Cloud DNS ও Certificate Manager

Cloud DNS হলো managed DNS service — domain-এর zone তৈরি করে A, AAAA, CNAME, MX, TXT record পরিচালনা করুন। Load Balancer-এর static IP-তে A record দিন (`www.technova.example → 203.0.113.10` — উদাহরণ IP, আসল IP `gcloud compute addresses` থেকে নিন)। DNS propagation-এ সময় লাগে (TTL অনুযায়ী) — record বদলানোর পরপরই সবাই নতুন IP দেখবে না। **Certificate Manager** দিয়ে Google-managed TLS certificate তৈরি করুন — Google স্বয়ংক্রিয়ভাবে renew করে, আপনাকে renew করতে হয় না। Certificate-এর জন্য domain ownership যাচাই করতে DNS-এ CNAME record যোগ করতে হয় (Google-managed certificate-এর ক্ষেত্রে Load Balancer-এর সঙ্গে যুক্ত করলে স্বয়ংক্রিয়)।

```
# Cloud DNS zone ও record তৈরি
gcloud dns managed-zones create technova-zone \
  --dns-name=technova.example. --description="TechNova public zone"
# Load Balancer-এর static IP-তে A record
gcloud dns record-sets create www.technova.example. \
  --zone=technova-zone --type=A --ttl=300 \
  --rrdatas=203.0.113.10
# Zone-এর name server দেখুন (registrar-এ বসাতে হবে)
gcloud dns managed-zones describe technova-zone --format="value(nameServers)"
```

```
# Google-managed certificate (Load Balancer-এর জন্য)
gcloud certificate-manager certificates create technova-cert \
  --domains=www.technova.example,technova.example \
  --scope=default
# Certificate-এর status দেখুন (ACTIVE হতে সময় লাগতে পারে)
gcloud certificate-manager certificates describe technova-cert
# Classic SSL certificate (Load Balancer-এর সঙ্গে সরাসরি)
# gcloud compute ssl-certificates create technova-cert-classic \
#   --domains=www.technova.example --global
```

Domain কেনার পর registrar-এ Cloud DNS-এর name server বসাতে হয় — তা না করলে DNS কাজ করবে না। TLS certificate-এর status ACTIVE না হওয়া পর্যন্ত HTTPS কাজ করবে না — ধৈর্য ধরুন, DNS propagation-এর জন্য ঘণ্টা লাগতে পারে।

১৬.১১ HTTPS Architecture — End-to-End TLS

HTTPS-এর প্রবাহ: ব্যবহারকারী `https://www.technova.example`-এ যায় → DNS Load Balancer IP দেয় → TLS handshake-এ Load Balancer certificate দেখায়, ব্যবহারকারী যাচাই করে → encrypted সংযোগে request যায় → Load Balancer backend-এ (Cloud Run / bucket) পাঠায়। Load Balancer থেকে backend-এর সংযোগও encrypt করা উচিত — Cloud Run-এর ক্ষেত্রে Google-এর ভেতরে TLS হয়। HTTP (port 80) থেকে HTTPS-এ redirect করুন — Load Balancer-এ HTTP-to-HTTPS redirect সেট করুন, যাতে কেউ ভুল করে HTTP-তে গেলেও HTTPS-এ চলে আসে। HSTS header দিয়ে ব্রাউজারকে বলুন সবসময় HTTPS ব্যবহার করতে।

> **বাস্তব উদাহরণ:** TechNova-র প্রথম deploy-এ HTTPS কাজ করছিল না — কারণ certificate ACTIVE হয়নি। কারণ ছিল DNS-এ A record ভুল IP-তে ছিল। A record ঠিক করার পর certificate ACTIVE হলো, আর HTTPS চালু হলো। শিক্ষা: TLS সমস্যা হলে প্রথমে DNS যাচাই করুন — `dig www.technova.example` ও `nslookup` দিয়ে।

১৬.১২ IaC — Terraform দিয়ে Load Balancer, CDN ও DNS

```
# Terraform — Cloud Run + Load Balancer + DNS (সংক্ষিপ্ত, মূল resource)
resource "google_cloud_run_v2_service" "api" {
  name     = "technova-api"
  location = "asia-south1"
  ingress  = "INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER"
  template {
    containers {
      image = "asia-south1-docker.pkg.dev/technova-prod/technova-repo/api:v1"
      resources { limits = { cpu = "1", memory = "512Mi" } }
    }
    scaling { min_instance_count = 0 max_instance_count = 10 }
  }
}
resource "google_compute_global_address" "lb_ip" { name = "technova-lb-ip" }
resource "google_dns_managed_zone" "technova" {
  name     = "technova-zone"
  dns_name = "technova.example."
}
resource "google_dns_record_set" "www" {
  name         = "www.technova.example."
```

```
  managed_zone = google_dns_managed_zone.technova.name
  type         = "A"
  ttl          = 300
  rrdatas      = [google_compute_global_address.lb_ip.address]
}
# URL map, backend service, target HTTPS proxy ও forwarding rule-ও
# Terraform-এ লিখুন — Console-এ হাতে তৈরি Load Balancer audit করা কঠিন।
```

Load Balancer-এর Terraform বেশ দীর্ঘ — `google_compute_url_map`, `google_compute_backend_service`, `google_compute_target_https_proxy`, `google_compute_global_forwarding_rule` — প্রতিটি resource আলাদাভাবে লিখুন, আর `terraform plan` দিয়ে যাচাই করুন।

১৬.১৩ Production Scenario — TechNova-র Full Hosting Stack

TechNova-র production stack: `www.technova.example` → Cloud DNS → Global Load Balancer (static IP, Google-managed certificate) → URL map: `/api/*` → Cloud Run API (serverless NEG, min 0, max 10, concurrency 80), `/*` → backend bucket (static site, CDN চালু) → Cloud Run থেকে Cloud SQL (private IP), Firestore ও Memorystore। Deploy: নতুন image Artifact Registry-তে push, Cloud Run-এ canary (১০% traffic), monitoring-এ error rate দেখে পুরো traffic সরানো। Rollback: traffic আগের revision-এ ফেরানো — এক কমান্ডে। CDN invalidation deploy-এর পর। Billing alert ও Cloud Monitoring dashboard — request count, latency, error rate, CDN hit ratio। এই ডিজাইনে static asset দ্রুত, API scale হয়, TLS স্বয়ংক্রিয়, আর rollback সহজ।

১৬.১৪ Security, Cost ও Common Mistakes

Security: HTTPS সর্বত্র, HTTP-to-HTTPS redirect, HSTS; Cloud Run internal service-এ unauthenticated বন্ধ; Load Balancer-এ Cloud Armor (WAF) যেখানে প্রয়োজন — DDoS ও SQL injection থেকে সুরক্ষা; Secret Manager-এ credential; IAM least-privilege। **Cost:** Cloud Run-এ request সংখ্যা, CPU/memory সময়, egress; Load Balancer-এ forwarding rule ও data processing; CDN-এ cache egress; DNS-এ zone ও query সংখ্যা। Scale-to-zero ব্যবহার করুন যেখানে cold start acceptable; min-instances শুধু যেখানে latency-critical। দাম Pricing Calculator-এ যাচাই করুন।

- **ভুল ১:** Cloud Run-কে public করে internal API চালানো — IAM দিয়ে সীমিত করুন।
- **ভুল ২:** HTTP-তে সাইট চালানো — HTTPS বাধ্যতামূলক করুন।
- **ভুল ৩:** Cache-Control ছাড়া static asset deploy — CDN কাজ করে না।
- **ভুল ৪:** Certificate ACTIVE হওয়ার আগেই DNS বদলানো — ধৈর্য ধরুন, status যাচাই করুন।
- **ভুল ৫:** Min-instances বেশি রেখে খরচ বাড়ানো — load test দিয়ে নির্ধারণ করুন।
- **ভুল ৬:** Health check-এর path ভুল দেওয়া — backend অসুস্থ দেখায়।
- **ভুল ৭:** Canary ছাড়া সরাসরি ১০০% traffic নতুন revision-এ দেওয়া।

১৬.১৫ Troubleshooting

TROUBLESHOOTING: 502 Bad Gateway: backend অসুস্থ বা health check ব্যর্থ — Cloud Run-এর log দেখুন, health check path যাচাই করুন। **404 on static asset:** bucket-এ ফাইল আছে কি না, URL map-এর path rule ঠিক আছে কি না দেখুন। **Certificate not ACTIVE:** DNS A record ও domain ownership যাচাই করুন; propagation-এর জন্য অপেক্ষা করুন। **Cloud Run cold start ধীর:** min-instances বাড়ান বা container image ছোট করুন; startup time log-এ দেখুন। **CDN stale content:** invalidation করুন; Cache-Control header যাচাই করুন। **DNS কাজ করছে না:** registrar-এ name server ঠিক আছে কি না, `dig +trace` দিয়ে যাচাই করুন। **Traffic সব নতুন revision-এ যাচ্ছে না:** `update-traffic` সঠিক revision-এ দিয়েছেন কি না দেখুন।

১৬.১৬ সারাংশ

Application hosting-এর স্তরগুলো: DNS নাম দেয়, Load Balancer traffic গ্রহণ করে, CDN static asset দ্রুত দেয়, Certificate Manager TLS দেয়, Cloud Run কোড চালায়, App Engine পুরনো PaaS workload চালায়। সঠিক পছন্দ workload অনুযায়ী — static site-এ bucket + CDN, API-তে Cloud Run, পুরনো monolith-এ App Engine। HTTPS সর্বত্র, canary deploy, rollback পরিকল্পনা, আর monitoring — এই চারটি ছাড়া production hosting সম্পূর্ণ নয়। Part III-এ আপনি শিখেছেন storage, database ও hosting — তিনটি মিলেই একটি সম্পূর্ণ application architecture তৈরি হয়।

১৬.১৭ Review Questions

- App Engine Standard, Flexible ও Cloud Run-এর মধ্যে কখন কোনটি বেছে নেবেন?
- Cloud Run-এ revision ও traffic splitting কীভাবে canary deploy সম্ভব করে?
- Load Balancer-এ URL map ও health check-এর ভূমিকা কী?
- Cloud CDN কখন কাজ করে, কখন করে না? Cache-Control-এর ভূমিকা কী?
- Google-managed certificate কীভাবে renew হয়? DNS-এর সঙ্গে এর সম্পর্ক কী?
- Static site-এর জন্য bucket + CDN কেন VM-এ web server চালানোর চেয়ে ভালো?
- Cloud Run-এ cold start কী? Min-instances কীভাবে তা কমায়, আর খরচ কী?

১৬.১৮ Practical Exercises

- **Exercise ১:** একটি container image Artifact Registry-তে push করে Cloud Run-এ deploy করুন; revision তালিকা দেখুন, নতুন version canary (১০%) দিয়ে deploy করুন।
- **Exercise ২:** Static HTML site Cloud Storage bucket-এ sync করুন, backend bucket + CDN দিয়ে Load Balancer তৈরি করুন, Cache-Control header যাচাই করুন।
- **Exercise ৩:** Cloud DNS-এ একটি test subdomain (যেমন `test.technova.example`) তৈরি করে A record দিন; `dig` দিয়ে যাচাই করুন।
- **Exercise ৪:** Google-managed certificate তৈরি করে Load Balancer-এ HTTPS proxy যোগ করুন; HTTP-to-HTTPS redirect সেট করুন।

- **Exercise ৫:** পুরো stack-এর Terraform লিখুন (Cloud Run + backend bucket + URL map + DNS) — `terraform plan` পড়ে প্রতিটি resource বুঝুন। Pricing Calculator-এ Cloud Run + Load Balancer + CDN-এর মাসিক খরচের অনুমান করুন।

PART IV • DEVOPS ENGINEERING MASTERY

Containers, Serverless & Delivery

GKE Container Engineering, Serverless & Event-Driven Architecture, Cloud Build & CI/CD, Infrastructure as Code

অধ্যায় ১৭ • PART IV — Containers, Serverless & Delivery

Chapter 17 — Container Engineering with GKE

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Artifact Registry-তে নিরাপদে কনটেইনার ইমেজ সংরক্ষণ ও স্ক্যান করতে; GKE-র কন্ট্রোল প্লেন, নোড, নোড পুল ও নেটওয়ার্কিংয়ের ভেতরের কাজ ব্যাখ্যা করতে; Autopilot ও Standard মোডের মধ্যে সিদ্ধান্ত নিতে; Workload Identity দিয়ে Pod-কে least-privilege GCP পরিচয় দিতে; Ingress, PersistentVolume ও autoscaling সাজাতে; ক্লাস্টার আপগ্রেড নিরাপদে করতে; এবং GKE-র খরচ ও নিরাপত্তা নিয়ন্ত্রণ করতে।

১৭.২ পূর্বজ্ঞান ঝালাই

কনটেইনার মানে একটি প্রক্রিয়া ও তার নির্ভরতা — কোড, লাইব্রেরি, কনফিগারেশন — একটি বিচ্ছিন্ন পরিবেশে প্যাকেজ করা, যা যেকোনো হোস্টে একইভাবে চলে। Docker ইমেজ হলো লেয়ারযুক্ত, অপরিবর্তনীয় (immutable) ফাইলসিস্টেম স্ন্যাপশট; কনটেইনার হলো সেই ইমেজের চলমান ইনস্ট্যান্স। Kubernetes হলো কনটেইনার অর্কেস্ট্রেটর: এটি সিদ্ধান্ত নেয় কোন Pod কোন নোডে চলবে, কত রেপ্লিকা থাকবে, ব্যর্থ Pod কীভাবে বদলাবে। আগের অধ্যায়গুলোতে আপনি Compute Engine, VPC, IAM ও Cloud Storage শিখেছেন — GKE সেই সবকিছুর ওপরে দাঁড়িয়ে আছে। একটি GKE ক্লাস্টার আসলে পরিচালিত (managed) Kubernetes, যেখানে Google কন্ট্রোল প্লেন চালায় আর আপনি workload চালান।

> **CONCEPT:** কনটেইনার ভার্চুয়াল মেশিন নয়। VM নিজের কার্নেল নিয়ে চলে; কনটেইনার হোস্টের কার্নেল ভাগ করে নেয়, কিন্তু namespace ও cgroup দিয়ে প্রক্রিয়া, নেটওয়ার্ক ও রিসোর্স আলাদা করে। তাই কনটেইনার হালকা ও দ্রুত শুরু হয়, কিন্তু কার্নেল-স্তরের বিচ্ছিন্নতা VM-এর মতো শক্ত নয় — এজন্যই GKE-তে sandboxing, নেটওয়ার্ক পলিসি ও IAM-এর মতো অতিরিক্ত নিয়ন্ত্রণ দরকার।

১৭.৩ প্রথম নীতি: GKE কেন দরকার?

একটি অ্যাপ্লিকেশন যখন একটি সার্ভারে চলে, তখন স্কেল, ডিপ্লয়মেন্ট ও ব্যর্থতা সামলানো মানুষের হাতে থাকে। দশটি সার্ভার হলে হাতে করা যায়; একশোটি মাইক্রোসার্ভিস, হাজারো Pod হলে আর যায় না। সমস্যাটি হলো *desired state* বনাম *actual state*: আপনি চান পাঁচটি রেপ্লিকা চলুক, একটি নোড মারা গেলে সেগুলো অন্য নোডে উঠুক, নতুন ভার্সন ধীরে ধীরে আসুক, ট্রাফিক বাড়লে রেপ্লিকা বাড়ুক। Kubernetes এই ঘোষণামূলক (declarative) মডেলটি বাস্তবায়ন করে: আপনি YAML-এ কাঙ্ক্ষিত অবস্থা লেখেন, কন্ট্রোল প্লেন ক্রমাগত বাস্তব অবস্থাকে সেই দিকে টেনে নেয়। GKE সেই কন্ট্রোল প্লেনকে Google-এর পরিচালনায় দেয়, যাতে আপনি etcd ব্যাকআপ, API সার্ভারের high availability বা কন্ট্রোল প্লেন প্যাচ নিয়ে না ভেবে workload-এ মন দিতে পারেন। TechNova-র MERN অ্যাপের ক্ষেত্রে: API সার্ভার, worker ও frontend আলাদা Deployment হিসেবে চলে, একই ক্লাস্টারে, আলাদা namespace-এ।

১৭.৪ Artifact Registry ও কনটেইনার ইমেজ

ইমেজ ছাড়া কনটেইনার চলে না, আর ইমেজ রাখার জায়গা ছাড়া ডিপ্লয়মেন্ট হয় না। Artifact Registry হলো GCP-র আর্টিফ্যাক্ট সংরক্ষণাগার — Docker ইমেজ ছাড়াও Maven, npm, Python প্যাকেজ রাখা যায়। পুরোনো Container Registry (gcr.io) এখন Artifact Registry-র পক্ষে নিরুৎসাহিত করা হয়; নতুন কাজে Artifact Registry ব্যবহার করুন। রিপোজিটরি আঞ্চলিক (regional) হয় — ক্লাস্টারের অঞ্চলের কাছাকাছি রাখলে pull দ্রুত হয় ও egress কমে।

একটি ভালো ইমেজের বৈশিষ্ট্য: ছোট বেস ইমেজ (যেমন distroless বা slim), নির্দিষ্ট ট্যাগের বদলে digest দিয়ে পিন করা, non-root ব্যবহারকারী, এবং নিয়মিত vulnerability স্ক্যান। `latest` ট্যাগ কখনো production-এ ব্যবহার করবেন না — এটি কোন ভার্সন চলছে তা জানতে দেয় না, rollback-কে অনুমাননির্ভর করে।

```
# Artifact Registry repository তৈরি (Console: Artifact Registry → Repositories → Create)
gcloud artifacts repositories create technova-images \
  --repository-format=docker \
  --location=asia-south1 \
  --description="TechNova container images"

# Docker-কে Artifact Registry-তে authenticate করা
gcloud auth configure-docker asia-south1-docker.pkg.dev

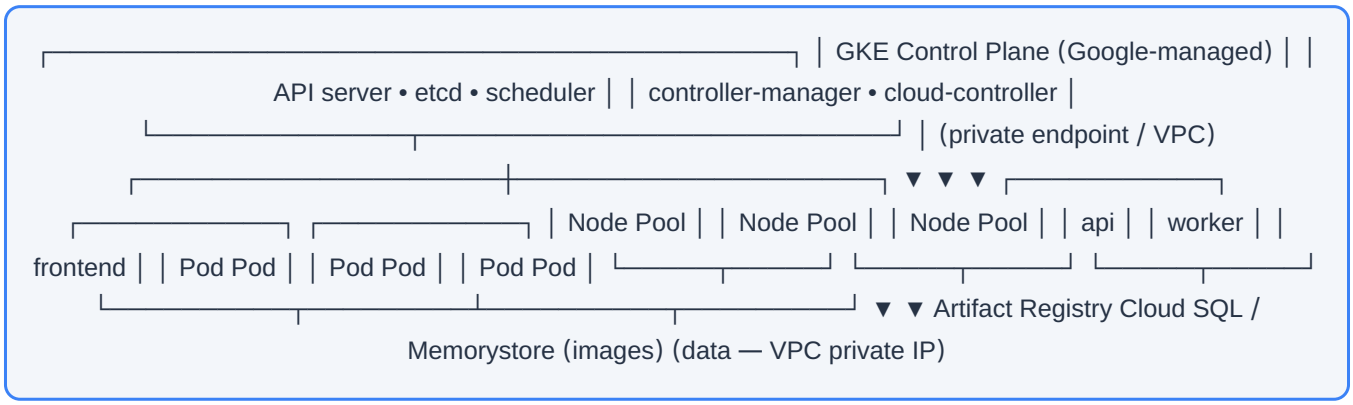
# ইমেজ build, tag ও push
docker build -t asia-south1-docker.pkg.dev/technova-prod/technova-images/api:v1.4.2 .
docker push asia-south1-docker.pkg.dev/technova-prod/technova-images/api:v1.4.2

# Vulnerability স্ক্যানের ফল দেখা (Artifact Analysis / On-Demand Scanning)
gcloud artifacts docker images list asia-south1-docker.pkg.dev/technova-prod/technova-images/
api
```

ইমেজ pull করার সময় GKE নোড Artifact Registry থেকে ইমেজ টানে। নোডের service account-কে রিপোজিটরিতে `roles/artifactregistry.reader` দিতে হয় — ক্লাস্টার তৈরির সময় ডিফল্ট নোড service account-কে এই অনুমতি আছে কি না যাচাই করুন। ব্যক্তিগত রেজিস্ট্রি থেকে pull করতে `imagePullSecrets` লাগে না যদি নোডের পরিচয় যথেষ্ট হয়; এটিই GCP-তে স্বাভাবিক পথ।

> **সতর্কতা:** পাবলিক ইমেজ (Docker Hub) থেকে সরাসরি pull করলে rate limit, supply-chain ঝুঁকি ও অপ্রত্যাশিত পরিবর্তনের মুখে পড়বেন। উৎপাদনে বেস ইমেজ নিজের Artifact Registry-তে mirror করে, স্ক্যান করে, তারপর ব্যবহার করুন। Secret কখনো ইমেজ লেয়ারে রাখবেন না — Dockerfile-এ `ENV API_KEY=...` লেখা মানে সেই secret ইমেজ ইতিহাসে চিরকাল থেকে যাওয়া।

১৭.৫ GKE আর্কিটেকচার: ভেতরে কী আছে



কন্ট্রোল প্লেনে থাকে: **kube-apiserver** (সব অনুরোধের প্রবেশদ্বার ও একমাত্র etcd-র সঙ্গে কথা বলা উপাদান), **etcd** (ক্লাস্টারের অবস্থার উৎস — desired state-এর সত্য), **scheduler** (কোন Pod কোন নোডে বসবে), **controller-manager** (Deployment, ReplicaSet-এর মতো কন্ট্রোলার চালায়) এবং **cloud-controller-manager**

(LoadBalancer, PersistentVolume-র মতো GCP রিসোর্সের সঙ্গে সংযোগ)। নোডে থাকে kubelet (Pod চালানো ও রিপোর্ট করা), kube-proxy (Service নেটওয়ার্কিং) ও কনটেইনার রানটাইম (containerd)।

অনুরোধের প্রবাহ: আপনি `kubectl apply` করেন → API server অনুরোধ যাচাই করে etcd-তে লেখে → scheduler Pod-এর জন্য নোড বেছে নেয় → সেই নোডের kubelet ইমেজ pull করে কনটেইনার চালায় → kubelet অবস্থা ফিরে জানায়। কোনো Pod মারা গেলে controller তা দেখে নতুন Pod তৈরি করে — এটাই reconciliation loop, Kubernetes-এর হৃদয়।

Regional বনাম Zonal ক্লাস্টার: Zonal ক্লাস্টারের কন্ট্রোল প্লেন একটিমাত্র জোনে থাকে; Regional ক্লাস্টারের কন্ট্রোল প্লেন তিনটি জোনে প্রতিলিপি করা থাকে এবং নোডও একাধিক জোনে ছড়ানো যায়। Production-এ Regional ক্লাস্টার বেছে নিন — একটি জোনের বিভ্রাটে কন্ট্রোল প্লেন বেঁচে থাকে।

১৭.৬ Autopilot বনাম Standard

বিষয়	Autopilot	Standard
নোড পরিচালনা	Google পরিচালনা করে; আপনি নোড দেখেন না	আপনি নোড পুল, মেশিন টাইপ, আপগ্রেড পরিচালনা করেন
বিলিং	Pod-এর অনুরোধ করা CPU/মেমরি অনুযায়ী	নোডের (VM) জন্য, Pod ব্যবহার করুক বা না করুক
নমনীয়তা	সীমিত: নির্দিষ্ট Pod spec, সুরক্ষা ডিফল্ট কঠোর	পূর্ণ: কাস্টম নোড, taint, বিশেষ হার্ডওয়্যার (GPU/TPU)
উপযুক্ত	বেশিরভাগ সাধারণ workload, ছোট দল	বিশেষ নেটওয়ার্কিং, GPU, সূক্ষ্ম নিয়ন্ত্রণ দরকার হলে

নতুন দলের জন্য Autopilot সাধারণত সঠিক শুরু: কম পরিচালন-ভার, নিরাপত্তা ডিফল্ট ভালো, খরচ Pod-ভিত্তিক হওয়ায় অব্যবহৃত নোডের খরচ নেই। Standard দরকার হয় যখন আপনার নির্দিষ্ট মেশিন টাইপ, স্থানীয় SSD, GPU, বা নোড-স্তরের কাস্টমাইজেশন লাগে। মনে রাখবেন: Autopilot-এ Pod-এর resource request-ই বিলিংয়ের ভিত্তি, তাই অতিরিক্ত বড় request লিখলে খরচ বাড়ে; request বাস্তব ব্যবহারের কাছাকাছি রাখুন।

```
# Autopilot ক্লাস্টার তৈরি
gcloud container clusters create-auto technova-prod \
  --region=asia-south1 \
  --release-channel=regular

# Standard ক্লাস্টার তৈরি (private nodes, VPC-native)
gcloud container clusters create technova-std \
  --region=asia-south1 \
  --num-nodes=2 \
  --enable-private-nodes \
  --master-ipv4-cidr=172.16.0.0/28 \
  --network=technova-vpc --subnetwork=technova-subnet \
  --workload-pool=technova-prod.svc.id.goog

# kubeconfig নেওয়া
gcloud container clusters get-credentials technova-prod --region=asia-south1
kubectl get nodes
```

১৭.৭ নোড পুল, লেবেল ও Taint

নোড পুল হলো একই মেশিন টাইপ ও কনফিগারেশনের নোডের দল। Production-এ অন্তত দুটি পুল ভাবুন: সাধারণ workload-এর জন্য একটি, আর memory-heavy worker বা GPU-র জন্য আলাদা। **Node affinity / selector** দিয়ে Pod-কে নির্দিষ্ট পুলে পাঠান; **taint ও toleration** দিয়ে নিশ্চিত করুন শুধু অনুমোদিত Pod-ই সেই পুলে বসে। উদাহরণ: `workload=worker:NoSchedule` taint দেওয়া পুলে শুধু toleration-সহ worker Pod বসবে।

নোডে SSH করে হাতে কিছু ঠিক করা Kubernetes-পন্থা নয় — নোড অপরিবর্তনীয় (immutable) ভাবুন: সমস্যা হলে নোড বদলে ফেলুন, হাতে সারাই করবেন না। নোড auto-repair চালু রাখুন, যাতে অসুস্থ নোড স্বয়ংক্রিয়ভাবে প্রতিস্থাপিত হয়।

১৭.৮ Workload Identity: Pod-কে GCP পরিচয় দেওয়া

আগে Pod-কে GCP API ব্যবহার করতে নোডের service account-এর key ফাইলে দিতে হতো — সেটি ছিল বিপজ্জনক: যেকোনো Pod সেই key চুরি করতে পারত। **Workload Identity** সেই সমস্যার সমাধান: Kubernetes Service Account (KSA)-কে Google Service Account (GSA)-র সঙ্গে যুক্ত করা হয়; Pod চলার সময় GKE একটি স্বল্পমেয়াদি token দেয়, key ফাইল কোথাও থাকে না।

```
# 1. Kubernetes Service Account তৈরি
kubectl create serviceaccount api-sa -n prod

# 2. Google Service Account তৈরি ও অনুমতি দেওয়া (least privilege)
gcloud iam service-accounts create technova-api \
  --display-name="TechNova API service account"
gcloud projects add-iam-policy-binding technova-prod \
  --member="serviceAccount:technova-api@technova-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# 3. দুটিকে যুক্ত করা (Workload Identity binding)
gcloud iam service-accounts add-iam-policy-binding technova-api@technova-
prod.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser \
  --member="serviceAccount:technova-prod.svc.id.goog[prod/api-sa]"

# 4. Pod-এ ব্যবহার (Deployment-এ serviceAccountName: api-sa)
kubectl annotate serviceaccount api-sa -n prod \
  iam.gke.io/gcp-service-account=technova-api@technova-prod.iam.gserviceaccount.com
```

এই পদ্ধতিতে প্রতিটি workload আলাদা পরিচয় পায়: API শুধু Storage পড়ে, worker শুধু Pub/Sub-তে প্রকাশ করে — একটি Pod আপস হলেও অন্যগুলোর অনুমতি চুরি হয় না।

১৭.৯ নেটওয়ার্কিং ও Ingress

GKE-তে তিন স্তরের নেটওয়ার্কিং: **Pod নেটওয়ার্ক** (প্রতিটি Pod-এর নিজস্ব IP, VPC-native ক্লাস্টারে Pod IP সরাসরি VPC-র সাবনেট থেকে আসে), **Service** (স্থিতিশীল ClusterIP / NodePort / LoadBalancer), এবং **Ingress** (HTTP(S) রাউটিং, TLS সমাপ্তি)। Production-এ VPC-native ক্লাস্টার ব্যবহার করুন — Pod IP VPC-তে দৃশ্যমান হওয়ায় firewall নিয়ম ও Cloud NAT পরিষ্কারভাবে কাজ করে।

Ingress-এর জন্য Google Cloud Load Balancer ব্যবহার করুন (GKE Ingress controller), অথবা Gateway API। TLS-এর জন্য Google-managed certificate ব্যবহার করুন — সার্টিফিকেট নবায়ন Google করে। বাইরের ট্রাফিকের

সামনে Cloud Armor (অধ্যায় ২২) বসান। ভেতরের Service-to-Service যোগাযোগে NetworkPolicy দিয়ে অনুমতি সীমাবদ্ধ করুন: ডিফল্টভাবে সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে — এটি production-এ গ্রহণযোগ্য নয়।

```
# Ingress উদাহরণ (সরলীকৃত)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-ingress
  namespace: prod
  annotations:
    kubernetes.io/ingress.class: "gce"
spec:
  rules:
  - host: api.technova.example
    http:
      paths:
      - path: /*
        pathType: ImplementationSpecific
        backend:
          service:
            name: api-service
            port:
              number: 80
```

১৭.১০ Persistent Storage

Pod ক্ষণস্থায়ী — মারা গেলে তার স্থানীয় ডিস্ক যায়। স্থায়ী ডেটার জন্য **PersistentVolumeClaim (PVC)** ব্যবহার করুন, যার পেছনে থাকে Compute Engine Persistent Disk বা Filestore। Stateful workload (যেমন নিজে চালানো ডেটাবেস) ছাড়া সাধারণত ডেটাবেস Cloud SQL-এ রাখাই ভালো (অধ্যায় ২৩) — ক্লাস্টারে ডেটাবেস চালানো মানে ব্যাকআপ, প্রতিলিপি ও আপগ্রেডের দায় আপনার। PVC-তে `storageClassName`, access mode (ReadWriteOnce বনাম ReadWriteMany) ও reclaim policy স্পষ্ট করুন; Filestore ছাড়া ReadWriteMany পাবেন না।

১৭.১১ Autoscaling: HPA, VPA ও Cluster Autoscaler

তিনটি ভিন্ন স্কেলিং স্তর: **Horizontal Pod Autoscaler (HPA)** — CPU/মেমরি বা কাস্টম মেট্রিকের ভিত্তিতে রেপ্লিকা বাড়ায়-কমায়; **Vertical Pod Autoscaler (VPA)** — Pod-এর request/limit বদলায় (HPA-র সঙ্গে একই মেট্রিকে একসঙ্গে চালাবেন না); **Cluster Autoscaler** — Pod বসার জায়গা না থাকলে নোড যোগ করে, নোড ফাঁকা হলে সরায়। Autopilot-এ নোড স্কেলিং Google করে, আপনি শুধু Pod-স্তর দেখেন। HPA-র জন্য Pod-এ resource request বাধ্যতামূলক — request ছাড়া HPA কাজ করে না।

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
  namespace: prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 10
```

```
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

minReplicas ২ রাখুন — একটি রেপ্লিকা মানে একক ব্যর্থতার বিন্দু। PodDisruptionBudget (PDB) সেট করুন যাতে আপগ্রেডের সময় অন্তত একটি রেপ্লিকা চলতে থাকে।

১৭.১২ আপগ্রেড ও রোলআউট কৌশল

GKE-তে দুটি আপগ্রেড: কন্ট্রোল প্লেন (Google করে, release channel অনুযায়ী) ও নোড পুল। Release channel বেছে নিন: `Regular` বেশিরভাগ production-এর জন্য উপযুক্ত; `Rapid` নতুন ফিচার চাইলে, `Stable` সর্বোচ্চ রক্ষণশীলতা চাইলে। নোড আপগ্রেডে surge upgrade ব্যবহার করুন — নতুন নোড আগে ওঠে, পুরোনো Pod সেখানে সরে, তারপর পুরোনো নোড যায়; এতে downtime থাকে না। Deployment-এ `strategy: RollingUpdate` (ডিফল্ট), `maxUnavailable: 0` বা ১, readiness probe ও liveness probe সঠিকভাবে সেট করুন — readiness probe ছাড়া নতুন Pod ট্রাফিক পাওয়ার আগেই Service-এ ঢুকে পড়ে।

> **বাস্তব উদাহরণ:** TechNova-র API ডিপ্লয়মেন্টে একবার readiness probe ছিল না। নতুন ভার্সন ডিপ্লয় হতেই LoadBalancer ট্রাফিক পাঠাল, কিন্তু অ্যাপ তখনো ডেটাবেস সংযোগ তৈরি করছিল — ব্যবহারকারীরা ৫০০ এরর পেলেন। সমাধান: `/healthz` endpoint-এ readiness probe, যেটি শুধু ডেটাবেস সংযোগ প্রস্তুত হলেই সফল হয়; liveness probe আলাদা, হালকা endpoint-এ, যাতে সাময়িক ধীরগতিতে Pod মেরে না ফেলা হয়।

১৭.১৩ নিরাপত্তা ও খরচ

নিরাপত্তা চেকলিস্ট: private ক্লাস্টার (কন্ট্রোল প্লেনের public endpoint বন্ধ বা authorized networks সীমিত), Binary Authorization (শুধু স্বাক্ষরিত/স্ক্যান-পাস ইমেজ ডিপ্লয়), Workload Identity, NetworkPolicy, Pod Security Standards (restricted), Secret Manager থেকে secret (etcd-তে Kubernetes Secret base64-এ থাকে — এটি encryption নয়), নিয়মিত vulnerability স্ক্যান। **খরচের সতর্কতা:** GKE-র কন্ট্রোল প্লেনের জন্য প্রতি ক্লাস্টারে ঘণ্টাভিত্তিক ফি আছে; Standard মোডে অব্যবহৃত নোডের খরচ চলতে থাকে; LoadBalancer প্রতি ঘণ্টায় বিল হয়; Persistent Disk প্রভিশন করা আকার অনুযায়ী বিল হয়, ব্যবহার অনুযায়ী নয়। খরচ দেখুন Cloud Billing-এর cost breakdown-এ, লেবেল দিয়ে প্রতি namespace/env আলাদা করুন। সঠিক খরচ জানতে Google Cloud Pricing Calculator দেখুন — এখানে লেখা কোনো সংখ্যাই চিরস্থায়ী নয়।

১৭.১৪ সাধারণ ভুল

- `latest` ট্যাগ ব্যবহার করা ও digest পিন না করা।
- resource request/limit না দেওয়া — HPA কাজ করে না, একটি Pod পুরো নোড খেয়ে ফেলে।
- readiness/liveness probe না দেওয়া বা একই ভারী endpoint দুটোতেই ব্যবহার করা।
- নোডের service account-কে অতিরিক্ত অনুমতি দেওয়া (Workload Identity-র বদলে)।
- Secret YAML-এ লেখা ও Git-এ কমিট করা।

- একটি মাত্র রেপ্লিকা চালানো ও PDB না থাকা।
- Log-এ সংবেদনশীল তথ্য ছাপানো — Cloud Logging-এ সেগুলো সংরক্ষিত হয়।

১৭.১৫ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Pod `ImagePullBackOff`	ভুল ইমেজ নাম, অনুমতি নেই, রেজিস্ট্রি অঞ্চল ভুল	`kubectl describe pod`; Artifact Registry reader অনুমতি যাচাই; digest দিয়ে pull পরীক্ষা
Pod `Pending`	রিসোর্স নেই, node selector মেলেনি, taint	`kubectl describe pod`-এর Events; Cluster Autoscaler লগ
Pod `CrashLoopBackOff`	অ্যাপ শুরুতে ব্যর্থ, ভুল env/secret	`kubectl logs --previous`; env ও Secret যাচাই
Service-এ পৌঁছানো যায় না	selector ভুল, Pod not ready, NetworkPolicy	`kubectl get endpoints`; Pod-এর readiness; Policy পরীক্ষা
Ingress ৫০২	backend unhealthy, Pod প্রস্তুত নয়	Backend Service-এর health check; readiness probe

১৭.১৬ সারাংশ

GKE হলো পরিচালিত Kubernetes: Google কন্ট্রোল প্লেন চালায়, আপনি workload চালান। Artifact Registry থেকে স্ক্যান করা ইমেজ, Workload Identity দিয়ে least-privilege পরিচয়, VPC-native নেটওয়ার্কিং, Ingress দিয়ে প্রবেশ, PVC দিয়ে স্থায়ী ডেটা, HPA/Cluster Autoscaler দিয়ে স্কেলিং, আর surge upgrade দিয়ে নিরাপদ আপগ্রেড — এই স্তরগুলো একসঙ্গে একটি উৎপাদন-উপযোগী প্ল্যাটফর্ম তৈরি করে। Autopilot দিয়ে শুরু করুন; বিশেষ প্রয়োজন হলে Standard-এ যান।

১৭.১৭ পর্যালোচনা প্রশ্ন

- Autopilot ও Standard মোডের বিলিং পার্থক্য কী, এবং কোন পরিস্থিতিতে কোনটি সস্তা?
- Workload Identity কীভাবে service account key ফাইলের ঝুঁকি দূর করে?
- Regional ক্লাস্টার কেন production-এ পছন্দনীয়?
- HPA, VPA ও Cluster Autoscaler কোন স্তরে কাজ করে?
- readiness probe ও liveness probe-এর ভূমিকা আলাদা কেন?

১৭.১৮ ব্যবহারিক অনুশীলন

- একটি Autopilot ক্লাস্টার তৈরি করে Artifact Registry থেকে একটি নমুনা অ্যাপ ডিপ্লয় করুন; digest দিয়ে ইমেজ পিন করুন।
- Workload Identity দিয়ে একটি Pod-কে শুধু একটি নির্দিষ্ট Cloud Storage bucket পড়ার অনুমতি দিন; অন্য bucket পড়ার চেষ্টা ব্যর্থ হয় কি না পরীক্ষা করুন।

- HPA সেট করে লোড টেস্ট দিয়ে রেপ্লিকা বাড়া দেখুন; PDB-সহ rolling update করুন।

অধ্যায় ১৮ • PART IV — Containers, Serverless & Delivery

Chapter 18 — Serverless and Event-Driven Architecture

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Cloud Run ও Cloud Run functions-এর মধ্যে পার্থক্য ব্যাখ্যা করতে; HTTP ও ইভেন্ট-চালিত workload-এর জন্য সঠিক সার্ভারলেস পরিষেবা বেছে নিতে; Pub/Sub, Eventarc ও Cloud Tasks দিয়ে নির্ভরযোগ্য event-driven সিস্টেম সাজাতে; retry, dead-letter ও idempotency দিয়ে ব্যর্থতা সামলাতে; concurrency ও cold start বুঝে টিউন করতে; এবং সার্ভারলেস খরচ ও পর্যবেক্ষণ নিয়ন্ত্রণ করতে।

১৮.২ পূর্বজ্ঞান ঝালাই

সার্ভারলেস মানে সার্ভার নেই — তা নয়; মানে সার্ভার পরিচালনার দায় আপনার নয়। আপনি কোড দেন, প্ল্যাটফর্ম চালানো, স্কেলিং ও প্যাচিং করে। আগের অধ্যায়ের GKE-তে আপনি নোড ও Pod দেখতে পান; সার্ভারলেসে সেই স্তর লুকানো। বিনিময়ে আপনি নিয়ন্ত্রণ ছাড়েন: রানটাইম, নেটওয়ার্ক ও সময়সীমা প্ল্যাটফর্মের নিয়মে চলে। মূল প্রশ্ন সবসময় একটাই — workload কি অনুরোধ-ভিত্তিক ও স্বল্পস্থায়ী, নাকি দীর্ঘস্থায়ী ও অবস্থাযুক্ত (stateful)? প্রথমটি সার্ভারলেসের জায়গা, দ্বিতীয়টি সাধারণত নয়।

> **CONCEPT:** Event-driven architecture-এ উপাদানগুলো সরাসরি একে অপরকে ডাকে না; তারা ইভেন্ট প্রকাশ করে, আর আগ্রহী উপাদান সেগুলো গ্রহণ করে। প্রকাশক জানে না কে শুনছে, গ্রাহক জানে না কে প্রকাশ করেছে। এই বিচ্ছিন্নতা (decoupling) সিস্টেমকে স্বাধীনভাবে স্কেল ও পরিবর্তন করতে দেয় — কিন্তু ডেলিভারি, ক্রম ও পুনরাবৃত্তি নিয়ে নতুন জটিলতা আনে, যা এই অধ্যায়ের মূল বিষয়।

১৮.৩ প্রথম নীতি: কেন সার্ভারলেস?

একটি ওয়েবসাইটে ট্রাফিক সমান থাকে না — সকালে শূন্য, দুপুরে ঝড়। VM বা নোড দিয়ে এই ওঠানামা সামলাতে হয় আগে থেকে ক্ষমতা কিনে (অপচয়) অথবা দেরিতে স্কেল করে (ব্যর্থতা)। সার্ভারলেসের প্রতিশ্রুতি: শূন্য থেকে হাজার ইনস্ট্যান্সে স্কেল, ব্যবহার না হলে শূন্য খরচ (scale-to-zero), আর প্রতি অনুরোধে বিলিং। কিন্তু এই প্রতিশ্রুতির দাম আছে: cold start (নতুন ইনস্ট্যান্স শুরু হতে সময়), সময়সীমা (timeout), ও স্থানীয় অবস্থার অনুপস্থিতি। TechNova-র ক্ষেত্রে: ব্যবহারকারী সাইন-আপ করলে একটি স্বাগত ইমেইল পাঠানো, ছবি আপলোড হলে থাম্বনেইল বানানো, রাতের রিপোর্ট তৈরি — এগুলো সার্ভারলেসের নিখুঁত কাজ; কিন্তু দীর্ঘস্থায়ী WebSocket সংযোগ বা ভারী ব্যাচ প্রসেসিং নয়।

১৮.৪ Cloud Run: কনটেইনার, সার্ভারলেসভাবে

Cloud Run যেকোনো কনটেইনার চালায় — ভাষা বা ফ্রেমওয়ার্কের বাধা নেই — এবং HTTP অনুরোধে স্বয়ংক্রিয়ভাবে স্কেল করে, শূন্যে নেমে যায়। প্রতিটি revision অপরিবর্তনীয়; নতুন ডিপ্লয় নতুন revision তৈরি করে, আর আপনি ট্রাফিক ভাগ করে দিতে পারেন (যেমন ৯০% পুরোনো, ১০% নতুন — canary)। Cloud Run-এর ভেতরে: অনুরোধ এলে Google-এর infrastructure একটি ইনস্ট্যান্স বরাদ্দ করে, আপনার কনটেইনার চালায়, অনুরোধ পাঠায়; নিষ্ক্রিয় থাকলে ইনস্ট্যান্স সরিয়ে নেয়। একটি ইনস্ট্যান্স একসঙ্গে একাধিক অনুরোধ সামলাতে পারে — এটাই concurrency সেটিং।

```
# Console: Cloud Run → Create Service → Artifact Registry থেকে ইমেজ বেছে নিন
# CLI দিয়ে ডিপ্লয়
gcloud run deploy technova-api \
  --image=asia-south1-docker.pkg.dev/technova-prod/technova-images/api:v1.4.2 \
  --region=asia-south1 \
  --platform=managed \
  --allow-unauthenticated \
  --min-instances=1 --max-instances=10 \
  --concurrency=80 \
  --memory=512Mi --cpu=1 \
  --set-env-vars="NODE_ENV=production" \
  --service-account=technova-api@technova-prod.iam.gserviceaccount.com

# ট্রাফিক ভাগ (canary)
gcloud run services update-traffic technova-api --region=asia-south1 \
  --to-revisions=v1-4-2=90,v1-5-0=10
```

কখন Cloud Run, কখন GKE? Cloud Run যখন workload HTTP-চালিত, স্কেল ওঠানামা করে, দল ছোট, পরিচালনা কম চায়। GKE যখন অনেক মাইক্রোসার্ভিসের জটিল নেটওয়ার্কিং, বিশেষ হার্ডওয়্যার, দীর্ঘস্থায়ী সংযোগ, বা Kubernetes-নির্দিষ্ট ফিচার (DaemonSet, StatefulSet) দরকার। দুটিকে মেশানোও যায়: মূল API GKE-তে, স্পাইক সামলানো worker Cloud Run-এ।

Cold start কমানো: `min-instances=1` রাখলে একটি ইনস্ট্যান্স সবসময় গরম থাকে (খরচ আছে), ছোট ইমেজ ও দ্রুত startup কোড লিখুন, ভারী নির্ভরতা lazy-load করুন। CPU সবসময় বরাদ্দ (CPU always allocated) সেটিং ব্যাকগ্রাউন্ড কাজের জন্য দরকার — ডিফল্টে অনুরোধের বাইরে CPU থ্রটল হয়।

> **সতর্কতা:** `--allow-unauthenticated` মানে যে কেউ ইন্টারনেট থেকে সার্ভিসটি ডাকতে পারে। পাবলিক API ছাড়া সবসময় authenticated রাখুন, আর সামনে API Gateway বা Identity-Aware Proxy বসান। Cloud Run-এর URL অনুমানযোগ্য — নিরাপত্তা গোপনীয়তার ওপর নির্ভর করবেন না।

১৮.৫ Cloud Run functions: একটিমাত্র ফাংশন

Cloud Run functions (আগে Cloud Functions নামে পরিচিত) হলো একটি ফাংশন-স্তরের সার্ভারলেস: আপনি একটি ফাংশন লেখেন (Node.js, Python, Go, Java ইত্যাদি), Google বাকিটা চালায়। কনটেইনার বানাতে হয় না — সোর্স কোড দিলেই buildpack ইমেজ বানায়। HTTP ট্রিগার বা ইভেন্ট ট্রিগার (যেমন Storage-এ ফাইল ওঠা, Pub/Sub বার্তা) দুটোই চলে। দ্বিতীয় প্রজন্মের (2nd gen) functions আসলে Cloud Run-এর ওপরে চলে, তাই Cloud Run-এর অনেক সুবিধা (বড় মেমরি, দীর্ঘ timeout, concurrency) পাওয়া যায়।

```
# একটি HTTP ফাংশন ডিপ্লয় (Node.js)
gcloud functions deploy thumbnail-maker \
  --gen2 --runtime=nodejs20 --region=asia-south1 \
  --source=. --entry-point=makeThumbnail \
  --trigger-http --allow-unauthenticated \
  --memory=512MB --timeout=120s

# Pub/Sub ট্রিগারযুক্ত ফাংশন
gcloud functions deploy order-processor \
  --gen2 --runtime=nodejs20 --region=asia-south1 \
```

```
  --source=. --entry-point=processOrder \
  --trigger-topic=orders-created
```

নিয়ম: ফাংশন stateless রাখুন — স্থানীয় ফাইলে কিছু লিখবেন না (শুধু `/tmp` সাময়িক), প্রতিটি invocation স্বাধীন ভাবুন, ভারী কাজ Cloud Tasks বা Pub/Sub-এ ঠেলে দিন।

১৮.৬ Pub/Sub: নির্ভরযোগ্য বার্তা পরিবহন



Pub/Sub হলো asynchronous বার্তা পরিষেবা: প্রকাশক (publisher) একটি topic-এ বার্তা পাঠায়, গ্রাহক (subscriber) subscription দিয়ে তা পায়। প্রকাশক ও গ্রাহক একে অপরকে জানে না, আলাদা গতিতে চলে, আর বার্তা অন্তত একবার (at-least-once) পৌঁছে দেওয়ার নিশ্চয়তা থাকে। ভেতরে: বার্তা topic-এ লেখা হয়, subscription-এ সারিবদ্ধ হয়, subscriber pull বা push করে নেয়; স্বীকৃতি (ack) না দিলে বার্তা আবার দেওয়া হয়।

```
# Topic ও subscription তৈরি
gcloud pubsub topics create orders-created
gcloud pubsub subscriptions create orders-worker-sub \
  --topic=orders-created \
  --ack-deadline=60 \
  --max-delivery-attempts=5 \
  --dead-letter-topic=orders-dead-letter

# বার্তা প্রকাশ (পরীক্ষা)
gcloud pubsub topics publish orders-created --message='{"orderId":"ORD-101","total":2500}'
```

Ordering: ডিফল্টে ক্রমের নিশ্চয়তা নেই; একই ordering key-এর বার্তাগুলোর ক্রম চাইলে message ordering চালু করুন — তবে তাতে throughput কমে। **Push বনাম Pull:** Push-এ Pub/Sub আপনার HTTPS endpoint ডাকে (Cloud Run-এর জন্য উপযুক্ত); Pull-এ worker নিজে টেনে নেয় (GKE worker-এর জন্য উপযুক্ত)।

১৮.৭ Eventarc ও Cloud Tasks

Eventarc হলো GCP পরিষেবাগুলোর ইভেন্ট রাউটার: Cloud Storage-এ ফাইল ওঠা, Cloud SQL পরিবর্তন, Audit Log-এর ঘটনা — যেকোনো ইভেন্ট ধরে Cloud Run, functions বা GKE workload-এ পাঠায়। Pub/Sub আপনি নিজে বার্তা পাঠান; Eventarc প্ল্যাটফর্মের ঘটনা আপনার কাছে আনে। দুটো প্রায়ই একসঙ্গে চলে: Eventarc ইভেন্ট ধরে Pub/Sub topic-এ ফেলে, সেখান থেকে worker নেয়।

Cloud Tasks হলো পরিচালিত টাস্ক কিউ — যখন কাজটি পরে, নির্দিষ্ট হারে, নির্দিষ্ট সময়ে করতে হয়। উদাহরণ: পেমেন্ট সফল হলে ২৪ ঘণ্টা পর রিমাইন্ডার ইমেইল; HTTP endpoint-কে rate-limit করে ডাকা। Pub/Sub-এর সঙ্গে পার্থক্য: Tasks-এ আপনি কাজের সময়সূচি, হার সীমা ও retry নীতি সূক্ষ্মভাবে নিয়ন্ত্রণ করেন; Pub/Sub সাধারণ বার্তা প্রবাহের জন্য।

```
# Cloud Tasks কিউ তৈরি
gcloud tasks queues create email-reminders \
  --location=asia-south1 \
  --max-dispatches-per-second=10 \
```

```
  --max-concurrent-dispatches=5 \
  --max-attempts=5 --min-backoff=30s --max-backoff=3600s
```

১৮.৮ Retry, Dead-Letter ও Idempotency: ব্যর্থতা সামলানোর ত্রয়ী

বিতরণকৃত সিস্টেমে ব্যর্থতা স্বাভাবিক: নেটওয়ার্ক ছিঁড়বে, ডাউনস্ট্রিম ধীর হবে, কোডে বাগ থাকবে। তিনটি নীতি ছাড়া event-driven সিস্টেম টেকে না।

১. Retry — exponential backoff সহ: ব্যর্থ হলে সঙ্গে সঙ্গে আবার চেষ্টা করবেন না; অপেক্ষা বাড়িয়ে চেষ্টা করুন (৩০ সেকেন্ড, ১ মিনিট, ২ মিনিট...)। নাহলে ব্যর্থ ডাউনস্ট্রিমকে আরও চাপ দিয়ে পুরো সিস্টেম ফেলে দেবেন (retry storm)। সর্বোচ্চ চেষ্টার সংখ্যা ঠিক করুন।

২. Dead-letter: নির্দিষ্ট সংখ্যকবার ব্যর্থ বার্তা একটি dead-letter topic/queue-তে যায়, যেখানে মানুষ দেখে সিদ্ধান্ত নেয় — ফেরত পাঠানো, ঠিক করে আবার চালানো, বা বাদ দেওয়া। Dead-letter ছাড়া ব্যর্থ বার্তা চিরকাল ঘুরতে থাকে বা নীরবে হারায়।

৩. Idempotency: Pub/Sub অন্তত একবার ডেলিভারি দেয় — একই বার্তা দুবার আসতে পারে। আপনার handler এমন হতে হবে যাতে দুবার চালালেও ফল একই হয়: যেমন orderId দিয়ে ডেটাবেসে unique constraint, বা প্রসেস করা বার্তার ID সংরক্ষণ করে দ্বিতীয়বার এড়িয়ে যাওয়া। ইমেইল পাঠানো, পেমেন্ট চার্জ করা — এগুলো idempotent না হলে গ্রাহক দুবার চার্জ হবেন।

> **বাস্তব উদাহরণ:** TechNova-র অর্ডার worker একবার একই অর্ডার দুবার প্রসেস করে গ্রাহককে দুটি স্বাগত ইমেইল পাঠিয়েছিল — কারণ Pub/Sub বার্তাটি ack-এর আগেই timeout হয়ে আবার ডেলিভারি দিয়েছিল। সমাধান: worker প্রথমে `processed_orders` টেবিলে orderId ঢোকায় (unique key); দ্বিতীয়বার ঢোকাতে গিয়ে conflict হলে সঙ্গে সঙ্গে সফল বলে ack দেয়, কাজ আবার করে না।

১৮.৯ Concurrency, Timeout ও Observability

Cloud Run-এ concurrency বেশি রাখলে ইনস্ট্যান্স কম লাগে (খরচ কমে) কিন্তু একটি ধীর অনুরোধ অন্যগুলোকে আটকায়; CPU-ভারী কাজে concurrency ১–১০, I/O-ভারী কাজে ৮০–১০০০ পর্যন্ত যুক্তিসঙ্গত। Timeout সেট করুন ডাউনস্ট্রিমের সঙ্গে মিলিয়ে: Cloud Run অনুরোধের সর্বোচ্চ সময়সীমা আছে (ডকুমেন্টেশন দেখুন — সীমা পরিবর্তন হতে পারে), তার বেশি কাজ Cloud Tasks-এ ভাগ করুন। Observability: প্রতিটি অনুরোধের log Cloud Logging-এ, মেট্রিক (request count, latency, instance count) Cloud Monitoring-এ, আর distributed trace-এর জন্য Cloud Trace — অধ্যায় ২১-এ বিস্তারিত। প্রতিটি বার্তায় correlation ID রাখুন, যাতে API থেকে worker পর্যন্ত একই অনুরোধ খুঁজে পাওয়া যায়।

১৮.১০ নিরাপত্তা ও খরচ

নিরাপত্তা: প্রতিটি সার্ভিসের নিজস্ব service account, least-privilege IAM; Pub/Sub topic-এ প্রকাশ ও গ্রহণের অনুমতি আলাদা; Cloud Run-এ ingress সীমিত করুন (internal-only, বা VPC-র ভেতর থেকে শুধু); secret Secret Manager থেকে, env var-এ নয়; VPC Connector দিয়ে Cloud Run-কে VPC-র ভেতরের Cloud SQL-এ (private IP) পৌঁছাতে দিন, পাবলিক ইন্টারনেট দিয়ে নয়। **খরচের সতর্কতা:** সার্ভারলেস বিল হয় অনুরোধ, CPU-সময় ও মেমরি-সময় অনুযায়ী — min-instances, always-allocated CPU ও egress খরচ বাড়ায়; Pub/Sub-এ বার্তার আকার ও সংখ্যা, retention-এর

জন্য সংরক্ষণ বিল হয়; অপ্রত্যাশিত ট্রাফিক স্পাইক বিল স্পাইক করে — billing alert ও max-instances সীমা সেট করুন। সঠিক মূল্য Google Cloud Pricing Calculator-এ যাচাই করুন।

১৮.১১ সাধারণ ভুল

- handler-কে non-idempotent লেখা — দুবার ডেলিভারিতে দ্বিগুণ কাজ।
- retry ছাড়া বা অসীম retry দিয়ে ব্যর্থতা সামলানো।
- dead-letter ছাড়া Pub/Sub চালানো — ব্যর্থ বার্তা অদৃশ্য হয়।
- Cloud Run-এ দীর্ঘ ব্যাকগ্রাউন্ড কাজ চালানো (অনুরোধ শেষে CPU থ্রটল হয়)।
- সব সার্ভিসকে `--allow-unauthenticated` দেওয়া।
- বার্তায় বড় payload পাঠানো — বার্তায় শুধু ID, ডেটা Storage/ডেটাবেসে।

১৮.১২ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Cloud Run cold start ধীর	বড় ইমেজ, ভারী startup	min-instances, ছোট ইমেজ, lazy-load
বার্তা দুবার প্রসেস হচ্ছে	at-least-once ডেলিভারি, ack দেরি	idempotency key; ack-deadline বাড়ান
বার্তা হারিয়ে যাচ্ছে	ack ছাড়া ড্রপ, retention শেষ	subscription-এর ack ও retention; dead-letter যাচাই
Function timeout	দীর্ঘ কাজ, ধীর ডাউনস্ট্রিম	কাজ ভাগ করে Cloud Tasks-এ; timeout বাড়ান
Eventarc ইভেন্ট আসছে না	ভুল ট্রিগার, অনুমতি নেই	Eventarc trigger ও service account অনুমতি যাচাই

১৮.১৩ সারাংশ

Cloud Run কনটেইনার-ভিত্তিক সার্ভারলেস, Cloud Run functions ফাংশন-ভিত্তিক; Pub/Sub বার্তা পরিবহন করে, Eventarc প্ল্যাটফর্মের ঘটনা আনে, Cloud Tasks সময়সূচিবদ্ধ কাজ চালায়। এই স্থাপত্যের সাফল্য নির্ভর করে তিনটি শৃঙ্খলায়: exponential backoff-সহ retry, dead-letter, ও idempotent handler। সার্ভারলেস মানে দায়মুক্তি নয় — পরিচয়, নেটওয়ার্ক ও খরচের নিয়ন্ত্রণ আপনারই।

১৮.১৪ পর্যালোচনা প্রশ্ন

- Pub/Sub কেন at-least-once ডেলিভারি দেয়, exactly-once নয়?
- Cloud Run ও Cloud Run functions-এর মধ্যে কখন কোনটি বেছে নেবেন?
- Idempotency ছাড়া retry কেন বিপজ্জনক?
- Push ও Pull subscription-এর পার্থক্য কী?
- Cold start কমানোর তিনটি উপায় কী?

১৮.১৫ ব্যবহারিক অনুশীলন

- Cloud Storage-এ ফাইল আপলোড হলে Eventarc দিয়ে একটি function চালান, যেটি Pub/Sub-এ বার্তা পাঠায়; একটি Cloud Run worker সেটি গ্রহণ করে idempotency-সহ প্রসেস করে।
- ইচ্ছাকৃতভাবে worker-কে ব্যর্থ করে dead-letter topic-এ বার্তা যাওয়া দেখুন।
- একই বার্তা দুবার পাঠিয়ে দ্বিতীয়বার কাজ না হওয়া পরীক্ষা করুন।

অধ্যায় ১৯ • PART IV — Containers, Serverless & Delivery

Chapter 19 — CI/CD with Google Cloud

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Cloud Build দিয়ে ইমেজ build ও টেস্ট চালাতে; build trigger দিয়ে Git push-এ স্বয়ংক্রিয় pipeline চালাতে; Cloud Deploy দিয়ে dev → staging → production-এ নিরাপদ প্রমোশন করতে; Workload Identity Federation দিয়ে GitHub Actions থেকে key ছাড়া GCP-তে authenticate করতে; canary ও blue-green রিলিজ কৌশল প্রয়োগ করতে; rollback করতে; এবং পুরো CI/CD pipeline-কে সুরক্ষিত করতে।

১৯.২ পূর্বজ্ঞান ঝালাই

CI (Continuous Integration) মানে প্রতিটি পরিবর্তন স্বয়ংক্রিয়ভাবে build ও পরীক্ষা হওয়া; CD (Continuous Delivery/Deployment) মানে পরীক্ষায় পাস করা পরিবর্তন নির্ভরযোগ্যভাবে পরিবেশে পৌঁছানো। Git-এ push করা মানে কোড জমা দেওয়া; pipeline-এর কাজ হলো সেই কোডকে যাচাই করা আর্টিফ্যাক্টে (ইমেজ) বদলে, তারপর পরিবেশে পৌঁছে দেওয়া — মানুষের হাতে নয়, ঘোষণামূলক ধাপে। আগের অধ্যায়ে আপনি Artifact Registry ও Cloud Run/GKE শিখেছেন; এই অধ্যায়ে সেগুলোকে একসঙ্গে বেঁধে একটি pipeline বানাবেন।

> **CONCEPT:** ভালো pipeline-এর তিনটি গুণ: **পুনরুৎপাদনযোগ্য** (একই কমিট সবসময় একই আর্টিফ্যাক্ট দেয়), **পর্যবেক্ষণযোগ্য** (কোন ধাপে কেন ব্যর্থ হলো, তা দেখা যায়), এবং **নিরাপদ** (শুধু অনুমোদিত পরিবর্তনই production ছোঁয়, প্রতিটি ধাপের পরিচয় যাচাই করা যায়)। গতি গুরুত্বপূর্ণ, কিন্তু এই তিনটির বিনিময়ে নয়।

১৯.৩ প্রথম নীতি: কেন আলাদা build ও deploy?

নতুনরা প্রায়ই build ও deploy এক ধাপে করে ফেলে — সার্ভারে গিয়ে `git pull` করে চালানো। সমস্যা: সার্ভারে কী চলছে তা কেউ জানে না, একই কোড দুই সার্ভারে দুইভাবে চলে, rollback মানে আবার `git pull`-এর আশা করা। সঠিক বিভাজন: **build একবার, deploy অনেকবার।** একটি কমিট থেকে একটি ইমেজ তৈরি হয়, সেই একই ইমেজ (একই digest) dev-এ পরীক্ষা হয়, staging-এ যাচাই হয়, production-এ যায়। পরিবেশের পার্থক্য থাকে শুধু কনফিগারেশনে (env var, secret, replica সংখ্যা) — কোডে নয়। এতে production-এ যা চলছে তা ঠিক জানা যায়, আর rollback মানে আগের digest-এ ফেরত যাওয়া — দ্রুত ও নিশ্চিত।

১৯.৪ Cloud Build: পরিচালিত build পরিষেবা

Cloud Build হলো serverless build পরিষেবা: আপনি build ধাপের তালিকা দেন, Google worker চালায়। প্রতিটি ধাপ একটি কনটেইনার — `docker`, `node`, `gcloud` ইত্যাদি অফিসিয়াল builder ইমেজ ব্যবহার করা যায়। Build কনফিগারেশন `cloudbuild.yaml`-এ লেখা হয়, রিপোজিটরিতেই থাকে — ফলে build-এর সংজ্ঞা কোডের সঙ্গে version হয়।

```
# cloudbuild.yaml — TechNova API
steps:
  # 1. নির্ভরতা ও টেস্ট
  - name: 'node:20'
    entrypoint: 'npm'
```

```
    args: ['ci']
  - name: 'node:20'
    entrypoint: 'npm'
    args: ['test']
  # 2. ইমেজ build ও push (SHORT_SHA দিয়ে ট্যাগ)
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'build'
      - '-t'
      - 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/api:$SHORT_SHA'
      - '.'
  # 3. ইমেজ push
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'push'
      - 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/api:$SHORT_SHA'
images:
  - 'asia-south1-docker.pkg.dev/$PROJECT_ID/technova-images/api:$SHORT_SHA'
options:
  logging: CLOUD_LOGGING_ONLY
```

Cloud Build-এর নিজস্ব service account আছে — তাকে শুধু প্রয়োজনীয় অনুমতি দিন: Artifact Registry-তে লেখা (`roles/artifactregistry.writer`), Cloud Run/GKE-তে ডিপ্লয় (আলাদা ধাপে, আলাদা পরিচয়ে হলে আরও ভালো)। Build লগ Cloud Logging-এ যায়; build ইতিহাস Console-এ দেখা যায়।

১৯.৫ Build Trigger: Git push থেকে pipeline

Trigger হলো ঘটনা ও pipeline-এর সংযোগ: `main` ব্রাঞ্চে push হলে build চলবে, pull request-এ শুধু টেস্ট চলবে, tag `v*` হলে release build চলবে। Console (Cloud Build → Triggers) বা `gcloud builds triggers create github` দিয়ে তৈরি করুন। GitHub রিপোজিটরি সংযুক্ত করতে Cloud Build GitHub App ইনস্টল করতে হয় — এতে Google আপনার রিপোজিটরির webhook পায়, কিন্তু আপনার GitHub পাসওয়ার্ড পায় না।

```
# Trigger তৈরি (GitHub সংযুক্ত থাকলে)
gcloud builds triggers create github \
  --name=technova-api-main \
  --repo-owner=technova --repo-name=api \
  --branch-pattern=^main$ \
  --build-config=cloudbuild.yaml

# হাতে build চালানো
gcloud builds submit --config=cloudbuild.yaml --region=asia-south1 .
```

ট্রিগারের নিয়ম: `main`-এ push → build + staging ডিপ্লয়; production ডিপ্লয় শুধু Cloud Deploy-এর অনুমোদন (approval) ধাপ দিয়ে — স্বয়ংক্রিয় production ডিপ্লয় ছোট দলে আকর্ষণীয়, কিন্তু অনুমোদন ছাড়া একটি ভুল কমিট সরাসরি ব্যবহারকারীর কাছে যায়।

১৯.৬ Cloud Deploy: পরিবেশ প্রমোশন ও রিলিজ

Cloud Build আর্টিফ্যাক্ট বানায়; **Cloud Deploy** সেই আর্টিফ্যাক্টকে পরিবেশে পৌঁছে দেয় — dev, staging, production-এর একটি pipeline (delivery pipeline) হিসেবে। আপনি একটি release তৈরি করেন (ইমেজ digest + কনফিগ), সেটি প্রথম target-এ যায়, অনুমোদন পেলে পরের target-এ প্রমোট হয়। প্রতিটি প্রমোশন রেকর্ড থাকে — কে, কখন, কোন

সংস্করণ অনুমোদন করল। Cloud Deploy GKE, Cloud Run ও (সীমিতভাবে) অন্যান্য target সমর্থন করে; deploy-এর জন্য Skaffold কনফিগারেশন ব্যবহার করে।

```
# clouddeploy.yaml (সরলীকৃত)
apiVersion: deploy.cloud.google.com/v1
kind: DeliveryPipeline
metadata:
  name: technova-api-pipeline
description: TechNova API delivery pipeline
serialPipeline:
  stages:
    - targetId: staging
      profiles: [staging]
    - targetId: prod
      profiles: [prod]
---
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata:
  name: staging
run:
  location: projects/technova-prod/locations/asia-south1
---
apiVersion: deploy.cloud.google.com/v1
kind: Target
metadata:
  name: prod
requireApproval: true
run:
  location: projects/technova-prod/locations/asia-south1
```

```
# Release তৈরি ও প্রমোট
gcloud deploy releases create rel-2026-11-09 \
  --delivery-pipeline=technova-api-pipeline \
  --region=asia-south1 \
  --images=api=asia-south1-docker.pkg.dev/technova-prod/technova-images/api:abc1234
gcloud deploy releases promote --release=rel-2026-11-09 \
  --delivery-pipeline=technova-api-pipeline --region=asia-south1 --to-target=prod
# অনুমোদন (Console-এ বা CLI-তে)
gcloud deploy rollouts approve prod-rollout --release=rel-2026-11-09 \
  --delivery-pipeline=technova-api-pipeline --region=asia-south1
```

১৯.৭ রিলিজ কৌশল ও Rollback

কৌশল	কী হয়	কখন
Rolling update	ধীরে ধীরে পুরোনো Pod বদলে নতুন আসে	ডিফল্ট; বেশিরভাগ ক্ষেত্রে
Canary	অল্প ট্রাফিক (৫–১০%) নতুন সংস্করণে, দেখে বাড়ানো	ঝুঁকিপূর্ণ পরিবর্তন, নতুন ফিচার
Blue-green	নতুন পরিবেশ পুরো প্রস্তুত, তারপর ট্রাফিক বদল	ডেটাবেস মাইগ্রেশন-সহ বড় পরিবর্তন
Feature flag	কোড ডিপ্লয় হয়, ফিচার flag-এ চালু/বন্ধ	ধীরে ধীরে ব্যবহারকারীকে দেওয়া

Rollback মানে আগের release-এ ফেরত যাওয়া — Cloud Deploy-এ আগের release-কে আবার প্রমোট করলেই হয়; GKE/Cloud Run-এ আগের revision-এ ট্রাফিক ফেরানো। Rollback পরিকল্পনা ডিপ্লয়ের আগেই ঠিক করুন: ডেটাবেস মাইগ্রেশন কি backward-compatible? পুরোনো কোড কি নতুন স্কিমায় চলবে? Expand-and-contract পদ্ধতি ব্যবহার করুন: প্রথমে নতুন কলাম যোগ (পুরোনো কোড চলে), তারপর কোড বদল, পরে পুরোনো কলাম সরানো।

১৯.৮ GitHub Actions ও Workload Identity Federation

অনেক দল CI-এর জন্য GitHub Actions ব্যবহার করে। তখন GCP-তে authenticate করার নিরাপদ উপায় **Workload Identity Federation (WIF)**: GitHub-এর OIDC token দিয়ে Google-এর কাছে স্বল্পমেয়াদি credential নেওয়া — কোনো service account key ফাইল নেই, ঘোরানো (rotation) নেই, ফাঁস হওয়ার ভয় নেই। সেটআপ: GCP-তে Workload Identity Pool ও provider তৈরি, GitHub রিপোজিটরিকে attribute mapping দিয়ে যুক্ত করা, তারপর Actions-এ `google-github-actions/auth` ব্যবহার করা।

```
# Workload Identity Pool ও Provider (একবারের সেটআপ)
gcloud iam workload-identity-pools create github-pool \
  --location=global --display-name="GitHub Actions pool"
gcloud iam workload-identity-pools providers create-oidc github-provider \
  --location=global \
  --workload-identity-pool=github-pool \
  --issuer-uri=https://token.actions.githubusercontent.com \
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=assertion.repository"
# GitHub Actions workflow-এ:
# - uses: google-github-actions/auth@v2
#   with:
#     workload_identity_provider: 'projects/123456789/locations/global/workloadIdentityPools/
github-pool/providers/github-provider'
#     service_account: 'technova-ci@technova-prod.iam.gserviceaccount.com'
```

> **সতর্কতা:** CI-এর service account-কে কখনো `roles/owner` বা `roles/editor` দেবেন না। শুধু যা দরকার: Artifact Registry writer, Cloud Run deployer, নির্দিষ্ট GKE ক্লাস্টারে deploy অনুমতি। Key ফাইল (JSON) GitHub secret-এ রাখা পুরোনো ও বিপজ্জনক পদ্ধতি — WIF ব্যবহার করুন।

১৯.৯ CI/CD নিরাপত্তা

Pipeline নিজেই আক্রমণের লক্ষ্য — এতে production-এর চাবি থাকে। নিয়মগুলো: (১) প্রতিটি build ধাপে নির্দিষ্ট ইমেজ digest ব্যবহার করুন; (২) Artifact Registry-তে vulnerability স্ক্যান চালু রাখুন, গুরুতর (critical) দুর্বলতায় build ব্যর্থ করুন; (৩) Binary Authorization দিয়ে শুধু অনুমোদিত ইমেজ ডিপ্লয় হতে দিন; (৪) secret Secret Manager থেকে, build log-এ কখনো secret ছাপাবেন না; (৫) pull request-এর build-কে production credential দেবেন না (fork থেকে আসা PR বিপজ্জনক); (৬) SLSA / provenance: Cloud Build build provenance তৈরি করে — কোন সোর্স থেকে কোন আর্টিফ্যাক্ট এলো, তা যাচাইযোগ্য থাকে।

১৯.১০ উৎপাদন পরিস্থিতি: TechNova-র pipeline

TechNova-র প্রবাহ: ডেভেলপার feature ব্রাঞ্চে push করে → PR-এ Cloud Build টেস্ট ও lint চালায় → merge হলে `main` trigger ইমেজ build করে staging-এ Cloud Deploy দিয়ে ডিপ্লয় করে → QA যাচাই করে → release-কে production-এ প্রমোটের অনুরোধ যায়, দলের একজন অনুমোদন করে → canary ১০% ট্রাফিক, মেট্রিক ভালো থাকলে

১০০%। ব্যর্থ হলে এক ক্লিকে আগের release-এ ফেরত। প্রতিটি ধাপের লগ ও অনুমোদন রেকর্ড থাকে — অডিটের সময় কে কী করল, তা দেখানো যায়।

১৯.১১ সাধারণ ভুল

- প্রতিটি পরিবেশে আলাদা করে build করা (build একবার, deploy অনেকবার — ভুলে যাওয়া)।
- `latest` ট্যাগ ডিপ্লয় করা।
- production ডিপ্লয়ে অনুমোদন ধাপ না রাখা।
- CI credential-কে অতিরিক্ত অনুমতি দেওয়া বা key ফাইল ব্যবহার করা।
- টেস্ট ছাড়া শুধু build করা pipeline — build সবুজ মানেই কোড সঠিক নয়।
- ডেটাবেস মাইগ্রেশন rollback-এর কথা না ভাবা।

১৯.১২ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Build ব্যর্থ: permission denied (Artifact Registry)	Cloud Build SA-র writer অনুমতি নেই	IAM যাচাই; সঠিক প্রজেক্ট/অঞ্চল
Trigger চলছে না	webhook/সংযোগ ভাঙা, branch pattern মেলেনি	Trigger ইতিহাস; GitHub App সংযোগ যাচাই
Deploy-এ ভুল ইমেজ	ট্যাগ বদলে গেছে	digest দিয়ে release তৈরি; `latest` এড়ান
GitHub Actions auth ব্যর্থ	WIF attribute mapping ভুল	provider-এর mapping ও SA binding যাচাই
Canary-তে এরর বাড়ছে	নতুন কোডে বাগ	ট্রাফিক আগের revision-এ ফেরান; লগ দেখুন

১৯.১৩ সারাংশ

Cloud Build আর্টিফ্যাক্ট বানায়, trigger Git-এর ঘটনায় তা চালায়, Cloud Deploy পরিবেশে প্রমোশন পরিচালনা করে, আর Workload Identity Federation key ছাড়া CI-কে নিরাপদ পরিচয় দেয়। Build একবার, deploy অনেকবার; অনুমোদনসহ প্রমোশন; canary দিয়ে ঝুঁকি কমানো; rollback-এর পরিকল্পনা আগেই — এই শৃঙ্খলাই নির্ভরযোগ্য ডেলিভারি।

১৯.১৪ পর্যালোচনা প্রশ্ন

- কেন প্রতিটি পরিবেশে আলাদা build করা ভুল?
- Cloud Build ও Cloud Deploy-এর দায়িত্বের পার্থক্য কী?
- Workload Identity Federation কীভাবে service account key-এর প্রয়োজন দূর করে?
- Canary ও blue-green-এর মধ্যে কখন কোনটি বেছে নেবেন?
- Backward-compatible মাইগ্রেশন কেন rollback-এর জন্য জরুরি?

১৯.১৫ ব্যবহারিক অনুশীলন

- একটি নমুনা রিপোজিটরিতে `cloudbuild.yaml` লিখে trigger দিয়ে staging-এ ডিপ্লয় করুন।
- Cloud Deploy pipeline বানিয়ে staging → prod প্রমোশনে অনুমোদন ধাপ যোগ করুন।
- GitHub Actions + WIF দিয়ে key ছাড়া Artifact Registry-তে ইমেজ push করুন।

অধ্যায় ২০ • PART IV — Containers, Serverless & Delivery

Chapter 20 — Infrastructure as Code

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Terraform দিয়ে GCP রিসোর্স ঘোষণামূলকভাবে তৈরি ও পরিচালনা করতে; provider, resource, variable, output ও module ব্যাখ্যা করতে; remote state নিরাপদে (Cloud Storage + state lock) সাজাতে; plan/apply চক্রে পরিবর্তন যাচাই করতে; drift শনাক্ত ও সারাতে; Infrastructure Manager ব্যবহার করতে; এবং infrastructure-এর জন্য CI/CD pipeline বানাতে।

২০.২ পূর্বজ্ঞান ঝালাই

এতদিন আপনি Console ও gcloud-এ রিসোর্স বানিয়েছেন — হাতে করা (imperative) পদ্ধতি। সমস্যা: কে কী বানাল, তা কোথাও লেখা থাকে না; একই পরিবেশ আবার বানাতে গেলে ভুল হয়; পরিবর্তনের ইতিহাস নেই। Infrastructure as Code (IaC) মানে infrastructure-কে কোড হিসেবে লেখা — version-controlled, পর্যালোচনাযোগ্য (reviewable), পুনরুৎপাদনযোগ্য। কোডই সত্যের উৎস (source of truth); Console-এ হাতে বদলানো মানে কোড থেকে সরে যাওয়া — যাকে drift বলে।

> **CONCEPT:** IaC-র মূল ধারণা *desired state*। আপনি বলেন "এই VPC, এই দুটি সাবনেট, এই ফায়ারওয়াল নিয়ম থাকবে" — টুলটি বর্তমান অবস্থা পড়ে, পার্থক্য (diff) বের করে, শুধু প্রয়োজনীয় পরিবর্তন করে। এটি Kubernetes-এর reconciliation-এর মতোই দর্শন, শুধু স্তরটি infrastructure।

২০.৩ প্রথম নীতি: কেন কোড?

একটি পরিবেশ হাতে বানানো যায়; তিনটি পরিবেশ (dev, staging, prod) হাতে বানানো মানে তিন রকম ভুল। কোডে লিখলে: (১) পরিবেশগুলো একই সংজ্ঞা থেকে তৈরি হয়, শুধু variable বদলায়; (২) পরিবর্তন pull request-এ পর্যালোচনা হয় — ভুল ফায়ারওয়াল নিয়ম production-এ যাওয়ার আগে ধরা পড়ে; (৩) দুর্যোগে পুরো পরিবেশ আবার বানানো যায়; (৪) কে কখন কী বদলাল, Git ইতিহাসে থাকে। TechNova-র ক্ষেত্রে: VPC, GKE ক্লাস্টার, Cloud SQL, Artifact Registry — সব Terraform-এ; নতুন অঞ্চলে সম্প্রসারণ মানে একটি variable বদলানো, নতুন করে Console ঘাঁটা নয়।

২০.৪ Terraform on GCP: মূল উপাদান

Terraform HashiCorp-এর ওপেন-সোর্স IaC টুল। GCP-র জন্য `google` provider ব্যবহার করে — provider হলো Terraform ও GCP API-র মধ্যে অনুবাদক। মূল ব্লক: `resource` (তৈরি করার জিনিস), `data` (পড়ার জিনিস), `variable` (ইনপুট), `output` (ফল), `module` (পুনর্ব্যবহারযোগ্য প্যাকেজ)। Terraform state ফাইলে বর্তমান বাস্তব অবস্থার মানচিত্র রাখে — কোন resource কোন GCP রিসোর্সের সঙ্গে মেলে।

```
# main.tf — TechNova নেটওয়ার্ক (সরলীকৃত)
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
```

```
    }
  }
  backend "gcs" {
    bucket = "technova-tfstate"
    prefix = "prod"
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
}

variable "project_id" { type = string }
variable "region" {
  type    = string
  default = "asia-south1"
}

resource "google_compute_network" "technova_vpc" {
  name                    = "technova-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_subnetwork" "technova_subnet" {
  name          = "technova-subnet"
  ip_cidr_range = "10.10.0.0/24"
  region        = var.region
  network       = google_compute_network.technova_vpc.id
}

output "vpc_name" {
  value = google_compute_network.technova_vpc.name
}
```

```
# দৈনন্দিন চক্র
terraform init        # provider ও backend প্রস্তুত
terraform fmt         # ফরম্যাট ঠিক করা
terraform validate    # সিনট্যাক্স যাচাই
terraform plan        # কী বদলাবে, তা দেখা (ধ্বংসাত্মক পরিবর্তন খুঁজুন)
terraform apply       # অনুমোদন দিয়ে প্রয়োগ
terraform destroy     # পরিবেশ মোছা (সাবধানে!)
```

`terraform plan`-এর আউটপুট মনোযোগ দিয়ে পড়ুন: `+` নতুন, `~` বদল, `-` মোছা। কোনো resource-এ `-/+` (ধ্বংস করে আবার তৈরি) দেখলে থামুন — এর মানে ডেটা হারাতে পারে (যেমন ডেটাবেস প্রতিস্থাপন)। সেক্ষেত্রে `lifecycle { prevent_destroy = true }` দিয়ে গুরুত্বপূর্ণ resource রক্ষা করুন।

২০.৫ State Management: সবচেয়ে গুরুত্বপূর্ণ সিদ্ধান্ত

State ফাইলে সব resource-এর ID ও কিছু ক্ষেত্রে সংবেদনশীল মান থাকে — এটি কখনো Git-এ কমিট করবেন না, কখনো স্থানীয় ডিস্কে একমাত্র কপি রাখবেন না। GCP-তে সঠিক পদ্ধতি: **Cloud Storage bucket-এ remote backend**, versioning চালু, আর state lock-এর জন্য Cloud Storage-এর native locking (Terraform Google backend-এ সমর্থিত) ব্যবহার। একসঙ্গে দুজন apply করলে lock দ্বিতীয়জনকে থামায় — নাহলে state নষ্ট হয়।

```
# State bucket তৈরি (একবার, হাতে বা আলাদা bootstrap Terraform দিয়ে)
gsutil mb -p technova-prod -l asia-south1 gs://technova-tfstate/
gsutil versioning set on gs://technova-tfstate/
# bucket-এ public access বন্ধ, শুধু Terraform চালানো পরিচয়গুলোকে
# roles/storage.objectAdmin দিন; IAM-এ uniform bucket-level access চালু করুন।
```

State-এর নিয়ম: (১) প্রতি পরিবেশে আলাদা state (prefix বা আলাদা bucket); (২) state-এ secret রাখবেন না — secret Secret Manager-এ, Terraform শুধু রেফারেন্স করে; (৩) state ফাইল সরাসরি সম্পাদনা করবেন না — `terraform import` দিয়ে হাতে তৈরি resource-কে Terraform-এর অধীনে আনুন, `terraform state mv/rm` শুধু প্রয়োজনে।

২০.৬ Module ও নির্ভরতা

Module হলো পুনর্ব্যবহারযোগ্য Terraform প্যাকেজ: একই VPC সংজ্ঞা dev/staging/prod-এ ব্যবহার করা যায়, শুধু variable বদলে। ভালো module-এর স্পষ্ট ইনপুট, ন্যূনতম আউটপুট, ও ডিফল্ট নিরাপদ মান থাকে। নিজের module লিখুন (যেমন `modules/network`, `modules/gke`), অথবা Google-এর অফিসিয়াল Cloud Foundation Toolkit module ব্যবহার করুন — তবে যেকোনো বাইরের module-এর কোড পড়ে নিন, অন্ধভাবে ব্যবহার করবেন না।

নির্ভরতা: Terraform resource-এর মধ্যে রেফারেন্স দেখে স্বয়ংক্রিয়ভাবে ক্রম ঠিক করে (সাবনেট VPC-র পরে তৈরি হয়)। স্পষ্ট নির্ভরতা দরকার হলে `depends_on` ব্যবহার করুন — তবে অতিরিক্ত ব্যবহার graph-কে জটিল করে। `count` / `for_each` দিয়ে একই resource একাধিকবার তৈরি করুন; `for_each` map-ভিত্তিক হওয়ায় নাম বদলালে resource নষ্ট হয় না, `count`-এ ইনডেক্স সরে গেলে হয়।

২০.৭ Infrastructure Manager: GCP-র নিজস্ব IaC

Infrastructure Manager হলো Google-এর পরিচালিত Terraform পরিষেবা: আপনি Terraform কনফিগারেশন Cloud Storage-এ রাখেন, Infrastructure Manager deployment তৈরি করে plan/apply চালায় — আপনার নিজের CI runner বা state bucket সামলাতে হয় না। এটি Terraform-এর বিকল্প নয়, বরং Terraform চালানোর পরিচালিত উপায়; state Google পরিচালনা করে। ছোট দল বা যারা Terraform runner পরিচালনা করতে চায় না, তাদের জন্য উপযুক্ত। বড় দলে সাধারণত নিজস্ব CI pipeline-এ Terraform চালানোই বেশি নমনীয়তা দেয়।

```
# Infrastructure Manager deployment (সরলীকৃত)
gcloud infra-manager deployments create technova-prod-deploy \
  --location=asia-south1 \
  --service-account=technova-tf@technova-prod.iam.gserviceaccount.com \
  --git-source-repo=https://github.com/technova/infra \
  --git-source-directory=terraform/prod \
  --git-source-ref=main \
  --input-values="project_id=technova-prod"
```

২০.৮ Drift Detection ও CI/CD for Infrastructure

Drift মানে বাস্তব অবস্থা কোড থেকে সরে গেছে — কেউ Console-এ হাতে ফায়ারওয়াল খুলে দিয়েছে, বা Google কোনো ডিফল্ট বদলেছে। শনাক্তকরণ: নিয়মিত `terraform plan` (CI-তে নির্ধারিত সময়ে) চালিয়ে পার্থক্য দেখা; পার্থক্য থাকলে হয় কোড আপডেট করুন (ইচ্ছাকৃত পরিবর্তন হলে), নয়তো `apply` করে বাস্তব অবস্থা কোডে ফেরান। Console-এ হাতে বদলানো নিষেধ করুন — Organization Policy বা প্রক্রিয়া দিয়ে।

Infrastructure CI/CD: pull request-এ `terraform plan` চালিয়ে ফল PR-এ মন্তব্য হিসেবে দেখান (Atlantis বা GitHub Actions); `main`-এ merge হলে `apply` — production-এ অনুমোদন ধাপসহ। Plan-এর আউটপুটে ধ্বংসাত্মক পরিবর্তন থাকলে apply থামান। Terraform-এর service account-কে least-privilege দিন: শুধু সেই resource-গুলো তৈরি/বদলের অনুমতি, `roles/owner` নয়।

> **সতর্কতা:** Terraform-এর service account-এর হাতে পুরো প্রজেক্টের চাবি থাকলে একটি ভুল `apply` সব মুছে দিতে পারে। গুরুত্বপূর্ণ resource-এ `prevent_destroy`, production apply-তে মানুষের অনুমোদন, আর state bucket-এর versioning — এই তিনটি রক্ষাকবচ ছাড়া production Terraform চালাবেন না।

২০.৯ IaC টুল বেছে নেওয়া

টুল	শক্তি	দুর্বলতা / কখন নয়
Terraform	মাল্টি-ক্লাউড, বিশাল ecosystem, ঘোষণামূলক	state সামলাতে হয়; GCP-নির্দিষ্ট গভীর ফিচারে পিছিয়ে থাকতে পারে
Infrastructure Manager	পরিচালিত, state Google-এর দায়িত্বে	Terraform-এর ওপরে আবরণ; নমনীয়তা কম
Deployment Manager	GCP-নেটিভ (পুরোনো)	নতুন কাজে সুপারিশ করা হয় না; Terraform-এ যান
Pulumi / CDK	সাধারণ প্রোগ্রামিং ভাষায় IaC	দল Python/TypeScript-এ স্বচ্ছন্দ হলে ভালো; ecosystem ছোট
Config Connector / Crossplane	Kubernetes YAML দিয়ে GCP রিসোর্স	GitOps-কেন্দ্রিক দলে উপযুক্ত; শেখার বক্ররেখা আছে

TechNova-র জন্য সুপারিশ: Terraform মূল IaC, Infrastructure Manager দিয়ে শুরু করা যেতে পারে, আর Kubernetes-ভেতরের রিসোর্সের জন্য Config Connector বিবেচনা করা যায়। একটি টুলে স্থির থাকুন — প্রতিটি পরিবেশে ভিন্ন টুল মানে দ্বিগুণ জ্ঞান ও দ্বিগুণ ভুল।

২০.১০ সাধারণ ভুল

- state ফাইল Git-এ কমিট করা বা স্থানীয়ভাবে রাখা।
- plan না পড়ে apply করা; ধ্বংসাত্মক পরিবর্তন উপেক্ষা করা।
- secret Terraform variable-এ লেখা (tfvars Git-এ)।
- একটি বিশাল monolithic state — ভাগ করুন (network, data, apps আলাদা state)।
- Console-এ হাতে বদলানো ও drift উপেক্ষা করা।
- `count` দিয়ে নামযুক্ত resource-এর তালিকা বদলানো।

২০.১১ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
State lock আটকে আছে	আগের apply মাঝপথে থেমেছে	lock ধারক নিশ্চিত হয়ে `force-unlock` (সাবধানে)

Plan-এ অপ্রত্যাশিত ধ্বংস	নাম/আর্গুমেন্ট বদলেছে	plan পড়ুন; `terraform state mv` বিবেচনা করুন
Apply-এ permission denied	SA-র অনুমতি কম	কোন API/role দরকার, তা লগ থেকে দেখুন
Drift প্রতিবার দেখা যাচ্ছে	কেউ হাতে বদলাচ্ছে বা ডিফল্ট বদলেছে	পরিবর্তনের উৎস খুঁজুন; `ignore_changes` শুধু যৌক্তিক ক্ষেত্রে

২০.১২ সারাংশ

IaC মানে infrastructure-কে version-controlled কোড হিসেবে দেখা। Terraform-এ provider, resource, variable, module দিয়ে desired state লিখুন; Cloud Storage-এ remote state ও lock রাখুন; plan পড়ে apply করুন; drift নিয়মিত যাচাই করুন; infrastructure-এর পরিবর্তনও CI/CD ও অনুমোদনের মধ্য দিয়ে যাক। Infrastructure Manager একই Terraform-কে পরিচালিতভাবে চালায়।

২০.১৩ পর্যালোচনা প্রশ্ন

- State ফাইল কেন Git-এ রাখা যায় না?
- `plan`-এ `-/+` দেখলে কী করবেন?
- Module ব্যবহারের সুবিধা কী?
- Drift কীভাবে শনাক্ত ও সারাবেন?
- Terraform ও Infrastructure Manager-এর সম্পর্ক কী?

২০.১৪ ব্যবহারিক অনুশীলন

- Terraform দিয়ে একটি VPC + সাবনেট + Cloud Storage bucket তৈরি করুন; remote state সাজান।
- Console-এ হাতে একটি ফায়ারওয়াল নিয়ম বদলে `plan`-এ drift দেখুন, তারপর সারান।
- একটি module লিখে দুটি পরিবেশে (dev/prod) ভিন্ন variable দিয়ে ব্যবহার করুন।

PART V • DEVOPS ENGINEERING MASTERY

Production GCP Engineering

Monitoring, Logging & Tracing, GCP Security Engineering, Reliability, Backup & Disaster Recovery, Production Architecture & FinOps I

অধ্যায় ২১ • PART V — Production GCP Engineering

Chapter 21 — Monitoring, Logging and Tracing

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Cloud Monitoring-এ মেট্রিক, ড্যাশবোর্ড ও alerting policy সাজাতে; Cloud Logging-এ লগ খুঁজতে, ফিল্টার করতে ও log-based metric বানাতে; Cloud Trace দিয়ে distributed অনুরোধের ধীরগতি খুঁজতে; Error Reporting ও Profiler ব্যবহার করতে; OpenTelemetry দিয়ে নিজের অ্যাপ instrument করতে; এবং ঘটনার সময় দ্রুত তদন্ত করতে।

২১.২ পূর্বজ্ঞান ঝালাই

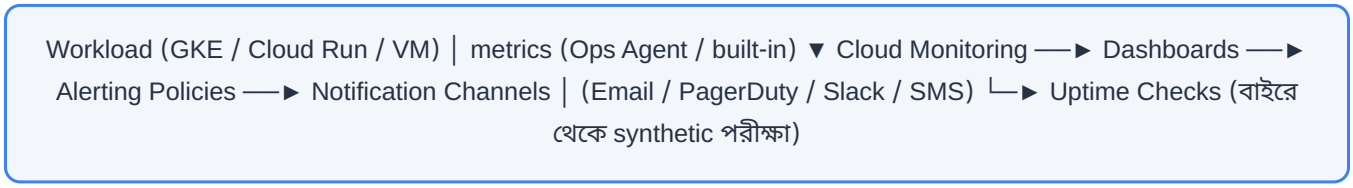
তিনটি স্তম্ভ (three pillars) মনে রাখুন: **Metrics** (সংখ্যা — কত অনুরোধ, কত দেরি, কত CPU), **Logs** (ঘটনা — কী ঘটল, কখন, কার জন্য), **Traces** (পথ — একটি অনুরোধ কোন পরিষেবা দিয়ে কোথায় সময় কাটাল)। Metrics বলে "কিছু ভুল হচ্ছে", logs বলে "কী ভুল হচ্ছে", traces বলে "কোথায় ভুল হচ্ছে"। GCP-তে এগুলো Cloud Operations suite-এ একত্রিত: Monitoring, Logging, Trace, Error Reporting, Profiler। আগের অধ্যায়গুলোতে আপনি workload চালিয়েছেন; এখন সেগুলোকে দেখতে শিখবেন।

> **CONCEPT:** পর্যবেক্ষণ (monitoring) মানে ড্যাশবোর্ডে তাকিয়ে থাকা নয়; মানে ব্যবহারকারীর অভিজ্ঞতা পরিমাপ করা। SLI (Service Level Indicator) হলো পরিমাপ — যেমন "৯৯% অনুরোধ ৩০০ মিলিসেকেন্ডের মধ্যে সফল"। SLO হলো লক্ষ্য — যেমন "মাসে ৯৯.৯% সময় এই SLI ধরে রাখা"। Alert বাজবে SLI খারাপ হলে, CPU ৮০% হলেই নয় — CPU বেশি হওয়া সমস্যা নয়, যদি ব্যবহারকারী ভালো অভিজ্ঞতা পায়।

২১.৩ প্রথম নীতি: কী পরিমাপ করবেন?

Google-এর SRE বইয়ের "four golden signals" দিয়ে শুরু করুন: **Latency** (অনুরোধ কত দেরিতে ফেরে), **Traffic** (কত অনুরোধ আসছে), **Errors** (কত শতাংশ ব্যর্থ), **Saturation** (সম্পদ কতটা ভরা — CPU, মেমরি, ডিস্ক, সংযোগ)। প্রতিটি পরিষেবার জন্য এই চারটি মেট্রিক থাকা চাই। তারপর ব্যবসায়িক মেট্রিক: TechNova-র ক্ষেত্রে সফল অর্ডার প্রতি মিনিটে, পেমেন্ট ব্যর্থতার হার। মেট্রিক ছাড়া আপনি অনুমানে চালাচ্ছেন; অতিরিক্ত মেট্রিক সংগ্রহ করলে খরচ ও শব্দ (noise) বাড়ে — যা সিদ্ধান্তে লাগে, শুধু তা-ই সংগ্রহ করুন।

২১.৪ Cloud Monitoring: মেট্রিক ও ড্যাশবোর্ড



Cloud Monitoring GCP পরিষেবাগুলোর মেট্রিক স্বয়ংক্রিয়ভাবে সংগ্রহ করে (যেমন Cloud Run-এর request count, GKE-র Pod CPU)। VM-এর ভেতরের মেট্রিকের জন্য Ops Agent ইনস্টল করুন — এটি Logging ও Monitoring দুটোর agent একত্রিত করে। কাস্টম মেট্রিক নিজের অ্যাপ থেকে লিখতে পারেন (OpenTelemetry বা Monitoring API দিয়ে)।

ড্যাশবোর্ডে golden signals + ব্যবসায়িক মেট্রিক রাখুন; প্রতিটি ড্যাশবোর্ডের একজন মালিক থাকা চাই, নাহলে সেটি পুরোনো হয়ে যায়।

```
# Console: Monitoring → Dashboards → Create Dashboard
# CLI: alerting policy তৈরি (উদাহরণ — এরর হার বেশি হলে)
gcloud alpha monitoring policies create --policy-from-file=alert-policy.json
# alert-policy.json-এ: condition — Cloud Run request count-এর মধ্যে
# 5xx অনুপাত 5% ছাড়ালে 5 মিনিট ধরে, notification channel-এ alert যাবে।
```

Uptime check: বাইরে থেকে আপনার endpoint-কে নিয়মিত ডাকা হয় — এটি সত্যিকারের ব্যবহারকারীর দৃষ্টিভঙ্গি দেয়। ভেতরের মেট্রিক ভালো থাকলেও DNS বা LoadBalancer ভাঙলে uptime check ধরে।

২১.৫ Alerting Policy: কখন ঘণ্টা বাজবে?

ভালো alert-এর নিয়ম: (১) কাজ করার মতো (actionable) — alert পেলে কী করবেন, তা জানা থাকতে হবে; (২) ব্যবহারকারী-প্রভাবিত — ভেতরের CPU নয়, এরর হার বা latency; (৩) সঠিক threshold ও duration — এক মিনিটের স্পাইকে জাগাবেন না, পাঁচ মিনিট ধরে খারাপ থাকলে জাগান; (৪) severity আলাদা করুন — page (জরুরি, মানুষ জাগবে) বনাম ticket (সকালে দেখা যাবে)। Alert fatigue মারাত্মক: প্রতিদিন দশটি অপ্রয়োজনীয় alert এলে আসল alert-ও উপেক্ষিত হয়। প্রতিটি alert-এর সঙ্গে runbook-এর লিঙ্ক দিন (অধ্যায় ২৩)।

Notification channel: Email, SMS, Slack, PagerDuty, webhook। Production-এর জরুরি alert অন্তত দুটি চ্যানেলে যাক, আর on-call rotation থাকুক — একজন মানুষ সবসময় দায়িত্বে।

২১.৬ Cloud Logging: লগ খোঁজা ও বোঝা

Cloud Logging সব GCP পরিষেবা, GKE Pod, Cloud Run ও VM-এর লগ এক জায়গায় জমা করে। Logs Explorer-এ query লিখে খুঁজুন — যেমন একটি নির্দিষ্ট request ID, একটি নির্দিষ্ট Pod-এর এরর, বা গত এক ঘণ্টার সব ৫০০ এরর। ভালো লগের বৈশিষ্ট্য: structured (JSON) — যাতে `severity`, `trace`, `userId` (সংবেদনশীল নয় এমন) ফিল্ডে খোঁজা যায়; প্রতিটি লাইনে correlation/trace ID; পাসওয়ার্ড, token, ব্যক্তিগত তথ্য কখনো লগে নয়।

```
# Logs Explorer query উদাহরণ
resource.type="cloud_run_revision"
resource.labels.service_name="technova-api"
severity>=ERROR
timestamp>="2026-11-09T00:00:00Z"

# gcloud দিয়ে লগ পড়া
gcloud logging read 'resource.type="cloud_run_revision" severity>=ERROR' \
  --limit=20 --format=json --freshness=1h
```

খরচের সতর্কতা: লগ ingestion (জমা করা) ও সংরক্ষণ বিল হয় — verbose debug লগ production-এ চালু রাখলে বিল বাড়ে। Log exclusion দিয়ে অপ্রয়োজনীয় লগ (যেমন স্বাস্থ্য পরীক্ষার সফল লগ) বাদ দিন, retention প্রয়োজন অনুযায়ী সেট করুন, আর গুরুত্বপূর্ণ লগ BigQuery-তে export করে দীর্ঘমেয়াদে বিশ্লেষণ করুন। সঠিক মূল্য Google Cloud Pricing Calculator-এ যাচাই করুন।

Log-based metric: লগের প্যাটার্ন থেকে মেট্রিক বানান — যেমন "payment failed" লগের সংখ্যা প্রতি মিনিটে — তারপর তাতে alert বসান। এটি কোড বদল না করে নতুন মেট্রিক পাওয়ার উপায়।

২১.৭ Cloud Trace, Error Reporting ও Profiler

Cloud Trace: একটি অনুরোধ API → worker → ডেটাবেস — কোথায় কত সময় গেল, তা দেখায়। ধীর অনুরোধের ক্ষেত্রে trace খুললে দেখা যায় ডেটাবেস query-তে ২ সেকেন্ড গেছে, নাকি ডাউনস্ট্রিম API-তে। Trace-এর জন্য প্রতিটি পরিষেবায় trace context (traceparent header) এগিয়ে দিতে হয় — OpenTelemetry এটি স্বয়ংক্রিয় করে।

Error Reporting: সব এরর একত্রিত করে গোষ্ঠীভুক্ত করে — একই স্ট্যাক ট্রেসের হাজারটি ঘটনা একটি গ্রুপে, প্রথম কবে দেখা গেল, কতবার হলো। নতুন এরর গ্রুপ দেখা দিলে alert দিন।

Profiler: production-এ চলা কোডের CPU ও মেমরি প্রোফাইল নেয় — কোন ফাংশন সবচেয়ে বেশি CPU খাচ্ছে, তা দেখায়। লোড টেস্টের অনুমানের বদলে বাস্তব ব্যবহারের চিত্র দেয়।

২১.৮ OpenTelemetry Integration

OpenTelemetry (OTel) হলো vendor-neutral instrumentation মান: একবার instrument করলে trace/metric/log যেকোনো backend-এ (Cloud Trace/Monitoring সহ) পাঠানো যায়। Node.js অ্যাপে OTel SDK বসিয়ে স্বয়ংক্রিয়ভাবে HTTP অনুরোধ, ডেটাবেস query ও বাইরের API কলের span তৈরি হয়। GCP-তে OTel-এর exporter Cloud Trace ও Cloud Monitoring-এ পাঠায়। নতুন অ্যাপে শুরু থেকেই OTel বসান — পরে যোগ করা কঠিন।

```
// Node.js — OpenTelemetry SDK (সরলীকৃত ধারণা)
// npm install @opentelemetry/sdk-node @opentelemetry/auto-instrumentations-node
// SDK চালু করলে HTTP, Express ও ডেটাবেস কল স্বয়ংক্রিয়ভাবে traced হয়;
// trace ID লগের সঙ্গে যুক্ত হয়, ফলে Logs Explorer থেকে Trace-এ যাওয়া যায়।
```

২১.৯ ঘটনা তদন্তের কর্মপ্রবাহ

Alert বাজল — এখন কী? (১) ড্যাশবোর্ডে golden signals দেখুন: কখন শুরু, কোন পরিষেবা, কতটা প্রভাব। (২) Error Reporting-এ নতুন এরর গ্রুপ আছে কি না দেখুন। (৩) Trace-এ ধীর অনুরোধের নমুনা খুলুন। (৪) Logs Explorer-এ সেই সময়ের, সেই trace ID-র লগ দেখুন। (৫) সাম্প্রতিক পরিবর্তন দেখুন: কোন ডিপ্লয়, কনফিগ বদল, স্কেলিং ঘটনা ঘটেছে? বেশিরভাগ ঘটনার মূলে থাকে সাম্প্রতিক পরিবর্তন। তদন্তের প্রতিটি ধাপ লিখে রাখুন — পরে postmortem-এ (অধ্যায় ২৩) কাজে লাগবে।

> **বাস্তব উদাহরণ:** TechNova-র API-র latency হঠাৎ বেড়ে গেল, কিন্তু CPU স্বাভাবিক। Trace খুলে দেখা গেল ডেটাবেস query-তে সময় যাচ্ছে; Logs-এ দেখা গেল একই সময়ে একটি নতুন ডিপ্লয় একটি N+1 query চালু করেছে — প্রতিটি অর্ডারের জন্য আলাদা query। সমাধান: query একত্রিত করা, আর alert-এ latency-ভিত্তিক SLO যোগ করা, যাতে ভবিষ্যতে ধীরগতি আগেই ধরা পড়ে।

২১.১০ সাধারণ ভুল

- শুধু infrastructure মেট্রিক (CPU/মেমরি) দেখা, ব্যবহারকারীর SLI না দেখা।
- প্রতিটি জিনিসে alert বসানো — alert fatigue।
- অসংগঠিত (unstructured) লগ লেখা, trace ID না রাখা।
- লগে secret বা ব্যক্তিগত তথ্য ছাপানো।
- ড্যাশবোর্ড বানিয়ে আর কখনো না দেখা।

- debug লগ production-এ চালু রেখে খরচ বাড়ানো।

২১.১১ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
মেট্রিক দেখা যাচ্ছে না	Ops Agent নেই, অনুমতি নেই	Agent অবস্থা; SA-র monitoring অনুমতি যাচাই
Alert বাজছে না	condition/duration ভুল, channel ভুল	Policy পরীক্ষা; test notification পাঠান
লগ খুঁজে পাওয়া যাচ্ছে না	ভুল resource ফিল্টার, exclusion	ফিল্টার সরিয়ে দেখুন; exclusion যাচাই
Trace ভাঙা (span মেলেনি)	trace context এগিয়ে দেওয়া হয়নি	OTel instrumentation ও header propagation যাচাই
লগ বিল বেশি	verbose লগ, দীর্ঘ retention	exclusion, sampling, retention কমান

২১.১২ সারাংশ

Metrics বলে কী হচ্ছে, logs বলে কেন হচ্ছে, traces বলে কোথায় হচ্ছে। Golden signals দিয়ে SLI/SLO সাজান, actionable alert বসান, structured লগে correlation ID রাখুন, OpenTelemetry দিয়ে instrument করুন। পর্যবেক্ষণ কোনো অতিরিক্ত কাজ নয় — এটি production-এর অংশ, ডিপ্লয়ের আগেই সাজাতে হয়।

২১.১৩ পর্যালোচনা প্রশ্ন

- SLI, SLO ও alert-এর সম্পর্ক কী?
- Four golden signals কী কী?
- Log-based metric কখন ব্যবহার করবেন?
- Trace কেন শুধু মেট্রিক দিয়ে বদলানো যায় না?
- Alert fatigue কীভাবে এড়াবেন?

২১.১৪ ব্যবহারিক অনুশীলন

- একটি Cloud Run সার্ভিসের জন্য latency ও error-rate ড্যাশবোর্ড ও alerting policy বানান।
- একটি ধীর অনুরোধের trace খুঁজে কোন span-এ সময় গেছে, তা চিহ্নিত করুন।
- একটি log-based metric বানিয়ে তাতে alert বসান।

অধ্যায় ২২ • PART V — Production GCP Engineering

Chapter 22 — GCP Security Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: least-privilege IAM ডিজাইন করতে; service account-কে নিরাপদে পরিচালনা করতে (key ছাড়া); Workload Identity Federation ব্যবহার করতে; Cloud KMS দিয়ে encryption key পরিচালনা করতে; Secret Manager দিয়ে secret-এর জীবনচক্র সামলাতে; Security Command Center দিয়ে ঝুঁকি দেখতে; Cloud Armor দিয়ে অ্যাপ্লিকেশন রক্ষা করতে; VPC Service Controls দিয়ে ডেটা সীমানা বাঁধতে; organization policy দিয়ে নিয়ন্ত্রণ আরোপ করতে; এবং নিরাপত্তা ঘটনায় সাড়া দিতে।

২২.২ পূর্বজ্ঞান ঝালাই

নিরাপত্তার মূলনীতি তিনটি: **গোপনীয়তা** (শুধু অনুমোদিতরা ডেটা দেখবে), **অখণ্ডতা** (ডেটা অনুমোদন ছাড়া বদলাবে না), **প্রাপ্যতা** (অনুমোদিতরা প্রয়োজনে ডেটা পাবে)। GCP-তে এই তিনটি বাস্তবায়িত হয় IAM (কে কী করতে পারে), encryption (ডেটা কীভাবে সুরক্ষিত), network controls (কে কার সঙ্গে কথা বলতে পারে) ও audit logging (কে কী করল) দিয়ে। আগের অধ্যায়গুলোতে আপনি প্রতিটি পরিষেবায় IAM ছুঁয়েছেন; এই অধ্যায়ে সেগুলোকে একটি পূর্ণ নিরাপত্তা স্থাপত্যে বাঁধবেন। মনে রাখবেন: নিরাপত্তা কোনো একক পণ্য নয় — এটি স্তরে স্তরে (defense in depth) সাজানো নিয়ন্ত্রণের সমষ্টি।

> **CONCEPT:** Least privilege মানে প্রতিটি পরিচয় (মানুষ বা service account) শুধু তার কাজের জন্য প্রয়োজনীয় ন্যূনতম অনুমতি পায়, শুধু যতক্ষণ দরকার ততক্ষণ। `roles/owner` দেওয়া সহজ, কিন্তু একটি আপস হওয়া পরিচয় পুরো প্রজেক্ট মুছে দিতে পারে। নিরাপত্তা-প্রকৌশলীর কাজ হলো সহজ পথটি কঠিন করে সঠিক পথটি সহজ করা — যেমন Workload Identity Federation দিয়ে key-এর ঝামেলা দূর করা, যাতে কেউ key ব্যবহারের অজুহাত না পায়।

২২.৩ প্রথম নীতি: আস্থার সীমানা আঁকুন

প্রতিটি সিস্টেমে আস্থার সীমানা (trust boundary) থাকে: ইন্টারনেট বনাম আপনার VPC, একটি microservice বনাম আরেকটি, একজন ডেভেলপার বনাম production ডেটা। নিরাপত্তা নকশা মানে এই সীমানাগুলো চিহ্নিত করা ও প্রতিটি পারাপারে যাচাই বসানো: authentication (আপনি কে?), authorization (আপনি কী করতে পারেন?), audit (আপনি কী করলেন?)। GCP-তে পরিচয় হলো Google account, service account বা federated identity; যাচাই হয় IAM policy-তে; রেকর্ড থাকে Cloud Audit Logs-এ। সীমানা না আঁকলে সবকিছু সবকিছুকে বিশ্বাস করে — আর একটি ছিদ্রই যথেষ্ট।

২২.৪ IAM Security: নীতি, ভূমিকা ও শর্ত

IAM policy তিনটি অংশে: **who** (member — user, group, service account, domain), **what** (role — অনুমতির সমষ্টি), **where** (resource — প্রজেক্ট, ফোল্ডার, organization, বা নির্দিষ্ট resource)। নীতি উত্তরাধিকারসূত্রে নিচে নামে: organization-এ দেওয়া অনুমতি সব প্রজেক্টে প্রযোজ্য হয় — তাই organization-স্তরে সাবধানে দিন। Predefined role ব্যবহার করুন (যেমন `roles/storage.objectViewer`); basic role (`roles/owner`, `roles/editor`, `roles/viewer`) production-এ এড়িয়ে চলুন — এগুলো অত্যন্ত বিস্তৃত। Custom role শুধু তখনই, যখন predefined role-এ ঠিক মেলে না।

IAM Conditions দিয়ে অনুমতি সীমিত করুন: যেমন একটি role শুধু নির্দিষ্ট সময়ে, নির্দিষ্ট IP থেকে, বা নির্দিষ্ট resource ট্যাগ থাকলে প্রযোজ্য। **IAM Recommender** দেখায় কোন অনুমতি অব্যবহৃত — নিয়মিত পর্যালোচনা করে সরিয়ে দিন। **Policy Analyzer** দিয়ে জিজ্ঞাসা করুন: "এই ব্যবহারকারী এই bucket-এ লিখতে পারে কি?" — অনুমানের বদলে যাচাই করুন।

```
# Least-privilege উদাহরণ: শুধু একটি bucket পড়ার অনুমতি
gcloud storage buckets add-iam-policy-binding gs://technova-uploads \
  --member="serviceAccount:technova-api@technova-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"

# কার কী অনুমতি আছে, তা পরীক্ষা করা
gcloud projects get-iam-policy technova-prod --format=json | grep -A2 technova-api
```

২২.৫ Service Account Security ও Workload Identity Federation

Service account হলো অ্যাপ্লিকেশনের পরিচয়। নিয়ম: (১) প্রতিটি workload-এর আলাদা service account — একটি আপস হলে অন্যগুলো নিরাপদ; (২) key ফাইল তৈরি করবেন না — Workload Identity (GKE), attached service account (Compute Engine/Cloud Run), বা Workload Identity Federation (বাইরের সিস্টেম, যেমন GitHub Actions — অধ্যায় ১৯) ব্যবহার করুন; (৩) key যদি থাকেই (legacy), তবে নিয়মিত ঘোরান, ব্যবহার নিরীক্ষা করুন, আর দ্রুত সরিয়ে ফেলুন; (৪) service account-কে মানুষের মতো ব্যবহার করবেন না — মানুষের জন্য মানব পরিচয়, অ্যাপের জন্য service account।

Workload Identity Federation (WIF): বাইরের পরিচয় প্রদানকারী (GitHub, AWS, on-premises OIDC) থেকে token নিয়ে Google স্বল্পমেয়াদি credential দেয়। দীর্ঘমেয়াদি key কোথাও সংরক্ষিত হয় না, ফাঁস হওয়ার কিছু নেই। বাইরের CI/CD, on-premises সার্ভার, বা অংশীদার সিস্টেম থেকে GCP-তে আসার একমাত্র সঠিক পথ এটি।

২২.৬ Cloud KMS ও Secret Manager

Cloud KMS (Key Management Service): encryption key পরিচালনা করে — key তৈরি, ঘোরানো (rotation), নিষ্ক্রিয় করা, আর কে key ব্যবহার করতে পারে তা IAM দিয়ে নিয়ন্ত্রণ করা। GCP-তে ডেটা বিশ্রামে (at rest) ডিফল্টভাবে Google-managed key দিয়ে এনক্রিপ্ট হয়; CMEK (Customer-Managed Encryption Key) ব্যবহার করলে আপনি নিজের key নিয়ন্ত্রণ করেন — key নিষ্ক্রিয় করলে ডেটা অপঠনযোগ্য হয়। Key-র উদ্দেশ্য আলাদা রাখুন (একটি key সব জায়গায় নয়), rotation চালু করুন, আর key-র ব্যবহার Audit Log-এ নিরীক্ষা করুন।

Secret Manager: API key, পাসওয়ার্ড, টোকেনের মতো secret সংরক্ষণ, versioning ও access নিয়ন্ত্রণের পরিষেবা। Secret-এর প্রতিটি সংস্করণ আলাদা; পুরোনো সংস্করণ নিষ্ক্রিয় করা যায়; access শুধু `roles/secretmanager.secretAccessor` পাওয়া পরিচয় পায়। অ্যাপ Secret Manager থেকে runtime-এ secret নেয় — env var-এ, Git-এ, বা ইমেজে secret থাকে না।

```
# Secret তৈরি ও ব্যবহার
gcloud secrets create db-password --replication-policy=automatic
echo -n "CHANGE-ME-strong-password" | gcloud secrets versions add db-password --data-file=-
# Cloud Run-এ secret হিসেবে mount করা
gcloud run services update technova-api --region=asia-south1 \
  --update-secrets=DB_PASSWORD=db-password:latest
# GKE-তে Secret Manager থেকে secret নিতে Secret Store CSI Driver বা
# Workload Identity-সহ অ্যাপ-স্তরের client ব্যবহার করুন।
```

> **সতর্কতা:** Secret Manager-এর secret-ও যদি কোনো service account-কে `secretAccessor` দেওয়া হয়, তবে সেই পরিচয় আপস হলে secret ফাঁস হয়। Secret-এর access-ও least-privilege: শুধু যে workload-এর দরকার, শুধু সেই secret-এ। আর secret ঘোরানোর পরিকল্পনা রাখুন — ডেটাবেস পাসওয়ার্ড বদলালে অ্যাপ কীভাবে নতুন secret পাবে?

২২.৭ Security Command Center (SCC)

SCC হলো GCP-র নিরাপত্তা ও ঝুঁকি ব্যবস্থাপনার কেন্দ্রীয় ড্যাশবোর্ড: ভুল কনফিগারেশন (যেমন পাবলিক bucket, খোলা ফায়ারওয়াল), দুর্বলতা (অনিরাপদ ইমেজ), সক্রিয় হুমকি (অস্বাভাবিক API কল) — সব এক জায়গায়, তীব্রতা অনুযায়ী সাজানো। SCC-র ফাইন্ডিংগুলো নিয়মিত পর্যালোচনা করুন, গুরুতরগুলো ঠিক করার দায়িত্ব ও সময়সীমা নির্ধারণ করুন। Premium tier-এ আরও গভীর হুমকি শনাক্তকরণ থাকে; খরচ ও প্রয়োজন যাচাই করে সিদ্ধান্ত নিন (মূল্য Pricing Calculator-এ দেখুন)।

২২.৮ Cloud Armor ও VPC Service Controls

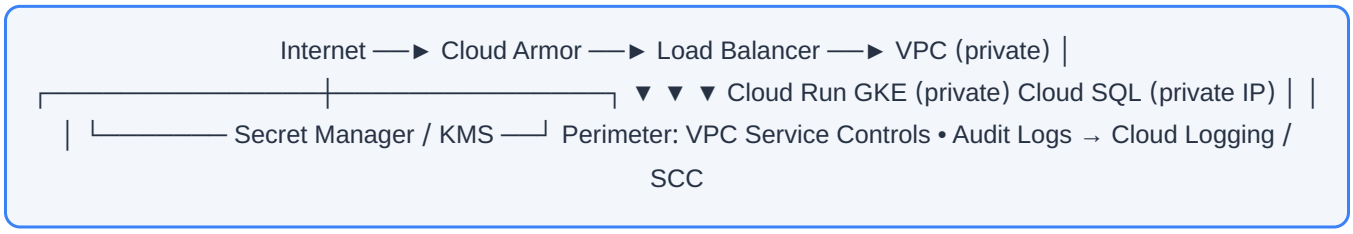
Cloud Armor: LoadBalancer-এর সামনে ওয়েব অ্যাপ্লিকেশন ফায়ারওয়াল (WAF) ও DDoS সুরক্ষা। নিয়ম দিয়ে SQL injection, cross-site scripting-এর মতো পরিচিত আক্রমণ আটকান, নির্দিষ্ট দেশ/IP ব্লক করুন, rate limit বসান। Cloud Armor নিয়ম পরীক্ষামূলকভাবে (preview mode) চালিয়ে প্রথমে দেখুন কী ব্লক হতো, তারপর প্রয়োগ করুন — ভুল নিয়ম বৈধ ব্যবহারকারীকে আটকে দিতে পারে।

VPC Service Controls: GCP পরিষেবাগুলোর চারপাশে একটি নিরাপত্তা পরিধি (perimeter) আঁকে — পরিধির ভেতরের ডেটা বাইরে যাবে না, এমনকি চুরি হওয়া credential দিয়েও নয়। উদাহরণ: Cloud Storage bucket-এর ডেটা শুধু আপনার VPC থেকে পড়া যাবে, ইন্টারনেট থেকে নয়। এটি ডেটা exfiltration-এর বিরুদ্ধে শক্তিশালী নিয়ন্ত্রণ, কিন্তু সেটআপে সতর্কতা দরকার — ভুল পরিধি বৈধ integration ভেঙে দেয়। প্রথমে dry-run mode-এ চালান।

২২.৯ Organization Policy ও Encryption

Organization Policy: পুরো organization-এ নিয়ম আরোপ করে — যেমন "কোনো VM-এর public IP থাকবে না", "শুধু নির্দিষ্ট অঞ্চলে resource তৈরি হবে", "service account key তৈরি নিষিদ্ধ"। এটি প্রতিরোধমূলক নিয়ন্ত্রণ: ভুল করলেও নীতি তা আটকায়। নতুন নিয়ম dry-run করে প্রভাব দেখে তারপর প্রয়োগ করুন।

Encryption: ডেটা বিশ্রামে (at rest) — GCP ডিফল্টভাবে এনক্রিপ্ট করে, CMEK দিয়ে নিজের key নিয়ন্ত্রণ করুন; ডেটা চলাচলে (in transit) — সবসময় TLS, অভ্যন্তরীণ যোগাযোগেও; Cloud SQL-এর সংযোগে TLS বাধ্যতামূলক করুন। Encryption কোনো বিকল্প নয় — এটি ভিত্তি।



২২.১০ নিরাপত্তা নিরীক্ষা ও ঘটনায় সাড়া

Audit: Cloud Audit Logs (Admin Activity, Data Access, System Event) চালু রাখুন — কে কী করল, কখন, কোথা থেকে। Data Access লগ বিস্তারিত ও ব্যয়বহুল হতে পারে; সংবেদনশীল resource-এ সীমিত করুন। লগ BigQuery-তে export করে অস্বাভাবিক আচরণ খুঁজুন (যেমন রাত ৩টায় নতুন service account key তৈরি)। **ঘটনায় সাড়া:** সন্দেহজনক credential পেলে প্রথমে তা নিষ্ক্রিয়/ঘোরান, তারপর প্রভাব যাচাই করুন (কী access করা হয়েছে?), তারপর মূলে ঠিক করুন। Incident response-এর বিস্তারিত অধ্যায় ২৩-এ।

২২.১১ সাধারণ ভুল

- `roles/owner` বা `roles/editor` দেওয়া।
- service account key ফাইল তৈরি করে Git/ইমেইলে রাখা।
- secret কোডে, env ফাইলে বা ইমেজে রাখা।
- পাবলিক bucket বা খোলা ফায়ারওয়াল (0.0.0.0/0) রাখা।
- Audit Log বন্ধ করা বা কখনো না দেখা।
- Cloud Armor নিয়ম পরীক্ষা ছাড়া প্রয়োগ করা।

২২.১২ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
403 Permission denied	role নেই বা ভুল resource-এ	Policy Troubleshooter; সঠিক member/role যাচাই
Secret পড়া যাচ্ছে না	secretAccessor নেই	Secret-স্তরের IAM যাচাই
VPC Service Controls-এ বৈধ কল ব্লক	পরিধি/ingress নিয়ম	dry-run লগ; access level যোগ করুন
Cloud Armor বৈধ ট্রাফিক আটকাচ্ছে	নিয়ম খুব কঠোর	preview mode-এ লগ দেখুন; নিয়ম শিথিল করুন
SCC-তে গুরুতর ফাইন্ডিং	ভুল কনফিগারেশন	ফাইন্ডিংয়ের নির্দেশনা অনুযায়ী সারান; পুনরায় স্ক্যান

২২.১৩ সারাংশ

নিরাপত্তা হলো স্তর: least-privilege IAM, key-ছাড়া পরিচয় (Workload Identity / WIF), KMS-এ key ও Secret Manager-এ secret, SCC দিয়ে দৃশ্যমানতা, Cloud Armor দিয়ে প্রান্তরক্ষা, VPC Service Controls দিয়ে ডেটা পরিধি, Organization Policy দিয়ে প্রতিরোধ, আর Audit Log দিয়ে জবাবদিহি। কোনো একটি স্তরই যথেষ্ট নয়; একসঙ্গে তারা একটি আপসকে বিপর্যয়ে পরিণত হতে বাধা দেয়।

২২.১৪ পর্যালোচনা প্রশ্ন

- Least privilege কেন `roles/editor` দেওয়ার চেয়ে নিরাপদ?
- WIF কীভাবে key ফাঁসের ঝুঁকি দূর করে?
- CMEK কখন ব্যবহার করবেন, কখন ডিফল্ট encryption যথেষ্ট?
- VPC Service Controls কোন ধরনের আক্রমণ আটকায়?

- Cloud Armor ও VPC Service Controls-এর পার্থক্য কী?

২২.১৫ ব্যবহারিক অনুশীলন

- একটি service account-কে শুধু একটি bucket পড়ার অনুমতি দিয়ে অন্য bucket পড়ার চেষ্টা ব্যর্থ করুন।
- Secret Manager-এ একটি secret তৈরি করে Cloud Run থেকে তা পড়ুন; Git-এ secret নেই তা যাচাই করুন।
- SCC-র ফাইন্ডিং তালিকা দেখে একটি গুরুতর ফাইন্ডিং সারান।

অধ্যায় ২৩ • PART V — Production GCP Engineering

Chapter 23 — Reliability, Backup and Disaster Recovery

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: high availability ও fault tolerance-এর পার্থক্য ব্যাখ্যা করতে; RPO ও RTO দিয়ে পুনরুদ্ধার লক্ষ্য নির্ধারণ করতে; ব্যাকআপ ও পুনরুদ্ধার কৌশল সাজাতে; cross-region স্থাপত্য ডিজাইন করতে; ডেটাবেস পুনরুদ্ধার পরীক্ষা করতে; incident response ও runbook লিখতে; blameless postmortem করতে; এবং reliability testing (chaos/game day) চালাতে।

২৩.২ পূর্বজ্ঞান ঝালাই

কোনো সিস্টেমই কখনো ভাঙে না — এমন প্রতিশ্রুতি মিথ্যা। জোন বিভ্রাট হয়, ডিস্ক নষ্ট হয়, মানুষ ভুল কমান্ড চালায়, ডেটাবেসে ভুল delete যায়। নির্ভরযোগ্যতা-প্রকৌশলের প্রশ্ন "ভাঙবে কি না" নয়, "ভাঙলে কতটা ক্ষতি হবে, আর কত দ্রুত ফিরব"। আগের অধ্যায়গুলোতে আপনি multi-zone GKE, regional ক্লাস্টার, alerting শিখেছেন; এই অধ্যায়ে সেগুলোকে একটি পুনরুদ্ধার পরিকল্পনায় বাঁধবেন। মনে রাখবেন: পরীক্ষা না করা ব্যাকআপ ব্যাকআপ নয় — এটি আশা মাত্র।

> **CONCEPT:** দুটি সংখ্যা সব পুনরুদ্ধার পরিকল্পনার ভিত্তি। **RPO (Recovery Point Objective)** — কতটা ডেটা হারানো গ্রহণযোগ্য (যেমন ১ ঘণ্টা: ব্যাকআপ প্রতি ঘণ্টায় নিলে সর্বোচ্চ ১ ঘণ্টার ডেটা হারাবে)। **RTO (Recovery Time Objective)** — কত দ্রুত ফিরতে হবে (যেমন ৪ ঘণ্টা: বিভ্রাটের ৪ ঘণ্টার মধ্যে সেবা ফিরবে)। RPO ছোট করতে ঘন ঘন ব্যাকআপ/প্রতিলিপি লাগে, RTO ছোট করতে প্রস্তুত standby লাগে — দুটোই খরচ বাড়ায়। ব্যবসার সঙ্গে বসে এই দুটি সংখ্যা ঠিক করুন, অনুমানে নয়।

২৩.৩ প্রথম নীতি: ব্যর্থতার ধরন বুঝুন

ব্যর্থতা চার স্তরে আসে: (১) **প্রক্রিয়া/ Pod ব্যর্থতা** — Kubernetes নিজে সারায়; (২) **জোন ব্যর্থতা** — multi-zone স্থাপত্য সারায়; (৩) **অঞ্চল (region) ব্যর্থতা** — cross-region স্থাপত্য লাগে; (৪) **ডেটা ব্যর্থতা** — ভুল delete, দুর্নীতি, ransomware — শুধু ব্যাকআপ সারায়, প্রতিলিপি নয় (প্রতিলিপি ভুলটিও কপি করে!)। প্রতিটি স্তরের জন্য আলাদা প্রতিরক্ষা দরকার; একটি স্তরের সমাধান অন্য স্তরকে বাঁচায় না। এটাই সবচেয়ে সাধারণ ভুল: "আমাদের তো replica আছে" বলে ব্যাকআপ না রাখা — replica ভুল delete-ও replicate করে।

২৩.৪ High Availability ও Fault Tolerance

High Availability (HA) মানে ব্যর্থতা হলেও সেবা চালু থাকে — সাধারণত redundancy দিয়ে (একাধিক জোনে রেপ্লিকা)। **Fault Tolerance** আরও শক্তিশালী: ব্যর্থতা ব্যবহারকারী টেরই পায় না (যেমন স্বয়ংক্রিয় failover)। GCP-তে HA-র উপাদান: regional GKE ক্লাস্টার, multi-zone Managed Instance Group, Cloud SQL high availability (standby ইনস্ট্যান্স অন্য জোনে, স্বয়ংক্রিয় failover), Cloud Storage-এর multi-region replication, Global Load Balancer (একটি অঞ্চল ডাউন হলে ট্রাফিক অন্য অঞ্চলে)। Single point of failure খুঁজুন: একটি মাত্র জোন, একটি মাত্র ডেটাবেস ইনস্ট্যান্স, একটি মাত্র NAT gateway — প্রতিটির জন্য প্রশ্ন করুন, "এটি মারা গেলে কী হয়?"

Users ──► Global Load Balancer ├─► asia-south1 (primary): GKE regional + Cloud SQL HA └─► asia-southeast1 (standby): GKE + Cloud SQL read replica Cloud Storage (multi-region) • Backups → Cloud Storage (separate region) DNS / Traffic Director: failover to standby region when primary is down

২৩.৫ ব্যাকআপ ও পুনরুদ্ধার

ব্যাকআপের নিয়ম **3-2-1**: ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন স্থানে (অন্য অঞ্চলে)। GCP-তে: Cloud SQL-এর automated backup (point-in-time recovery সহ) ও on-demand backup; Persistent Disk snapshot (নিয়মিত, অন্য অঞ্চলে কপি); Cloud Storage-এর versioning ও Object Versioning; GKE-র জন্য Backup for GKE (PersistentVolume-সহ ক্লাস্টার ব্যাকআপ); Firestore-এর scheduled export। প্রতিটি ব্যাকআপের retention ঠিক করুন, আর **পুনরুদ্ধার পরীক্ষা করুন** — একটি staging পরিবেশে ব্যাকআপ থেকে ডেটাবেস ফিরিয়ে আনুন, ডেটা যাচাই করুন, সময় মাপুন। পরীক্ষা ছাড়া RTO একটি অনুমান মাত্র।

```
# Cloud SQL automated backup যাচাই ও on-demand backup
gcloud sql instances describe technova-db --format="value(settings.backupConfiguration)"
gcloud sql backups create --instance=technova-db

# Persistent Disk snapshot (নিয়মিত — Cloud Scheduler + Cloud Function দিয়ে স্বয়ংক্রিয় করুন)
gcloud compute snapshots create technova-disk-snap-20261109 \
  --source-disk=technova-disk --source-disk-zone=asia-south1-a \
  --storage-location=asia-southeast1

# GKE Backup for GKE — BackupPlan তৈরি (Console: GKE → Backup → Create plan)
```

ব্যাকআপের নিরাপত্তা: ব্যাকআপেও সংবেদনশীল ডেটা থাকে — encryption, access নিয়ন্ত্রণ, আর ransomware থেকে বাঁচতে immutable/retention lock বিবেচনা করুন। ব্যাকআপের access শুধু পুনরুদ্ধারের দায়িত্বে থাকা পরিচয়ের।

২৩.৬ Cross-Region স্থাপত্য

একটি অঞ্চল পুরোপুরি ডাউন হলে কী হবে? দুটি পদ্ধতি: **Active-passive** — প্রাথমিক অঞ্চল সক্রিয়, দ্বিতীয় অঞ্চল standby (কম খরচ, failover-এ সময় লাগে); **Active-active** — দুই অঞ্চলেই ট্রাফিক চলে (দ্রুত failover, দ্বিগুণ খরচ, ডেটা সিঙ্ক জটিল)। বেশিরভাগ দলের জন্য active-passive যথেষ্ট: Cloud SQL cross-region replica, Storage multi-region, GKE দ্বিতীয় অঞ্চলে standby (ছোট আকারে, প্রয়োজনে স্কেল)। Failover-এর সিদ্ধান্ত কে নেবে, DNS কীভাবে বদলাবে, ডেটা কীভাবে সিঙ্ক হবে — এগুলো আগেই লিখে রাখুন, বিভ্রাটের সময় আবিষ্কার করবেন না। ডেটা সিঙ্কে lag থাকে — RPO সেই lag-এর চেয়ে ছোট হতে পারে না।

২৩.৭ ডেটাবেস পুনরুদ্ধার

ডেটাবেস পুনরুদ্ধারের তিনটি পরিস্থিতি: (১) **ইনস্ট্যান্স ব্যর্থতা** — Cloud SQL HA স্বয়ংক্রিয়ভাবে standby-তে failover করে (কয়েক মিনিট); (২) **ভুল ডেটা পরিবর্তন** — point-in-time recovery (PITR) দিয়ে নির্দিষ্ট সময়ের আগের অবস্থায় ফেরা; (৩) **পুরো অঞ্চল ব্যর্থতা** — cross-region replica-কে প্রমোট করা। PITR-এর জন্য transaction log retention চালু রাখুন; replica প্রমোশন পরীক্ষা করুন — প্রমোট করা replica আর আগের primary-র সঙ্গে সিঙ্ক হয় না, ফিরে আসার পরিকল্পনা (failback) আলাদা করে ভাবুন।

> **সতর্কতা:** `DROP TABLE` বা ভুল `DELETE`-এর পর প্রথম কাজ প্যানিক করে কিছু না করা। লেখা বন্ধ করুন, ব্যাকআপ/PITR বিকল্প যাচাই করুন, তারপর পরিকল্পিতভাবে ফেরান। তাড়াহুড়ো করে production-এ পরীক্ষা করা দ্বিতীয় বিপর্যয় ডেকে আনে।

২৩.৮ Incident Response ও Runbook

Incident-এর পাঁচটি পর্যায়: **শনাক্ত** (alert, ব্যবহারকারীর অভিযোগ), **ঘোষণা** (incident ঘোষণা, commander নিয়োগ), **সমন্বয়** (যোগাযোগের চ্যানেল, অবস্থা আপডেট), **প্রশমন** (সেবা ফেরানো — মূল কারণ খোঁজা নয়, আগে সেবা ফেরান), **সমাধান ও পর্যালোচনা** (postmortem)। প্রতিটি সাধারণ incident-এর জন্য **runbook** লিখুন: লক্ষণ, যাচাইয়ের ধাপ, প্রশমনের ধাপ (যেমন ট্রাফিক ফেরানো, স্কেল বাড়ানো, feature flag বন্ধ করা), কাকে ডাকবেন। Runbook ছাড়া incident মানে প্রতিবার নতুন করে আবিষ্কার।

যোগাযোগ: ব্যবহারকারীকে নিয়মিত জানান — কী ভেঙেছে, কারা প্রভাবিত, কখন পরের আপডেট। নীরবতা আস্থা নষ্ট করে, সততা নয়।

২৩.৯ Blameless Postmortem

Incident শেষে postmortem লিখুন — দোষারোপ ছাড়া (blameless)। প্রশ্নগুলো: কী ঘটল (সময়রেখা), কেন ঘটল (মূল কারণ — সাধারণত একাধিক), কী ভালো হলো, কী খারাপ হলো, ভবিষ্যতে কী বদলাব (action item, মালিক ও সময়সীমাসহ)। মানুষকে দোষ দিলে মানুষ ভুল লুকায়; সিস্টেমকে দোষ দিলে সিস্টেম ঠিক হয়। Postmortem পড়া হয় কি না, তা যাচাই করুন — action item-এর অগ্রগতি পর্যালোচনা করুন।

২৩.১০ Reliability Testing: Game Day ও Chaos

নির্ভরযোগ্যতা পরীক্ষা না করলে জানা যায় না। **Game day:** নির্ধারিত সময়ে ইচ্ছাকৃতভাবে একটি ব্যর্থতা ঘটানো (যেমন একটি জোনের Pod বন্ধ করা, একটি ডেটাবেস failover ঘটানো) ও দলের প্রতিক্রিয়া দেখা। **Chaos engineering:** নিয়মিত, ছোট, নিয়ন্ত্রিত ব্যর্থতা ঢুকিয়ে সিস্টেমের দুর্বলতা খুঁজে বের করা — production-এ, কিন্তু blast radius সীমিত রেখে (একটি namespace, অল্প ট্রাফিক)। শুরু করুন staging-এ, তারপর production-এ ছোট পরিসরে। প্রতিটি পরীক্ষার আগে rollback পরিকল্পনা ও abort শর্ত ঠিক করুন।

২৩.১১ সাধারণ ভুল

- ব্যাকআপ নেওয়া কিন্তু পুনরুদ্ধার কখনো পরীক্ষা না করা।
- replica-কে ব্যাকআপ ভাবা।
- RPO/RTO ব্যবসার সঙ্গে ঠিক না করে অনুমান করা।
- একটি মাত্র জোন/অঞ্চলে সব রাখা।
- incident-এর সময় মূল কারণ খুঁজতে গিয়ে সেবা ফেরানো দেরি করা।
- postmortem লিখে action item-এর খোঁজ না নেওয়া।

২৩.১২ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
Failover হচ্ছে না	HA চালু নেই, replica lag বেশি	Cloud SQL HA সেটিং; replica lag মেট্রিক
ব্যাকআপ থেকে ফেরানো যাচ্ছে না	ব্যাকআপ নষ্ট/পুরোনো, অনুমতি নেই	staging-এ নিয়মিত restore পরীক্ষা
PITR-এর সময় পাওয়া যাচ্ছে না	log retention কম	retention বাড়ান; backup নীতি যাচাই
Cross-region failover-এ ডেটা পুরোনো	replication lag	lag মনিটর; RPO বাস্তবসম্মত করুন
Incident-এ বিভ্রান্তি	runbook নেই, ভূমিকা অস্পষ্ট	runbook লিখুন; game day-তে অনুশীলন

২৩.১৩ সারাংশ

নির্ভরযোগ্যতা মানে ব্যর্থতা এড়ানো নয় — ব্যর্থতার প্রভাব সীমিত করা ও দ্রুত ফেরা। RPO/RTO ঠিক করুন, 3-2-1 ব্যাকআপ রাখুন ও পুনরুদ্ধার পরীক্ষা করুন, multi-zone/multi-region সাজান, runbook লিখুন, blameless postmortem করুন, আর game day-তে নিয়মিত অনুশীলন করুন। যে পরিকল্পনা পরীক্ষা করা হয়নি, সেটি পরিকল্পনা নয়।

২৩.১৪ পর্যালোচনা প্রশ্ন

- RPO ও RTO-র পার্থক্য কী, এবং এগুলো খরচকে কীভাবে প্রভাবিত করে?
- Replica কেন ব্যাকআপের বিকল্প নয়?
- Active-passive ও active-active-এর মধ্যে কখন কোনটি?
- Incident-এ প্রশমন ও মূল কারণ অনুসন্ধানের ক্রম কেন গুরুত্বপূর্ণ?
- Blameless postmortem কেন দোষারোপের চেয়ে কার্যকর?

২৩.১৫ ব্যবহারিক অনুশীলন

- একটি Cloud SQL ইনস্ট্যান্সের ব্যাকআপ থেকে staging-এ পুনরুদ্ধার করে সময় মাপুন।
- TechNova-র একটি পরিষেবার জন্য RPO/RTO লিখে ব্যাকআপ ও failover পরিকল্পনা বানান।
- একটি game day চালান: একটি Pod/জোন বন্ধ করে দলের প্রতিক্রিয়া ও runbook পরীক্ষা করুন।

অধ্যায় ২৪ • PART V — Production GCP Engineering

Chapter 24 — Production Architecture and FinOps

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: একটি production-উপযোগী GCP স্থাপত্যের সম্পূর্ণ ছবি আঁকতে; organization governance ও multi-project কৌশল ডিজাইন করতে; multi-region নকশা করতে; খরচ অপ্টিমাইজ করতে (FinOps); কর্মক্ষমতা-প্রকৌশল করতে; নিরাপত্তা governance সাজাতে; operational readiness যাচাই করতে; TechNova-র MERN স্থাপত্য GCP-তে বসাতে; architecture decision record (ADR) লিখতে; এবং একটি capstone walkthrough-এ পুরো প্ল্যাটফর্ম একত্র করতে।

২৪.২ পূর্বজ্ঞান ঝালাই

এই খণ্ডের আগের অধ্যায়গুলোতে আপনি প্রতিটি ইট শিখেছেন: GKE (১৭), সার্ভারলেস ও ইভেন্ট (১৮), CI/CD (১৯), IaC (২০), পর্যবেক্ষণ (২১), নিরাপত্তা (২২), নির্ভরযোগ্যতা (২৩)। এই অধ্যায়ে ইটগুলো দিয়ে বাড়ি বানাবেন। মূল প্রশ্ন বদলে গেছে: "এই পরিষেবাটি কীভাবে কাজ করে?" থেকে "এই সবকিছু একসঙ্গে কীভাবে একটি ব্যবসা চালায় — নিরাপদে, নির্ভরযোগ্যভাবে, ও সাশ্রয়ে?" উত্তরটি কোনো একক চিত্রে নেই; এটি governance, স্থাপত্য, খরচ ও পরিচালনার সমন্বয়।

> **CONCEPT:** Production-উপযোগী (production-ready) মানে "চলে" নয়; মানে "ভাঙলে বোঝা যায়, ভাঙলে ফেরা যায়, কে কী করেছে জানা যায়, খরচ নিয়ন্ত্রণে থাকে, আর নিরাপত্তা কারও সদিচ্ছার ওপর নির্ভর করে না"। এই পাঁচটি পরীক্ষায় পাস না করলে সিস্টেম production-এ যাওয়ার যোগ্য নয় — যতই সুন্দর কোড হোক।

২৪.৩ প্রথম নীতি: স্থাপত্য মানে বিনিময় (Trade-off)

প্রতিটি স্থাপত্য সিদ্ধান্ত একটি বিনিময়: multi-region নির্ভরযোগ্যতা দেয়, খরচ ও জটিলতা বাড়ায়; microservices স্বাধীনতা দেয়, পর্যবেক্ষণ কঠিন করে; সার্ভারলেস পরিচালনা কমায়, নিয়ন্ত্রণ কমায়। ভালো স্থপতি সবচেয়ে "আধুনিক" নকশা বেছে নেন না — ব্যবসার সীমাবদ্ধতায় (বাজেট, দলের আকার, নিয়ম, RPO/RTO) সবচেয়ে উপযুক্ত নকশা বেছে নেন, আর বিনিময়টি লিখে রাখেন। এজন্যই এই অধ্যায়ের শেষে ADR (Architecture Decision Record) শিখবেন: সিদ্ধান্ত, প্রসঙ্গ, বিকল্প, ও পরিণতি — যাতে ছয় মাস পর কেউ জিজ্ঞাসা করলে উত্তর থাকে।

২৪.৪ Organization Governance ও Multi-Project Strategy

একটি প্রজেক্টে সবকিছু রাখা শুরুতে সহজ, পরে বিপজ্জনক: dev-এর ভুল prod-কে স্পর্শ করে, খরচ আলাদা করা যায় না, IAM জটিল হয়। সঠিক কাঠামো: **Organization → Folder (পরিবেশ/দল অনুযায়ী) → Project (প্রতি পরিবেশে আলাদা)**। উদাহরণ: `technova-prod`, `technova-staging`, `technova-dev` আলাদা প্রজেক্ট; shared services (যেমন নেটওয়ার্ক, লগ সংরক্ষণ)-এর জন্য আলাদা প্রজেক্ট (host project — Shared VPC)। প্রতিটি প্রজেক্টের নিজস্ব billing, quota ও IAM সীমানা থাকে — একটি পরিবেশের বিভ্রাট অন্য পরিবেশকে স্পর্শ করে না।

Governance-এর উপাদান: Organization Policy (অধ্যায় ২২) — যেমন শুধু অনুমোদিত অঞ্চল, public IP নিষেধ; IAM-এ Google Group ব্যবহার (ব্যক্তি নয়, দলকে role দিন — দল বদলালে group বদলায়, policy নয়); Audit Logs কেন্দ্রীয় প্রজেক্টে সংগ্রহ; Billing-এ budget alert প্রতি প্রজেক্টে। নতুন প্রজেক্ট তৈরির প্রক্রিয়া লিখিত থাকুক — কে অনুরোধ করে, কে অনুমোদন করে, কোন নীতি প্রযোজ্য হয়।

২৪.৫ Multi-Region Design

Global: Cloud DNS • Cloud Armor • Global Load Balancer • Cloud CDN asia-south1 (primary, active): VPC → GKE regional (Autopilot) → Cloud Run (spiky workloads) Cloud SQL (HA) • Memorystore (Redis) • Artifact Registry Pub/Sub • Cloud Storage (multi-region) • Secret Manager • KMS asia-southeast1 (standby): GKE (scaled-down) • Cloud SQL cross-region replica • Storage replica Observability: Cloud Monitoring/Logging/Trace (central) • SCC Delivery: Cloud Build → Artifact Registry → Cloud Deploy (staging → prod) IaC: Terraform (remote state in Cloud Storage) • Infrastructure Manager

Multi-region মানে শুধু দ্বিতীয় অঞ্চলে কপি নয় — মানে ডেটা সিঙ্ক, ট্রাফিক রাউটিং, failover সিদ্ধান্ত ও failback পরিকল্পনা। প্রাথমিক অঞ্চলে সক্রিয় ট্রাফিক, দ্বিতীয় অঞ্চলে standby (অধ্যায় ২৩-এর active-passive)। Global Load Balancer স্বাস্থ্য পরীক্ষা করে ট্রাফিক সরায়; Cloud DNS failover-এর অংশ। ডেটা স্তরে: Cloud Storage multi-region, Cloud SQL cross-region replica, Firestore multi-region (প্রয়োজনে)। মনে রাখবেন: multi-region খরচ দ্বিগুণ করে না — standby ছোট রাখা যায় — কিন্তু জটিলতা বাড়ায়; RTO/RPO-র দাবি না থাকলে multi-region-এ যাবেন না।

২৪.৬ Production MERN Architecture on GCP

TechNova-র MERN অ্যাপ (MongoDB-ধরনের ডকুমেন্ট ডেটা, Express API, React frontend, Node.js) GCP-তে এভাবে বসে:

স্তর	GCP বাস্তবায়ন	কেন
Frontend (React)	Cloud Storage + Cloud CDN, অথবা Cloud Run	স্থির সম্পদ CDN থেকে দ্রুত, সস্তা; SSR দরকার হলে Cloud Run
API (Express/ Node)	GKE Autopilot (মূল) বা Cloud Run	স্থির ট্রাফিক GKE-তে, স্পাইক Cloud Run-এ; দুটোই Workload Identity ব্যবহার করে
ডেটা	Cloud SQL (PostgreSQL) বা Firestore / MongoDB Atlas (GCP-তে)	পরিচালিত ডেটাবেস: ব্যাকআপ, HA, প্যাচ Google-এর দায়িত্বে
Cache/Queue	Memorystore (Redis) + Pub/Sub + Cloud Tasks	সেশন/cache ও async কাজের জন্য
ফাইল	Cloud Storage (signed URL দিয়ে আপলোড)	সরাসরি ক্লায়েন্ট আপলোড, সার্ভার মধ্যস্থতা করে না
অনুসন্ধান/ বিশ্লেষণ	BigQuery (লগ/ইভেন্ট export)	দীর্ঘমেয়াদি বিশ্লেষণ, খরচ-কার্যকর

প্রবাহ: ব্যবহারকারী → Cloud Armor → Global Load Balancer → Cloud CDN (স্থির) / GKE Ingress (API) → API Pod → Cloud SQL / Memorystore; async কাজ Pub/Sub → Cloud Run worker; ফাইল আপলোড signed URL দিয়ে সরাসরি Storage-এ, Eventarc ধরে thumbnail function চালায়। সব যোগাযোগ private IP / VPC-র ভেতরে, যেখানে সম্ভব; secret Secret Manager থেকে; প্রতিটি স্তরে মেট্রিক, লগ ও trace।

২৪.৭ Cost Optimization ও FinOps

FinOps মানে খরচকে প্রকৌশল সিদ্ধান্ত হিসেবে দেখা — মাস শেষে বিল দেখে অবাক হওয়া নয়। অনুশীলনগুলো:

- **দৃশ্যমানতা:** প্রতিটি resource-এ লেবেল (env, team, service); Billing export BigQuery-তে; প্রতি পরিষেবার খরচ ড্যাশবোর্ডে; budget alert ৫০%, ৮০%, ১০০%-এ।
- **অপচয় দূর করা:** অব্যবহৃত Persistent Disk, পুরোনো snapshot, idle LoadBalancer, অতিরিক্ত বড় VM, অপ্রয়োজনীয় multi-region replication — নিয়মিত পর্যালোচনা।
- **সঠিক আকার (right-sizing):** Recommender-এর পরামর্শ (idle VM, অতিরিক্ত CPU) অনুসরণ; GKE-তে request বাস্তব ব্যবহারের কাছাকাছি; Cloud Run-এ min-instances শুধু যেখানে দরকার।
- **প্রতিশ্রুতি (commitment):** স্থির workload-এ Committed Use Discount (CUD) — ১ বা ৩ বছরের প্রতিশ্রুতিতে ছাড়; তবে শুধু স্থির অংশে, স্পাইকে নয়।
- **সার্ভারলেসের খরচ:** scale-to-zero সস্তা, কিন্তু min-instances ও always-allocated CPU খরচ বাড়ায়; Pub/Sub-এ বার্তার আকার, Logging ingestion, egress — এগুলো নীরবে বাড়ে।
- **সীমা:** max-instances, quota, billing alert — যাতে একটি বাগ বা আক্রমণ বিলকে আকাশে না তোলে।

সঠিক মূল্য সবসময় Google Cloud Pricing Calculator-এ যাচাই করুন; এই বইয়ের কোনো সংখ্যাই চিরস্থায়ী নয়, আর free credit, ছাড় ও quota পরিবর্তন হতে পারে। খরচের সিদ্ধান্ত নেওয়ার আগে Calculator-এ মডেল করুন।

> **সতর্কতা:** খরচ কমানোর নামে নির্ভরযোগ্যতা কাটবেন না — একটি replica কমিয়ে বাঁচানো টাকা একটি বিভ্রাটে বহুগুণ খরচ হয়। প্রথমে অপচয় কাটুন, তারপর আকার ঠিক করুন, তারপর প্রতিশ্রুতি দিন। আর egress (অঞ্চল/ক্লাউডের বাইরে ডেটা পাঠানো) সবসময় পরীক্ষা করুন — এক অঞ্চল থেকে অন্য অঞ্চলে অপ্রয়োজনীয় ডেটা প্রবাহ নীরবে বিল বাড়ায়।

২৪.৮ Performance Engineering

কর্মক্ষমতা মানে শুধু দ্রুত কোড নয় — পুরো পথ: DNS → CDN → LoadBalancer → অ্যাপ → ডেটাবেস। পরিমাপ করুন: p50/p95/p99 latency (গড় নয় — p99 বলে সবচেয়ে খারাপ অভিজ্ঞতা), throughput, এরর হার। অপ্টিমাইজেশনের ক্রম: (১) ডেটাবেস query ও ইনডেক্স (সবচেয়ে সাধারণ বোতল-ঘাড়); (২) cache (Memorystore, CDN) — ডেটাবেসে যাওয়া কমান; (৩) সংযোগ পুলিং — প্রতিটি অনুরোধে নতুন সংযোগ নয়; (৪) অ্যাপ-স্তর — N+1 query, অপ্রয়োজনীয় serialization; (৫) নেটওয়ার্ক — অঞ্চলের কাছাকাছি রাখুন, অপ্রয়োজনীয় cross-region কল কমান। Load test করুন production-এর মতো পরিবেশে, production-এর মতো ডেটায় — staging-এ ছোট ডেটায় পরীক্ষা মিথ্যা আশ্বাস দেয়।

২৪.৯ Security Governance ও Operational Readiness

Security governance: নিয়মিত SCC পর্যালোচনা, IAM পর্যালোচনা (অব্যবহৃত অনুমতি সরানো), vulnerability স্ক্যান (Artifact Registry, VM), secret rotation, incident response পরিকল্পনা (অধ্যায় ২২–২৩)। **Operational readiness checklist** — production-এ যাওয়ার আগে প্রতিটি পরিষেবার জন্য: runbook আছে? alert আছে? on-call কে? rollback কীভাবে? ব্যাকআপ পরীক্ষা হয়েছে? খরচের সীমা আছে? লগে secret নেই? এই তালিকা ছাড়া launch নয়। নতুন পরিষেবার launch-এ একটি ছোট পর্যালোচনা (readiness review) করুন — পাঁচ মিনিটের চেকলিস্ট একটি রাতের incident বাঁচায়।

২৪.১০ Architecture Decision Records (ADR)

ADR হলো একটি ছোট নথি (সাধারণত Markdown, রিপোজিটরিতে): শিরোনাম, তারিখ, অবস্থা (প্রস্তাবিত/গৃহীত/বাতিল), প্রসঙ্গ, সিদ্ধান্ত, বিবেচিত বিকল্প, পরিণতি। উদাহরণ:

```
# ADR-001: API-র জন্য GKE Autopilot বনাম Cloud Run
- Date: 2026-11-09 | Status: Accepted
- Context: TechNova API-র ট্রাফিক দিনে স্থির, রাতে কম; দলে Kubernetes অভিজ্ঞতা আছে।
- Decision: মূল API GKE Autopilot-এ; স্পাইক সামলানো worker Cloud Run-এ।
- Alternatives: সব Cloud Run (cold start ও দীর্ঘ সংযোগে ঝুঁকি);
 সব GKE Standard (পরিচালন-ভার বেশি)।
- Consequences: দুটি রানটাইম পরিচালনা করতে হবে; Workload Identity
 দুই জায়গায় একইভাবে কাজ করে; খরচ মাসে পর্যালোচনা করা হবে।
```

ADR-এর মূল্য ভবিষ্যতে: কেউ যখন জিজ্ঞাসা করে "কেন এভাবে?", উত্তর নথিতে থাকে — মানুষের স্মৃতিতে নয়।

২৪.১১ Capstone: TechNova Production-Ready GCP Platform Walkthrough

এবার পুরো ছবিটি একসঙ্গে দেখুন — TechNova-র production প্ল্যাটফর্ম, প্রথম অনুরোধ থেকে পুনরুদ্ধার পর্যন্ত:

১. ডেভেলপার পরিবর্তন করে: feature ব্রাঞ্চে push → PR-এ Cloud Build টেস্ট চালায় → merge হলে `main` trigger ইমেজ build করে Artifact Registry-তে push করে, vulnerability স্ক্যান চালায় → Cloud Deploy staging-এ ডিপ্লয় করে → অনুমোদন পেলে canary-সহ production-এ যায়। Infrastructure-এর পরিবর্তন একইভাবে Terraform PR-এ plan দেখায়, অনুমোদন পেলে apply হয়।

২. ব্যবহারকারী আসে: Cloud DNS → Cloud Armor (WAF + rate limit) → Global Load Balancer → Cloud CDN (স্থির সম্পদ) বা GKE Ingress (API)। API Pod Workload Identity দিয়ে Cloud SQL (private IP, HA) ও Memorystore-এ যায়; secret Secret Manager থেকে আসে; ফাইল signed URL-এ Storage-এ যায়।

৩. কিছু ভুল হয়: এরর হার বাড়ে → alert on-call-কে জাগায় → ড্যাশবোর্ডে golden signals, Error Reporting-এ নতুন গ্রুপ, Trace-এ ধীর span, Logs-এ trace ID ধরে কারণ → runbook অনুযায়ী ট্রাফিক আগের revision-এ ফেরানো হয় (প্রশমন) → postmortem লেখা হয়, action item তৈরি হয়।

৪. অঞ্চল ডাউন হয়: LoadBalancer স্বাস্থ্য পরীক্ষায় ব্যর্থতা দেখে ট্রাফিক standby অঞ্চলে সরায়; Cloud SQL cross-region replica প্রমোট করা হয়; RPO/RTO অনুযায়ী ডেটা হার ও downtime জানা থাকে — কারণ এগুলো আগেই পরীক্ষা করা হয়েছে।

৫. মাস শেষে: Billing ড্যাশবোর্ডে প্রতি পরিষেবার খরচ দেখা হয়; Recommender-এর পরামর্শে idle resource সরানো হয়; CUD পর্যালোচনা করা হয়; SCC-র ফাইন্ডিং ও IAM পর্যালোচনা করা হয়। প্ল্যাটফর্মটি শুধু চলে না — এটি পরিচালিত হয়।

> **বাস্তব উদাহরণ:** এই walkthrough-এর প্রতিটি উপাদানই এই খণ্ডে শেখা হয়েছে — কোনো জাদুকরী নতুন পরিষেবা নেই। পার্থক্য তৈরি করে শৃঙ্খলা: প্রতিটি সিদ্ধান্ত লেখা (ADR), প্রতিটি পরিবর্তন পর্যালোচিত (PR), প্রতিটি ব্যর্থতা পরীক্ষিত (game day), প্রতিটি খরচ দৃশ্যমান (FinOps)। TechNova-র প্ল্যাটফর্ম ছোট দিয়ে শুরু হয়েছিল — একটি Cloud Run সার্ভিস, একটি Cloud SQL ইনস্ট্যান্স — আর প্রয়োজন অনুযায়ী স্তর যোগ হয়েছে। আপনার প্ল্যাটফর্মও এভাবেই বাড়বে: ছোট শুরু, সঠিক ভিত্তি, ধাপে ধাপে।

২৪.১২ সাধারণ ভুল

- একটি প্রজেক্টে dev/staging/prod মেশানো।
- স্থাপত্য সিদ্ধান্ত না লেখা — ছয় মাস পর কেউ জানে না কেন এভাবে।
- খরচ শুধু মাস শেষে দেখা; লেবেল ছাড়া resource চালানো।
- multi-region-এ যাওয়া RPO/RTO না ঠিক করে।
- readiness checklist ছাড়া launch করা।
- গড় latency দেখে সন্তুষ্ট থাকা (p99 দেখুন)।

২৪.১৩ সমস্যা সমাধান

লক্ষণ	সম্ভাব্য কারণ	পরীক্ষা / সমাধান
খরচ হঠাৎ বেড়েছে	নতুন resource, স্পাইক, egress	Billing breakdown; লেবেল অনুযায়ী খরচ দেখুন
Cross-region latency বেশি	দূরের অঞ্চলে কল	কল পথ যাচাই; cache/CDN; অঞ্চল কাছাকাছি রাখুন
পরিবেশের মধ্যে পার্থক্য	হাতে বদল, drift	Terraform plan; IaC-তে ফেরান
Launch-এ প্রস্তুতি নেই	checklist নেই	readiness review; runbook ও alert যাচাই

২৪.১৪ সারাংশ

Production স্থাপত্য হলো governance (multi-project, organization policy), স্থাপত্য (multi-region, MERN on GCP), খরচ (FinOps), কর্মক্ষমতা (p99, cache, ডেটাবেস), নিরাপত্তা (স্তরে স্তরে), ও পরিচালনা (readiness, runbook, postmortem)-এর সমন্বয়। সিদ্ধান্তগুলো ADR-এ লিখুন, পরিবর্তনগুলো CI/CD দিয়ে যাক, ব্যর্থতাগুলো পরীক্ষা করুন, খরচগুলো দেখুন। এই খণ্ড এখানেই শেষ — কিন্তু আপনার প্ল্যাটফর্মের কাজ এখান থেকে শুরু।

২৪.১৫ পর্যালোচনা প্রশ্ন

- কেন dev/staging/prod আলাদা প্রজেক্টে রাখা উচিত?
- FinOps-এর তিনটি মূল অনুশীলন কী?
- ADR কেন লেখা হয়, এবং এতে কী থাকে?
- p99 latency কেন গড়ের চেয়ে গুরুত্বপূর্ণ?
- Operational readiness checklist-এ কী কী থাকা উচিত?

২৪.১৬ ব্যবহারিক অনুশীলন (Capstone)

- TechNova-র MERN অ্যাপের জন্য একটি পূর্ণ স্থাপত্য চিত্র আঁকুন: frontend, API, ডেটা, queue, observability, নিরাপত্তা ও ব্যাকআপসহ।
- একটি ADR লিখুন: GKE বনাম Cloud Run — আপনার প্রসঙ্গে কোনটি, কেন।
- একটি FinOps পর্যালোচনা করুন: একটি নমুনা প্রজেক্টের খরচ লেবেল অনুযায়ী ভাগ করে তিনটি সাশ্রয়ের সুযোগ চিহ্নিত করুন।

- একটি readiness checklist বানিয়ে একটি পরিষেবাকে production-এর জন্য যাচাই করুন।